

FR-1

Transaction Management

จดยรายการ เพิ่ม แก้ จัดหมวด ลบ
เข้าใจระบบทั้งหมดตั้งแต่ศูนย์ จนอธิบายโค้ดและตอบ
คำถามอาจารย์ได้ เหมือนเป็นคนสร้างระบบนี้เอง

ภาค ก · พื้นฐานทั้งระบบ (สำหรับคนไม่เคยเขียนโค้ด)

ภาค ข · FR-1 ละเอียดย่อยทุกส่วน

โค้ดจริงพร้อมคำอธิบายทีละบรรทัด



วิธีใช้เล่มนี้

เล่มนี้ทำให้คนที่รับผิดชอบ FR-1 Transaction Management เข้าใจระบบทั้งหมด ตั้งแต่ "แอปนี้สร้างด้วยอะไร" ไปจนถึง "บรรทัดนี้ของโค้ดทำอะไร" แม้ไม่เคยเขียนโปรแกรมมาก่อน

เล่มนี้แบ่งเป็น 2 ภาค

i ภาค ก • พื้นฐานทั้งระบบ

เหมือนกันทุกเล่ม (FR-1 ถึง FR-6) สอนตั้งแต่ศูนย์: เครื่องมือที่ใช้ (VS Code, Node.js, Git), ภาษา TypeScript, React, การทำแอปและเว็บจากโค้ดชุดเดียว, เซิร์ฟเวอร์ Supabase, การเชื่อม AI, การออกแบบหน้าจอ (UX/UI), การทดสอบ และ GitHub

★ ภาค ข • FR-1 Transaction Management

เฉพาะ FR ของเล่มนี้ เขียนในมุม "ฉันเป็นคนสร้างระบบนี้" มีหน้าจอจริง หลักการ ตัวอย่างคิดมือ โค้ดจริงพร้อมคำอธิบายทีละบรรทัด การทดสอบ เหตุผล การออกแบบ คำถามอาจารย์ แบบฝึกหัด และสรุปแนะนำ

เปิดโค้ดใน VS Code ได้ทุกจุดที่เห็นในเล่ม

โค้ดทุกก้อนในเล่มนี้ ดึงมาจากไฟล์จริงในโปรเจกต์ ไม่ได้พิมพ์ใหม่ เลขบรรทัดทางซ้ายจึงตรงกับในไฟล์ทุกตัว ทุกก้อนมีแถบล่างแบบนี้:

src/domain/runway.ts

ตัวอย่าง • บรรทัด 17-23

```
17 * so the status is "no_spending" and no date is shown.
18 */
19 import { addDays, bkkDayKey, daysBetween } from './dates';
20 import { confirmedOnly } from './summary';
21 import type { Transaction } from './types';
22
23 export const DEFAULT_FLOOR_SATANG = 50_000; // ฿500
```

▶ เปิดใน VS Code (เว็บ) GitHub

VS Code ในเครื่อง: Ctrl+P → src/domain/runway.ts:17

- 1 **แตะชื่อไฟล์ หรือ "เปิดใน VS Code (เว็บ)"** ระบบจะเปิด VS Code ในเบราว์เซอร์ (vscode.dev) พร้อมโปรเจกต์ MindPay และกระโดดไปที่บรรทัดนั้นเลย ไม่ต้องติดตั้งอะไร ใช้ได้ทั้งคอมและแท็บเล็ต ครั้งแรกอาจถามให้เข้าสู่ระบบ GitHub ก่อน
- 2 **แตะ "GitHub"** เปิดไฟล์เดียวกันบนเว็บไซต์ GitHub และไฮไลต์บรรทัดเหล่านั้น (ใช้ได้บนมือถือ)
- 3 **ถ้าโคลนโปรเจกต์ลงเครื่องแล้ว** (วิธีอยู่ในบท A3) เปิด VS Code กด Ctrl+P (Mac: Cmd+P) แล้วพิมพ์ข้อความตัวหน้าที่มุมขวา เช่น **src/domain/runway.ts:17** แล้วกด Enter จะเปิดไฟล์และไปที่บรรทัด 17 ทันที

ชื่อไฟล์สีเขียวขีดเส้นใต้ในเนื้อหา เช่น [src/domain/runway.ts](#) ก็แตะเปิดได้เหมือนกัน

i ลิงก์ล็อกไว้ที่เวอร์ชันของเล่มนี้

ทุกลิงก์ชี้ไปที่โค้ดเวอร์ชันวันที่ 3 ต.ค. 2569 (commit 646b485) ซึ่งเป็นเวอร์ชันที่ใช้เขียนเล่มนี้ ถ้าทีมแก้ไขโค้ดหลังจากนี้ ลิงก์ในเล่มยังเปิดเวอร์ชันเดิมที่เลขบรรทัดตรงกับในเล่ม ถ้าอยากดูเวอร์ชันล่าสุด ให้เปลี่ยนไปที่ branch main ใน VS Code หรือ GitHub

อ่านคำอธิบายโค้ดยังไ

หลังโค้ดแต่ละก้อนจะมีตาราง "อ่านทีละบรรทัด" คอลัมน์ซ้ายคือเลขบรรทัด คอลัมน์ขวาคือคำอธิบายเป็นภาษาคน ให้เปิดโค้ดกับตารางคู่กัน อ่านบรรทัดในโค้ดก่อน แล้วค่อยอ่านคำอธิบาย ถ้าเจอคำที่ไม่รู้จัก ให้กลับไปดูบท A4 ถึง A7 ซึ่งสอนพื้นฐานภาษาตั้งแต่ศูนย์

บรรทัด	ความหมาย
17	ตัวอย่าง: คำอธิบายของบรรทัด 17 จะอยู่แถวนี้
18–20	ถ้าหลายบรรทัดทำงานเรื่องเดียวกัน จะอธิบายรวมกันเป็นช่วง

กล่องสีต่าง ๆ ในเล่ม

≈ เปรียบเทียบ

เทียบกับของในชีวิตจริง ให้เห็นภาพก่อนลงรายละเอียด

★ จำให้ได้

เรื่องสำคัญที่อาจารย์มักถาม

!ระวัง

จุดที่คนมักเข้าใจผิด หรือนึกที่เคยเกิดขึ้นจริง

i เพิ่มเติม

รายละเอียดเสริม ข้ามได้ถ้ารีบ

ถ้ามีเวลาน้อย อ่านตามนี้

เวลาที่มี	อ่านอะไร
15 นาที	บท A1 (MindPay คืออะไร), บท A2 (แผนที่ระบบ), แล้วภาค ข บทที่ 1 และบทสุดท้าย (สรุปต้นน้ำเสนอ)
1–2 ชั่วโมง	ภาค ก บท A1, A2, A4, A9 แล้วอ่านภาค ข ทั้งหมดยกเว้นตารางโค้ดทีละบรรทัด
เตรียมตอบอาจารย์	ภาค ข บท "ทำไมฉันออกแบบแบบนี้", "ข้อจำกัด", "คำถามที่อาจารย์อาจถาม" และ "แบบฝึกหัด"
อยากอธิบายโค้ดได้จริง	อ่านทั้งเล่มตามลำดับ เปิด VS Code คู่กัน และลองทำแบบฝึกหัดโดยไม่ดูเฉลย

i ตัวเลขในเล่ม

ตัวอย่างขีดมือและเฉลยแบบฝึกหัดทุกข้อ ตรวจสอบโดยรันฟังก์ชันจริงของแอปแล้ว ภาพหน้าจอต่่ายจากเว็บเวอร์ชันจริงในโหมดทดลอง (ข้อมูลตัวอย่าง) วันที่ 9 ต.ค. 2569 ข้อมูลตัวอย่างสร้างตามวันที่เปิดแอป ถ้าเปิดดูเองตัวเลขจะต่างไปเล็กน้อย แต่หลักการเหมือนเดิมทุกอย่าง

สารบัญ

วิธีใช้เล่มนี้	1
เล่มนี้แบ่งเป็น 2 ภาค	1
เปิดโค้ดใน VS Code ได้ทุกจุดที่เห็นในเล่ม	1
อ่านคำอธิบายโค้ดอย่างไร	2
กล่องสีต่าง ๆ ในเล่ม	2
ถ้ามีเวลาน้อย อ่านตามนี้	2
พื้นฐานทั้งระบบ MINDPAY	9
MindPay คืออะไร และทีมเราสร้างอะไร	10
ปัญหาที่ทีมเราอยากแก้	10
6 ฟังก์ชันหลัก (Functional Requirements)	10
หน้าต่างแอปโดยรวม	11
ใช้แอปได้ 3 แบบ	12
ตัวเลขของโปรเจกต์	12
คำศัพท์ที่จะเจอบ่อยในเล่มนี้	13
แผนที่ระบบ: ชิ้นส่วนทั้งหมดต่อกันอย่างไร	14
ระบบมี 4 ส่วนใหญ่	14
ข้างในแอป: แบ่งโค้ดเป็น 5 ชั้น	14
ตามข้อมูล 1 รายการ: ตั้งแต่กดบันทึก จนถึงเลขบนหน้าหลักเปลี่ยน	15
ตามข้อมูลที่ใช้ AI: สแกนสลิป 1 ใบ	19
เครื่องมือ: เขียนโค้ดที่ไหน ใช้โปรแกรมอะไร และ VS Code ตั้งแต่ศูนย์	21
กล่องเครื่องมือของทีมเรา	21
ติดตั้งเครื่องมือ (ถ้าครั้งเดียว)	21
เอาโค้ด MindPay มาไว้ในเครื่อง	21
ไฟล์ package.json: บัตรประชาชนของโปรเจกต์	22
คำสั่งที่ใช้บ่อย	25
รู้จักหน้าจอส VS Code	25
คีย์ลัดที่ทำให้อ่านโค้ดเร็วขึ้น 10 เท่า	26
ส่วนเสริม (Extensions) ที่แนะนำ	27
ไม่อยากติดตั้งอะไร: ใช้ VS Code บนเว็บ	27
ฝึกใช้: ตามรอย Money Runway ใน 5 นาที	27
พื้นฐานการเขียนโปรแกรม สำหรับคนที่ไม่เคยเขียนเลย	28
โปรแกรมคืออะไร	28
1. ค่า (value) และชนิดของค่า	28
2. ตัวแปร: กล่องที่มีชื่อ	28
3. ตัวดำเนินการ: คำนวณและเปรียบเทียบ	29
4. เงื่อนไข if: ถ้า... ให้ทำ...	29
5. ฟังก์ชัน: เครื่องจักรรับของเข้า ส่งของออก	30
6. Array: รายการของหลายค่า	30
7. Object: ของ 1 ชิ้นที่มีหลายช่อง	32
8. วงซ้ำ (loop)	33
9. ข้อความ (string) ที่ใช้บ่อย	33
10. คณิตศาสตร์ที่ใช้	33
แบบฝึกหัด A4	33

TypeScript: ภาษาที่ทีมเราใช้ และชนิดข้อมูลของ MindPay	35
ทำไมต้อง TypeScript	35
การบอชนิดข้อมูล (type annotation)	35
ไฟล์ชนิดข้อมูลกลาง: src/domain/types.ts	35
ชนิดข้อมูลแบบพิเศษที่เจอบ่อย	37
import และ export: แคร่โค้ดระหว่างไฟล์	38
งานที่ต้องรอ: async, await และ Promise	39
จัดการความผิดพลาด: try, catch, throw	39
ทำไมเก็บเงินเป็นสตางค์ (จำนวนเต็ม)	40
อ่านโค้ดจริง: แปลงข้อความที่ผู้ใช้พิมพ์เป็นสตางค์	40
Regular expression (regex) อ่านยังงี้	42
แปลงสตางค์กลับเป็นข้อความบาท	43
React: วาดหน้าจอจากข้อมูล	45
แนวคิดหลัก: หน้าจอเป็นผลลัพธ์ของข้อมูล	45
Component: ชิ้นส่วนหน้าจอที่เป็นฟังก์ชัน	45
State: ข้อมูลที่เปลี่ยนได้ และทำให้หน้าจอวาดใหม่	47
Hook ที่ใช้ในโปรเจกต์	48
Context: แคร่ข้อมูลให้ทุกหน้าโดยไม่ต้องส่งต่อเป็นทอด ๆ	50
ทุกอย่างประกอบกัน: หน้าจอใช้ข้อมูลกลางยังงี้	51
React Native และ Expo: โค้ดชุดเดียว ได้ทั้งแอปและเว็บ	53
React Native คืออะไร	53
Expo: ชุดเครื่องมือที่ทำให้ React Native ง่ายขึ้น	54
Expo Router: ไฟล์ 1 ไฟล์ = หน้าจอ 1 หน้า	54
อ่านโค้ด: _layout.tsx โครงของทั้งแอป	56
แถบเมนูล่าง: (tabs)/_layout.tsx	60
โค้ดที่ต่างกันระหว่างมือถือกับเว็บ	61
จากโค้ด สู่เว็บและแอปที่ผู้ใช้ลง	63
วิธีอ่านโค้ด MindPay ด้วยตัวเอง	65
ขั้นตอนอ่านไฟล์ที่ไม่เคยเห็น	65
รูปแบบที่เจอซ้ำทั้งแอป (ตารางโกง)	65
ฟังก์ชันวันเวลาที่ต้องรู้ (ทุก FR ใช้)	66
แผนที่: อยากรู้อะไรไปเปิดไฟล์ไหน	67
เซิร์ฟเวอร์และฐานข้อมูล: Supabase	69
ฐานข้อมูลคืออะไร	69
Supabase ให้อะไรทีมเราบ้าง	69
Migration: ไฟล์สร้างฐานข้อมูล	70
Row Level Security (RLS): หัวใจความปลอดภัย	73
กฎแฉ 2 แบบ: อันไหนอยู่ในแอปได้ อันไหนห้าม	75
แอปต่อ Supabase ยังงี้: src/data/supabase.ts	75
การสมัครและเข้าสู่ระบบ	77
Repo: ชั้นที่ซ่อนว่าข้อมูลอยู่ที่ไหน	78
เปิดแอปแล้วเกิดอะไรขึ้น: AppProvider เลือกโหมด	83
การเชื่อม AI: ทำไมต้องผ่านเซิร์ฟเวอร์ และทำงานยังงี้	86
AI ในแอปคืออะไร	86
ทำไมแอปไม่เรียก AI เอง	86
เส้นทางของคำถาม 1 ข้อถึงน้องกล่า	86

ฝั่งแอป: src/services/coach.ts	87
ฝั่งเซิร์ฟเวอร์: Edge Function coach ที่ละบัสกัถ	88
ตัวช่วยกลาง: supabase/functions/_shared/common.ts	90
เลือก AI ตัวไหน: Claude หรือ Gemini	93
ส่งคำขอไปที่ Claude: callClaude	95
ตั้งค่า AI (ผู้ดูแลทำครั้งเดียว)	98
หลักการใช้ AI อย่างรับผิดชอบที่ทีมเราใช้	98
UX/UI: การออกแบบหน้าจอและ Design System	99
หลักการออกแบบของ MindPay	99
Design System คืออะไร	99
สี: Green + Gold	99
ความอ่านง่ายของสี (WCAG) และเทสที่บังคับ	101
ตัวอักษร	103
ชิ้นส่วนสำเร็จรูป: ปุ่ม (Button)	103
Accessibility: ทุกคนใช้ได้	106
แอนิเมชัน: สวยแต่ไม่กินแบต	106
เช็กลิสต์ก่อนเพิ่มหน้าจอใหม่ (จาก DESIGN_SYSTEM.md)	106
การทดสอบ: พิสูจน์ว่าระบบทำงานถูก	108
ทำไมต้องเขียนเทส	108
Unit test: ตรวจสอบฟังก์ชันทีละตัว	108
E2E: ทดสอบทั้งแอปเหมือนคนใช้จริง	110
Manual test: ทดสอบบนมือถือจริง	113
Traceability: ผูก Requirement → โค้ด → เทส	114
ตรวจสอบคุณภาพอื่น ๆ	114
Git และ GitHub: เก็บโค้ด ทำงานเป็นทีม และ build อัตโนมัติ	116
Git: ระบบบันทึกประวัติ	116
ใช้ Git ประจำวัน	116
ประวัติของ MindPay UU GitHub	117
GitHub Actions: หุ่นยนต์ที่ทำงานทุกครั้งที่เรา push	117
Secrets UU GitHub	122
ไฟล์เอกสารใน repo	122
6 FR ทำงานร่วมกันอย่างไร และคำถามระดับระบบ	124
สายข้อมูลของทั้งระบบ	124
ใครใช้ของใคร	124
ไฟล์ที่หลาย FR ใช้ร่วมกัน (แก้แล้วกระทบหลายคน)	124
คำถามระดับระบบ ที่อาจารย์ถามใครก็ได้	125
คำศัพท์รวม	126
FR-1 TRANSACTION MANAGEMENT	128
FR-1 ในหน้าเดียว	129
อยู่ตรงไหนในแอป	129
3 ข้อที่ต้องจำ	129
ปัญหาที่ฉันทอยากแก้	131
เป้าหมายที่ฉันทตั้งไว้ตอนออกแบบ	131
Requirement และเงื่อนไขผ่าน	132

รายการ 1 รายการ มีอะไรบ้าง	133
หมวดหมู่ 15 หมวด	133
ทิวทัศน์: จดรายการและเพิ่มเงินเข้า	135
"เพิ่มเงินเข้า" ต่างจาก "จดรายการ" ยังไง	136
แก้ไข ลบ และเลิกทำ	136
ทิวทัศน์: แท็บ "รายการ"	138
หลักการทํางาน: ตรวจสอบจำนวนเงิน เดาหมวด และวันเวลา	140
1. ตรวจสอบจำนวนเงิน (parseBahtToSatang)	140
2. เดาหมวดจากชื่อ (suggestCategory)	140
3. วันและเวลาแบบไทย	141
เบื้องหลัง: บันทึก แก้ ลบ ไปยังฐานข้อมูลยังไ	143
แผนที่โค้ดของ FR-1	144
อ่านโค้ด: หน้าจอรายการ (transaction.tsx) ตอนที่ 1 – สมอง	146
ส่วนที่ 1: คอมเมนต์หัวไฟล์และ import (บรรทัด 1–22)	146
ส่วนที่ 2: หน้าเปิดขึ้นมา รู้ได้ยังไงว่าเป็นหน้าแบบไหน (บรรทัด 24–32)	148
ส่วนที่ 3: ความจำเป็นของแบบฟอร์ม — state ทั้ง 14 ตัว (บรรทัด 34–49)	148
ส่วนที่ 4: ตรวจสอบวันที่ที่พิมพ์ และค่าที่คำนวณจาก state (บรรทัด 50–60)	150
ส่วนที่ 5: เปลี่ยนประเภท เปลี่ยนชื่อ และตัวเลือกวันที่ Android (บรรทัด 62–85)	151
ส่วนที่ 6: save — หัวใจของการบันทึก (บรรทัด 87–130)	152
ส่วนที่ 7: doDelete — ลบแล้วเลิกทำได้ (บรรทัด 132–155)	154
ส่วนที่ 8: ตัวช่วยวาด — flagged และ pill (บรรทัด 157–178)	155
ตัวช่วยจากไฟล์อื่น: nav.ts (ทั้งไฟล์)	156
อ่านโค้ด: หน้าจอรายการ ตอนที่ 2 – หน้าตา (JSX)	158
ส่วนที่ 9: โครงหน้าและหัวหน้าจอ (บรรทัด 180–193)	158
ส่วนที่ 10: การ์ดตรวจสอบ (บรรทัด 194–230) — เชื่อม FR-4	159
ส่วนที่ 11: ปุ่มสลับ ช่องเงิน และชิปเงินเข้า (บรรทัด 232–256)	160
ส่วนที่ 12: ชื่อรายการและหมวด (บรรทัด 258–284)	161
ส่วนที่ 13: วันและเวลา 3 แบบตามแพลตฟอร์ม (บรรทัด 286–336)	162
ส่วนที่ 14: บันทึกเพิ่มเติม ปุ่มล่าง และหน้าต่างยืนยันลบ (บรรทัด 338–364)	164
อ่านโค้ด: src/domain/categories.ts ทั้งไฟล์	166
ส่วนที่ 1: รูปร่างของหมวด (บรรทัด 1–14)	166
ส่วนที่ 2: รายการหมวดทั้ง 15 หมวด (บรรทัด 16–42)	166
ส่วนที่ 3: ฟังก์ชันช่วยหาหมวด (บรรทัด 44–56)	168
ส่วนที่ 4: ตรวจสอบว่าเป็นชื่อคนใหม่ และหาคำสำคัญ (บรรทัด 58–78)	168
ส่วนที่ 5: suggestCategory — เดาหมวด (บรรทัด 80–111)	169
อ่านโค้ด: แท็บรายการ (transactions.tsx)	172
ส่วนที่ 1: import และชนิดตัวกรอง (บรรทัด 1–24)	172
ส่วนที่ 2: ความจำเป็นหน้าและเดือนในปฏิทิน (บรรทัด 26–44)	173
ส่วนที่ 3: กรอง ค้นหา และแยกตามวัน — หัวใจของแท็บ (บรรทัด 46–54)	174
ตัวช่วยจาก FR-2: groupByDay (summary.ts บรรทัด 110–126)	175
ส่วนที่ 4: หัวแท็บ ช่องค้นหา ตัวกรอง และการ์ดสรุปยืนยัน (บรรทัด 56–112)	176
ส่วนที่ 5: รายการแบบมีหัวข้อวัน และหน้าว่าง 3 แบบ (บรรทัด 114–167)	178
อ่านโค้ด: แถวรายการ TxRow.tsx ทั้งไฟล์	181

อ่านโค้ด: ข้อมูล 1 รายการหน้าตาเป็นยังงัย (types.ts)	184
อ่านโค้ด: AppProvider — ศูนย์กลางที่หน้าจอเรียก	186
ตัวช่วยที่ใช้ร่วมกัน (บรรทัด 118–125 และ 320–323)	186
ฟังก์ชันจัดการรายการ 4 ตัว (บรรทัด 378–427)	186
อ่านโค้ด: Repo — ประตูดึงข้อมูล (repo.ts)	189
ส่วนที่ 1: สัญญาของ Repo (บรรทัด 1–38)	189
ส่วนที่ 2: แปลงชื่อช่องระหว่างแอปกับฐานข้อมูล (บรรทัด 40–93)	190
ส่วนที่ 3: CloudRepo — คำสั่งของ FR-1 ที่คุยกับ Supabase (บรรทัด 217–237)	192
ส่วนที่ 4: DemoRepo — โหมดทดลองในเครื่อง (บรรทัด 283 และ 313–362)	193
อ่านโค้ด: ตาราง transactions ในฐานข้อมูล (SQL)	195
กฎความปลอดภัย RLS ของตารางนี้	196
อ่านโค้ด: ชิ้นส่วนหน้าตาที่ FR-1 ใช้ (inputs.tsx, feedback.tsx)	198
ช่องกรอกทั่วไป Field (inputs.tsx บรรทัด 6–96)	198
ช่องกรอกเงิน AmountField (inputs.tsx บรรทัด 156–209)	200
toast และ ConfirmSheet (feedback.tsx)	202
อ่านโค้ด: หน้าจอรายการถูกเปิดจากไหน	207
ปุ่มบนหน้าหลัก (index.tsx บรรทัด 471–482)	207
ลงทะเบียนหน้าใน _layout.tsx (บรรทัด 97–126)	207
อ่านโค้ด: รายการประจำเดือน (recurring.ts ทั้งไฟล์)	210
ส่วนที่ 1: รูปร่างของผลลัพธ์และค่าคงที่ (บรรทัด 1–38)	210
ส่วนที่ 2: วันในเดือนแบบ "วงกลม" (บรรทัด 40–81)	211
ส่วนที่ 3: findRecurring — หารายการประจำ (บรรทัด 83–135)	212
ส่วนที่ 4: ตัวช่วยแสดงผล (บรรทัด 137–148)	214
การทดสอบ FR-1	216
Unit test: เงินและหมวด (domain.test.ts)	216
Unit test: เงินที่ผู้ใช้พิมพ์ (audit.test.ts TC-79)	217
Unit test: รายการประจำ (recurring.test.ts TC-72)	218
E2E: เพิ่ม แกล้ง เลิกทำ ในเบราว์เซอร์จริง	219
เช็กลิสต์ทดสอบเองบนมือถือ (MT ในเอกสาร TRACEABILITY)	222
FR-1 เชื่อมกับ FR อื่นยังงัย	223
ทำไมฉันออกแบบแบบนี้	224
ข้อจำกัด และสิ่งที่จะทำต่อ	225
คำถามที่อาจารย์อาจถาม พร้อมคำตอบ	226
แบบฝึกหัดเช็กความเข้าใจ	228
เฉลย	228
สคริปต์นำเสนอ 1 นาที และคำศัพท์	229
คำศัพท์ของ FR-1	229

ภาค ก

พื้นฐานทั้งระบบ MindPay

สอนตั้งแต่ศูนย์ สำหรับคนที่ไม่เคยเขียนโค้ด ภาคนี้เหมือนกับทุกเล่ม เพราะไม่ว่าจะรับผิดชอบ FR ไหน ทุกคนต้องเข้าใจว่าแอปทั้งตัวสร้างด้วยอะไร ทำงานที่ไหน และชิ้นส่วนต่างๆ คอยกันยังไง

- เครื่องมือ ภาษา TypeScript และ React ตั้งแต่พื้นฐาน
- โค้ดชุดเดียวกลายเป็นทั้งเว็บและแอป Android ได้ยังไง
- เซิร์ฟเวอร์ Supabase ฐานข้อมูล และความปลอดภัย
- การเชื่อม AI, การออกแบบ UX/UI, การทดสอบ และ GitHub

MindPay คืออะไร และทีมเราสร้างอะไร

ก่อนดูโค้ดสักบรรทัด ต้องรู้ก่อนว่าแอปนี้แก้ปัญหอะไร ใครใช้ และมีส่วนไหนบ้าง

ปัญหาที่ทีมเราอยากแก้

- นักศึกษาและคนเพิ่งเริ่มทำงาน (First Jobber) ส่วนใหญ่รับและจ่ายเงินผ่านแอปธนาคารบนมือถือ ทุกครั้งที่โอนเงินจะได้ "สลิป" เป็นรูปเก็บไว้ในแกลเลอรี แต่ปัญหาคือ
- **ขี้เกียจจด** แอปจดรายรับรายจ่ายทั่วไปต้องพิมพ์เองทุกรายการ ทั้งที่ข้อมูลอยู่ในสลิปแล้ว คนจึงเลื่อนการจดไปเรื่อย ๆ จนข้อมูลไม่ครบ
 - **เห็นแค่ยอดเงิน แต่ไม่รู้ว่าพอหรือเปล่า** แอปธนาคารบอกว่าเหลือ ฿3,500 แต่ไม่ได้บอกว่าเงินก้อนนี้จะอยู่ได้ถึงวันไหน
 - **รู้ตัวตอนเงินหมดแล้ว** ไม่มีใครช่วยเตือนหรือช่วยคิดก่อนจ่าย

ทีมเราจึงสร้าง **MindPay** แอปการเงินส่วนตัวที่ "ไม่ได้แค่จดว่าใช้ไปเท่าไร แต่บอกว่าเงินที่มีจะพอใช้อีกกี่วัน และช่วยคิดก่อนจ่าย" (ประโยคแรกของไฟล์ `README.md` ในโปรเจกต์)

6 ฟังก์ชันหลัก (Functional Requirements)

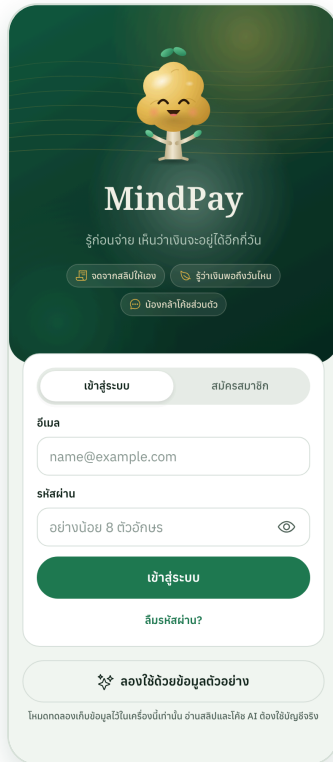
ในวิชา Software Engineering เราเรียกสิ่งที่ระบบ "ต้องทำได้" ว่า **Functional Requirement** หรือ **FR** แต่ละ FR มีเจ้าของ 1 คนในกลุ่ม MindPay มีทั้งหมด 6 ข้อ

FR	ชื่อ	ทำอะไร (ภาษาคน)	อยู่ตรงไหนในแอป
FR-1	Transaction Management	จดรายการเอง แก้ไข เลือกหมวด และลบรายการได้	แท็บ "รายการ" และหน้า "จดรายการ"
FR-2	Overview Dashboard	สรุปรายรับ รายจ่าย ยอดคงเหลือ และยอดแยกตามหมวด	แท็บ "หน้าหลัก"
FR-3	Voice Entry	พูดว่า "ข้าว 50 บาท" แล้วแอปจดให้	ปุ่มไมค์ทองบนหน้าหลัก
FR-4	Automatic Slip Detection	หาสลิปในแกลเลอรีเองแล้วให้ AI อ่านยอดเงิน วันที่ และชื่อผู้รับ	ปุ่มทวงกลางแถบล่าง "สแกนสลิป" และสแกนอัตโนมัติตอนเปิดแอป
FR-5	AI Coach (น้องกล้า)	โค้ช AI ที่อธิบายตัวเลขการเงินเป็นภาษาคน และพูดออกเสียงได้	แท็บ "โค้ช"
FR-6	Money Runway	คำนวณว่าเงินที่มีจะพอใช้อีกกี่วัน และหมดวันไหน	แท็บ "เงินพอถึง" และการ์ดเขียบบนหน้าหลัก

★ **จำให้ได้: ทุก FR ต่อกันเป็นสายเดี่ยว**

FR-1, FR-3 และ FR-4 คือ **ทางเข้าของข้อมูล** (จุดเอง พุดจกด สแกนสลิป) ส่วน FR-2 และ FR-6 คือ **สมองที่คิดตัวเลข** (ยอดคงเหลือ และเงินพอถึงวันไหน) และ FR-5 คือ **ปากที่อธิบาย** ตัวเลขเหล่านั้นให้ผู้ฟัง ถ้า FR ไหนพัง FR ที่อยู่ปลายน้ำจะได้ตัวเลขผิดตามไปด้วย

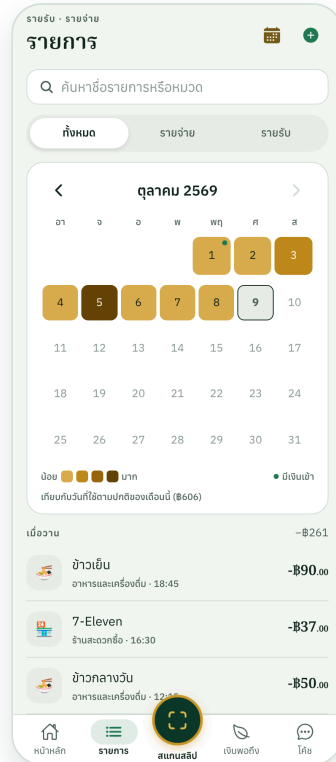
หน้าตาแอปโดยรวม



หน้าต้อนรับ: สมัคร เข้าสู่ระบบ หรือ "ลองใช้ด้วยข้อมูลตัวอย่าง"



หน้าหลัก (FR-2): ยอดคงเหลือ, การ์ดเงินพอถึง (FR-6), น้องกล่า พุดสรุป



แท็บรายการ (FR-1): รายการ ทั้งหมด ค้นหา กรองรายรับ/รายจ่าย



ด้านล่างของทุกหน้ามี **แถบเมนู 5 ช่อง**: หน้าหลัก, รายการ, ปุ่มทวง "สแกนสลิป" ตรงกลาง, เงินพอถึง และไอซ์ ปุ่มสแกนอยู่ตรงกลางและใหญ่ที่สุด เพราะเป็นสิ่งที่ผู้ใช้ทำบ่อยที่สุด (โค้ดอยู่ใน `src/app/(tabs)/_layout.tsx` ซึ่งจะอ่านทีละบรรทัดใน un A7)

ใช้แอปได้ 3 แบบ

แบบ	เปิดยังไ้	ต่างกันยังไ้
เว็บ	https://2550expo-tech.github.io/Socrates-and-Skeletons/ (เปิดในเบราว์เซอร์ได้ทั้งคอม iPhone และ Android)	ทำได้ทุกอย่าง ยกเว้นสแกนแคเลเลอร์ทั้งหมดเอง (บนเว็บต้องกดเลือกรูปสลิป)
แอป Android (.apk)	ดาวน์โหลดจากหน้า Releases ของ GitHub ชื่อ latest-apk	สแกนแคเลเลอร์อัตโนมัติ พุดจดย่อยด้วยไมค์ นื่องคล้ายพุดออกเสียง
Expo Go	แอปตัวช่วยตอนพัฒนา สแกน QR จากคอมของนักพัฒนา	ใช้ทดสอบระหว่างเขียนโค้ดเท่านั้น

และไม่ว่าเปิดแบบไหน ผู้ใช้เลือกได้ 2 โหมด

- **บัญชีจริง** สมัครด้วยอีเมล ข้อมูลเก็บบนเซิร์ฟเวอร์ (Supabase) เปิดจากเครื่องไหนก็เห็นข้อมูลเดียวกัน ใช้ AI อ่านสลิปและไอซ์ได้
- **โหมดทดลอง** และ "ลองใช้ด้วยข้อมูลตัวอย่าง" แอปสร้างรายการตัวอย่างประมาณ 75 วันให้ เก็บไว้ในเครื่องอย่างเดียว ไม่ต้องสมัคร ใช้ตอนฟรีเซนต์หรือลองเล่น (AI ยังใช้ไม่ได้ในโหมดนี้)

ตัวเลขของโปรเจกต์

~18,700

บรรทัดโค้ดของแอปและเซิร์ฟเวอร์ (ไม่นับ
เทสต์)

157

unit test ผ่านทั้งหมด

145

ข้อตรวจในเบราวเซอร์ (E2E) ผ่านทั้งหมด

โฟลเดอร์	มีอะไร	ไฟล์	บรรทัด
src/app	หน้าจอทุกหน้า	19	5,266
src/ui	ชิ้นส่วนหน้าตา: ปุ่ม การ์ด สี กราฟ ภาพน้องกล้า	31	6,042
src/domain	กฎและการคำนวณทั้งหมด (สมองของแอป)	24	3,457
src/data	การเก็บและโหลดข้อมูล	9	1,019
src/services	งานที่ต้องคุยกับโลกภายนอก: แกลเลอรี ไมค์ เสียง AI	15	1,776
supabase	ฐานข้อมูลและโค้ดฝั่งเซิร์ฟเวอร์	8	1,160
e2e, scripts	เทสต์ในเบราวเซอร์ และสคริปต์ช่วย (ไอคอน ภาพหน้าจอ)	10	1,867

นับจากโค้ดเวอร์ชัน 3 ต.ค. 2569 ไฟล์ .ts, .tsx, .sql และ .mjs ไม่รวมไฟล์เทสต์ (ไฟล์เทสต์อีกประมาณ 2,000 บรรทัด)

≈ เปรียบแอปเป็นร้านอาหาร

หน้าจอ (src/app, src/ui) คือหน้าร้านที่ลูกค้าเห็น **กฎการคำนวณ** (src/domain) คือสูตรอาหารของเชฟ **ข้อมูล** (src/data) คือตู้เย็นเก็บวัตถุดิบ **เซิร์ฟเวอร์** (supabase) คือโถงกลางที่ทุกสาขาใช้ร่วมกัน และ **AI** คือผู้เชี่ยวชาญภายนอกที่เราโทรถามเมื่อต้องอ่านสลิปหรืออธิบายตัวเลข บทต่อไปจะวาดแผนที่ว่าทั้งหมดนี้ต่อกันยังไง

คำศัพท์ที่จะเจอบ่อยในเล่มนี้

คำ	ความหมายแบบง่าย
รายการ (Transaction)	เงินเข้าหรือเงินออก 1 ครั้ง เช่น ข้าวมันไก่ ฿50
แบบร่าง (Draft)	รายการจากสลิปที่ AI อ่านแล้วแต่ยังไม่แน่ใจ รอผู้ตรวจ ยังไม่นับในยอดเงิน
ยืนยันแล้ว (Confirmed)	รายการที่นับในยอดเงินแล้ว
สตางค์ (satang)	หน่วยที่แอปใช้เก็บเงินในระบบ 100 สตางค์ = 1 บาท (เหตุผลอยู่บท A5)
เส้นเงินสำรอง (Floor)	เงินขั้นต่ำที่ผู้ใช้ไม่อยากให้ต่ำกว่า ค่าเริ่มต้น ฿500
น้องกล้า	มาสคอตของแอป เป็นต้นกล้าจากต้นไม้เงิน และเป็นโค้ช AI ใน FR-5
Supabase	บริการเซิร์ฟเวอร์ที่ให้ทั้งฐานข้อมูล ระบบบัญชีผู้ใช้ และที่รันโค้ดฝั่งเซิร์ฟเวอร์
Expo	เครื่องมือที่ทำให้โค้ดชุดเดียวเป็นได้ทั้งแอป Android, iPhone และเว็บ

แผนที่ระบบ: ชิ้นส่วนทั้งหมดต่อกันยังงี้

ภาพใหญ่ของ MindPay ตั้งแต่นิ้วแตะจอ จนข้อมูลไปถึงฐานข้อมูลบนคลาวด์และกลับมาเป็นตัวเลขบนหน้าจอ

ระบบมี 4 ส่วนใหญ่

① แอปบนมือถือหรือเว็บ (โค้ดในโฟลเดอร์ src/)

สร้างด้วย Expo + React Native ภาษา TypeScript โค้ดชุดเดียวได้ทั้งแอป Android, iPhone และเว็บ ทำหน้าที่แสดงหน้าจอ รับการแตะ คำนวณตัวเลขทั้งหมด (ยอดคงเหลือ, Runway) อ่าน QR บนสลิป และแปลงเสียงพูดเป็นรายการ

↕ อินเทอร์เน็ต (HTTPS) ด้วย "publishable key" ซึ่งเป็นกุญแจสาธารณะ

② Supabase: ฐานข้อมูล + ระบบบัญชีผู้ใช้ (โค้ดใน supabase/migrations/)

ฐานข้อมูล PostgreSQL เก็บตาราง profiles, transactions, savings_goals, ai_usage ทุกตารางเปิด Row Level Security (RLS) ทำให้แต่ละคนอ่านและแก้ไขเฉพาะแถวของตัวเอง ระบบ Auth จัดการสมัคร เข้าสู่ระบบ ยืนยันอีเมล

↕ เฉพาะงานที่ต้องใช้ AI

③ Supabase Edge Functions: โค้ดฝั่งเซิร์ฟเวอร์ (โค้ดใน supabase/functions/)

ฟังก์ชัน 2 ตัว: **parse-slip** (อ่านสลิป, FR-4) และ **coach** (โค้ชน้องกล้า, FR-5) ตรวจสอบว่าผู้ใช้เข้าสู่ระบบจริง นับโควตาต่อวัน แล้วคือ "กุญแจลับ" ของ AI ไว้เรียกแทนแอป

↕ API key ลับ (อยู่บนเซิร์ฟเวอร์เท่านั้น)

④ AI ภายนอก: Claude (Anthropic) หรือ Gemini (Google)

อ่านรูปสลิปแล้วตอบเป็น JSON และเขียนคำอธิบายของโค้ช ระบบเลือกใช้ตัวที่มีกุญแจ ถ้ามีทั้งสองตัวจะใช้ Claude ก่อน

★ **จำให้ได้: 4 กฎของสถาปัตยกรรม (จาก README)**

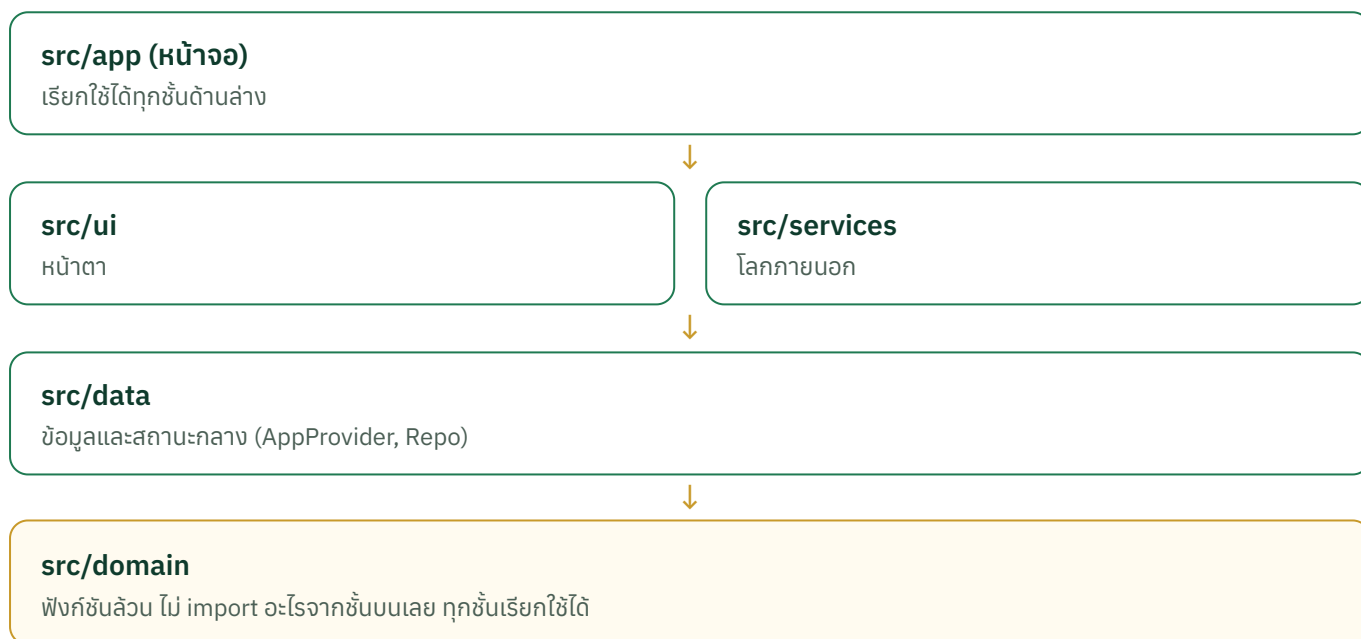
- เงินเก็บเป็นสตางค์ (จำนวนเต็ม) ไม่มีปัญหาทศนิยมเพี้ยน
- เวลาอิงเวลาไทย (UTC+7) ทุกจุด ทั้งการจัดกลุ่มรายวันและการอ่านวันที่บนสลิป
- สลิปที่ AI อ่านได้ไม่ชัดเป็น "แบบร่าง" จนกว่าผู้ใช้จะยืนยัน ไม่เข้ายอดเงินก่อน
- กุญแจ AI อยู่บนเซิร์ฟเวอร์เท่านั้น ในแอปไม่มีกุญแจลับเลยสักตัว

ข้างในแอป: แบ่งโค้ดเป็น 5 ชั้น

ถ้าเอาโค้ดทุกอย่างไปกองในไฟล์เดียว จะแก้ยากและทดสอบไม่ได้ ทีมเราจึงแยกตามหน้าที่ แต่ละโฟลเดอร์ใน src/ มีหน้าที่ชัดเจน และมีกฎว่าใครเรียกใครได้

ชั้น	โฟลเดอร์	หน้าที่	ตัวอย่างไฟล์
หน้าจอ	src/app/	แต่ละไฟล์คือ 1 หน้า (Expo Router) วางชิ้นส่วน รับการแตะ แล้วเรียกชั้นอื่นทำงาน	<code>src/app/transaction.tsx</code> หน้าจอดรายการ
ชิ้นส่วน หน้าตา	src/ui/	ปุ่ม การ์ด ช่องกรอก สี ตัวอักษร กราฟ ภาพวาด ใช้ซ้ำได้ทุกหน้า	<code>src/ui/components.tsx</code> , <code>src/ui/theme.ts</code>

ชั้น	โฟลเดอร์	หน้าที่	ตัวอย่างไฟล์
กฎและการคำนวณ	src/domain/	"สมอง" ของแอป เป็นฟังก์ชันล้วน รับตัวเลขเข้า ส่งตัวเลขออก ไม่ยุ่งกับหน้าจอหรืออินเทอร์เน็ท จึงทดสอบได้ง่าย มี unit test ทุกไฟล์	<code>src/domain/runway.ts</code> , <code>src/domain/money.ts</code>
ข้อมูล	src/data/	โหลดและบันทึกข้อมูล ทั้งบนคลาวด์ (Supabase) และในเครื่อง (โหมดทดลอง) และเก็บ "สถานะกลาง" ที่ทุกหน้าใช้ร่วมกัน	<code>src/data/repo.ts</code> , <code>src/data/AppProvider.tsx</code>
บริการภายนอก	src/services/	งานที่คุยกับโลกนอกแอป: แกลเลอรี่รูป ไมโครโฟน เสียงอ่าน และเรียก Edge Functions	<code>src/services/slips.ts</code> , <code>src/services/coach.ts</code>



≈ ทำไมแยก domain ออกมา

เหมือนแยก "สูตรคำนวณภาษี" ออกจาก "แบบฟอร์มกรอกภาษี" ถ้าสูตรเปลี่ยน แค่นี้เดียว แบบฟอร์มทุกฉบับได้ผลใหม่ทันที และเราตรวจสอบสูตรได้ด้วยการใส่ตัวเลขทดลองโดยไม่ต้องกรอกแบบฟอร์มจริง โค้ดใน src/domain ก็เช่นกัน มีเทสต์ 157 ข้อที่รันจบในไม่ถึง 2 วินาทีโดยไม่ต้องเปิดแอป

ตามข้อมูล 1 รายการ: ตั้งแต่กดบันทึก จนตัวเลขบนหน้าหลักเปลี่ยน

วิธีที่ดีที่สุดในการเข้าใจระบบ คือเลือกการกระทำหนึ่งอย่าง แล้วตามดูว่ามันวิ่งผ่านไฟล์ไหนบ้าง สมมติผู้ใช้กด "ข้ามมันไ้฿85" แล้วกด "บันทึกรายการ"

1. หน้าจอรายการ [src/app/transaction.tsx](#)

ฟังก์ชัน `save()` แปลง "85" เป็น 8,500 สตางค์ ด้วย `parseBahtToSatang` แล้วเรียก `addTx(...)`



2. สถานะกลาง [src/data/AppProvider.tsx](#)

`addTx` ส่งต่อให้ Repo บันทึก รวบรวมกลับมา แล้วเพิ่มรายการใหม่เข้าไปในรายการทั้งหมด (`setTxs`)



3. Repo [src/data/repo.ts](#)

บัญชีจริง: แปลงชื่อช่องเป็นแบบฐานข้อมูล (`amountSatang` → `amount_satang`) แล้วส่งคำสั่ง `insert` ไปที่ Supabase · โหมดทดลอง: เก็บลงเครื่องแทน



4. ฐานข้อมูล Supabase [supabase/migrations/20260927000000_init.sql](#)

ตรวจกฎ: จำนวนเงินต้อง > 0, ผู้บันทึกต้องเป็นเจ้าของแถว (RLS) แล้วเพิ่มแถวใหม่ในตาราง `transactions` ส่งแถวที่บันทึกแล้วกลับไป



5. คำนวณใหม่อัตโนมัติ `useMoney()` ใน [src/data/AppProvider.tsx](#)

เพราะรายการเปลี่ยน React จึงคำนวณยอดคงเหลือ (FR-2) และ Money Runway (FR-6) ใหม่ด้วยฟังก์ชันใน `src/domain` แล้ววาดหน้าหลักใหม่ ตัวเลขลดลง ฿85 กันที

ดูโค้ดจริงของแต่ละชั้น (ยังไม่ต้องเข้าใจทุกคำ บท A4–A9 จะสอนทีละอย่าง ตอนนีดูแค่ว่ามันต่อกันยังไง)

ชั้น 1: หน้าจอรายการเรียก `addTx`

`src/app/transaction.tsx`

บรรทัด 87–124

```
87  async function save(confirm: boolean) {
88    if (busy) return;
89    const satang = parseBahtToSatang(amount);
90    if (satang === null) {
91      setAmountError('ใส่จำนวนเงินมากกว่า 0 เช่น 85 หรือ 85.50');
92      return;
93    }
94    setAmountError(null);
95    if (dayError || timeError) return;
96    const input = {
97      kind,
98      amountSatang: satang,
99      categoryKey: category,
100     title: title.trim(),
101     note: note.trim() || null,
102     occurredAt: bkkToIso(day, time),
103   };
104   setBusy(true);
105   try {
106     if (existing) {
107       await updateTx(existing.id, {
108         ...input,
109         ...(confirm ? { status: 'confirmed' as const, reviewFlags: [] } : {}),
110       });
111       toast({ message: confirm && isDraft ? 'ยืนยันรายการแล้ว รวมในยอดเงินแล้ว' : 'บันทึกการแก้ไขแล้ว' });
112     } else {
```

```

113     await addTx({
114         ...input,
115         source: 'manual',
116         status: 'confirmed',
117         slipRef: null,
118         slipImageHash: null,
119         ocrConfidence: null,
120         reviewFlags: [],
121     });
122     toast({ message: kind === 'income' ? 'เพิ่มเงินเข้าแล้ว' : 'บันทึกรายการแล้ว' });
123 }
124 leaveWhenDone();

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/transaction.tsx:87

บรรทัด	ความหมาย
87	ประกาศฟังก์ชัน save ซึ่งทำงานเมื่อกดปุ่มบันทึก คำว่า async แปลว่าข้างในมีงานที่ต้องรอ (เช่น รออินเทอร์เน็ต)
88	ถ้ากำลังบันทึกอยู่แล้ว (busy) ให้ออกเลย กันคนกดปุ่มรัว ๆ แล้วได้รายการซ้ำ
89–93	แปลงข้อความในช่องเงินเป็นสตริง ถ้าแปลงไม่ได้ (เช่น พิมพ์ "abc" หรือ 0) จะได้ null แล้วขึ้นข้อความเตือนใต้ช่อง และหยุดไม่บันทึก
95	ถ้าวันที่หรือเวลาที่พิมพ์ (บนเว็บ) ผิดรูปแบบ ก็ไม่บันทึก
96–103	รวมค่าจากทุกช่องในฟอร์มเป็นก้อนเดียวชื่อ input: ประเภท จำนวนเงิน หมวด ชื่อ บันทึก และเวลา (แปลงวันและเวลาไทยเป็นรูปแบบสากลด้วย bkkToIso)
104	ตั้ง busy เป็นจริง ปุ่มจะหมุนรอและกดซ้ำไม่ได้
106–111	ถ้าเป็นการแก้ไขรายการเดิม เรียก updateTx แทน
113–121	ถ้าเป็นรายการใหม่ เรียก addTx พร้อมบอกว่ามาจากการกดเอง (manual) และยืนยันแล้ว (confirmed) จึงนับในยอดเงินทันที
122	ขึ้นข้อความสั้น ๆ ด้านล่าง (toast) ว่า "บันทึกรายการแล้ว"
124	ปิดหน้าฟอร์มกลับไปหน้าก่อน

ขั้น 2: AppProvider ส่งต่อให้ Repo แล้วอัปเดตรายการ

src/data/AppProvider.tsx

บรรทัด 385–392

```

385     async addTx(input) {
386         const r = need();
387         const tx = await r.insert(input);
388         if (repoRef.current !== r) return tx;
389         noteLocal(tx.id, tx);
390         setTxs((list) => [tx, ...list]);
391         return tx;
392     },

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/AppProvider.tsx:385

บรรทัด	ความหมาย
386	หยาบ Repo ตัวที่ใช้อยู่ (บัญชีจริงหรือโหมดทดลอง) ถ้ายังไม่ได้เข้าสู่ระบบ need () จะโยน error
387	สั่ง Repo บันทึก แล้วรอผล (await) ได้รายการที่มี id จริงกลับมา
388	ถ้าระหว่างรอ ผู้ใช้ลบบัญชีไปแล้ว ไม่ต้องเอารายการนี้ไปใส่ในบัญชีใหม่
389	จดไว้ว่ามีการเปลี่ยนแปลงในเครื่อง (กันรายการหายถ้ากำลังโหลดข้อมูลชุดใหม่พร้อมกัน)
390	เอารายการใหม่ไปไว้หน้าสุดของรายการทั้งหมด การเรียก setTxS ทำให้ React วาดทุกหน้าที่ใช้รายการนี้ใหม่

ขั้น 3: Repo ส่งคำสั่งไป Supabase

src/data/repo.ts	บรรทัด 217–222
<pre> 217 async insert(input) { 218 const { data, error } = await db.from('transactions').insert(toRow(input)).select().single(); 219 if (isUniqueViolation(error)) throw new DuplicateSlipError(); 220 if (error) throw error; 221 return fromRow(data as TxRow); 222 }, </pre>	
▶ เปิดใน VS Code (เว็บ) GitHub VS Code ในเครื่อง: Ctrl+P → src/data/repo.ts:217	

บรรทัด	ความหมาย
218	อ่านเหมือนประโยคภาษาอังกฤษ: "จากตาราง transactions, เพิ่ม (insert) แถวนี้, แล้วเลือก (select) แถวที่เพิ่ง เพิ่มกลับมา 1 แถว (single)" toRow แปลงชื่อช่องให้ตรงกับฐานข้อมูล
219	ถ้าฐานข้อมูลตอบว่า "ซ้ำ" (รหัส 23505) แปลว่าสลิปนี้เคยบันทึกแล้ว โยน error แบบพิเศษให้หน้าจอบอกผู้ใช้ว่า "มีรายการจากสลิปนี้อยู่แล้ว"
220	error อื่น ๆ (เช่น อินเทอร์เน็ตหลุด) โยนต่อให้หน้าจอขึ้น "บันทึกไม่สำเร็จ ลองอีกครั้ง"
221	แปลงแถวจากฐานข้อมูลกลับเป็นรูปแบบที่แอปใช้ (amount_satang → amountSatang) แล้วส่งคืน

ขั้น 5: ตัวเลขคำนวณใหม่เอง

src/data/AppProvider.tsx	บรรทัด 442–459
<pre> 442 /** Derived money numbers used across screens (FR-2 balance, FR-6 runway). */ 443 export function useMoney() { 444 const { profile, txs, carrySatang, goals, today } = useApp(); 445 return useMemo(() => { 446 const opening = profile?.openingBalanceSatang ?? 0; 447 const floor = profile?.runwayFloorSatang ?? 50_000; 448 // Money from before the loaded window (over ~13 months ago) still counts. 449 const balance = computeBalance(opening + carrySatang, txs); 450 const average = averageDailyExpense(txs); 451 // Money in savings goals is set aside: it does not count as money to spend. 452 const reserved = reservedSatang(goals); 453 const runway = computeRunway({ balanceSatang: balance, floorSatang: floor + reserved, averageSatang: average.averageSatang }); 454 const drafts = txs.filter((t) => t.status === 'draft'); 455 return { balance, average, runway, drafts, reserved, floor }; </pre>	

```

456 // `today` makes the runway and averages start over at midnight.
457 // eslint-disable-next-line react-hooks/exhaustive-deps
458 }, [profile, txs, carrySatang, goals, today]);
459 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/data/AppProvider.tsx:442`

บรรทัด	ความหมาย
443	useMoney คือ "hook" ที่ทุกหน้าเรียกใช้เมื่ออยากได้ตัวเลขเงิน (บท A6 อธิบายว่า hook คืออะไร)
444	ดึงข้อมูลดิบจากสถานะกลาง: โปรไฟล์ รายการ ยอดยกมา กระปุกลูก และวันนี้
445	useMemo แปลว่า "คำนวณแล้วจำไว้" คำนวณใหม่เฉพาะเมื่อข้อมูลในบรรทัด 458 เปลี่ยน
446–447	ยอดเงินตอนเริ่มใช้แอป และเส้นเงินสำรอง (ถ้ายังไม่ตั้ง ใช้ 50,000 สตางค์ = ฿500)
449	คำนวณยอดคงเหลือ (FR-2) จากรายการทั้งหมด
450–453	คำนวณค่าเฉลี่ยรายจ่าย เงินในกระปุก และ Money Runway (FR-6)
454	หารายการแบบร่างที่รอตรวจ (FR-4)
455	ส่งตัวเลขทั้งหมดออกไปให้หน้าจอใช้
458	รายการ "สิ่งที่ต้องจับตา" ถ้า txs เปลี่ยน (เพิ่งเพิ่มข้าวมันไก่) จะคำนวณใหม่ทั้งหมด และ today ทำให้คำนวณใหม่ตอนเที่ยงคืน

★ จำให้ได้: ตัวเลขบนหน้าจอไม่ได้เก็บไว้ แต่คิดใหม่ทุกครั้ง

ยอดคงเหลือ ค่าเฉลี่ย และ Runway **ไม่ได้เก็บในฐานข้อมูล** ฐานข้อมูลเก็บแค่ "รายการ" แล้วแอปคำนวณตัวเลขทั้งหมดสด ๆ จากรายการทุกครั้งที่มีอะไรเปลี่ยน ข้อดีคือตัวเลขไม่มีทางขัดกันเอง (ไม่มีกรณียอดในฐานข้อมูลออกอย่างหนึ่ง แต่รายการบอกอีกอย่าง) คอมเมนต์บนสุดของ [src/data/AppProvider.tsx](#) เขียนไว้ว่า "Numbers shown on screen are always derived from this state ... never stored twice"

ตามข้อมูลที่ใช้ AI: สแกนสลิป 1 ใบ

1. มือถือหารูปใหม่ในแคลเลอร์ [src/services/AutoScanProvider.tsx](#)

อ่าน QR บนรูปในเครื่องก่อน (ฟรี ไม่ใช้อินเทอร์เน็ต) รูปไหนไม่มี QR ของสลิปธนาคารจะไม่ถูกส่งไปไหนเลย

↓ เฉพาะรูปที่เป็นสลิป

2. ส่งรูปไป Edge Function [src/services/slips.ts](#) → [supabase/functions/parse-slip/index.ts](#)

เซิร์ฟเวอร์ตรวจว่าเข้าสู่ระบบจริง ตรวจโควตา (300 ใบต่อวัน) แล้วส่งรูปให้ AI อ่าน 2 ครั้งพร้อมกันด้วย 2 รุ่น

↓

3. AI ตอบเป็น JSON

ยอดเงิน วันที่ ชื่อผู้รับ ธนาคาร และ "ความมั่นใจ" ของแต่ละช่อง เซิร์ฟเวอร์รวมผลสองครั้ง ถ้าไม่ตรงกันจะให้ผู้ใช้ตรวจ

↓

4. แอปบันทึกรายการ

AI อ่านตรงกันทั้ง 2 ครั้งและมั่นใจทุกช่อง $\geq 80\%$ → เข้ายอดเงินเลย (ผู้ใช้ปิดตัวเลือกนี้ได้ในหน้าตั้งค่า) · ไม่มั่นใจหรืออ่านไม่ตรงกัน → เป็นแบบร่าง รอผู้ใช้ตรวจ แล้วตัวเลขทั้งแอปคำนวณใหม่เหมือนขึ้น 5 ด้านบน

รายละเอียดของ AI อยู่ในบท A10 และรายละเอียดเพิ่มเติมของการสแกนสลิปอยู่ในเล่ม FR-4

เครื่องมือ: เขียนโค้ดที่ไหน ใช้โปรแกรมอะไร และ VS Code ตั้งแต่ศูนย์

ทุกอย่างที่ต้องมีในเครื่อง วิธีเอาโค้ด MindPay มาเปิด รับแอป และใช้ VS Code อ่านโค้ดอย่างมืออาชีพ

กล่องเครื่องมือของทีมเรา

เครื่องมือ	คืออะไร	ใช้ทำอะไรใน MindPay
VS Code	โปรแกรมเขียนโค้ด (code editor) ฟรีจาก Microsoft	เปิด อ่าน แก้โค้ดทุกไฟล์ และรับคำสั่งผ่าน terminal ในตัว
Node.js (เวอร์ชัน 22 ขึ้นไป)	ตัวรับภาษา JavaScript นอกเบราว์เซอร์	รันเครื่องมือทั้งหมด: Expo, เทสต์, สคริปต์ build
npm	ตัวจัดการ "แพ็คเกจ" (โค้ดสำเร็จรูปที่คนอื่นเขียนไว้) มากับ Node.js	ติดตั้งไลบรารี 38 ตัวที่แอปใช้ (npm install) และรับคำสั่งใน package.json
Git	ระบบเก็บประวัติการแก้ไขโค้ด ย้อนเวอร์ชันได้	บันทึกการเปลี่ยนแปลงทุกครั้งเป็น "commit"
GitHub	เว็บที่เก็บโค้ดของ Git บนคลาวด์ ทำงานเป็นทีมได้	เก็บโค้ดที่ github.com/2550expo-tech/Socrates-and-Skeletons - และ build แอปให้อัตโนมัติ (un A13)
Expo	ชุดเครื่องมือทำแอปมือถือด้วย React Native	รันแอประหว่างพัฒนา (npx expo start), ทำเว็บ, build APK, ส่งอัปเดต
Expo Go	แอปบนมือถือสำหรับลองแอปที่กำลังเขียน	สแกน QR จากคอมแล้วเห็นแอปบนมือถือทันที
Supabase Dashboard	หน้าเว็บจัดการเซิร์ฟเวอร์ที่ supabase.com	ดูตาราง รับ SQL ใส่query แอ AI ดู log ของ Edge Functions
เบราว์เซอร์ Chrome	-	เปิดเว็บเวอร์ชัน และใช้ DevTools (F12) ดู error

ติดตั้งเครื่องมือ (ทำครั้งเดียว)

- VS Code:** ดาวน์โหลดจาก code.visualstudio.com ติดตั้งแบบค่าเริ่มต้น (Windows คลิก "Add to PATH" และ "Open with Code" ไปด้วย)
- Node.js:** ดาวน์โหลดรุ่น LTS (22 ขึ้นไป) จาก nodejs.org แล้วเปิด terminal พิมพ์ node -v ต้องขึ้น v22 หรือสูงกว่า
- Git:** ดาวน์โหลดจาก git-scm.com (Mac มีมาให้แล้ว) ตรวจสอบด้วย git --version
- บัญชี GitHub:** สมัครที่ github.com และขอให้เจ้าของ repo เพิ่มเป็นสมาชิกถ้าต้องแก้ไข (อ่านอย่างเดียวไม่ต้องขอ เพราะ repo เป็นสาธารณะ)
- Expo Go** บนมือถือ (ไม่บังคับ): ดาวน์โหลดจาก Play Store หรือ App Store

เอาโค้ด MindPay มาไว้ในเครื่อง

เปิด terminal (Windows: PowerShell, Mac: Terminal หรือใช้ terminal ใน VS Code) แล้วพิมพ์ทีละบรรทัด

Terminalคำสั่งที่พิมพ์จริง

```
1 git clone https://github.com/2550expo-tech/Socrates-and-Skeletons-.git
2 cd Socrates-and-Skeletons-
3 npm install
4 cp .env.example .env
5 code .
```

บรรทัด	ความหมาย
1	clone = คัดลอกโค้ดทั้งหมดพร้อมประวัติจาก GitHub มาไว้ในโฟลเดอร์ใหม่ชื่อ Socrates-and-Skeletons-
2	cd (change directory) = เข้าไปในโฟลเดอร์นั้น
3	อ่านรายชื่อแพ็คเกจใน package.json แล้วดาวน์โหลดทั้งหมดลงโฟลเดอร์ node_modules (ประมาณ 1-3 นาที ใช้พื้นที่หลายร้อย MB)
4	คัดลอกไฟล์ตั้งค่าตัวอย่างเป็น .env ซึ่งบอกแอปว่าเซิร์ฟเวอร์ Supabase อยู่ที่ไหน (Windows PowerShell ใช้ copy .env.example .env)
5	เปิด VS Code ที่โฟลเดอร์นี้ (จุด . แปลว่า "โฟลเดอร์ปัจจุบัน")

ไฟล์ .env มีอะไร

.env.exampleบรรทัด 1-6

```
1 # Copy this file to .env and fill in the values from
2 # Supabase Dashboard -> Project Settings -> API (or the green "Connect" button).
3 # Only the PUBLISHABLE (anon) key goes here. Never put the secret/service_role key
4 # or any AI key (Gemini / Anthropic) in the app: those live in Supabase Edge Function secrets.
5 EXPO_PUBLIC_SUPABASE_URL=https://dufvcdswcaqsfbxjwat.supabase.co
6 EXPO_PUBLIC_SUPABASE_PUBLISHABLE_KEY=sb_publishable_LKQP3TU8Fr3t8JD6bzg7TA_5Uy6zcm
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)VS Code ในเครื่อง: Ctrl+P → .env.example:1

บรรทัด	ความหมาย
1-4	คอมเมนต์ (บรรทัดที่ขึ้นต้นด้วย # คอมพิวเตอร์ไม่อ่าน) บอกว่าให้ใส่เฉพาะกุญแจ "publishable" ห้ามใส่กุญแจลับหรือกุญแจ AI ในแอปเด็ดขาด
5	ที่อยู่ของเซิร์ฟเวอร์ Supabase ของทีม
6	กุญแจสาธารณะ (publishable key) ใช้บอกว่า "แอปนี้เป็นของโปรเจกต์ไหน" ปลอดภัยที่จะอยู่ในแอป เพราะฐานข้อมูลใช้ RLS กันไว้ (บท A9) ชื่อที่ขึ้นต้นด้วย EXPO_PUBLIC_ คือค่าที่ Expo ยอมใส่เข้าไปในแอป

ไฟล์ package.json: บัตรประชาชนของโปรเจกต์

ไฟล์นี้บอกว่าโปรเจกต์ชื่ออะไร ใช้แพ็คเกจอะไร และมีคำสั่งลัดอะไรบ้าง

package.jsonบรรทัด 1-59

```

1 {
2   "name": "mindpay",
3   "version": "1.0.0",
4   "main": "expo-router/entry",
5   "dependencies": {
6     "@expo-google-fonts/anuphan": "^0.4.1",
7     "@expo-google-fonts/noto-serif-thai": "^0.4.2",
8     "@expo/vector-icons": "^15.0.2",
9     "@react-native-community/datetimepicker": "9.1.0",
10    "@supabase/supabase-js": "^2.117.2",
11    "expo": "~57.0.25",
12    "expo-camera": "~57.0.5",
13    "expo-constants": "~57.0.19",
14    "expo-crypto": "~57.0.3",
15    "expo-file-system": "~57.0.7",
16    "expo-font": "~57.0.4",
17    "expo-haptics": "~57.0.3",
18    "expo-image-manipulator": "~57.0.20",
19    "expo-image-picker": "~57.0.20",
20    "expo-linear-gradient": "~57.0.2",
21    "expo-linking": "~57.0.11",
22    "expo-media-library": "~57.0.5",
23    "expo-router": "~57.0.23",
24    "expo-speech": "~57.0.3",
25    "expo-speech-recognition": "^57.1.0",
26    "expo-splash-screen": "~57.0.9",
27    "expo-sqlite": "~57.0.3",
28    "expo-status-bar": "~57.0.1",
29    "expo-system-ui": "~57.0.4",
30    "expo-updates": "~57.0.23",
31    "react": "19.2.3",
32    "react-dom": "19.2.3",
33    "react-native": "0.86.3",
34    "react-native-safe-area-context": "~5.7.0",
35    "react-native-screens": "~4.26.0",
36    "react-native-svg": "15.15.4",
37    "react-native-web": "~0.21.0"
38  },
39  "devDependencies": {
40    "@types/react": "~19.2.2",
41    "eslint": "^9.0.0",
42    "eslint-config-expo": "~57.0.2",
43    "playwright": "^1.56.0",
44    "typescript": "~6.0.3",
45    "vitest": "^5.0.2"
46  },
47  "scripts": {
48    "start": "expo start",
49    "android": "expo start --android",
50    "ios": "expo start --ios",
51    "web": "expo start --web",
52    "test": "vitest run",
53    "typecheck": "tsc --noEmit",
54    "lint": "expo lint",
55    "e2e:web": "EXPO_WEB_BASE_URL=/Socrates-and-Skeletons- expo export --platform web --output-dir dist && EXPO_WEB_BASE_URL=/Socrates-and-Skeletons- node scripts/web-home-screen.mjs dist && cp dist/index.html dist/404.html && node e2e/web-auth.mjs dist e2e-shots",
56    "demo:video": "node e2e/demo-video.mjs dist demo-video light android && node e2e/demo-video.mjs dist demo-video dark android && node e2e/demo-video.mjs dist demo-video light web"
57  },

```

```
58 "private": true
59 }
```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → package.json:1

บรรทัด	ความหมาย
2–3	ชื่อโปรเจกต์ mindpay และเวอร์ชัน
4	จุดเริ่มต้นของแอป: ให้ Expo Router เป็นคนเริ่ม แล้วมันจะไปหาหน้าจอในโฟลเดอร์ src/app เอง
5–38	dependencies แพ็กเกจที่แอปต้องใช้ตอนทำงานจริง ตัวเลขหลังชื่อคือเวอร์ชัน (^ = ยอมรุ่นย่อยที่ใหม่กว่า, ~ = ยอมเฉพาะรุ่นแก้บั๊ก)
6–7	ฟอนต์ไทย Anuphan (ข้อความทั่วไป) และ Noto Serif Thai (ตัวเลขเงินและหัวข้อ)
8	ไอคอนสำเร็จรูป (Icons) เช่น รูปบ้าน รูปใบไม้ในแถบล่าง
9	ตัวเลือกวันที่และเวลาแบบของมือถือ (FR-1)
10	ตัวเชื่อมต่อ Supabase (ฐานข้อมูล, เข้าสู่ระบบ, Edge Functions)
11	Expo ตัวหลัก เวอร์ชัน 57
12	expo-camera ใช้ อ่าน QR จากรูป (ไม่ได้ใช้ถ่ายรูป แอปบล็อกสิทธิ์กล้องไว้)
14	expo-crypto ทำ "ลายนิ้วมือ" (hash) ของรูปสลิป กันสแกนซ้ำ (FR-4)
15, 18	อ่านไฟล์ และย่อ/ตัดรูปก่อนส่งให้ AI
17	expo-haptics สั่นเบา ๆ เวลากดปุ่ม
19, 22	เลือกรูปเอง และเข้าถึงแคเลอรี่ทั้งหมด (FR-4)
20	สีไล่ระดับ (gradient) ของการ์ดเขียว
21	เปิดลิงก์ เช่น ลิงก์ยืนยันอีเมลที่พาเข้าแอป
23	Expo Router ระบบเปลี่ยนหน้าจอ (บท A7)
24–25	expo-speech ให้น้องกล่าวพูดออกเสียง (FR-5), expo-speech-recognition ฟังเสียงพูดจด (FR-3)
26	หน้าจอแรกตอนเปิดแอป (splash)
27	expo-sqlite ฐานข้อมูลเล็ก ๆ ในเครื่อง ใช้เก็บการตั้งค่าและข้อมูลโหมดทดลอง
30	expo-updates รับอัปเดตโค้ดใหม่โดยไม่ต้องลงแอปใหม่ (EAS Update)
31–33	React (ตัววาดหน้าจอ), react-dom (สำหรับเว็บ), React Native (สำหรับมือถือ)
34–35	จัดการขอมจ (รอยบาก แถบสถานะ) และหน้าจอแบบ native
36	วาดภาพเวกเตอร์ (SVG) เช่น ต้นไม้เงิน น้องกล่าวกราฟ
37	react-native-web แปลงชิ้นส่วนของมือถือเป็น HTML ให้แสดงบนเว็บได้
39–46	devDependencies ใช้ตอนพัฒนาเท่านั้น ไม่ได้อยู่ในแอปที่ผู้ใช้งาน: ตัวตรวจภาษา TypeScript, ESLint (ตรวจรูปแบบโค้ด), Playwright (ควบคุมเบราว์เซอร์อัตโนมัติสำหรับ E2E), Vitest (รัน unit test)

บรรทัด	ความหมาย
47-57	scripts คำสั่งลัด พิมพ์ <code>npm run</code> ชื่อ (หรือ <code>npm test</code> , <code>npm start</code>) แทนคำสั่งยาว ๆ ทางขวา
58	<code>private</code> = ไม่เผยแพร่เป็นแพ็คเกจให้คนอื่นติดตั้ง

คำสั่งที่ใช้บ่อย

คำสั่ง	ทำอะไร	ผลที่ควรเห็น
<code>npm run web</code>	เปิดแอปเวอร์ชันเว็บในเบราว์เซอร์ แก้โค้ดแล้วหน้าเว็บเปลี่ยนเอง	เบราว์เซอร์เปิด localhost:8081
<code>npx expo start</code>	เปิด "dev server" มี QR ให้สแกนด้วย Expo Go	QR ใน terminal กด w = เปิดเว็บ, r = โหลดใหม่
<code>npm test</code>	รัน unit test ทั้งหมด	"Tests 157 passed (157)"
<code>npm run typecheck</code>	ให้ TypeScript ตรวจสอบข้อมูลทั้งโปรเจกต์	ไม่มีข้อความ = ไม่มีปัญหา
<code>npx expo lint</code>	ตรวจสอบรูปแบบโค้ดตามกฎ ESLint	ไม่มี error
<code>npm run e2e:web</code>	build เว็บแล้วรันเทสต์ในเบราว์เซอร์ 145 ข้อ ถ่ายภาพหน้าจอเก็บใน e2e-shots	"145/145 checks passed"

! ถ้าเจอ error ตอนติดตั้ง

"node: command not found" = ยังไม่ได้ติดตั้ง Node.js หรือต้องปิดแล้วเปิด terminal ใหม่ · **npm install** ซ้ำ หรือค้าง = รอได้ หรือลบโฟลเดอร์ node_modules แล้วลองใหม่ · **แอปขึ้นแต่เข้าสู่ระบบไม่ได้** = ลืมสร้างไฟล์ .env (บรรทัด 4) · ห้ามใช้ `npm install` ชื่อแพ็คเกจ ตรง ๆ ในโปรเจกต์ Expo ให้ใช้ `npx expo install` ชื่อแพ็คเกจ เพื่อให้ได้รุ่นที่เข้ากับ Expo 57 (กฎใน AGENTS.md)

รู้จักหน้าจอ VS Code

ส่วน	อยู่ตรงไหน	ใช้ทำอะไร
Activity Bar	แถบไอคอนซ้ายสุด	สลับระหว่าง Explorer, Search, Source Control, Run, Extensions
Explorer	ไอคอนรูปกระดาษ (Ctrl+Shift+E)	ต้นไม้โฟลเดอร์และไฟล์ทั้งโปรเจกต์ คลิกเพื่อเปิดไฟล์
Editor	พื้นที่ใหญ่ตรงกลาง	อ่านและแก้ไขโค้ด เปิดหลายไฟล์เป็นแท็บได้ ลากแท็บไปข้าง ๆ เพื่อดู 2 ไฟล์ คู่กัน
Search	ไอคอนแว่นขยาย (Ctrl+Shift+F)	ค้นคำในทุกไฟล์ เช่นค้น <code>computeRunway</code> จะเห็นทุกที่ที่ใช้
Source Control	ไอคอนกิ่งไม้ (Ctrl+Shift+G)	ดูไฟล์ที่แก้ไข, commit, push (un A13)

ส่วน	อยู่ตรงไหน	ใช้ทำอะไร
Terminal	ด้านล่าง (Ctrl+` ปุ่มตัวหนอนใต้ Esc)	พิมพ์คำสั่ง npm, git ได้โดยไม่ต้องออกจาก VS Code
Problems	แถบข้าง Terminal (Ctrl+Shift+M)	รวม error จาก TypeScript และ ESLint คลิกแล้วกระโดดไปบรรทัดที่ผิด
Status Bar	แถบล่างสุด	บอก branch ของ Git, บรรทัด/คอลัมน์ที่เคอร์เซอร์อยู่, ภาษาของไฟล์

คีย์ลัดที่ทำให้อ่านโค้ดเร็วขึ้น 10 เท่า

Mac ใช้ Cmd แทน Ctrl และ Option แทน Alt

คีย์	ทำอะไร	ลองกับ MindPay
Ctrl+P	เปิดไฟล์จากชื่อ พิมพ์ : เลขบรรทัด ต่อท้ายเพื่อกระโดดไปบรรทัดนั้น	พิมพ์ runway.ts : 72 แล้ว Enter
Ctrl+G	ไปบรรทัดที่ต้องการในไฟล์ที่เปิดอยู่	Ctrl+G แล้วพิมพ์ 89
F12	Go to Definition ไปดูว่าฟังก์ชันหรือตัวแปรนี้ถูกสร้างไว้ที่ไหน (หรือ Ctrl+คลิก)	ใน AppProvider.tsx คลิกที่ computeRunway แล้วกด F12
Alt+F12	Peek Definition: ดูโค้ดปลายทางในกล่องเล็ก ๆ โดยไม่ต้องเปลี่ยนไฟล์	-
Shift+F12	Find All References หากทุกที่ที่เรียกใช้สิ่งนี้	ที่ computeRunway จะเห็นทั้งในแอปและในเทสต์
Alt+← / Alt+→	ย้อนกลับ/ไปข้างหน้า หลังกระโดดไปมาด้วย F12	-
เอาเมาส์ชี้ (hover)	เห็นชนิดข้อมูลและคอมเมนต์อธิบายของสิ่งนั้น	ชี้ที่ Transaction จะเห็นทุกช่องของรายการ
Ctrl+Shift+0	รายการฟังก์ชันทั้งหมดในไฟล์นี้ (outline)	ใน runway.ts จะเห็น 6 ฟังก์ชัน
Ctrl+T	ค้นหาชื่อฟังก์ชันทั้งโปรเจกต์	พิมพ์ parseSpokenEntry
Ctrl+Shift+F	ค้นข้อความในทุกไฟล์	ค้น "เงินพอใช้อีก" เพื่อหาว่าข้อความบนจอมาจากไฟล์ไหน
Ctrl+F	ค้นในไฟล์นี้	-
F2	เปลี่ยนชื่อตัวแปรทุกที่พร้อมกันอย่างปลอดภัย	-
Ctrl+/,	เปลี่ยนบรรทัดเป็นคอมเมนต์ (ปิดชั่วคราว) หรือกลับคืน	-
Ctrl+Shift+P	Command Palette เรียกคำสั่งทุกอย่างของ VS Code	พิมพ์ "Format Document"
Ctrl+B	ซ่อน/แสดงแถบด้านข้าง ให้มีที่อ่านโค้ดมากขึ้น	-

★ เทคนิคหาว่าข้อความบนจอมาจากโค้ดตรงไหน

เห็นคำว่า "เงินพอใช้อีก 16 วัน" บนหน้าจอหลัก แล้วอยากรู้ว่าโค้ดอยู่ไหน: กด **Ctrl+Shift+F** พิมพ์ "เงินพอใช้อีก" จะเจอไฟล์ที่มีข้อความนี้ทั้งหมด คลิกผลลัพธ์แล้วจะไปบรรทัดนั้นทันที วิธีนี้ใช้ได้กับทุกข้อความภาษาไทยในแอป เพราะข้อความทั้งหมดเขียนอยู่ในโค้ดตรง ๆ

ส่วนเสริม (Extensions) ที่แนะนำ

กด Ctrl+Shift+X แล้วค้นชื่อ กด Install

ชื่อ	ช่วยอะไร
ESLint (Microsoft)	ขีดเส้นใต้โค้ดที่ผิดกฎทันทีที่พิมพ์ ใช้กฎเดียวกับ <code>npm run lint</code>
Expo Tools	ช่วยเติมคำและตรวจไฟล์ <code>app.json</code> / <code>eas.json</code>
Vitest	มีปุ่ม ▶ ข้างเกสต์แต่ละข้อ กดรันทีละข้อได้
GitLens (ไม่บังคับ)	ขีดบรรทัดไหนก็เห็นว่าใครแก้ล่าสุด เมื่อไร ใน commit อะไร
Error Lens (ไม่บังคับ)	แสดงข้อความ error ถ้ายกบรรทัดเลย ไม่ต้องเอาเมาส์ชี้

ส่วน TypeScript ไม่ต้องลงเพิ่ม VS Code รองรับในตัวอยู่แล้ว และใช้ TypeScript รุ่นในโปรเจกต์ (`node_modules/typescript`) ตามไฟล์ `tsconfig.json`

ไม่อยากติดตั้งอะไร: ใช้ VS Code บนเว็บ

มี 2 ทางที่เปิดโค้ด MindPay ด้วย VS Code ได้ทันทีในเบราว์เซอร์

- **ลิงก์ในเล่มนี้** แต่ชื่อไฟล์บนหัวโค้ดค่อนข้างโหดก็ได้ จะเปิด **vscode.dev** ที่ไฟล์และบรรทัดนั้น
- **จากหน้า GitHub** เปิด repo บน github.com แล้วกดปุ่ม . (จุด) บนคีย์บอร์ด หน้าเว็บจะกลายเป็น VS Code (github.dev)

VS Code บนเว็บอ่านโค้ด ค้นหา (Ctrl+Shift+F) กด F12 ไปดูฟังก์ชัน และแก้ไขได้ แต่ **รันคำสั่งไม่ได้** (ไม่มี terminal จึงรัน `npm test` หรือเปิดแอปไม่ได้) ถ้าจะรันแอปต้องโคลนลงเครื่องตามขั้นตอนด้านบน

ฝึกใช้: ตามรอย Money Runway ใน 5 นาที

1. Ctrl+P พิมพ์ `index.tsx` เลือกไฟล์ใน `src/app/` (tabs) (หน้าจอหลัก)
2. Ctrl+F ค้น `useMoney` แล้วกด F12 ที่คำนั้น จะเดินไป `src/data/AppProvider.tsx` บรรทัด 443
3. ที่บรรทัด 453 คลิก `computeRunway` แล้วกด F12 จะไปที่ `src/domain/runway.ts` บรรทัด 72 ซึ่งเป็นสูตรของ FR-6
4. กด Shift+F12 ที่ชื่อฟังก์ชัน จะเห็นว่ามีเกสต์เรียกใช้ใน `src/domain/__tests__/domain.test.ts` ด้วย
5. กด Alt+← สองครั้ง ย้อนกลับไปหน้าจอหลัก

ทำแบบนี้ได้ แปลว่าพร้อมอ่านโค้ดส่วนไหนของแอปก็ได้แล้ว

พื้นฐานการเขียนโปรแกรม สำหรับคนที่ไม่เคยเขียนเลย

ทุกอย่างที่ต้องรู้เพื่ออ่านโค้ด MindPay ออก: ค่า ตัวแปร เงื่อนไข ฟังก์ชัน array และ object พร้อมตัวอย่างจากโค้ดจริง

โปรแกรมคืออะไร

โปรแกรมคือ รายการคำสั่งที่คอมพิวเตอร์ทำตามทีละบรรทัด จากบนลงล่าง เหมือนสูตรทำอาหาร ทีมเราเขียนด้วยภาษา **TypeScript** ซึ่งคือภาษา **JavaScript** (ภาษาของเว็บ) ที่เพิ่ม "ชนิดข้อมูล" เข้าไป (บท A5) ไฟล์ที่ลงท้ายด้วย `.ts` เป็น TypeScript ล้วน ส่วน `.tsx` คือ TypeScript ที่มีโค้ดหน้าจอบน React อยู่นะ (บท A6)

1. ค่า (value) และชนิดของค่า

ชนิด	ตัวอย่าง	ใช้ใน MindPay
number ตัวเลข	8500, -3, 0.8	จำนวนเงินเป็นสตางค์ 8500 = ฿85, ความมั่นใจ 0.8 = 80%
string ข้อความ	'ข้าวมันไก่', "2026-10-09"	ชื่อรายการ วันที่ ข้อความบนจอ (ใส่ในเครื่องหมายคำพูด ' หรือ ")
boolean จริง/เท็จ	true, false	เข้าสู่ระบบแล้วหรือยัง, ตั้งค่าเสร็จหรือยัง (onboarded)
null	null	"ไม่มีค่า" โดยตั้งใจ เช่น รายการที่จองเองไม่มีเลขอ้างอิงสลิป (slipRef: null)
undefined	undefined	"ยังไม่ได้ใส่ค่า" เช่น ไม่ได้ส่งพารามิเตอร์นั้นมา

2. ตัวแปร: กล่องที่มีชื่อ

ตัวแปรคือ "กล่องที่ติดป้ายชื่อ" ไว้ใส่ค่า เพื่อเรียกใช้ทีหลัง

ตัวแปร

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 const floor = 50_000;           // const = ค่าคงที่ ใส่แล้วเปลี่ยนไม่ได้
2 let total = 0;                  // let = เปลี่ยนค่าได้ภายหลัง
3 total = total + 8500;           // ตอนนี้ total = 8500
4 const title = 'ข้าวมันไก่';
```

- `const` ใช้บ่อยที่สุดในโค้ด MindPay เพราะค่าส่วนใหญ่ไม่ควรเปลี่ยน ใช้ `let` เฉพาะตอนต้องเปลี่ยนจริง ๆ
- `50_000` คือ 50000 เครื่องหมาย `_` ช่วยให้อ่านง่ายเหมือนคอมมา คอมพิวเตอร์ไม่สนใจมัน
- ข้อความหลัง `//` คือ **คอมเมนต์** คอมพิวเตอร์ข้าม มีไว้ให้คนอ่าน ส่วน `/* ... */` และ `/** ... */` เป็นคอมเมนต์หลายบรรทัด (แบบ `/**` VS Code จะเอาไปแสดงตอนเอาเมาส์ชี้)
- ตั้งชื่อแบบ **camelCase**: คำแรกตัวเล็ก คำต่อไปขึ้นต้นตัวใหญ่ เช่น `balanceSatang`, `averageDailyExpense` คำคงที่สำคัญใช้ตัวใหญ่ทั้งหมด เช่น `RUNWAY_CAP_DAYS`

★ กฎการตั้งชื่อของทิมเรา: ตัวแปรเงินลงท้ายด้วย Satang

ทุกตัวแปรที่เป็นจำนวนเงิน ลงท้ายด้วย Satang เสมอ เช่น amountSatang, floorSatang เห็นชื่อแล้วรู้ทันทีว่าต้องหาร 100 ก่อนแสดงเป็นบาท กันความผิดพลาดแบบเอาบาทไปบวกกับสตางค์

3. ตัวดำเนินการ: คำนวณและเปรียบเทียบ

เขียนว่า	ความหมาย	ตัวอย่าง → ผล
+ - * /	บวก ลบ คูณ หาร	300000 / 145 → 2068.96...
%	เศษจากการหาร	8550 % 100 → 50 (50 สตางค์)
=== !==	เท่ากับ / ไม่เท่ากับ (แบบเคร่ง ชนิดต้องตรงด้วย)	t.kind === 'income'
< <= > >=	น้อยกว่า น้อยกว่าหรือเท่ากับ ...	days < 7
&&	"และ" จริงเมื่อทั้งสองข้างจริง	d >= windowStart && d <= today
	"หรือ" จริงเมื่อข้างใดข้างหนึ่งจริง	event === 'SIGNED_IN' ...
!	"ไม่" กลับจริงเป็นเท็จ	!existing → ไม่มีรายการเดิม
ก ? ข : ค	ถ้า ก จริง ได้ ข ไม่จริงได้ ค (ternary)	kind === 'income' ? 'เพิ่มเงินเข้า' : 'ลดรายการ'
ก ?? ข	ใช้ ก ถ้า ก ไม่ใช่ null/undefined ไม่ขึ้นใช้ ข (ค่าสำรอง)	profile?.runwayFloorSatang ?? 50_000
ก?.ข	เข้าถึง ข ใน ก แต่ถ้า ก เป็น null ไม่ต้อง error ได้ undefined	profile?.openingBalanceSatang

ลองอ่านบรรทัดนี้จาก [src/data/AppProvider.tsx:447](#) ซึ่งใช้ทั้ง ?. และ ??

src/data/AppProvider.tsx

บรรทัด 447-447

```
447 const floor = profile?.runwayFloorSatang ?? 50_000;
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/AppProvider.tsx:447

อ่านว่า: "เส้นเงินสำรอง = ค่าที่ผู้ตั้งไว้ในโปรไฟล์ แต่ถ้ายังไม่มีโปรไฟล์ (?.) หรือไม่มีค่า ให้ใช้ 50,000 สตางค์ (??)"

4. เงื่อนไข if: ถ้า... ให้ทำ...

if / else

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 if (spendable <= 0) {
2   // ทำส่วนนี้เมื่อเงื่อนไขจริง
3 } else if (averageSatang <= 0) {
4   // ไม่จริง ถ้าเงื่อนไขนี้จริง ทำส่วนนี้
5 } else {
6   // ไม่ตรงอะไรเลย ทำส่วนนี้
```

ในโค้ด MindPay จะเห็นรูปแบบ "เช็กกรณีพิเศษก่อน แล้วออกเลย" (early return) บ่อยมาก เช่นในสูตร Money Runway

src/domain/runway.ts

บรรทัด 83-88

```
83  if (spendable <= 0) {
84    return { ...base, status: 'below_floor', days: 0, depletionDay: today, capped: false };
85  }
86  if (averageSatang <= 0) {
87    return { ...base, status: 'no_spending', days: null, depletionDay: null, capped: false };
88  }
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/runway.ts:83

อ่านว่า: "ถ้าเงินที่ใช้ได้ ≤ 0 ให้ส่งผลกลับเลยว่าต่ำกว่าเงินสำรอง · ถ้าค่าเฉลี่ย ≤ 0 ส่งกลับว่ายังไม่มีรายจ่าย" พอผ่าน 2 ด่านนี้ไปได้ โค้ดด้านล่างมั่นใจได้ว่าหารได้แน่นอน ไม่หารด้วยศูนย์

5. ฟังก์ชัน: เครื่องจักรรับของเข้า ส่งของออก

ฟังก์ชันคือ "ชุดคำสั่งที่ตั้งชื่อไว้" เรียกใช้ซ้ำได้ รับ พารามิเตอร์ (ของที่ส่งเข้า) และ return ผลลัพธ์ออกมา

ฟังก์ชัน 2 แบบ

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1  // แบบที่ 1: function
2  function toBaht(satang: number): number {
3    return satang / 100;
4  }
5
6  // แบบที่ 2: arrow function (ลูกศร =>) ทำงานเหมือนกัน เขียนสั้นกว่า
7  const toBaht2 = (satang: number) => satang / 100;
8
9  toBaht(8500); // ได้ 85
```

- (satang: number) = รับ 1 พารามิเตอร์ชื่อ satang ต้องเป็นตัวเลข
- : number หลังวงเล็บ = ฟังก์ชันนี้จะคืนค่าเป็นตัวเลข
- arrow function ที่ไม่มี { } จะ return ค่าทางขวาของ => ให้อัตโนมัติ
- export หน้าฟังก์ชัน = ให้ไฟล์อื่น import ไปใช้ได้

★ Pure function: หัวใจของ src/domain

ฟังก์ชันใน src/domain ทุกตัวเป็น **pure function** คือ (1) ใส่ค่าเดิม ได้ผลเดิมทุกครั้ง และ (2) ไม่ไปทำอะไรข้างนอก ไม่เรียกอื่นเตอร์เน็ต ไม่แตะหน้าจอ เหมือนเครื่องคิดเลข จึงทดสอบง่ายมาก: ใส่ตัวเลขแล้วตรวจคำตอบ อาจารย์มักถามว่า "ทำไมแยก domain" ให้ตอบเรื่องนี้

6. Array: รายการของหลายค่า

Array คือรายการเรียงลำดับ เขียนใน [] ค้นด้วยคอมมา ตำแหน่งแรกคือ 0

Array พื้นฐาน

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```

1 const amounts = [12000, 0, 8500, 30000];
2 amounts[0];           // 12000 (ตัวแรก)
3 amounts.length;       // 4 (มีกี่ตัว)
4 amounts.includes(0);   // true

```

MindPay เก็บรายการเงินทั้งหมดเป็น array ชื่อ txs (ย่อจาก transactions) และใช้ "เมธอด" ของ array ต่อไปนี้ตลอด ต้องจำให้ได้

เมธอด	ทำอะไร	ตัวอย่าง
filter	คัดเฉพาะตัวที่ผ่านเงื่อนไข ได้ array ใหม่ที่สั้นลง	txs.filter((t) => t.status === 'confirmed')
map	แปลงทุกตัว ได้ array ยาวเท่าเดิม	txs.map((t) => t.amountSatang)
reduce	รวมทุกตัวให้เหลือค่าเดียว เช่น ผลรวม	list.reduce((s, t) => s + t.amountSatang, 0)
find	หาตัวแรกที่ตรงเงื่อนไข (ไม่เจอได้ undefined)	txs.find((t) => t.id === id)
some	มีอย่างน้อย 1 ตัวที่ตรงไหม (true/false)	list.some((t) => t.slipRef === ref)
sort	เรียงลำดับ	days.sort()
slice	ตัดเอาบางส่วน	list.slice(0, 5) = 5 ตัวแรก

ตัว (t) => ... ที่ส่งเข้าไปในเมธอดเหล่านี้คือ arrow function ที่จะถูกเรียกกับสมาชิกแต่ละตัว t คือ "รายการที่กำลังดูอยู่"

ตัวอย่างจริง: ยอดคงเหลือคำนวณยัง (FR-2)

src/domain/summary.ts

บรรทัด 9-17

```

9 export const confirmedOnly = (txs: Transaction[]) => txs.filter((t) => t.status === 'confirmed');
10
11 /** Balance now = money at start + all confirmed income - all confirmed expense. */
12 export function computeBalance(openingBalanceSatang: number, txs: Transaction[]): number {
13   return confirmedOnly(txs).reduce(
14     (sum, t) => sum + (t.kind === 'income' ? t.amountSatang : -t.amountSatang),
15     openingBalanceSatang,
16   );
17 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/summary.ts:9

บรรทัด	ความหมาย
9	สร้างฟังก์ชัน confirmedOnly: รับรายการทั้งหมด คัดเฉพาะรายการที่ยืนยันแล้ว (กรองแบบร่างทิ้ง) ด้วย filter
11	คอมเมนต์อธิบายสูตร: ยอดตอนนี้ = เงินตอนเริ่ม + รายรับที่ยืนยัน - รายจ่ายที่ยืนยัน
12	ฟังก์ชัน computeBalance รับ 2 อย่าง: เงินตอนเริ่ม (สตางค์) และรายการทั้งหมด คำนวณเป็นตัวเลข
13	เอาเฉพาะรายการที่ยืนยันแล้ว แล้ว reduce รวมให้เหลือตัวเลขเดียว

บรรทัด	ความหมาย
14	สำหรับแต่ละรายการ t: ถ้าเป็นรายรับ บวกเข้า (+t.amountSatang) ถ้าเป็นรายจ่าย ลบออก (-t.amountSatang) แล้วเก็บผลไว้ใน sum
15	ค่าเริ่มต้นของ sum คือเงินตอนเริ่มใช้แอป

ลองคิดตาม: เริ่ม ฿3,000 (300,000 สตางค์) มีรายรับ ฿10,000 ที่ยืนยันแล้ว และรายจ่าย ฿85 กับแบบร่าง ฿200

```
sum เริ่ม          = 300,000
+ รายรับ 1,000,000 = 1,300,000
- รายจ่าย 8,500    = 1,291,500
(แบบร่าง 20,000 ถูกกรองทิ้งตั้งแต่บรรทัด 13)
ผล = 1,291,500 สตางค์ = ฿12,915.00
```

7. Object: ของ 1 ชั้นที่มีหลายช่อง

Object คือกลุ่มของ "ชื่อช่อง: ค่า" อยู่ใน { } รายการเงิน 1 รายการใน MindPay คือ object แบบนี้

รายการ 1 รายการ (Transaction)

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 const tx = {
2   id: 'demo-12',
3   kind: 'expense',
4   amountSatang: 8500,
5   categoryKey: 'food',
6   title: 'ข้าวมันไก่',
7   status: 'confirmed',
8 };
9 tx.title;           // 'ข้าวมันไก่' (เข้าถึงช่องด้วยจุด)
10 tx.amountSatang;   // 8500
```

เทคนิค object ที่เจอบ่อยในโค้ด

เขียนว่า	ชื่อเรียก	ความหมาย
<code>const { a, b } = obj;</code>	destructuring (แกะของ)	ดึงช่อง a และ b ออกมาเป็นตัวแปรในบรรทัดเดียว เช่น <code>const { balance, runway } = useMoney();</code>
<code>{ ...obj, status: 'confirmed' }</code>	spread (กระจาย)	คัดลอกทุกช่องของ obj มาเป็น object ใหม่ แล้วเปลี่ยนเฉพาะช่อง status ของเดิมไม่ถูกแก้
<code>[tx, ...list]</code>	spread กับ array	array ใหม่ที่มี tx อยู่หน้าสุด ตามด้วยของเดิมทั้งหมด (ใช้ตอนเพิ่มรายการใหม่ในบท A2)
<code>{ kind, title }</code>	shorthand	ย่อจาก <code>{ kind: kind, title: title }</code>

! ห้ามโค้ดซ่อมสร้าง object ใหม่แทนการแก้ไขเดิม

React (บท A6) รู้ว่าข้อมูลเปลี่ยนด้วยการดูว่า "เป็น object ตัวใหม่หรือเปล่า" ถ้าเราแก้ไขใน object เดิมตรง ๆ React อาจไม่รู้ และหน้าจอก็ไม่อัปเดต โค้ดจึงใช้ { ...t, status: 'confirmed' } สร้างของใหม่เสมอ (ดูตัวอย่างใน [src/data/AppProvider.tsx:418](#))

8. วงซ้ำ (loop)

for ... of

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 let total = 0;
2 for (const t of txs) {           // ทำซ้ำกับรายการทีละตัว
3   if (t.kind !== 'expense') continue; // continue = ข้ามตัวนี้ ไปตัวถัดไป
4   total += t.amountSatang;       // += แปลว่า total = total + ...
5 }
```

ส่วน for (let i = 0; i < 20; i++) คือวนแบบนับรอบ: เริ่ม i = 0 ทำซ้ำตราบที่ i < 20 และเพิ่ม i ทีละ 1 (i++)
ส่วน break คือหยุดวนทันที

9. ข้อความ (string) ที่ใช้บ่อย

เขียนว่า	ผล
`เงินพอใช้อีก \${days} วัน`	template string (ใช้ backtick) แทรกค่าตัวแปรใน \${ } ได้ เช่น "เงินพอใช้อีก 16 วัน"
'2026-10-09'.slice(0, 7)	'2026-10' (ตัดตัวอักษรตำแหน่ง 0 ถึงก่อน 7)
'a,b'.split(',')	['a', 'b'] (แยกเป็น array)
' ข้าว '.trim()	'ข้าว' (ตัดช่องว่างหัวท้าย)
'KFC'.toLowerCase()	'kfc'
String(5).padStart(2, '0')	'05' (เติม 0 ข้างหน้าให้ครบ 2 ตัว ใช้กับวันที่)
s.replace(/บาง/g, '')	ลบคำว่า "บาง" ทุกที่ (/.../g คือ regular expression ดูบท A5)

10. คณิตศาสตร์ที่ใช้

คำสั่ง	ผล	ใช้ที่ไหน
Math.floor(20.69)	20 (ปัดลงเสมอ)	จำนวนวัน Money Runway
Math.round(680.2857)	680 (ปัดตามปกติ)	ค่าเฉลี่ยต่อวันเป็นสตางค์
Math.max(0, -500)	0 (ตัวที่มากกว่า)	กันเงินที่ใช้ได้ติดลบ
Math.min(7, 3)	3 (ตัวที่น้อยกว่า)	จำนวนวันที่ใช้หารค่าเฉลี่ย ไม่เกิน 7
Math.abs(-85)	85 (ค่าสัมบูรณ์)	แสดงยอดเงินโดยไม่สนเครื่องหมาย

แบบฝึกหัด A4

1. ถ้า `profile` เป็น `null` ค่าของ `profile?.runwayFloorSatang ?? 50_000` คืออะไร
2. `[100, 200, 300].filter((x) => x > 150).map((x) => x / 100)` ได้อะไร
3. เริ่ม ฿500 มีรายการยืนยันแล้ว: รายรับ ฿1,000 รายจ่าย ฿300 และแบบร่างรายจ่าย ฿200 `computeBalance` คืนค่าสตางค์
4. `const t2 = { ...t, title: 'ข้าวผัด' }` แล้ว `t.title` เปลี่ยนไหม

i เฉลย

- 1) 50000 · 2) `[2, 3]` (กรองเหลือ 200, 300 แล้วหาร 100) · 3) $50,000 + 100,000 - 30,000 = 120,000$ สตางค์ (แบบร่างไม่นับ) · 4) ไม่เปลี่ยน `t2` เป็น object ใหม่ `t` ยังเป็นค่าเดิม

TypeScript: ภาษาที่ทีมเราใช้ และชนิดข้อมูลของ MindPay

TypeScript คือ JavaScript ที่บังคับให้บอกว่าข้อมูลแต่ละตัวเป็นชนิดอะไร ทำให้จับผิดได้ตั้งแต่ตอนพิมพ์ ก่อนผู้ใช้จะเจอ

ทำไมต้อง TypeScript

ใน JavaScript ธรรมดา ถ้าพิมพ์ `tx.amountSatnag` (สะกดผิด) โปรแกรมจะรันได้ แต่ได้ค่า `undefined` แล้วยอดเงินเพี้ยนตอนผู้ใช้งานใช้ ส่วน TypeScript จะขีดเส้นแดงใน VS Code กันที่ว่า "ไม่มีช่องชื่อนี้" ทีมเราเปิดโหมดเข้มงวดที่สุด ("`strict`": `true` ในไฟล์ `tsconfig.json`) และรัน `npm run typecheck` ก่อน build APK ทุกครั้ง

≈ เปรียบเทียบ

TypeScript เหมือนแบบฟอร์มราชการที่มีช่องบังคับว่า "ช่องนี้ต้องเป็นตัวเลข 13 หลัก" กรอกผิดเจ้าหน้าที่ตักกลับทันทีที่โต๊ะ ตักว่าไปรู้ตอนเอกสารเข้าระบบแล้ว เมื่อ build แอป ชนิดข้อมูลทั้งหมดจะถูกลบออก เหลือ JavaScript ธรรมดา ชนิดข้อมูลจึงช่วยเราตอนเขียนโค้ดเท่านั้น ไม่ทำให้แอปช้าลง

การบอกชนิดข้อมูล (type annotation)

บอกชนิดด้วย :

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 const days: number = 16;
2 const title: string = 'ข้าวมันไก่';
3 let slipRef: string | null = null; // เป็น string หรือ null ก็ได้
4 function toBaht(satang: number): number { return satang / 100; }
```

หลายครั้งไม่ต้องเขียนชนิดเอง TypeScript เดาเองได้ (type inference) เช่น `const days = 16` รู้เองว่าเป็น number เอาเมาส์ชี้ใน VS Code จะเห็นชนิดที่มันเดา

ไฟล์ชนิดข้อมูลกลาง: `src/domain/types.ts`

ไฟล์นี้สำคัญที่สุดไฟล์หนึ่ง เพราะกำหนดว่า "รายการเงิน" และ "โปรไฟล์ผู้ใช้" หน้าตาเป็นยังไง ทุก FR ใช้ร่วมกัน

`src/domain/types.ts`

บรรทัด 1-55

```
1 /**
2  * Core data types for MindPay.
3  *
4  * Money is always stored as an integer number of satang (1 baht = 100 satang)
5  * so that adding many amounts never produces floating-point errors
6  * (0.1 + 0.2 !== 0.3 in JavaScript).
7  */
8
9 export type TxKind = 'income' | 'expense';
10
```

```

11 /** draft = not counted in any total until the user confirms it (Charter: slip results stay drafts). */
12 export type TxStatus = 'draft' | 'confirmed';
13
14 export type TxSource = 'manual' | 'slip';
15
16 export interface Transaction {
17   id: string;
18   kind: TxKind;
19   amountSatang: number;
20   categoryKey: string;
21   title: string;
22   note: string | null;
23   /** ISO 8601 instant, e.g. "2026-09-27T01:15:00.000Z" */
24   occurredAt: string;
25   source: TxSource;
26   status: TxStatus;
27   /** Transaction reference read from the slip (QR or text). Used for duplicate detection. */
28   slipRef: string | null;
29   /** SHA-256 of the processed slip image. Second line of duplicate detection. */
30   slipImageHash: string | null;
31   /** Lowest field confidence (0..1) reported by the slip reader, null for manual entries. */
32   ocrConfidence: number | null;
33   /** Fields the slip reader was unsure about, shown to the user during review. */
34   reviewFlags: string[];
35   createdAt: string;
36 }
37
38 /** Everything the user can type or change in the form. */
39 export type TransactionInput = Omit<Transaction, 'id' | 'createdAt'>;
40
41 export type CoachTone = 'friend' | 'coach' | 'senior';
42
43 export interface Profile {
44   id: string;
45   displayName: string;
46   /** Money the user had when they started using MindPay. Balance = opening + income - expense. */
47   openingBalanceSatang: number;
48   /** FR-6: the "low" line. Runway counts the days until balance would reach this amount. */
49   runwayFloorSatang: number;
50   monthlyBudgetSatang: number | null;
51   coachTone: CoachTone;
52   onboarded: boolean;
53 }
54
55 export type RangeKey = 'today' | '7d' | '1m' | '6m' | '1y';

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/types.ts:1

บรรทัด ความหมาย

- 1-7** คอมเมนต์อธิบายกฎสำคัญ: เงินเก็บเป็นจำนวนเต็มสตางค์เสมอ (1 บาท = 100 สตางค์) เพราะใน JavaScript 0.1 + 0.2 ไม่เท่ากับ 0.3
- 9** **union type:** ชนิด TxKind เป็นได้แค่ 2 ค่า คือข้อความ 'income' (รายรับ) หรือ 'expense' (รายจ่าย) เครื่องหมาย | แปลว่า "หรือ" ถ้าพิมพ์ 'incom' จะขึ้นเส้นแดงทันที

บรรทัด	ความหมาย
11–12	สถานะของรายการ: 'draft' (แบบร่าง ยังไม่นับ) หรือ 'confirmed' (ยืนยันแล้ว นับในยอด) คอมเมนต์อ้างอิงถึง Charter ของทีมว่าผลจากสลิปเป็นแบบร่างก่อน
14	รายการมาจากไหน: จดเอง ('manual') หรือจากสลิป ('slip') รายการที่พุดจด (FR-3) นับเป็น manual
16	interface = พิมพ์เขียวของ object บอกว่า "รายการ 1 รายการ ต้องมีช่องอะไรบ้าง และแต่ละช่องชนิดอะไร"
17	id สหสัมพันธ์ของรายการ (ฐานข้อมูลสร้างให้)
18	kind ใช้ชนิด TxKind จากบรรทัด 9
19	amountSatang จำนวนเงินเป็นสตางค์ (บวกเสมอ ส่วนเข้าหรือออกดูจาก kind)
20	categoryKey สหสัมพันธ์ เช่น 'food', 'transport' (รายชื่อหมวดอยู่ใน src/domain/categories.ts)
21–22	ชื่อรายการ และบันทึกเพิ่มเติม (string null = ไม่มีบันทึกก็ได้)
23–24	เวลาที่เกิดรายการ เก็บแบบสากล ISO 8601 เช่น "2026-09-27T01:15:00.000Z" (Z = เวลา UTC ซึ่งช้ากว่าไทย 7 ชั่วโมง ตัวอย่างนี้คือ 08:15 น. เวลาไทย)
25–26	ที่มาและสถานะ ใช้ชนิดจากบรรทัด 14 และ 12
27–30	เลขอ้างอิงจากสลิป และ "ลายนิ้วมือ" ของรูป (SHA-256) ใช้กันสลิปซ้ำ 2 ชั้น (FR-4)
31–32	ความมั่นใจต่ำสุดที่ AI อ่านได้ (0 ถึง 1) รายการที่จดเองเป็น null
33–34	รายชื่อช่องที่ AI ไม่แน่ใจ เช่น ['amount'] ให้นำมาตรวจสอบสลิปไฮไลต์ช่องนั้น string[] = array ของข้อความ
35	เวลาที่สร้างแถวในระบบ
38–39	TransactionInput = Transaction ที่ ไม่มี id และ createdAt (omit = ตัดช่องออก) ใช้ตอนสร้างรายการใหม่ เพราะสองช่องนี้ฐานข้อมูลเป็นคนสร้าง
41	โทโคชิ 3 แบบเดิม (ไม่ได้ใช้แล้ว เพราะ 29 ก.ย. เปลี่ยนเป็นน้องกล้าเสียงเดียว แต่ยังเก็บชนิดไว้ให้ตรงกับคอลัมน์ในฐานข้อมูล)
43–53	Profile ข้อมูลผู้ใช้ 1 คน: ชื่อ, เงินตอนเริ่มใช้แอป (บรรทัด 47), เงินสำรองของ FR-6 (บรรทัด 49), จบต่อเดือน (ไม่ตั้งก็ได้ = null), โทโคชิ และตั้งค่าครั้งแรกเสร็จหรือยัง
55	ช่วงเวลาที่เลือกดูได้ในหน้าหลัก: วันนี้, 7 วัน, 1 เดือน, 6 เดือน, 1 ปี (FR-2)

ชนิดข้อมูลแบบพิเศษที่เจอบ่อย

เขียนว่า	ความหมาย	ตัวอย่างใน MindPay
A B	เป็น A หรือ B (union)	'draft' 'confirmed'
ชื่อ?: ชนิด	ช่องนี้ไม่ใส่ก็ได้ (optional)	now?: Date
ชนิด[]	array ของชนิดนั้น	Transaction[]
Omit<T, 'a'>	T ที่ตัดช่อง a ออก	Omit<Transaction, 'id' 'createdAt'>

เขียนว่า	ความหมาย	ตัวอย่างใน MindPay
Partial<T>	T ที่ทุกช่องไม่บังคับ ใช้ตอน "แก้บางช่อง"	updateTx(id, patch: Partial<TransactionInput>)
Pick<T, 'a' 'b'>	T ที่เอาเฉพาะช่อง a และ b	Pick<TxRow, 'kind' 'amount_satang'>
Record<K, V>	object ที่ชื่อช่องเป็นชนิด K ค่าเป็นชนิด V	Record<string, unknown>
Promise<T>	"สัญญาว่าจะได้ T ในอนาคต" (งานที่ต้องรอ)	getProfile(): Promise<Profile>
type X = ...	ตั้งชื่อเล่นให้ชนิด	type TxKind = 'income' 'expense'
x as Y	บอก TypeScript ว่า "ชื่อฉัน x เป็นชนิด Y" (ใช้น้อย ๆ)	data as TxRow[]
'confirmed' as const	ให้ถือว่าเป็นค่าตายตัว ไม่ใช่ string ทั่วไป	status: 'confirmed' as const
unknown	ยังไม่รู้ชนิด ต้องตรวจสอบก่อนใช้ (ปลอดภัยกว่า any)	context: unknown

import และ export: แอร์โด้ระหว่างไฟล์

src/data/AppProvider.tsx

บรรทัด 6-17

```

6 import { Storage } from './storage';
7 import { createContext, useCallback, useContext, useEffect, useMemo, useRef, useState, type
  ReactNode } from 'react';
8 import { AppState } from 'react-native';
9 import { authErrorMessage, authLinkErrorMessage, parseAuthLink } from '../domain/auth';
10 import { bkkDayKey, msUntilNextBkkMidnight } from '../domain/dates';
11 import { applyDeposit, reservedSatang, type GoalInput, type SavingsGoal } from '../domain/goals';
12 import { averageDailyExpense, computeRunway } from '../domain/runway';
13 import { computeBalance } from '../domain/summary';
14 import type { Profile, Transaction, TransactionInput } from '../domain/types';
15 import { createCloudRepo, createDemoRepo, resetDemoData, type Repo } from './repo';
16 import { clearLinkFromAddressBar, onIncomingLink, openingLink } from './authLinks';
17 import { supabase } from './supabase';

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/AppProvider.tsx:6

บรรทัด	ความหมาย
6	นำ Storage มาจากไฟล์ ./storage (จุด = โฟลเดอร์เดียวกัน ไม่ต้องใส่نامสกุลไฟล์)
7	นำเครื่องมือของ React มาใช้ ชื่อที่ไม่มีจุดนำหน้า เช่น 'react' คือแพ็คเกจใน node_modules
9-13	../domain/... = ถอยออกไป 1 โฟลเดอร์ (จาก data ไป src) แล้วเข้า domain นำฟังก์ชันคำนวณมาใช้
14	import type = นำมาเฉพาะชนิดข้อมูล (หายไปตอน build ไม่เพิ่มขนาดแอป)

11 `type GoalInput` ในวงเล็บปีกกา = import ทั้งฟังก์ชันและชนิดจากไฟล์เดียวกันในบรรทัดเดียว

ฝั่งไฟล์ต้นทางต้องเขียน `export` หน้าสิ่งที่ยอมให้คนอื่นใช้ ส่วน `export default` ใช้กับ "ของหลัก" ของไฟล์ เช่นหน้าจอแต่ละหน้า (Expo Router หาหน้าจออกจาก default export)

งานที่ต้องรอ: `async`, `await` และ `Promise`

การคุยกับอินเทอร์เน็ตใช้เวลา (อาจครึ่งวินาทีหรือหลายวินาที) ถ้าแอปหยุดรอเฉย ๆ หน้าจอจะค้าง JavaScript จึงใช้ **Promise** คือ "ใบรับของ" ที่บอกว่าเดี๋ยวผลจะมา

async / await

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 async function save() {           // async = ฟังก์ชันนี้มีงานที่ต้องรอ
2   const tx = await repo.insert(input); // await = รอจนจับได้ผล แต่หน้าจอไม่ค้าง
3   setTxs((list) => [tx, ...list]); // ทำต่อเมื่อได้ผลแล้ว
4 }
```

ถ้ามีหลายงานที่ไม่ต้องรอกัน ใช้ `Promise.all` ทำพร้อมกัน เช่น ตอนโหลดข้อมูลครั้งแรก แอปโหลดโปรไฟล์ รายการ และกระปุกออม พร้อมกันทั้ง 3 อย่าง

src/data/AppProvider.tsx

บรรทัด 131–133

```
131   try {
132     // Goals never block the app: if they cannot be loaded, the rest still works.
133     const [p, list, g] = await Promise.all([r.getProfile(), r.listTransactions(),
      r.listGoals().catch(() => [] as SavingsGoal[])]);
```

▶ เปิดใน VS Code (เว็บ) GitHub

VS Code ในเครื่อง: Ctrl+P → `src/data/AppProvider.tsx:131`

บรรทัด 133: เรียก 3 งานพร้อมกัน แล้วรอจนเสร็จทั้งหมด ผลออกมาเป็น array 3 ช่อง แะใส่ตัวแปร `p`, `list`, `g` ส่วน `.catch(() => [])` ถ้างานที่ 3 แปลว่า "ถ้าโหลดกระปุกออมไม่ได้ ให้ถือว่าไม่มีกระปุก" แอปส่วนอื่นยังใช้ได้ (Supabase ฝั่งเซิร์ฟเวอร์ ใน Edge Function ก็ใช้ `Promise.allSettled` อ่านสลิป 2 ครั้งพร้อมกันแบบเดียวกัน un A10)

จัดการความผิดพลาด: `try`, `catch`, `throw`

try / catch / finally

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 try {
2   await addTx(input);           // ลองทำ
3   toast({ message: 'บันทึกรายการแล้ว' });
4 } catch (e) {                   // ถ้ามี error ระหว่างทาง กระโดดมาที่นี่
5   toast({ message: 'บันทึกไม่สำเร็จ ลองอีกครั้ง', tone: 'error' });
6 } finally {                    // ทำเสมอ ไม่ว่าสำเร็จหรือพลาด
7   setBusy(false);
8 }
```

`throw` คือ "โยน error" ออกไปให้ผู้เรียกจัดการ ทีมเราสร้าง error ชนิดของตัวเองเพื่อแยกเหตุผล เช่น `DuplicateSlipError` (สลิปซ้ำ) ใน [src/data/repo.ts:14](#)

```

14 export class DuplicateSlipError extends Error {
15   constructor() {
16     super('รายการนี้มีอยู่แล้ว');
17     this.name = 'DuplicateSlipError';
18   }
19 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/repo.ts:14

class ... extends Error = สร้างชนิด error ใหม่โดยต่อยอดจาก Error ธรรมดา super(...) ส่งข้อความภาษาไทยให้ Error ตัวแม่ และตั้ง name ไว้ให้หน้าจอลูกได้ว่าเป็น error แบบไหน (ดู [src/app/transaction.tsx:126](#))

ทำไมเก็บเงินเป็นสตางค์ (จำนวนเต็ม)

คอมพิวเตอร์เก็บทศนิยมเป็นเลขฐานสอง ซึ่งเก็บบางค่าได้ไม่เป๊ะ ทดลองรันใน Node.js จริง:

```

0.1 + 0.2      → 0.30000000000000004
19.99 * 100    → 1998.9999999999998

```

ถ้าบวกเงินเป็นบาททศนิยมหลายพันรายการ เศษเล็ก ๆ จะสะสมจนยอดเพี้ยน ทางแก้คือเก็บเป็น **สตางค์ (จำนวนเต็ม)** ทั้งหมด เช่น ฿85.50 เก็บเป็น 8550 จำนวนเต็มบวกหลบกันได้แม่นยำ 100% แล้วค่อยแปลงเป็นบาทตอนแสดงผลบนจอ ฐานข้อมูลก็เก็บเป็นจำนวนเต็ม (คอลัมน์ amount_satang bigint)

อ่านโค้ดจริง: แปลงข้อความที่ผู้ใช้พิมพ์เป็นสตางค์

ฟังก์ชัน parseBahtToSatang ใน [src/domain/money.ts](#) ใช้ทุกที่ที่ผู้ใช้กรอกจำนวนเงิน (FR-1 จดรายการ, ตั้งค่า, เช็กรหัสจ่ายของ FR-6, กระบุยอด) และใช้กับยอดเงินที่ AI อ่านจากสลิปด้วย

```

1 /**
2  * Money helpers. All amounts are integers in satang.
3  */
4
5 /** The largest amount the app takes: ฿1,000,000,000 (the database caps goals at the same value).
6  */
7 export const MAX_SATANG = 100_000_000_000;
8
9 /** "๑๒๓" -> "123": Thai digits typed on a Thai keyboard. */
10 export function toAsciiDigits(s: string): string {
11   return s.replace(/[\u0E50-\u0E59]/g, (d) => String(d.charCodeAt(0) - 0x0e50));
12 }
13
14 /**
15  * Parse what a user (or a slip) wrote into satang.
16  * Accepts "1,234.50", "1234.5", "฿85", "85 บาท", " 1 234.00 ", Thai digits "๘๕", and a decimal
17  * comma as typed on phones set to such regions ("85,50" = 85.50, "12,34" = 12.34; never 1,234).
18  * Other commas must group thousands ("1,234.50"); anything else ("1,23,4") is refused.
19  * Returns null for anything that is not a positive amount with at most 2 decimals, or above
20  * MAX_SATANG. With `allowZero`, "0" and "0.00" give 0 (an opening balance may be zero).

```

```

21 export function parseBahtToSatang(input: string | number | null | undefined, opts: { allowZero?:
    boolean } = {}): number | null {
22   if (input === null || input === undefined) return null;
23   const ok = (satang: number) => (satang > MAX_SATANG ? null : satang > 0 || (opts.allowZero &&
    satang === 0) ? satang : null);
24   if (typeof input === 'number') {
25     if (!Number.isFinite(input) || input < 0) return null;
26     return ok(Math.round(input * 100));
27   }
28   let cleaned = toAsciiDigits(input)
29     .replace(/฿|บาท|THB|baht/gi, '')
30     .replace(/,/g, '');
31   if (/^\d+,\d{1,2}$/.test(cleaned)) cleaned = cleaned.replace(',', '.');
32   else if (cleaned.includes(',')) {
33     if (!/^\d{1,3}(\,\d{3})+(\.\d{1,2})?$/.test(cleaned)) return null;
34     cleaned = cleaned.replace(/,/g, '');
35   }
36   if (!/^\d+(\.\d{0,2})?$/.test(cleaned)) return null;
37   const [whole, frac = ''] = cleaned.split('.');
38   if (whole.length > 12) return null;
39   return ok(Number(whole) * 100 + Number((frac + '00').slice(0, 2)));
40 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/domain/money.ts:1`

บรรทัด	ความหมาย
5–6	ค่าคงที่ MAX_SATANG = 100,000,000,000 สตางค์ = ฿1,000,000,000 เป็นจำนวนเงินสูงสุดที่แอปรับ (ฐานข้อมูลตั้งเพดานเดียวกัน กับค่าที่อ่านผิดใหญ่เกินจริง)
8–11	toAsciiDigits แปลงเลขไทย "๑๒๓" เป็น "123" คนที่ใช้แป้นพิมพ์ไทยอาจพิมพ์เลขไทยมา ทำงานโดยหาตัวอักษรในช่วงเลขไทย (รหัส U+0E50 ถึง U+0E59) แล้วลบด้วยรหัสของ "๐" (0x0e50) จะได้ 0–9
13–20	คอมเมนต์บอกว่ารับรูปแบบไหนได้บ้าง: "1,234.50", "1234.5", "฿85", "85 บาท", เลขไทย และทศนิยมแบบคอมมาที่มีมือถือบางประเทศใช้ ("85,50" = 85.50) แต่ "1,23,4" ไม่รับ
21	รับค่าเป็นข้อความหรือตัวเลขก็ได้ และมีตัวเลือก allowZero (ปกติ 0 ไม่ผ่าน เพราะรายการเงิน 0 บาทไม่มีความหมาย แต่ยอดเงินตั้งต้นเป็น 0 ได้) คืนค่าเป็นสตางค์ หรือ null ถ้าไม่ใช่จำนวนเงินที่ถูกต้อง
22	ไม่มีค่าเลย คืน null
23	ฟังก์ชันย่อย ok ตรวจสอบสุดท้าย: เกินเพดานไม่ผ่าน, มากกว่า 0 ผ่าน, เท่ากับ 0 ผ่านเฉพาะเมื่อ allowZero
24–27	ถ้าส่งเข้ามาเป็นตัวเลขอยู่แล้ว ต้องเป็นตัวเลขจริงและไม่ติดลบ แล้วคูณ 100 ปัดเป็นจำนวนเต็ม (ปัดเพื่อแก้ปัญหา $19.99 \times 100 = 1998.999\dots$)
28–30	ถ้าเป็นข้อความ: แปลงเลขไทย ลบสัญลักษณ์ ฿, คำว่า "บาท", THB, baht (ไม่สนตัวพิมพ์เล็กใหญ่) และลบช่องว่างทั้งหมด
31	ถ้ารูปแบบคือ "ตัวเลข,ตัวเลข 1–2 หลัก" เช่น "85,50" ถือว่าคอมมาคือจุดทศนิยม
32–35	ถ้ามีคอมมาแบบอื่น ต้องเป็นคอมมาคั่นหลักพันที่ถูกต้อง (เช่น 1,234 หรือ 1,234.50) ถึงจะลบคอมมาออก ไม่จิ้นคืน null
36	สุดท้ายต้องเหลือแค่ตัวเลข กับทศนิยมไม่เกิน 2 ตำแหน่ง ไม่จิ้นคืน null (เช่น "85.555" หรือ "abc")
37	แยกส่วนจำนวนเต็มกับทศนิยมที่จุด ถ้าไม่มีทศนิยมให้ frac เป็นข้อความว่าง

บรรทัด	ความหมาย
38	เกิน 12 หลักไม่รับ (กันตัวเลขยาวผิดปกติ)
39	คำนวณสตางค์ด้วยจำนวนเต็มล้วน: บาท × 100 + สตางค์ (เติม "00" แล้วตัด 2 ตัว ทำให้ ".5" กลายเป็น 50 สตางค์) ไม่มีการคูณทศนิยมเลย จึงไม่เพี้ยน

ผลจากการรับฟังค์ชันจริง

ผู้ใช้พิมพ์	ได้สตางค์	เหตุผล
85	8500	-
85.5	8550	".5" → 50 สตางค์
1,234.50	123450	คอมมาคั่นหลักพันถูกต้อง
฿85 / 85 บาท / ๘๕	8500	ลบสัญลักษณ์และแปลงเลขไทย
85,50	8550	คอมมาเป็นทศนิยม
1,23,4	null	คอมมาผิดรูปแบบ
0	null	ต้องมากกว่า 0 (เว้นแต่ allowZero)
85.555	null	ทศนิยมเกิน 2 ตำแหน่ง
-5	null	มีเครื่องหมายลบ ไม่ใช่ตัวเลขล้วน
1000000000.01	null	เกินเพดาน ฿1,000,000,000

Regular expression (regex) อ่านยังงี้

บรรทัด 10 และ 29–36 มีของแปลก ๆ ใน / / เรียกว่า **regular expression** คือ "แพตเทิร์นสำหรับค้นหาข้อความ" ไม่ต้องจำทั้งหมด แค่อ่านออก

สัญลักษณ์	แปลว่า
\d	ตัวเลข 0–9 หนึ่งตัว
\s	ช่องว่าง (space, tab, ขึ้นบรรทัด)
+	ตัวข้างหน้ามี 1 ตัวขึ้นไป (\d+ = ตัวเลขที่หลักก็ได้)
{1,3}	ตัวข้างหน้ามี 1 ถึง 3 ตัว
?	ตัวข้างหน้ามีหรือไม่มีก็ได้
^ และ \$	ต้นข้อความ และท้ายข้อความ (ต้องตรงทั้งข้อความ ไม่ใช่แค่บางส่วน)
()	จับกลุ่ม
a b	a หรือ b
[๐-๙]	ตัวอักษร 1 ตัวในช่วงเลขไทย ๐–๙

สัญลักษณ์	แปลว่า
g, i (ท้าย /)	g = หากจุด, i = ไม่สนตัวพิมพ์เล็กใหญ่

ตัวอย่าง `/^\d{1,3}(\,\d{3})+(\.\d{1,2})?$/` อ่านว่า: "ขึ้นต้นด้วยตัวเลข 1-3 หลัก ตามด้วย (คอมมา + ตัวเลข 3 หลัก) อย่างน้อย 1 ชุด แล้วอาจมี (จุด + ตัวเลข 1-2 หลัก) และจบข้อความ" ซึ่งก็คือรูปแบบ 1,234 หรือ 12,345,678.50

แปลงสตริงกลับเป็นข้อความบาท

src/domain/money.ts

บรรทัด 42-63

```

42 /** 123450 -> "1,234.50" */
43 export function formatSatang(satang: number, opts: { decimals?: boolean } = {}): string {
44   const decimals = opts.decimals ?? true;
45   const negative = satang < 0;
46   const abs = Math.abs(Math.round(satang));
47   // Whole baht only: round to the nearest baht (#99.99 reads #100, not #99).
48   const whole = decimals ? Math.floor(abs / 100) : Math.round(abs / 100);
49   const frac = abs % 100;
50   const grouped = String(whole).replace(/\B(?=(\d{3})+(?!\d))/g, ',');
51   const body = decimals ? `${grouped}.${String(frac).padStart(2, '0')}` : grouped;
52   return negative ? `-${body}` : body;
53 }
54
55 /** 123450 -> "฿1,234.50"; with sign: "+฿1,234.50" / "-฿1,234.50" */
56 export function formatBaht(
57   satang: number,
58   opts: { decimals?: boolean; sign?: boolean } = {},
59 ): string {
60   const body = formatSatang(Math.abs(satang), opts);
61   if (opts.sign) return `${satang < 0 ? '-' : '+'}฿${body}`;
62   return `${satang < 0 ? '-' : ''}฿${body}`;
63 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/money.ts:42

บรรทัด	ความหมาย
42-43	formatSatang แปลง 123450 เป็น "1,234.50"
44	ตัวเลือก decimals ถ้าไม่ส่งมาใช้ค่าเริ่มต้น true (แสดงสตางค์)
45-46	จำไว้ว่าติดลบไหม แล้วทำงานกับค่าบวกที่ปิดเป็นจำนวนเต็มแล้ว
47-48	ส่วนบาท: ถ้าแสดงสตางค์ด้วย ใช้ปัดลง (สตางค์จะแสดงแยก) ถ้าไม่แสดงสตางค์ ปิดตามปกติ #99.99 จึงแสดงเป็น #100 ไม่ใช่ #99
49	สตางค์คือเศษจากการหาร 100
50	ใส่คอมมาค้นหลักพันด้วย regex (หาตำแหน่งที่ด้านหลัมีตัวเลขเป็นชุดละ 3 ตัวพอดี)
51-52	ต่อสตางค์ 2 หลัก (5 สตางค์ → "05") และเติมเครื่องหมายลบถ้าติดลบ
55-63	formatBaht เพิ่ม "฿" ข้างหน้า และถ้าขอเครื่องหมาย (sign) จะใส่ + หรือ - เช่น formatBaht(-8500, {sign: true}) → "-฿85.00"

i ฅลจรง

```
formatBaht(123450) → "฿1,234.50" · formatBaht(-8500, { sign: true }) → "-฿85.00" ·  
formatBaht(9999, { decimals: false }) → "฿100"
```

React: วาดหน้าจจากข้อมูล

React คือวิธีคิดที่ว่า "หน้าจอ = ฟังก์ชันของข้อมูล" ข้อมูลเปลี่ยนเมื่อไร หน้าจอวาดใหม่เอง ไม่ต้องสั่งแก้ทีละจุด

แนวคิดหลัก: หน้าจอเป็นผลลัพธ์ของข้อมูล

สมัยก่อน ถ้ายอดเงินเปลี่ยน โปรแกรมเมอร์ต้องเขียนเองว่า "ไปหาข้อความยอดเงิน แก่ตัวเลข ไปหาต้นไม้ เปลี่ยนจำนวนใบ ไปหาป้ายสถานะ เปลี่ยนสี..." พลาดจุดเดียวหน้าจอก็ขัดกันเอง React เปลี่ยนวิธีคิด: เราเขียนแค่ว่า "ถ้าข้อมูลเป็นแบบนี้ หน้าจอหน้าตาแบบนี้" แล้ว React จะวาดใหม่ให้เองทุกครั้งที่ข้อมูลเปลี่ยน

ข้อมูล (state)

เช่น รายการทั้งหมด txs, ยอดคงเหลือ

↓ React เรียก component ใหม่

Component (ฟังก์ชันวาดหน้าจอ)

อ่านข้อมูล แล้วคืน "หน้าตา" เป็น JSX

↓ React เทียบกับของเดิม แก้เฉพาะจุดที่ต่าง

หน้าจอที่ผู้ใช้เห็น

Component: ชิ้นส่วนหน้าจอที่เป็นฟังก์ชัน

Component คือฟังก์ชันที่ชื่อขึ้นต้นด้วยตัวใหญ่ และคืนค่าเป็น **JSX** (หน้าตาที่เขียนคล้าย HTML อยู่ในโค้ด) หน้าจอทั้งหน้าก็เป็น component และปุ่มเล็ก ๆ ก็เป็น component ซ้อนกันเป็นชั้น ๆ

Component เล็กที่สุด

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 function Hello({ name }: { name: string }) {
2   return <Text>สวัสดี {name}</Text>;
3 }
4
5 // เรียกใช้ที่อื่น:
6 <Hello name="มินท์" />    // แสดง "สวัสดี มินท์"
```

ส่วน	ความหมาย
<Text>...</Text>	JSX แท็กเปิดและปิด เหมือน HTML (Text คือชิ้นส่วนแสดงข้อความของ React Native)
{name}	วงเล็บปีกกาใน JSX = "ใส่ค่าจากโค้ด JavaScript ตรงนี้"
{ name }: { name: string }	props คือค่าที่ผู้เรียกส่งเข้ามา (เหมือนพารามิเตอร์) ในที่นี้แกะเอาช่อง name ออกมา
<Hello name="มินท์" />	ใช้ component เหมือนแท็ก ส่ง props เป็น attribute แท็กที่ไม่มีของข้างในปิดด้วย />

ตัวอย่างจริง: T ชิ้นส่วนตัวหนังสือที่ทุกหน้าใช้

ทุกข้อความในแอปใช้ <T> แทนการตั้งขนาดและสีเอง เพื่อให้ทั้งแอปหน้าตาเหมือนกัน (กฎใน Design System un A11)

```

36 type Variant = 'display' | 'h1' | 'h2' | 'h3' | 'body' | 'small' | 'micro' | 'label';
37
38 const VARIANT_FONT: Record<Variant, string> = {
39   display: fonts.serif,
40   h1: fonts.serif,
41   h2: fonts.sansBold,
42   h3: fonts.sansSemi,
43   body: fonts.sans,
44   small: fonts.sans,
45   micro: fonts.sansMedium,
46   label: fonts.sansSemi,
47 };
48
49 export function T({
50   v = 'body',
51   color,
52   style,
53   children,
54   numberOfLines,
55   center,
56   selectable,
57   accessibilityRole,
58 }: {
59   v?: Variant;
60   color?: string;
61   style?: StyleProp<TextStyle>;
62   children: ReactNode;
63   numberOfLines?: number;
64   center?: boolean;
65   selectable?: boolean;
66   accessibilityRole?: 'header' | 'text' | 'alert';
67 }) {
68   const theme = useTheme();
69   const size = v === 'label' ? type.micro : type[v];
70   return (
71     <Text
72       accessibilityRole={accessibilityRole}
73       numberOfLines={numberOfLines}
74       selectable={selectable}
75       style={[
76         size,
77         {
78           fontFamily: VARIANT_FONT[v],
79           color: color ?? (v === 'small' || v === 'micro' ? theme.inkSoft : theme.ink),
80           textAlign: center ? 'center' : undefined,
81         },
82         // Labels are text: inkSoft (inkFaint is for placeholders, rules and quiet icons, below
4.5:1).
83         v === 'label' && { letterSpacing: 0.8, textTransform: 'uppercase', color: color ??
theme.inkSoft },
84         style,
85       ]}
86     >
87       {children}
88     </Text>
89   );

```

บรรทัด	ความหมาย
36	ระดับตัวหนังสือมี 8 แบบ ตั้งแต่ใหญ่สุด display (ยอดเงินบนการ์ด) ไปจนเล็กสุด micro
38–47	ตารางจับคู่ว่าแต่ละระดับใช้ฟอนต์อะไร: หัวข้อใหญ่ใช้ Noto Serif Thai (มีหัว) ที่เหลือใช้ Anuphan น้ำหนักต่างกัน Record<Variant, string> บังคับให้ต้องใส่ครบทั้ง 8 ระดับ
49–57	ประกาศ component ชื่อ T พร้อมแกะ props ออกมา <code>v = 'body'</code> แปลว่าถ้าไม่ระบุระดับ ใช้ body
58–67	ชนิดของ props แต่ละตัว (เครื่องหมาย ? = ไม่บังคับ) children คือสิ่งที่อยู่ระหว่าง <T> กับ </T> ส่วน accessibilityRole บอกโปรแกรมอ่านหน้าจอสำหรับคนตาบอดว่าเป็นหัวข้อหรือข้อความแจ้งเตือน
68	เรียก <code>useTheme()</code> เอาชุดสีของธีมที่ผู้ใช้เลือก (สว่าง/มืด, ป่าทอง/ม่วง ฯลฯ)
69	เลือกขนาดและความสูงบรรทัดจากตาราง type (label ใช้ขนาดเท่า micro)
70–86	คืน JSX: ชิ้นส่วน Text ของ React Native พร้อม style ที่ซ้อนกันเป็นชั้น (array ของ style ตัวหลังทับตัวหน้า)
78	ฟอนต์ตามระดับ
79	สี: ถ้าผู้เรียกส่งสีมาใช้สีนั้น ไม่จันข้อความเล็กใช้สีรอง (inkSoft) ข้อความอื่นใช้สีหลัก (ink)
80	จัดกึ่งกลางถ้าส่ง center
82–83	ระดับ label (หัวข้อเล็กตัวพิมพ์ใหญ่ เช่น "MONEY RUNWAY") ห่างตัวอักษรกว้างขึ้น และใช้สีที่ผ่านเกณฑ์อ่านง่าย
84	style ที่ผู้เรียกส่งมาเองใส่ท้ายสุด จึงทับค่าทั้งหมดได้
87	แสดงสิ่งที่อยู่ข้างใน (ข้อความ)

ใช้งานแบบนี้: `<T v="h3">จดรายการ</T>` หรือ `<T v="small" color={theme.watch}>อ่านวันที่ไม่ชัด</T>`

State: ข้อมูลที่เปลี่ยนได้ และทำให้หน้าจอลoadใหม่

ตัวแปรธรรมดาในฟังก์ชันจะหายไปทุกครั้งที React วาดใหม่ ถ้าอยากให้ component "จำ" ค่าได้ ต้องใช้ **state** ผ่าน hook ชื่อ `useState`

useState

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```
1 const [amount, setAmount] = useState(''); // ค่าเริ่มต้น = ข้อความว่าง
2 // amount    = ค่าปัจจุบัน
3 // setAmount = ฟังก์ชันเปลี่ยนค่า เรียกแล้ว React วาดหน้าจอใหม่
4
5 <AmountField value={amount} onChangeText={setAmount} />
6 // ผู้ใช้พิมพ์ "8" → setAmount('8') → วาดใหม่ ช่องแสดง 8
```

หน้าจอดรายการ (FR-1) ใช้ state จำค่าทุกช่องในฟอร์ม ดูจากโค้ดจริง:

src/app/transaction.tsx

บรรทัด 35–46

```
35 const startKind: TxKind = existing?.kind ?? (kindParam === 'income' ? 'income' : 'expense');
```

```

36 const [kind, setKind] = useState<TxKind>(startKind);
37 const [amount, setAmount] = useState(existing ? satangToInput(existing.amountSatang) : '');
38 const [title, setTitle] = useState(existing?.title ?? '');
39 const [category, setCategory] = useState(existing?.categoryKey ?? defaultCategory(startKind));
40 const [categoryTouched, setCategoryTouched] = useState(!existing);
41 const [day, setDay] = useState(existing ? bkkDayKey(existing.occurredAt) : bkkDayKey(new
Date()));
42 const [time, setTime] = useState(existing ? bkkTime(existing.occurredAt) : bkkTime(new
Date()));
43 const [note, setNote] = useState(existing?.note ?? '');
44 const [busy, setBusy] = useState(false);
45 const [askDelete, setAskDelete] = useState(false);
46 const [amountError, setAmountError] = useState<string | null>(null);

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/transaction.tsx:35

บรรทัด	ความหมาย
35	ประเภทเริ่มต้น: ถ้าแก้ไขรายการเดิม ใช้ประเภทเดิม ไม่จําเป็นดูว่าเปิดมาจากปุ่ม "เพิ่มเงินเข้า" ใหม่ (เป็นรายรับ) ไม่จําเป็นรายจ่าย
36	state ประเภท (รายรับ/รายจ่าย) useState<TxKind> บอกว่าเก็บได้แค่ 'income' หรือ 'expense'
37	state ช่องจํานวนเงิน เก็บเป็น ข้อความ ตามที่ผู้ใช้พิมพ์ (แปลงเป็นสตริงตอนกดบันทึก) ถ้าแก้ไขรายการเดิม เติมค่าเดิมให้
38–39	ชื่อรายการ และหมวด
40	จํานวนที่ใช้เลือกหมวดเองแล้วหรือยัง (ถ้ายัง แอปจะเดาหมวดจากชื่อที่พิมพ์ให้)
41–42	วันและเวลา (ค่าเริ่มต้นคือตอนนี้ ตามเวลาไทย)
43	บันทึกเพิ่มเติม
44–46	สถานะของหน้าจอเอง: กำลังบันทึกอยู่ไหม, กำลังถามยืนยันการลบไหม, ข้อความ error ใต้ช่องเงิน

! กฎของ Hook

ฟังก์ชันที่ขึ้นต้นด้วย use เรียกว่า **hook** มีกฎ 2 ข้อ: (1) เรียกได้เฉพาะใน component หรือใน hook อื่น (2) ต้องเรียกที่ระดับบนสุดของ component ห้ามอยู่ใน if หรือ loop เพราะ React จํา state ตามลำดับการเรียก ESLint ของโปรเจกต์ตรวจกฎนี้ให้อัตโนมัติ

Hook ที่ใช้ในโปรเจกต์

Hook	ทำอะไร	ตัวอย่างใน MindPay
useState	จําค่าที่เปลี่ยนได้ เปลี่ยนแล้ววาดใหม่	ทุกช่องในฟอร์ม, รายการทั้งหมดใน AppProvider
useEffect	ทำงาน "ข้างเคียง" หลังวาดเสร็จ เช่น ตั้งเวลา ฟัง event โหลดข้อมูล และเก็บกวาดเมื่อเลิกใช้	ตั้งนาฬิกาให้วันเปลี่ยนตอนเที่ยงคืน
useMemo	คำนวณแล้วจําผลไว้ คำนวณใหม่เฉพาะเมื่อข้อมูลที่ระบุเปลี่ยน	useMoney () คำนวณยอดเงินและ Runway

Hook	ทำอะไร	ตัวอย่างใน MindPay
useCallback	จำ "ฟังก์ชัน" ไว้ไม่ให้สร้างใหม่ทุกครั้ง	loadAll, activateRepo
useRef	กล่องจำค่าที่เปลี่ยนแล้ว ไม่ ต้องวาดใหม่	จำว่ากำลังใช้บัญชีไหนอยู่ (activeUserRef)
useContext	หยิบข้อมูลที่ "แชร์ทั้งแอป" มาใช้	useApp()
hook ของเรา	รวม hook หลายตัวเป็นชิ้นเดียว ตั้งชื่อขึ้นต้นด้วย use	useMoney, useTheme, useToast

ตัวอย่าง useEffect: วันใหม่เริ่มตอนเที่ยงคืนตามเวลาไทย

src/data/AppProvider.tsx

บรรทัด 182–212

```

182 // A new day starts at 00:00 Bangkok time: while the app is open (timer) or
183 // when it comes back to the screen the next morning (AppState / tab focus).
184 useEffect(() => {
185   let timer: ReturnType<typeof setTimeout>;
186   const check = () => {
187     const now = bkkDayKey(Date.now());
188     if (now !== todayRef.current) {
189       todayRef.current = now;
190       setToday(now);
191       setNewDay(true);
192       Storage.setItem(LAST_DAY_KEY, now).catch(() => {});
193     }
194     clearTimeout(timer);
195     timer = setTimeout(check, msUntilNextBkkMidnight() + 1000);
196   };
197   check();
198   // First open on a different day from the last one.
199   Storage.getItem(LAST_DAY_KEY)
200     .then((last) => {
201       if (last && last !== todayRef.current) setNewDay(true);
202       return Storage.setItem(LAST_DAY_KEY, todayRef.current);
203     })
204     .catch(() => {});
205   const sub = AppState.addEventListener('change', (s) => {
206     if (s === 'active') check();
207   });
208   return () => {
209     clearTimeout(timer);
210     sub.remove();
211   };
212 }, []);

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/AppProvider.tsx:182

บรรทัด	ความหมาย
182–183	คอมเมนต์: วันใหม่เริ่ม 00:00 เวลากรุงเทพ ทั้งตอนเปิดแอปค้างไว้ (ใช้นาฬิกาจับเวลา) และตอนกลับมาเปิดแอปตอนเช้า
184	useEffect(() => {...}, []) array วางท้ายสุดแปลว่า "ทำครั้งเดียวตอน component เริ่มทำงาน"
185	ตัวแปรเก็บนาฬิกาจับเวลา

บรรทัด	ความหมาย
186–196	ฟังก์ชัน check: ดูวันที่ตอนนี้ตามเวลาไทย ถ้าไม่ตรงกับ "วันนี้" ที่จำไว้ แปลว่าข้ามวันแล้ว ให้อัปเดตวันนี้ (setToday) ซึ่งทำให้ตัวเลขทุกอย่างที่ขึ้นกับวันนี้คำนวณใหม่ (เช่น ค่าเฉลี่ย 7 วันของ FR-6)
191	ตั้งธง "วันใหม่" ให้น้องกล่าวกักตายนและสรุปเมื่อวาน
192	จดวันที่ล่าสุดลงเครื่อง
194–195	ยกเลิกนาฬิกาเดิม แล้วตั้งใหม่ให้ปลุกตอนเที่ยงคืนครั้งหน้า (+1 วินาทีเพื่อ) msUntilNextBkkMidnight คำนวณว่าเหลืออีกกี่มิลิวินาที
197	เช็กกันทีหนึ่งครั้งตอนเริ่ม
198–204	อ่านวันที่เปิดแอปครั้งล่าสุดจากเครื่อง ถ้าไม่ใช่วันนี้ แปลว่าเป็นการเปิดครั้งแรกของวัน
205–207	ฟังเหตุการณ์ "แอปกลับมาอยู่หน้าจอ" (เช่น เปิดแอปค้างไว้ข้ามคืน แล้วเข้ามาสลับกลับมา) แล้วเช็กวันใหม่อีกครั้ง
208–211	ฟังก์ชันเก็บกวาด (cleanup): ถ้า component นี้ถูกปิด ให้ยกเลิกนาฬิกาและเลิกฟังเหตุการณ์ ไม่ให้ทำงานค้างกินแบตเตอรี่

Context: แชรข้อมูลให้ทุกหน้าโดยไม่ต้องส่งต่อเป็นทอด ๆ

ถ้าหน้าหลัก หน้ารายการ และหน้าเงินพอถึง ต้องใช้รายการเงินชุดเดียวกัน จะส่ง props ต่อกันเป็นทอด ๆ ก็ยุ่ง React มี **Context** เหมือน "ประกาศเสียงตามสาย" ตัวให้ข้อมูล (Provider) อยู่บนสุด ใครอยู่ข้างในจะหยิบไปใช้ได้เลย MindPay มี Provider หลายชั้นครอบคลุมทั้งแอปไว้ใน `src/app/_layout.tsx` (un A7) ตัวสำคัญที่สุดคือ **AppProvider**

`src/data/AppProvider.tsx`

บรรทัด 87–121

```

87 const AppContext = createContext<AppContextValue | null>(null);
88
89 export function useApp() {
90   const ctx = useContext(AppContext);
91   if (!ctx) throw new Error('useApp must be used inside AppProvider');
92   return ctx;
93 }
94
95 /** Same as useApp, but null outside the provider (for small shared pieces such as the
96     companion). */
97 export function useAppMaybe() {
98   return useContext(AppContext);
99 }
100 export function AppProvider({ children }: { children: ReactNode }) {
101   const [status, setStatus] = useState<AuthStatus>('loading');
102   const [repo, setRepo] = useState<Repo | null>(null);
103   const [userId, setUserId] = useState<string | null>(null);
104   const [profile, setProfile] = useState<Profile | null>(null);
105   const [txs, setTxs] = useState<Transaction[]>([]);
106   const [carrySatang, setCarry] = useState(0);
107   const [goals, setGoals] = useState<SavingsGoal[]>([]);
108   const [loadError, setLoadError] = useState<string | null>(null);
109   const [refreshing, setRefreshing] = useState(false);
110   const [authNotice, setAuthNotice] = useState<AuthNotice | null>(null);
111   const [today, setToday] = useState(() => bkkDayKey(Date.now()));
112   const [newDay, setNewDay] = useState(false);

```

```

113 const todayRef = useRef(today);
114 const repoRef = useRef<Repo | null>(null);
115 /** The account whose data is loaded (or being loaded): prevents loading the same account
    twice. */
116 const activeUserRef = useRef<string | null>(null);
117 const handledLinks = useRef(new Set<string>());
118 /** Each load gets a number; an older load that finishes late is ignored (it may be another
    account's). */
119 const loadGen = useRef(0);
120 /** Changes made on this phone while a load is running (null = deleted), put back over what it
    returns. */
121 const localChanges = useRef(new Map<string, Transaction | null>());

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/AppProvider.tsx:87

บรรทัด	ความหมาย
87	สร้าง Context เปล่า ๆ ไว้ (เริ่มต้นเป็น null)
89–93	useApp(): hook ที่ทุกหน้าใช้หับข้อมูลกลาง ถ้าเรียกนอก Provider จะ error กันที (กันลืมครอบ)
95–98	แบบที่ไม่ error แต่คืน null สำหรับชิ้นส่วนเล็กที่อาจอยู่นอก Provider
100	ประกาศ component AppProvider ซึ่งครอบหน้าจอทั้งหมด (children)
101	สถานะการเข้าสู่ระบบ: กำลังโหลด / ยังไม่เข้า / พร้อม
102–103	Repo ที่ใช้อยู่ (คลาวด์หรือเครื่อง) และ id ของผู้ใช้
104–107	ข้อมูลหลักของแอป: โปรไฟล์, รายการทั้งหมด, ยอดยกมาจากรายการเก่ามาก, กระปุกลูก
108–110	ข้อความ error ตอนโหลด, คำสั่งรีเฟรชใหม่, ข้อความแจ้งเรื่องบัญชี (เช่น "สมัครสำเร็จ")
111	วันนี้ตามเวลาไทย (ฟังก์ชันใน useState ทำงานครั้งเดียวตอนเริ่ม)
112	รวันใหม่
113–121	ค่าที่ใช้ useRef เพราะไม่ต้องวาดหน้าจอใหม่เมื่อเปลี่ยน: วันนี้ (ไว้ใช้ในนาฬิกา), Repo ปัจจุบัน, บัญชีที่กำลังโหลด, ลิงก์อีเมลที่จัดการแล้ว, เลขรอบการโหลด และการเปลี่ยนแปลงในเครื่องระหว่างโหลด

ส่วนท้ายของ AppProvider (บรรทัด 325–439) รวมข้อมูลและฟังก์ชันทั้งหมด (เช่น addTx, removeTx, saveProfile) เป็น object เดียว แล้วส่งให้ทุกหน้าผ่าน <AppContext.Provider value={value}> ที่บรรทัด 439

ทุกอย่างประกอบกัน: หน้าจอใช้ข้อมูลกลางยังไง

รูปแบบที่ทุกหน้าใช้

ตัวอย่างประกอบ (ไม่ใช่ไฟล์ในโปรเจกต์)

```

1 export default function SomeScreen() {
2   const theme = useTheme();           // สีของธีม
3   const { txs, addTx } = useApp();     // ข้อมูลและคำสั่งกลาง
4   const { balance, runway } = useMoney(); // ตัวเลขที่คำนวณแล้ว
5   const [open, setOpen] = useState(false); // state ของหน้านี้เอง
6
7   return (
8     <Screen>

```

```

9      <T v="h1">ยอด {formatBaht(balance)}</T>
10      {runway.days !== null ? <T>เงินพอใช้ซัก {runway.days} วัน</T> : null}
11    </Screen>
12  );
13 }

```

บรรทัด `{runway.days !== null ? ... : null}` คือ **การแสดงผลแบบมีเงื่อนไข** ใน JSX: ถ้ามีจำนวนวัน แสดงข้อความ ถ้าไม่มี (null) ไม่แสดงอะไร จะเจอรูปแบบนี้ทุกหน้า อีกแบบที่เจอบ่อยคือ `{list.map((x) => <Row key={x.id} ... />)}` แสดงรายการหลายแถวจาก array (ต้องใส่ key ที่ไม่ซ้ำ ให้ React รู้ว่าแถวไหนเป็นแถวไหน)

★ สรุปบท A6 ใน 4 ประโยค

(1) Component คือฟังก์ชันที่คืนหน้าจอ (JSX) (2) props คือของที่ส่งเข้า component, state คือความจำของ component (3) เปลี่ยน state ด้วย `setXxx` แล้ว React วาดใหม่เอง (4) ข้อมูลที่ทั้งแอปใช้ร่วมกันอยู่ใน `AppProvider` หยิบใช้ด้วย `useApp()` และตัวเลขเงินหยิบด้วย `useMoney()`

React Native และ Expo: โค้ดชุดเดียว ได้ทั้งแอปและเว็บ

ทำไมทีมเราเขียนโค้ดครั้งเดียว แล้วได้ทั้งแอป Android, iPhone และเว็บ และการเปลี่ยนหน้าจอด้วย Expo Router ทำงานยังไ้

React Native คืออะไร

React (un A6) เดิมใช้ทำเว็บ ซึ่งวาดหน้าจอด้วย HTML แต่แอปมือถือไม่มี HTML **React Native** จึงเปลี่ยน "ชิ้นส่วนพื้นฐาน" ให้เป็นของจริงบนมือถือ เราเขียน `<View>` แล้วบน Android กลายเป็นกล่องของ Android บน iPhone กลายเป็นกล่องของ iOS แอปจึงสิ้นเหมือนแอปที่เขียนด้วยภาษาของมือถือโดยตรง

สำหรับเว็บ มีแพ็คเกจ **react-native-web** (bussกั 37 ใน package.json) แปลชิ้นส่วนเดียวกันเป็น HTML เช่น `<View>` → `<div>` `<Text>` → `<span>` โค้ดชุดเดียวจึงได้ 3 แพลตฟอร์ม

ชิ้นส่วน React Native	คล้ายกับ (เว็บ)	ใช้ทำอะไร
View	div	กล่องสำหรับจัดวาง (การ์ด แถว คอลัมน์)
Text	span / p	ข้อความ (ข้อความทุกตัวต้องอยู่ใน Text)
Pressable	button	พื้นที่ที่แตะได้ มี onPress, onPressIn, onPressOut
ScrollView / FlatList / SectionList	หน้าที่เลื่อนได้	เลื่อนเนื้อหายาว ๆ (FlatList/SectionList เหมาะกับรายการยาวมาก เพราะวาดเฉพาะแถวที่เห็น)
TextInput	input	ช่องพิมพ์ (ห่ออยู่ใน Field / AmountField ของเรา)
Image	img	รูปภาพ
SafeAreaView	-	ไม่ให้เนื้อหาไปซ้อนกับรอยบาก แถบสถานะ หรือขอบล่างของมือถือ
Animated.View	-	กล่องที่ขยับได้ (แอนิเมชัน)

การจัดวางด้วย style

React Native ไม่มีไฟล์ CSS แต่ใส่ `style={{ ... }}` เป็น object ตรง ๆ ชื่อเหมือน CSS แต่เป็น camelCase เช่น `backgroundColor` หน่วยเป็นตัวเลขเฉย ๆ (หมายถึง "จุด" ที่ปรับตามความละเอียดจอเอง) การจัดวางใช้ **Flexbox**:

style	ความหมาย
<code>flexDirection: 'row'</code>	เรียงลูกในแนวนอน (ค่าเริ่มต้นคือแนวตั้ง 'column')
<code>flex: 1</code>	ขยายเต็มพื้นที่ที่เหลือ
<code>alignItems: 'center'</code>	จัดลูกให้อยู่กลาง (แกนขวาง)
<code>justifyContent: 'space-between'</code>	กระจายลูกให้ชิดหัวท้าย (แกนหลัก)

style	ความหมาย
gap: 8	ระยะห่างระหว่างลูกแต่ละตัว
padding / margin	ระยะห่างด้านใน / ด้านนอก
borderRadius	มุมโค้ง

ตัวอย่างจากชิ้นส่วน Row ที่ใช้ทั้งแอป: บรรทัดเดียวจบ

src/ui/components.tsx

บรรทัด 189–203

```

189 export function Row({
190   children,
191   gap = space.sm,
192   style,
193   align = 'center',
194   justify,
195 }: {
196   children: ReactNode;
197   gap?: number;
198   style?: StyleProp<ViewStyle>;
199   align?: ViewStyle['alignItems'];
200   justify?: ViewStyle['justifyContent'];
201 }) {
202   return <View style={[{ flexDirection: 'row', alignItems: align, justifyContent: justify, gap },
203     style] >{children}</View>;
204 }

```

▶ เปิดใน VS Code (เว็บ) GitHub

VS Code ในเครื่อง: Ctrl+P → src/ui/components.tsx:189

บรรทัด 202: สร้าง View ที่เรียงลูกแนวนอน (row) จัดกลางแนวตั้ง และเว้นระยะ gap ค่าเริ่มต้น 8 แล้วต่อด้วย style ที่ผู้เรียกส่งมา

Expo: ชุดเครื่องมือที่ทำให้ React Native ง่ายขึ้น

React Native เปล่า ๆ ต้องติดตั้ง Android Studio, Xcode และตั้งค่ามากมาย **Expo** ห่อทุกอย่างไว้ และมีโมดูลสำเร็จรูป (expo-camera, expo-speech ฯลฯ) ที่ใช้ได้เลย สิ่งที่ Expo ให้กับเรา:

- **Expo Go + dev server:** แก้โค้ดแล้วเห็นผลบนมือถือภายในไม่กี่วินาที
- **Expo Router:** ระบบเปลี่ยนหน้าจอจากโครงสร้างไฟล์ (หัวข้อถัดไป)
- **expo export:** ทำเว็บเวอร์ชันเป็นไฟล์นิ่ง ๆ เอาไปวางบนเว็บไซต์ได้
- **prebuild + EAS Build:** สร้างโปรเจกต์ Android/iOS และไฟล์ APK
- **EAS Update:** ส่งโค้ดใหม่ไปถึงแอปที่ลงแล้วโดยไม่ต้องลง APK ใหม่

! Expo ที่เราใช้เวอร์ชัน 57

Expo เปลี่ยนเร็วมาก ทุกเวอร์ชันมีคำสั่งที่เปลี่ยนชื่อหรือย้ายที่ ไฟล์ AGENTS.md ในโปรเจกต์จึงเตือนว่าก่อนใช้ API ของ Expo ให้เปิดเอกสารของเวอร์ชัน 57 (docs.expo.dev/versions/v57.0.0/) ไม่ใช่จากที่เคยเห็น

Expo Router: ไฟล์ 1 ไฟล์ = หน้าจอ 1 หน้า

ใน `src/app/` ทุกไฟล์คือหน้าจอ ชื่อไฟล์คือที่อยู่ (URL) ของหน้านั้น บนเว็บพิมพ์ที่อยู่ได้ตรง ๆ เลย

ไฟล์	ที่อยู่	หน้าจอ	FR
<code>_layout.tsx</code>	-	โครงทั้งแอป: ฟอนต์, Provider, กฎว่าเข้าหน้าไหนได้ (ไม่ใช่หน้าจอ)	ทุก FR
<code>welcome.tsx</code>	<code>/welcome</code>	สมัคร เข้าสู่ระบบ ลืมรหัสผ่าน ลองใช้ข้อมูลตัวอย่าง	บัญชี
<code>onboarding.tsx</code>	<code>/onboarding</code>	ตั้งค่าเงินครั้งแรก (ยอดเงินตอนนี้, เงินสำรอง, ...)	FR-2, FR-6
<code>(tabs)/_layout.tsx</code>	-	แถบเมนูล่าง 4 แท็บ + ปุ่มสแกน	-
<code>(tabs)/index.tsx</code>	<code>/</code>	หน้าหลัก	FR-2 (+FR-6)
<code>(tabs)/transactions.tsx</code>	<code>/transactions</code>	แท็บรายการ ปฏิทินการใช้จ่าย	FR-1, FR-2
<code>(tabs)/runway.tsx</code>	<code>/runway</code>	แท็บเงินพอถึง	FR-6
<code>(tabs)/coach.tsx</code>	<code>/coach</code>	แท็บโค้ชน้องกล้า	FR-5
<code>transaction.tsx</code>	<code>/transaction</code>	จด/แก้/ลบ/ตรวจสอบสลิป 1 รายการ	FR-1, FR-4
<code>voice.tsx</code>	<code>/voice</code>	พูดจด	FR-3
<code>scan.tsx</code>	<code>/scan</code>	สแกนสลิป	FR-4
<code>drafts.tsx</code>	<code>/drafts</code>	สลิปรอยืนยัน	FR-4
<code>settings.tsx</code>	<code>/settings</code>	ตั้งค่า (เงินสำรอง, ธีม, เสียงน้องกล้า ฯลฯ)	หลาย FR
<code>goals.tsx, goal.tsx</code>	<code>/goals, /goal</code>	กระปุกออม	FR-6
<code>recap.tsx</code>	<code>/recap</code>	สรุปเดือนแบบเรื่องเล่า	FR-2
<code>achievements.tsx, skins.tsx</code>	<code>/achievements, /skins</code>	เหรียญ การกิจ ตูสกินน้องกล้า	FR-5 (เสริม)
<code>whatsnew.tsx</code>	<code>/whatsnew</code>	มีอะไรใหม่	-

- โฟลเดอร์ในวงเล็บ เช่น `(tabs)` เป็น "กลุ่ม" ไม่โฟกัสในที่อยู่ ใช้จัดหน้าที่อยู่ใต้แถบเมนูเดียวกัน
- `index.tsx` คือหน้าแรกของโฟลเดอร์นั้น (ที่อยู่ `/`)
- ไฟล์ `_layout.tsx` ไม่ใช่หน้าจอ แต่เป็น "กรอบ" ที่ครอบหน้าจอในโฟลเดอร์เดียวกัน

เปลี่ยนหน้าในโค้ด

โค้ด	ผล
<code>router.push('/scan')</code>	เปิดหน้าจอสแกนซ้อนขึ้นมา (กดกลับได้)
<code>router.push({ pathname: '/transaction', params: { kind: 'income' } })</code>	เปิดหน้าจอรายการ พร้อมส่งค่า kind=income (ปุ่ม "เพิ่มเงินเข้า")
<code>useLocalSearchParams()</code>	หน้าปลายทางอ่านค่าที่ส่งมา (src/app/transaction.tsx:28)
<code>router.navigate('/runway')</code>	ไปที่เก็บเงินพอถึง
<code>goBack()</code>	กลับหน้าก่อน ถ้าไม่มี (เช่น เปิดลิงก์ตรงบนเว็บ) กลับหน้าหลัก (src/ui/nav.ts)

อ่านโค้ด: `_layout.tsx` โครงของทั้งแอป

ไฟล์นี้ทำงานเป็นไฟล์แรกเมื่อเปิดแอป ทุกหน้าจออยู่ข้างในมัน

`src/app/_layout.tsx`

บรรทัด 1-51

```
1 import { useFonts } from 'expo-font';
2 import { Stack } from 'expo-router';
3 import * as SplashScreen from 'expo-splash-screen';
4 import { StatusBar } from 'expo-status-bar';
5 import { useCallback, useState, type ReactNode } from 'react';
6 import { Platform, View } from 'react-native';
7 import { SafeAreaProvider } from 'react-native-safe-area-context';
8 import {
9   Anuphan_400Regular,
10  Anuphan_500Medium,
11  Anuphan_600SemiBold,
12  Anuphan_700Bold,
13 } from '@expo-google-fonts/anuphan';
14 import { NotoSerifThai_600SemiBold, NotoSerifThai_700Bold } from '@expo-google-fonts/noto-serif-thai';
15 import { AppProvider, useApp } from '../data/AppProvider';
16 import { AutoScanProvider } from '../services/AutoScanProvider';
17 import { AuthNoticeHost } from '../ui/AuthNotice';
18 import { CelebrationProvider } from '../ui/effects';
19 import { ToastProvider } from '../ui/feedback';
20 import { LaunchIntro } from '../ui/LaunchIntro';
21 import { UpdateBanner } from '../ui/UpdateBanner';
22 import { mix, useTheme } from '../ui/theme';
23
24 // The native splash stays until the opening animation (LaunchIntro) takes over.
25 SplashScreen.preventAutoHideAsync().catch(() => {});
26
27 export default function RootLayout() {
28   const [fontsLoaded, fontError] = useFonts({
29     Anuphan_400Regular,
30     Anuphan_500Medium,
31     Anuphan_600SemiBold,
32     Anuphan_700Bold,
33     NotoSerifThai_600SemiBold,
34     NotoSerifThai_700Bold,
```

```

35   });
36   return (
37     <SafeAreaProvider>
38       <WebFrame>
39         <CelebrationProvider>
40           <AppProvider>
41             <ToastProvider>
42               <AutoScanProvider>
43                 <Navigator ready={fontsLoaded || !!fontError} />
44               </AutoScanProvider>
45             </ToastProvider>
46           </AppProvider>
47         </CelebrationProvider>
48       </WebFrame>
49     </SafeAreaProvider>
50   );
51 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/_layout.tsx:1

บรรทัด	ความหมาย
1–22	นำเครื่องมือมาใช้: โหลดฟอนต์, Stack (การซ้อนหน้า), หน้าจอเปิดแอป, แถบสถานะ, ฟอนต์ Anuphan 4 น้ำหนัก และ Noto Serif Thai 2 น้ำหนัก, Provider ต่าง ๆ ของเรา
24–25	สั่งให้หน้าจอเปิดแอป (splash สีเขียวรูปน้องกล้า) ค้างไว้ก่อน จนกว่าแอนิเมชันเปิดแอปจะมารับช่วง .catch(() => {}) = ถ้าทำไม่ได้ก็ไม่เป็นไร
27	export default = นี่คือ component หลักของไฟล์ Expo Router จะเรียกตัวนี้
28–35	โหลดฟอนต์ทั้ง 6 ตัว ได้ผลว่าโหลดเสร็จหรือยัง (fontsLoaded) หรือมี error ไหม
36–50	ซ้อน Provider เป็นชั้น ๆ เหมือนตุ๊กตาแม่ลูกดก ชั้นนอกสุดถึงชั้นในสุด: ขอบจอปลอดภัย → กรอบเว็บ → ระบบพลูลอง → ข้อมูลกลาง (AppProvider) → ข้อความแจ้งเตือน (toast) → สแกนสลิปอัตโนมัติ (FR-4) → ตัวเปลี่ยนหน้า ชั้นในใช้ของชั้นนอกได้ เช่น AutoScanProvider ใช้ข้อมูลจาก AppProvider ได้เพราะอยู่ข้างใน
43	ส่ง ready = ฟอนต์โหลดเสร็จ หรือโหลดพังกี่ให้ไปต่อ (ไม่ค้างหน้าขาวตลอดไป)

src/app/_layout.tsx

บรรทัด 53–73

```

53  /** On a computer browser, show the app at phone width in the middle of the screen. */
54  function WebFrame({ children }: { children: ReactNode }) {
55    const theme = useTheme();
56    if (Platform.OS !== 'web') return <>{children}</>;
57    return (
58      <View style={{ flex: 1, alignItems: 'center', backgroundColor: theme.dark ? mix(theme.bg,
59        '#000000', 0.55) : theme.surfaceAlt }}>
60        <View
61          style={{
62            flex: 1,
63            width: '100%',
64            maxWidth: 440,
65            backgroundColor: theme.bg,
66            overflow: 'hidden',
67            boxShadow: '0 0 40px rgba(0, 0, 0, 0.18)',
68          }}
69        >
70          {children}
71        </View>
72      </View>
73    );
74  }

```

```

70     </View>
71   </View>
72 );
73 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/_layout.tsx:53

บรรทัด ความหมาย

53–54 WebFrame: บนคอม แสดงแอปกว้างเท่ามือถือไว้ตรงกลางจอ

56 ถ้าไม่ใช่เว็บ (Platform.OS !== 'web') ไม่ต้องทำอะไร ค้นของข้างในตรง ๆ (<>...</> คือกล่องที่มองไม่เห็น)

58 พื้นหลังเต็มจอ จัดกลาง สีเข้มขึ้นเล็กน้อยในโหมดมืด

59–68 กล่องแอป กว้างสูงสุด 440 จุด มีเงารอบ ๆ ให้ดูเหมือนมือถือวางบนโต๊ะ

src/app/_layout.tsx

บรรทัด 75–126

```

75 function Navigator({ ready }: { ready: boolean }) {
76   const theme = useTheme();
77   const { status, repo, profile } = useApp();
78   const loading = !ready || status === 'loading';
79   const [introDone, setIntroDone] = useState(false);
80   const finishIntro = useCallback(() => setIntroDone(true), []);
81
82   const signedIn = status === 'ready' && !!repo;
83   const needsOnboarding = signedIn && !!profile && !profile.onboarded;
84   const inApp = signedIn && !needsOnboarding;
85
86   return (
87     <>
88       <StatusBar style={!introDone || theme.dark ? 'light' : 'dark'} />
89       {loading ? null : <Screens signedIn={signedIn} needsOnboarding={needsOnboarding} inApp=
{inApp} />}
90       {loading || !introDone ? null : <AuthNoticeHost />}
91       {loading || !introDone ? null : <UpdateBanner />}
92       <LaunchIntro ready={!loading} fontsReady={ready} onDone={finishIntro} />
93     </>
94   );
95 }
96
97 function Screens({ signedIn, needsOnboarding, inApp }: { signedIn: boolean; needsOnboarding:
boolean; inApp: boolean }) {
98   const theme = useTheme();
99   return (
100     <>
101       /* Screens slide in from the side; scanning and adding a transaction rise from the bottom.
*/
102       <Stack screenOptions={{ headerShown: false, contentStyle: { backgroundColor: theme.bg },
animation: 'slide_from_right' }}>
103         <Stack.Protected guard={!signedIn}>
104           <Stack.Screen name="welcome" options={{ animation: 'fade' }} />
105         </Stack.Protected>
106         <Stack.Protected guard={needsOnboarding}>
107           <Stack.Screen name="onboarding" options={{ animation: 'fade' }} />
108         </Stack.Protected>
109         <Stack.Protected guard={inApp}>
110           <Stack.Screen name="(tabs)" options={{ animation: 'fade' }} />

```

```

111         <Stack.Screen name="transaction" options={{ presentation: 'modal', animation:
'slide_from_bottom' }} />
112         <Stack.Screen name="scan" options={{ presentation: 'fullScreenModal', animation:
'slide_from_bottom' }} />
113         <Stack.Screen name="voice" options={{ presentation: 'modal', animation:
'slide_from_bottom' }} />
114         <Stack.Screen name="achievements" />
115         <Stack.Screen name="skins" />
116         <Stack.Screen name="recap" options={{ presentation: 'fullScreenModal', animation:
'fade' }} />
117         <Stack.Screen name="goals" />
118         <Stack.Screen name="whatsnew" options={{ presentation: 'modal', animation:
'slide_from_bottom' }} />
119         <Stack.Screen name="goal" options={{ presentation: 'modal', animation:
'slide_from_bottom' }} />
120         <Stack.Screen name="drafts" />
121         <Stack.Screen name="settings" />
122     </Stack.Protected>
123 </Stack>
124 </>
125 );
126 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/app/_layout.tsx:75`

บรรทัด	ความหมาย
75–78	Navigator ดูสถานะจากข้อมูลกลาง: เข้าสู่ระบบหรือยัง มี Repo ใหม่ โปรไฟล์ตั้งค่าเสร็จไหม และยังโหลดอยู่ไหม
79–80	จำว่าแอนิเมชันเปิดแอปจบหรือยัง
82–84	คำนวณ 3 สถานะ: signedIn (เข้าสู่ระบบแล้ว), needsOnboarding (เข้าแล้วแต่ยังไม่ตั้งค่าเงิน), inApp (เข้าแล้วและตั้งค่าแล้ว ใช้แอปได้เต็มที่)
88	สไลด์คอนแทกสถานะด้านบน (สว้างบนพื้นเข้ม)
89	ถ้ายังโหลดอยู่ ยังไม่แสดงหน้าจอ
90–91	หลังแอนิเมชันจบ แสดงหน้าต่างแจ้งเรื่องบัญชี และแถบ "มีอัปเดตใหม่"
92	แอนิเมชันเปิดแอป (น้องกล้าไฟล์) ช้อนอยู่บนสุดจนจบ
97–102	Stack = หน้าจอซ้อนกันเป็นกอง หน้าใหม่กับหน้าเก่า กดกลับก็หิบบนออก ตั้งค่าทุกหน้า: ไม่มีหัวหน้าสำเร็จรูป, พื้นตามืด, เลื่อนเข้ามาจากขวา
103–105	Stack.Protected = ประตุมียาม: หน้า welcome เข้าได้เฉพาะตอน ยังไม่ เข้าสู่ระบบ
106–108	หน้า onboarding เข้าได้เฉพาะตอนเข้าแล้วแต่ยังไม่ตั้งค่าเงิน
109–122	หน้าในแอปทั้งหมดเข้าได้เฉพาะตอนพร้อมใช้งาน บางหน้าแสดงแบบ modal (เลื่อนขึ้นจากล่าง) เช่น จดรายการ พุดจด, บางหน้าเต็มจอ เช่น สแกนสลิป สรุปเดือน

★ ผลของ Stack.Protected

ผู้ใช้ไม่ต้องทำอะไร พอสมัครเสร็จ สถานะเปลี่ยนเป็น needsOnboarding ยามจะปิดประตู welcome และเปิดประตู onboarding แอปจึงพาไปหน้าตั้งค่าเงินเอง พอตั้งค่าเสร็จก็ไปหน้าหลักเอง และกดย้อนกลับก็ไม่เจอหน้าสมัครอีก (ตรงกับ acceptance criteria ในเอกสาร Traceability)

แถบเมนูล่าง: (tabs)/_layout.tsx

src/app/(tabs)/_layout.tsx

บรรทัด 23–28

```
23 const TABS: Record<string, { label: string; icon: IconName; iconActive: IconName }> = {
24   index: { label: 'หน้าหลัก', icon: 'home-outline', iconActive: 'home' },
25   transactions: { label: 'รายการ', icon: 'list-outline', iconActive: 'list' },
26   runway: { label: 'เงินพอถึง', icon: 'leaf-outline', iconActive: 'leaf' },
27   coach: { label: 'โค้ช', icon: 'chatbubble-ellipses-outline', iconActive: 'chatbubble-ellipses' },
28 };
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/(tabs)/_layout.tsx:23

บรรทัด 23–28: ตารางของ 4 แท็บ แต่ละแท็บมีชื่อไทย ไอคอนตอนไม่ได้เลือก และไอคอนตอนเลือก (แบบทึบ) ชื่อช่อง (index, transactions, runway, coach) ตรงกับชื่อไฟล์ในโฟลเดอร์ (tabs)

src/app/(tabs)/_layout.tsx

บรรทัด 134–186

```
134   {routes.slice(0, half).map((r, i) => tab(r, i))}
135   <View style={{ width: 76, alignItems: 'center' }}>
136     <View style={{ width: 60, height: 60, marginTop: -26, alignItems: 'center',
137       justifyContent: 'center' }}>
138       <PulseRing size={60} active times={auto.state.phase === 'scanning' ? 'always' : 3}
139         color={theme.accent} />
140       <Animated.View style={{ transform: [{ scale: press.scale }] }}>
141         <Pressable
142           accessibilityRole="button"
143           accessibilityLabel="สแกนสลิป"
144           onPressIn={press.onPressIn}
145           onPressOut={press.onPressOut}
146           onPress={() => {
147             Haptics.impactAsync(Haptics.ImpactFeedbackStyle.Medium).catch(() => {});
148             router.push('/scan');
149           }}
150           style={{
151             width: 60,
152             height: 60,
153             borderRadius: 30,
154             backgroundColor: theme.hero,
155             borderWidth: 3,
156             borderColor: theme.accent,
157             alignItems: 'center',
158             justifyContent: 'center',
159             shadowColor: theme.accent,
160             shadowOpacity: 0.35,
161             shadowRadius: 10,
162             shadowOffset: { width: 0, height: 4 },
163             elevation: 8,
164           }}
165         />
166     </View>
167   </View>
```

```

164         <Ionicons name="scan" size={26} color={theme.heroAccent} />
165     </Pressable>
166 </Animated.View>
167 </View>
168     <Text style={[type.micro, { fontFamily: fonts.sansSemi, color: theme.ink, marginTop: 4
    ]}]>สแกนสลิป</Text>
169 </View>
170     {routes.slice(half).map((r, i) => tab(r, i + half))}
171 </View>
172 );
173 }
174
175 export default function TabsLayout() {
176 // Records today's visit and hands out น้องกล้า's skins for finished missions and events.
177 useKlaSync();
178 return (
179     <Tabs tabBar={props} => <TabBar {...props} /> screenOptions={{ headerShown: false }}>
180         <Tabs.Screen name="index" />
181         <Tabs.Screen name="transactions" />
182         <Tabs.Screen name="runway" />
183         <Tabs.Screen name="coach" />
184     </Tabs>
185 );
186 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/(tabs)/_layout.tsx:134

บรรทัด	ความหมาย
134	วาดแท็บครั้งแรก (หน้าหลัก, รายการ) ทางซ้าย
135–169	ปุ่มทองตรงกลาง: วงกลม 60 จุด ลอยขึ้นมา 26 จุด (marginTop: -26) มีวงแหวนกระเพื่อม (PulseRing) 3 รอบ ตอนเปิดแอป และกระเพื่อมตลอดระหว่างสแกนอัตโนมัติ
144–147	แตะแล้วมือถือสั่นแรงปานกลาง และเปิดหน้าสแกน
170	วาดแท็บครั้งหลัง (เงินพอถึง, โค้ช) ทางขวา
175–186	component หลัก: ใช้แถบเมนูที่ทีมออกแบบเอง (tabBar=) แทนของสำเร็จรูป และประกาศ 4 แท็บ useKlaSync() บันทึกว่าวันนี้เปิดแอป (ใช้บันทึกการกิจ "เปิดแอป 30 วันติด")

โค้ดที่ต่างกันระหว่างมือถือกับเว็บ

บางอย่างทำได้ต่างกัน เช่น มือถือเก็บข้อมูลด้วย SQLite แต่เว็บใช้ localStorage ของเบราว์เซอร์ ทีมเราแก้ด้วย **ไฟล์คู่**: ตั้งชื่อ storage.ts สำหรับมือถือ และ storage.web.ts สำหรับเว็บ ตอน build ระบบ (Metro bundler) เลือกไฟล์ให้เองตามแพลตฟอร์ม ที่เหลือของแอป import แค่ './storage'

src/data/storage.ts

บรรทัด 1–12

```

1 /**
2  * Small key-value store used for app
  settings and demo data.
3  * Phone: SQLite-backed store (survives
  app restarts).
4  * Web: see storage.web.ts (browser
  localStorage).

```

src/data/storage.web.ts

บรรทัด 1–24

```

1 /** Web version of storage.ts: the
  browser's localStorage, wrapped so it
  never throws. */
2 export const Storage = {
3   async getItem(key: string):
    Promise<string | null> {
4     try {

```

```

5  */
6  import KV from 'expo-sqlite/kv-store';
7
8  export const Storage = {
9    getItem: (key: string) =>
10     KV.getItem(key),
11    setItem: (key: string, value: string) =>
12     KV.setItem(key, value),
13    removeItem: (key: string) =>
14     KV.removeItem(key),
15  };

```

► เปิดใน VS
Code (เว็บ)

GitHub

VS Code ในเครื่อง: Ctrl+P →
src/data/storage.ts:1

```

5      return
6      window.localStorage.getItem(key);
7    } catch {
8      return null;
9    },
10   async setItem(key: string, value:
11     string): Promise<void> {
12     try {
13       window.localStorage.setItem(key,
14         value);
15     } catch {
16       // Private mode or storage full:
17       // keep working in memory.
18     }
19   },
20   async removeItem(key: string):
21     Promise<void> {
22     try {
23       window.localStorage.removeItem(key);
24     } catch {
25       // ignore
26     }
27   },
28 };

```

► เปิดใน VS
Code (เว็บ)

GitHub

VS Code ในเครื่อง: Ctrl+P →
src/data/storage.web.ts:1

ทั้งสองไฟล์มี getItem, setItem, removeItem หน้าตาเดียวกัน ฉบับเว็บห่อด้วย try/catch เพราะเบราว์เซอร์โหมดส่วนตัวอาจห้ามเก็บข้อมูล คู่ไฟล์แบบนี้มีทั้งหมด 5 คู่:

ไฟล์	มือถือ	เว็บ
data/storage	SQLite (expo-sqlite)	localStorage
data/sessionStorageSetup	ที่เก็บการเข้าสู่ระบบของ Supabase บนมือถือ	ของเบราว์เซอร์
services/gallery	อ่านแกลเลอรีทั้งหมด (expo-media-library)	ให้ผู้ใช้เลือกรูปเอง (เบราว์เซอร์ไม่อนุญาตให้เว็บเปิดแกลเลอรี)
services/speech	ตัวฟังเสียงของมือถือ (Google/Apple)	Web Speech API ของเบราว์เซอร์
services/tts	เสียงอ่านของมือถือ (expo-speech)	speechSynthesis ของเบราว์เซอร์

ส่วนที่ต่างเล็กน้อยใช้ Platform.OS ในไฟล์เดียวกัน เช่น หน้าจอดรายการบนเว็บให้พิมพ์วันที่ แต่บน Android เปิดปฏิทินของเครื่อง ([src/app/transaction.tsx:288](#))

Native module แบบไม่บังคับ

src/services/speech.ts

บรรทัด 1–24

```

1  /**
2   * Speech to text on the phone ("พูดจด"), in Thai, with the phone's own speech

```

```

3  * recognizer (Google on Android, Apple on iOS) through expo-speech-recognition.
4  *
5  * The native module is looked up optionally: app builds made before it was
6  * added do not have it, and an over-the-air update must not crash them. There
7  * the voice screen simply offers typing instead (the keyboard's own mic works).
8  * The web version is speech.web.ts.
9  */
10 import { requireOptionalNativeModule } from 'expo';
11
12 interface Subscription {
13   remove(): void;
14 }
15 interface NativeSpeech {
16   start(options: Record<string, unknown>): void;
17   stop(): void;
18   abort(): void;
19   requestPermissionsAsync(): Promise<{ granted: boolean; canAskAgain?: boolean }>;
20   isRecognitionAvailable(): boolean;
21   addListener(event: string, listener: (e: never) => void): Subscription;
22 }
23
24 const Speech = requireOptionalNativeModule<NativeSpeech>('ExpoSpeechRecognition');

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/services/speech.ts:1

บรรทัด 24 ใช้ requireOptionalNativeModule = "ขอโมดูลฟังเสียง ถ้าไม่มีก็ให้ null ไม่ต้อง crash" เพราะแอปที่ลงไว้ก่อนเพิ่มฟีเจอร์พูดจดยังไม่มีโมดูลนี้ แต่ยังสามารถอัปเดตใหม่ผ่าน EAS Update (หัวข้อถัดไป) ถ้าเรียกแบบบังคับ แอปเก่าจะแจ้งเตือนแบบนี้แอปเก่าแค่แสดงช่องพิมพ์แทนไมค์

จากโค้ด สู่เว็บและแอปที่ผู้ใช้ลง

โค้ดใน src/ (TypeScript + React Native)



เว็บ

expo export --platform web รวมโค้ดเป็นไฟล์ JavaScript + HTML ในโฟลเดอร์ dist แล้ววางบน GitHub Pages

APK (Android)

expo prebuild สร้างโปรเจกต์ Android จาก app.json แล้ว Gradle build เป็นไฟล์ .apk

EAS Update

ส่งเฉพาะโค้ด JavaScript ใหม่ แอปที่ลงแล้วโหลดเองตอนเปิดแอป ใช้ตอนเปิดครั้งถัดไป

ทั้ง 3 ทางทำอัตโนมัติทุกครั้งที่เรา push โค้ดขึ้น branch main บน GitHub (บน A13) ไฟล์ตั้งค่าของแอปคือ app.json ส่วนสำคัญ:

ค่าใน app.json	ความหมาย
"name": "MindPay", "slug": "mindpay"	ชื่อแอปที่เห็นใต้ไอคอน
"version": "1.0.0" + "runtimeVersion": {"policy": "appVersion"}	EAS Update ส่งโค้ดใหม่ให้เฉพาะแอปที่เวอร์ชันตรงกัน ถ้าเพิ่ม native module ใหม่ ต้องเพิ่มเวอร์ชัน แล้วลง APK ใหม่
"android.package"	รหัสแอป com.socratesandskeletons.mindpay

ค่าใน app.json	ความหมาย
"blockedPermissions"	บล็อกสิทธิ์กล้อง วัตถุโอ เสียงในเครื่อง ที่ไม่ได้ใช้ (ความเป็นส่วนตัว)
"plugins"	ตั้งค่า native ของโมดูล เช่น ข้อความขอสิทธิ์แคลเลอร์/ไมโครโฟนเป็นภาษาไทย
"scheme": "mindpay"	ลิงก์ที่ขึ้นต้นด้วย mindpay:// เปิดแอปได้ (ใช้กับลิงก์ยืนยันอีเมล)
"updates"	ที่อยู่ของเซิร์ฟเวอร์ EAS Update และให้เช็คอัปเดตทุกครั้งที่เปิดแอป
"web": {"output": "single"}	เว็บเป็นหน้าเดียว (single page app) ให้ JavaScript เปลี่ยนหน้าเอง

ไฟล์ `app.config.js` เพิ่มค่าเฉพาะตอน build เว็บ: เว็บอยู่ที่ `/Socrates-and-Skeletons - /` บน GitHub Pages จึงต้องตั้ง `baseUrl` แต่ห้ามใส่ค่านี้นในแอปมือถือ (แอปจะหาฟอนต์และรูปผิดที่)

วิธีอ่านโค้ด MindPay ด้วยตัวเอง

โค้ดทั้งแอปมีเกือบ 19,000 บรรทัด ไม่มีใครจำได้หมด แต่ทุกไฟล์ใช้รูปแบบซ้ำ ๆ กัน รู้รูปแบบแล้วจะอ่านไฟล์ไหนก็ออก

ขั้นตอนอ่านไฟล์ที่ไม่เคยเห็น

1. **อ่านคอมเมนต์บนสุดของไฟล์ก่อน** ทุกไฟล์ในโปรเจกต์เริ่มด้วยคอมเมนต์ `/** ... */` บอกว่าไฟล์นี้ทำอะไร เป็นของ FR ไหน และมีกฎอะไรสำคัญ (ภาษาอังกฤษง่าย ๆ ใช้ Google Translate ช่วยได้)
2. **ดูบรรทัด import** บอกว่าไฟล์นี้พึ่งใครบ้าง ถ้า import จาก `../domain` แปลว่ามีการคำนวณ ถ้าจาก `../data/AppProvider` แปลว่าใช้ข้อมูลกลาง
3. **กด Ctrl+Shift+O** ดูรายชื่อฟังก์ชันทั้งหมดในไฟล์ เหมือนดูสารบัญ
4. **หาฟังก์ชันที่มี export** นั่นคือ "ประตูหน้าบ้าน" ที่ไฟล์อื่นเรียก ฟังก์ชันที่ไม่มี export คือผู้ช่วยภายใน
5. **อ่านชนิดข้อมูลก่อนอ่านตัวโค้ด** ดูว่ารับอะไรเข้า คืนอะไรออก ก็เข้าใจไปครึ่งหนึ่งแล้ว
6. **หาเทสต์ของมัน** กด Shift+F12 ที่ชื่อฟังก์ชัน ถ้ามีเทสต์ใน `__tests__` จะเห็นตัวอย่างการใช้งานจริงพร้อมคำตอบที่ถูกต้อง อ่านง่ายกว่าตัวโค้ดเสียอีก

รูปแบบที่เจอซ้ำทั้งแอป (ตารางโยง)

เห็นอะไร	แปลว่าอะไร
ชื่อลงท้าย Satang	จำนวนเงินเป็นสตางค์ (จำนวนเต็ม) หาร 100 ได้บาท
ชื่อขึ้นต้น bkk เช่น <code>bkkDayKey</code> , <code>bkkToIso</code>	ฟังก์ชันจัดการวันเวลาแบบเวลาไทย (กรุงเทพ UTC+7) อยู่ใน <code>src/domain/dates.ts</code>
ข้อความวันที่ <code>'2026-10-09'</code>	"day key" วันที่ไทยรูปแบบ ปี-เดือน-วัน (ค.ศ.) เอามาเทียบมากกว่า/น้อยกว่า กันได้ตรง ๆ เพราะเรียงตามตัวอักษรก็ตรงกับเรียงตามเวลา
<code>useApp()</code>	หยิบข้อมูลกลาง (โปรไฟล์ รายการ คำสั่ง addTx ฯลฯ)
<code>useMoney()</code>	หยิบตัวเลขที่คำนวณแล้ว: ยอดคงเหลือ ค่าเฉลี่ย Runway แบบร่าง
<code>useTheme()</code> แล้วตามด้วย <code>theme.ink</code> , <code>theme.primary</code>	สีจากธีมที่ผู้ใช้เลือก (ห้ามใส่รหัสสีเองในหน้าจอ)
<code><T v="h3"></code> , <code><Money satang={...}></code> , <code><Card></code> , <code><Button></code>	ชิ้นส่วนหน้าตาจาก <code>src/ui/components.tsx</code>
<code>space.lg</code> , <code>radius.md</code>	ระยะห่างและมุมโค้งมาตรฐาน (16 จุด, 14 จุด) จาก <code>src/ui/theme.ts</code>
<code>toast({ message })</code>	ข้อความแจ้งเตือนสั้น ๆ ด้านล่างจอ
<code>router.push('/xxx')</code>	เปิดหน้า <code>src/app/xxx.tsx</code>
<code>accessibilityLabel=""</code>	ข้อความที่โปรแกรมอ่านหน้าจออ่านให้คนตาบอดฟัง และเทสต์ E2E ใช้หาปุ่ม

เห็นอะไร	แปลว่าอะไร
status === 'confirmed' หรือ confirmedOnly(tx)	นับเฉพาะรายการที่ยืนยันแล้ว
Reveal, usePressSpring, useReduceMotion	แอนิเมชัน (ค่อย ๆ โฟกัส, ปุ่มยุบตอนกด, เคารพ "ลดการเคลื่อนไหว")
คอมเมนต์มีรหัส TC-12	รหัสเทสต์ ตรงกับในเอกสาร docs/TRACEABILITY.md
ไฟล์ชื่อ xxx.web.ts	ฉบับเว็บของไฟล์ xxx.ts
ชื่อแบบ snake_case เช่น amount_satang	ชื่อคอลัมน์ในฐานข้อมูล (ฝั่งแอปใช้ camelCase amountSatang) แปลงกันใน src/data/repo.ts

ฟังก์ชันวันเวลาที่ต้องรู้ (ทุก FR ใช้)

เวลาในฐานข้อมูลเก็บเป็นเวลาสากล (UTC) แต่ผู้ใช้ชาวไทย (เร็วกว่า 7 ชั่วโมง) ถ้าไม่แปลง รายการตอน 06:00 น. ของวันที่ 8 จะถูกนับเป็นวันที่ 7 (เพราะ UTC ยังเป็น 23:00 ของวันที่ 7) ทุก FR จึงใช้ฟังก์ชันจาก [src/domain/dates.ts](#)

src/domain/dates.ts

บรรทัด 1-51

```

1  /**
2   * Date helpers fixed to Thailand time (Asia/Bangkok, UTC+7, no daylight saving).
3   *
4   * Everything the user sees ("today", "this week", the date printed on a slip)
5   * is a Bangkok calendar day, so all grouping and range logic goes through
6   * `bkkDayKey`. Because Thailand has no DST, a fixed +7h offset is exact.
7   */
8  import type { RangeKey } from './types';
9
10 export const BKK_OFFSET_MS = 7 * 60 * 60 * 1000;
11 export const DAY_MS = 24 * 60 * 60 * 1000;
12
13 /** Instant -> "YYYY-MM-DD" in Bangkok. */
14 export function bkkDayKey(at: Date | string | number): string {
15   const ms = new Date(at).getTime() + BKK_OFFSET_MS;
16   return new Date(ms).toISOString().slice(0, 10);
17 }
18
19 /** "YYYY-MM-DD" (Bangkok day) -> the UTC instant of 00:00 that day in Bangkok. */
20 /** Milliseconds until the next 00:00 in Bangkok (when "วันนี้" starts over). */
21 export function msUntilNextBkkMidnight(nowMs: number = Date.now()): number {
22   const next = bkkDayStart(addDays(bkkDayKey(nowMs), 1)).getTime();
23   return Math.max(0, next - nowMs);
24 }
25
26 export function bkkDayStart(dayKey: string): Date {
27   return new Date(Date.parse(`${dayKey}T00:00:00.000Z`) - BKK_OFFSET_MS);
28 }
29
30 /** Add whole days to a "YYYY-MM-DD" key. */
31 export function addDays(dayKey: string, days: number): string {
32   return new Date(Date.parse(`${dayKey}T00:00:00.000Z`) + days * DAY_MS)
33     .toISOString()
34     .slice(0, 10);
35 }

```

```

36
37 /** Whole days from a to b (both "YYYY-MM-DD"). */
38 export function daysBetween(a: string, b: string): number {
39   return Math.round((Date.parse(`${b}T00:00:00Z`) - Date.parse(`${a}T00:00:00Z`)) / DAY_MS);
40 }
41
42 /** Combine a Bangkok day and "HH:MM" into an ISO instant. */
43 export function bkkToIso(dayKey: string, time: string | null = '12:00'): string {
44   const t = time && /\d{1,2}:\d{2}$/.test(time) ? time.padStart(5, '0') : '12:00';
45   return new Date(Date.parse(`${dayKey}T${t}:00.000Z`) - BKK_OFFSET_MS).toISOString();
46 }
47
48 /** "HH:MM" of an instant in Bangkok. */
49 export function bkkTime(at: Date | string): string {
50   return new Date(new Date(at).getTime() + BKK_OFFSET_MS).toISOString().slice(11, 16);
51 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/dates.ts:1

บรรทัด	ความหมาย
1–7	คอมเมนต์: ทุกอย่างที่ใช้ให้เห็น (วันนี้, สัปดาห์นี้, วันที่บนสลิป) เป็นวันตามปฏิทินกรุงเทพ และเพราะไทยไม่มีการปรับเวลา ฤดูร้อน (daylight saving) การบวก 7 ชั่วโมงตายตัวจึงถูกต้องเสมอ
10	7 ชั่วโมงในหน่วยมิลลิวินาที ($7 \times 60 \times 60 \times 1000 = 25,200,000$)
11	1 วันในหน่วยมิลลิวินาที ($86,400,000$)
13–17	bkkDayKey : รับเวลาแบบไหนก็ได้ (Date, ข้อความ, ตัวเลข) บวก 7 ชั่วโมง แล้วตัดเอา 10 ตัวอักษรแรกของรูปแบบ สากล ได้วันที่ไทย เช่น <code>bkkDayKey('2026-10-07T23:00:00Z')</code> → <code>'2026-10-08'</code> (23:00 UTC = 06:00 น. ของวันที่ 8 เวลาไทย)
20–24	msUntilNextBkkMidnight : เหลืออีกกี่มิลลิวินาทีถึงเที่ยงคืนไทยครั้งหน้า (เอาวันพรุ่งนี้ 00:00 ลบเวลาตอนนี้) ใช้ตั้ง นาฬิกาใน AppProvider (un A6)
26–28	bkkDayStart : วันที่ไทย → เวลาสากลของ 00:00 น. วันนั้น (ลบ 7 ชั่วโมงกลับ)
30–35	addDays : บวกหรือลบจำนวนวันจากวันที่ เช่น <code>addDays('2026-10-08', 20)</code> → <code>'2026-10-28'</code> ข้ามเดือน ข้ามปีให้เอง
37–40	daysBetween : นับจำนวนวันจาก a ถึง b เช่น <code>daysBetween('2026-10-08', '2026-10-31')</code> → 23
42–46	bkkToIso : เอาวันไทย + เวลา "HH:MM" มารวมเป็นเวลาสากลสำหรับเก็บในฐานข้อมูล ถ้าเวลาผิดรูปแบบใช้ 12:00 แทน เช่น <code>bkkToIso('2026-10-08', '06:00')</code> → <code>'2026-10-07T23:00:00.000Z'</code>
48–51	bkkTime : เวลาสากล → "HH:MM" ตามเวลาไทย ใช้แสดงเวลาของรายการ

แผนที่: อยากรู้อะไรไปเปิดไฟล์ไหน

อยากรู้	เปิดไฟล์
หน้าจอนี้หน้าตาแบบนี้เพราะอะไร	src/app/ชื่อหน้า.tsx (ตารางใน un A7)

อยากรู้	เปิดไฟล์
ตัวเลขนี้คิดยังไง	src/domain/: src/domain/summary.ts (ยอด), src/domain/runway.ts (Runway), src/domain/insights.ts (การ์ดโค้ช)
ข้อมูลโหลด/บันทึกยังไง	src/data/repo.ts , src/data/AppProvider.tsx
ตารางในฐานข้อมูลมีคอลัมน์อะไร	supabase/migrations/20260927000000_init.sql
AI ได้รับคำสั่งอะไร	supabase/functions/coach/index.ts , supabase/functions/parse-slip/index.ts
สี ขนาด ฟอนต์	src/ui/theme.ts , docs/DESIGN_SYSTEM.md
ฟีเจอร์นี้ตอบ requirement ข้อไหน มีทดสอบอะไร	docs/TRACEABILITY.md
ทีมใช้ AI ช่วยเขียนตรงไหน ตรวจอะไรไปแล้ว	docs/AI_USAGE_LOG.md
build / deploy อัตโนมัติ	.github/workflows/*.yaml

★ ถ้าอ่านแล้วงง

ใส่ `console.log('ค่า', ตัวแปร)` ไว้ในโค้ดแล้วเปิดแอปด้วย `npm run web` กด F12 ในเบราว์เซอร์ แถบ Console จะเห็นค่าที่พิมพ์ออกมา (อย่าลืมลบก่อน commit) หรือเขียนเทสต์เล็ก ๆ ลองส่งค่าเข้าฟังก์ชันใน `src/domain` แล้วดูผล

เซิร์ฟเวอร์และฐานข้อมูล: Supabase

ข้อมูลของผู้ใช้เก็บที่ไหน เก็บยังไง แอปต่อเซิร์ฟเวอร์ยังไง และทำไมคนอื่นแอบดูข้อมูลเราไม่ได้ ทั้งที่กุญแจอยู่ในแอป

ฐานข้อมูลคืออะไร

ฐานข้อมูล (database) คือที่เก็บข้อมูลแบบมีระเบียบ คิดว่าเป็น **สมุด Excel บนคลาวด์** ที่มีหลายชีต แต่ละชีตเรียกว่า **ตาราง (table)** แต่ละแถวคือข้อมูล 1 ชั่น (row) แต่ละคอลัมน์คือช่องข้อมูล (column) เช่น ตาราง transactions มี 1 แถวต่อรายการเงิน 1 รายการ

id	user_id	kind	amount_satang	title	status	occurred_at
a1b2...	มินท์	expense	8500	ข้าวมันไก่	confirmed	2026-10-08T05:10Z
c3d4...	มินท์	income	1000000	ค่าขนมจากที่บ้าน	confirmed	2026-10-01T02:00Z
e5f6...	ต้น	expense	125000	โอนให้ นาย ส***	draft	2026-10-07T11:42Z

ตัวอย่างสมมติ คอลัมน์ user_id จริงเป็นรหัสยาว (UUID) ไม่ใช่ชื่อ

ภาษาที่ใช้สั่งฐานข้อมูลคือ **SQL** คำสั่งพื้นฐานมี 4 คำ

SQL	ความหมาย	ในแอปเขียนว่า (supabase-js)
SELECT * FROM transactions	อ่านข้อมูล	db.from('transactions').select('*')
INSERT INTO transactions ...	เพิ่มแถว	db.from('transactions').insert(row)
UPDATE transactions SET ... WHERE id = ...	แก้แถว	db.from('transactions').update(row).eq('id', id)
DELETE FROM transactions WHERE id = ...	ลบแถว	db.from('transactions').delete().eq('id', id)

แอปไม่ได้เขียน SQL ตรง ๆ แต่ใช้ไลบรารี supabase-js ซึ่งแปลงคำสั่งเป็นคำขอผ่านอินเทอร์เน็ต แล้วเซิร์ฟเวอร์แปลงเป็น SQL ให้อีกที

Supabase ให้อะไรกับเราบ้าง

ส่วน	คืออะไร	ใช้ทำอะไรใน MindPay
PostgreSQL	ฐานข้อมูลแบบตาราง ที่นิยมที่สุดตัวหนึ่งของโลก	เก็บ 4 ตาราง: profiles, transactions, savings_goals, ai_usage
Auth	ระบบบัญชีผู้ใช้สำเร็จรูป	สมัคร เข้าสู่ระบบ ส่งรหัส 6 หลักทางอีเมล ลืมรหัสผ่าน

ส่วน	คืออะไร	ใช้ทำอะไรใน MindPay
REST API อัตโนมัติ	สร้าง "ประตู" ให้เข้าถึงตารางผ่านอินเทอร์เน็ทให้เอง	แอปอ่าน/เขียนตารางได้โดยไม่ต้องเขียนเซิร์ฟเวอร์เอง
Row Level Security	กฎความปลอดภัยรายแถว	แต่ละคนเห็นเฉพาะแถวของตัวเอง
Edge Functions	ที่รันโค้ด TypeScript ผังเซิร์ฟเวอร์ (Deno)	parse-slip และ coach ซึ่งถือกุญแจ AI (un A10)
Dashboard	หน้าเว็บจัดการทั้งหมด	ดูตาราง รัน SQL ตั้ง Secrets ดู log

โปรเจกต์ของทีมอยู่ภูมิภาค Singapore (ใกล้ไทย ตอบเร็ว) และใช้แพ็คเกจฟรี

Migration: ไฟล์สร้างฐานข้อมูล

แทนที่จะคลิกสร้างตารางในหน้าเว็บ ทีมเราเขียนคำสั่ง SQL ไว้ในไฟล์ เรียกว่า **migration** เรียงตามวันที่ในชื่อไฟล์ ใครจะสร้างฐานข้อมูลใหม่ก็แค่รันไฟล์ตามลำดับ ได้ฐานข้อมูลเหมือนกันทุกตัว และเก็บประวัติใน Git ได้

ไฟล์	ทำอะไร
<code>supabase/migrations/20260927000000_init.sql</code>	สร้างตารางหลัก 3 ตาราง กฎ RLS และ trigger (27 ก.ย. 2569)
<code>supabase/migrations/20260927000100_harden.sql</code>	แก้ตามคำแนะนำของตัวตรวจความปลอดภัย Supabase Advisor
<code>supabase/migrations/20260929000000_savings_goals.sql</code>	เพิ่มตารางกระปุกออม (FR-6)
<code>supabase/migrations/20261001000000_atomic_ai_quota.sql</code>	ฟังก์ชันนับโควตา AI แบบกันแย่งกัน และเพดานจำนวนเงิน

ตาราง profiles: ผู้ใช้ 1 คน = 1 แถว

supabase/migrations/20260927000000_init.sqlบรรทัด 1-22

```
1 -- MindPay database schema (Supabase / PostgreSQL)
2 -- Run once: Supabase Dashboard -> SQL Editor -> paste -> Run
3 -- or with the CLI: supabase db push
4 --
5 -- Security model: every row belongs to one user and Row Level Security (RLS)
6 -- makes sure a user can only ever read or change their own rows, even though
7 -- the app talks to the database directly with the public (publishable) key.
8
9 -- -----
10 -- profiles: one row per user (created automatically at sign-up)
11 -- -----
12 create table if not exists public.profiles (
13   id uuid primary key references auth.users (id) on delete cascade,
14   display_name text not null default '',
15   opening_balance_satang bigint not null default 0,
16   runway_floor_satang bigint not null default 50000 check (runway_floor_satang >= 0), -- FR-6 low
17   line, default #500
```

```

17 monthly_budget_satang bigint check (monthly_budget_satang is null or monthly_budget_satang >
0),
18 coach_tone text not null default 'friend' check (coach_tone in ('friend', 'coach', 'senior')),
-- FR-5
19 onboarded boolean not null default false,
20 created_at timestamptz not null default now(),
21 updated_at timestamptz not null default now()
22 );

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → supabase/migrations/20260927000000_init.sql:1

บรรทัด	ความหมาย
1-7	คอมเมนต์ SQL (ขึ้นต้นด้วย --) อธิบายวิธีรัน และหลักความปลอดภัย: ทุกแถวเป็นของผู้ใช้คนเดียว RLS ทำให้อ่านแก้ไขได้เฉพาะของตัวเอง แม้แอปจะคุยกับฐานข้อมูลตรง ๆ ด้วยคุณดูแลสาธารณะ
12	สร้างตาราง profiles ในส่วน public (ถ้ายังไม่มี)
13	id ชนิด uuid (รหัสยาวไม่ซ้ำ) เป็น primary key (ตัวระบุแถว) และ อ้างอิง ผู้ใช้ในระบบ Auth ถ้าลบผู้ใช้ โพรไฟล์ถูกลบตาม (on delete cascade)
14	ชื่อเล่น ห้ามว่าง (not null) ค่าเริ่มต้นเป็นข้อความว่าง
15	เงินตอนเริ่มใช้แอป เป็นสตางค์ ชนิด bigint (จำนวนเต็มขนาดใหญ่)
16	เส้นเงินสำรองของ FR-6 ค่าเริ่มต้น 50,000 สตางค์ (฿500) และมี check บังคับว่าต้อง ≥ 0 ถ้าแอปส่งค่าติดลบมา ฐานข้อมูลปฏิเสธเอง
17	งบท่อเดือน ไม่ตั้งก็ได้ (null) ถ้าตั้งต้องมากกว่า 0
18	โทนโค้ช (ของเดิม FR-5) ต้องเป็น 1 ใน 3 ค่าเท่านั้น
19	ตั้งค่าเงินครั้งแรกเสร็จหรือยัง (ใช้กับ Stack.Protected ใน un A7)
20-21	เวลาสร้างและแก้ไขล่าสุด ใส่เวลาปัจจุบันให้เอง (now())

ตาราง transactions: รายการเงิน (FR-1)

supabase/migrations/20260927000000_init.sql

บรรทัด 24-53

```

24 -- -----
25 -- transactions (FR-1). Money is stored in satang (integer) to avoid rounding errors.
26 -- -----
27 create table if not exists public.transactions (
28   id uuid primary key default gen_random_uuid(),
29   user_id uuid not null default auth.uid() references auth.users (id) on delete cascade,
30   kind text not null check (kind in ('income', 'expense')),
31   amount_satang bigint not null check (amount_satang > 0),
32   category_key text not null,
33   title text not null default '' check (char_length(title) <= 120),
34   note text check (note is null or char_length(note) <= 500),
35   occurred_at timestamptz not null,
36   source text not null default 'manual' check (source in ('manual', 'slip')),
37   status text not null default 'confirmed' check (status in ('draft', 'confirmed')),
38   slip_ref text, -- FR-4 duplicate key #1 (QR / printed reference)
39   slip_image_hash text, -- FR-4 duplicate key #2 (SHA-256 of the image)
40   ocr_confidence numeric(3, 2) check (ocr_confidence is null or (ocr_confidence >= 0 and
ocr_confidence <= 1)),

```

```

41 review_flags text[] not null default '{}',
42 created_at timestamptz not null default now(),
43 updated_at timestamptz not null default now()
44 );
45
46 create index if not exists transactions_user_time_idx on public.transactions (user_id,
    occurred_at desc);
47
48 -- The database itself refuses a second copy of the same slip, even if two
49 -- phones scan at the same moment (FR-4 "ตรวจสอบรายการซ้ำ").
50 create unique index if not exists transactions_user_slip_ref_uidx
51 on public.transactions (user_id, slip_ref) where slip_ref is not null;
52 create unique index if not exists transactions_user_slip_hash_uidx
53 on public.transactions (user_id, slip_image_hash) where slip_image_hash is not null;

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → supabase/migrations/20260927000000_init.sql:24

บรรทัด	ความหมาย
28	id สร้างให้อัตโนมัติด้วยฟังก์ชันสุ่ม (gen_random_uuid())
29	user_id เจ้าของรายการ ค่าเริ่มต้นคือ auth.uid() = id ของคนที่ส่งคำขอมา แอปจึงไม่ต้องส่ง user_id เอง (และปลอมเป็นคนอื่นไม่ได้)
30	ประเภทต้องเป็น income หรือ expense เท่านั้น (กฎเดียวกับ TxKind ในแอป)
31	จำนวนเงินเป็นสตางค์ ต้องมากกว่า 0
32	รหัสหมวด
33–34	ชื่อยาวไม่เกิน 120 ตัวอักษร บันทึกไม่เกิน 500 (ตรงกับ maxLength ในหน้าจอดรายการ)
35	เวลาที่เกิดรายการ ชนิด timestamptz (เวลาพร้อมเขตเวลา)
36–37	ที่มา (manual/slip) และสถานะ (draft/confirmed) ค่าเริ่มต้นคือยืนยันแล้ว
38–39	เลขอ้างอิงสลิป และ hash ของรูป สำหรับกันซ้ำ (FR-4)
40	ความมั่นใจของ AI ตัวเลข 0.00 ถึง 1.00
41	ช่องที่ต้องตรวจ เป็น array ของข้อความ (text[])
46	index = สารบัญของตาราง ช่วยให้ค้น "รายการของผู้ใช้คนนี้ เรียงตามเวลาล่าสุด" ได้เร็ว ไม่ต้องไล่ทุกแถว
48–53	unique index = ห้ามซ้ำ: ผู้ใช้คนเดียวกันจะมีเลขอ้างอิงสลิปซ้ำกันไม่ได้ และ hash รูปซ้ำไม่ได้ ถ้ามือถือ 2 เครื่องสแกนสลิปใบเดียวกันพร้อมกัน ฐานข้อมูลปฏิเสธใบที่ 2 เอง (ตอบรหัส 23505 ที่ repo.ts แปลงเป็น DuplicateSlipError)

ตาราง ai_usage: นับการใช้ AI

supabase/migrations/20260927000000_init.sql

บรรทัด 55–65

```

55 -- -----
56 -- ai_usage: counts calls to the AI so one account cannot run up the bill.
57 -- Written only by the Edge Functions (service role), readable by the owner.
58 -- -----
59 create table if not exists public.ai_usage (
60   id bigserial primary key,
61   user_id uuid not null references auth.users (id) on delete cascade,

```

```

62 kind text not null check (kind in ('slip', 'coach')),
63 created_at timestamptz not null default now()
64 );
65 create index if not exists ai_usage_user_day_idx on public.ai_usage (user_id, kind, created_at
desc);

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → supabase/migrations/20260927000000_init.sql:55

ทุกครั้งที่ AI ถูกเรียก (อ่านสลิป 1 ใบ หรือถามโค้ช 1 ครั้ง) Edge Function จะเพิ่ม 1 แถว เพื่อบันทึกว่าวันนี้ใช้ไปกี่ครั้งแล้ว กัน บัญชีเดียวใช้ AI จนค่าใช้จ่ายบานปลาย (ผู้ใช้อ่านของตัวเองได้ แต่เขียนได้เฉพาะเซิร์ฟเวอร์)

Row Level Security (RLS): หัวใจความปลอดภัย

≈ เปรียบเทียบ: ตู้ล็อกเกอร์ในห้องสมุด

กุญแจสาธารณะ (publishable key) ในแอปเหมือน "บัตรเข้าห้องสมุด" ใครมีก็เดินเข้าห้องได้ แต่ละคนมีล็อกเกอร์ของตัวเอง RLS คือ "ยามประจำล็อกเกอร์" ที่ดูบัตรประชาชน (การเข้าสู่ระบบ) ทุกครั้ง และเปิดให้เฉพาะล็อกเกอร์ที่มีชื่อเรา แม้แขกเกอร์จะโยมบัตรเข้าห้องสมุด (กุญแจในแอป) ไปได้ ก็เปิดล็อกเกอร์คนอื่นไม่ได้

supabase/migrations/20260927000000_init.sql

บรรทัด 67-87

```

67 -- -----
68 -- Row Level Security
69 -- -----
70 alter table public.profiles enable row level security;
71 alter table public.transactions enable row level security;
72 alter table public.ai_usage enable row level security;
73
74 drop policy if exists "own profile read" on public.profiles;
75 create policy "own profile read" on public.profiles
76   for select using (auth.uid() = id);
77 drop policy if exists "own profile update" on public.profiles;
78 create policy "own profile update" on public.profiles
79   for update using (auth.uid() = id) with check (auth.uid() = id);
80
81 drop policy if exists "own transactions" on public.transactions;
82 create policy "own transactions" on public.transactions
83   for all using (auth.uid() = user_id) with check (auth.uid() = user_id);
84
85 drop policy if exists "own ai usage read" on public.ai_usage;
86 create policy "own ai usage read" on public.ai_usage
87   for select using (auth.uid() = user_id);

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → supabase/migrations/20260927000000_init.sql:67

บรรทัด	ความหมาย
70-72	เปิด RLS ทั้ง 3 ตาราง พอเปิดแล้ว ค่าเริ่มต้นคือ ห้ามทุกอย่าง ต้องมี policy อนุญาตทีละเรื่อง
74-76	policy "own profile read": อ่านโปรไฟล์ได้ เฉพาะแถวที่ id ตรงกับคนที่เข้าสู่ระบบ (auth.uid() = id) drop policy if exists ด้านบนคือลบของเก่าก่อน ทำให้รันไฟล์ซ้ำได้
77-79	แก้โปรไฟล์ได้เฉพาะของตัวเอง using = ตรวจสอบแถวเดิม, with check = ตรวจสอบแถวหลังแก้ (กันแค่ id ให้เป็นของคนอื่น) ไม่มี policy insert/delete เพราะโปรไฟล์สร้างโดย trigger เท่านั้น

บรรทัด	ความหมาย
81–83	รายการเงิน: ทำได้ทุกอย่าง (for all = อ่าน เพิ่ม แก้ ลบ) เฉพาะแถวที่ user_id เป็นของตัวเอง
85–87	ai_usage อ่านได้อย่างเดียวและเฉพาะของตัวเอง

ทีมทดสอบเรื่องนี้ด้วยแผน MT-10 "บัญชี A ไม่เห็นรายการของบัญชี B" และรับตัวตรวจ Security Advisor ของ Supabase แล้วแก้ไขไม่เหลือคำเตือนเรื่องตาราง ไฟล์ `harden.sql` เปลี่ยน `auth.uid()` เป็น `(select auth.uid())` เพื่อให้ฐานข้อมูลคำนวณครั้งเดียวต่อคำสั่ง แทนที่จะคำนวณทุกแถว (เร็วขึ้นเมื่อมีรายการเยอะ)

supabase/migrations/20260927000100_harden.sql
บรรทัด 1–25

```

1 -- Hardening after running the Supabase security/performance advisors on the first migration.
2
3 -- 1. Fixed search_path on trigger functions (advisor 0011).
4 create or replace function public.touch_updated_at() returns trigger
5 language plpgsql set search_path = '' as $$
6 begin
7     new.updated_at := now();
8     return new;
9 end $$;
10
11 -- 2. The sign-up trigger must not be callable through the REST API (advisors 0028/0029).
12 revoke execute on function public.handle_new_user() from public, anon, authenticated;
13
14 -- 3. Evaluate auth.uid() once per query instead of once per row (advisor 0003).
15 drop policy if exists "own profile read" on public.profiles;
16 create policy "own profile read" on public.profiles
17     for select using ((select auth.uid()) = id);
18
19 drop policy if exists "own profile update" on public.profiles;
20 create policy "own profile update" on public.profiles
21     for update using ((select auth.uid()) = id) with check ((select auth.uid()) = id);
22
23 drop policy if exists "own transactions" on public.transactions;
24 create policy "own transactions" on public.transactions
25     for all using ((select auth.uid()) = user_id) with check ((select auth.uid()) = user_id);

```

▶ เปิดใน VS Code (เร็ว)
GitHub
VS Code ในเครื่อง: Ctrl+P → supabase/migrations/20260927000100_harden.sql:1

Trigger: งานที่ฐานข้อมูลทำเองอัตโนมัติ

supabase/migrations/20260927000000_init.sql
บรรทัด 89–118

```

89 -- -----
90 -- Triggers
91 -- -----
92 create or replace function public.touch_updated_at() returns trigger
93 language plpgsql as $$
94 begin
95     new.updated_at := now();
96     return new;
97 end $$;
98
99 drop trigger if exists profiles_touch on public.profiles;
100 create trigger profiles_touch before update on public.profiles
101     for each row execute function public.touch_updated_at();

```

```

102 drop trigger if exists transactions_touch on public.transactions;
103 create trigger transactions_touch before update on public.transactions
104   for each row execute function public.touch_updated_at();
105
106 -- Create the profile row when a user signs up.
107 create or replace function public.handle_new_user() returns trigger
108 language plpgsql security definer set search_path = public as $$
109 begin
110   insert into public.profiles (id, display_name)
111   values (new.id, coalesce(new.raw_user_meta_data ->> 'display_name', ''))
112   on conflict (id) do nothing;
113   return new;
114 end $$;
115
116 drop trigger if exists on_auth_user_created on auth.users;
117 create trigger on_auth_user_created after insert on auth.users
118   for each row execute function public.handle_new_user();

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → supabase/migrations/20260927000000_init.sql:89

บรรทัด	ความหมาย
92–97	ฟังก์ชัน touch_updated_at (เขียนด้วยภาษา plpgsql ของ PostgreSQL) ตั้งเวลาแก้ไขล่าสุดเป็นตอนนี้
99–104	trigger = "ทุกครั้งก่อนแก้ไขในตาราง profiles หรือ transactions ให้เรียกฟังก์ชันนั้น" updated_at จึงถูกต้องเสมอ ไม่ต้องพึ่งแอป
107–114	ฟังก์ชัน handle_new_user: สร้างแถวโปรไฟล์ใหม่ โดยเอาชื่อเล่นที่ผู้ใช้กรอกตอนสมัคร (เก็บใน raw_user_meta_data) ถ้าไม่มีใช้ข้อความว่าง และถ้ามีแถวนี้อยู่แล้วไม่ต้องทำ (on conflict do nothing) security definer = รันด้วยสิทธิ์ของเจ้าของฟังก์ชัน (เพราะผู้ใช้ไม่มีสิทธิ์ insert โปรไฟล์เอง)
116–118	trigger บนตารางผู้ใช้ของระบบ Auth: หลังจากมีคนสมัครใหม่ ให้สร้างโปรไฟล์ทันที แอปจึงไม่ต้องสร้างเอง

กฎแฉ 2 แบบ: อันไหนอยู่ในแอปได้ อันไหนห้าม

	Publishable key (anon)	Service role key (secret)
อยู่ที่ไหน	ในแอป (.env, eas.json) เผยแพร่ได้	บนเซิร์ฟเวอร์ใน Edge Functions เท่านั้น (Supabase ใส่ให้เอง)
ทำอะไรได้	ทุกคำขอต้องผ่าน RLS เห็นเฉพาะข้อมูลของคน que เข้าสู่ระบบ	ข้าม RLS ได้ทั้งหมด อ่านเขียนทุกแถว
ใช้ทำอะไร	แอปอ่าน/เขียนรายการของผู้ใช้	ตรวจตัวตนผู้ใช้ และนับโควตา AI ใน Edge Function
ถ้ารั่ว	ไม่เป็นไร (ออกแบบมาให้เปิดเผยได้)	อันตรายมาก ต้องเปลี่ยนกฎแฉทันที

กฎแฉ AI (Claude หรือ Gemini) ก็เป็นความลับระดับเดียวกับ service role เก็บใน Supabase → Edge Functions → Secrets เท่านั้น

แอปต่อ Supabase ยิงใจ: src/data/supabase.ts

src/data/supabase.ts

บรรทัด 1-38

```
1 /**
2  * Supabase client. Keys come from environment variables (see .env.example).
3  * The publishable key is safe to ship in the app: Row Level Security in the
4  * database decides what each signed-in user can read and write.
5  * AI keys are NOT here; they live only in the Edge Functions.
6  *
7  * detectSessionInUrl is off: email links are handled by AppProvider (via
8  * authLinks.ts) the same way on the web and in the phone app.
9  */
10 import './sessionStorageSetup';
11 import './authLinks'; // records the address the web page was opened with
12 import { createClient, type SupabaseClient } from '@supabase/supabase-js';
13 import { AppState } from 'react-native';
14
15 const url = process.env.EXPO_PUBLIC_SUPABASE_URL;
16 const key = process.env.EXPO_PUBLIC_SUPABASE_PUBLISHABLE_KEY;
17
18 /** null when the app was built without Supabase settings: only demo mode is available. */
19 export const supabase: SupabaseClient | null =
20   url && key
21     ? createClient(url, key, {
22       auth: {
23         storage: localStorage,
24         autoRefreshToken: true,
25         persistSession: true,
26         detectSessionInUrl: false,
27       },
28     })
29     : null;
30
31 if (supabase) {
32   AppState.addEventListener('change', (state) => {
33     if (state === 'active') supabase.auth.startAutoRefresh();
34     else supabase.auth.stopAutoRefresh();
35   });
36 }
37
38 export const cloudAvailable = supabase !== null;
```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/supabase.ts:1

บรรทัด	ความหมาย
1-9	คอมเมนต์: คุญแจมาจากตัวแปรสภาพแวดล้อม (.env) คุญแจ publishable ปลอดภัยเพราะ RLS และคุญแจ AI ไม่อยู่ที่นี่
10	ตั้งที่เก็บการเข้าสู่ระบบ (บนมือถือใช้ SQLite เก็บไว้ ปิดแอปแล้วยังเข้าสู่ระบบอยู่)
11	จดที่อยู่ของหน้าเว็บตอนเปิด (ไว้จัดการลิงก์ยืนยันอีเมล)
15-16	อ่านที่อยู่เซิร์ฟเวอร์และคุญแจจาก .env (process.env.EXPO_PUBLIC_...)
18-29	ถ้ามีทั้งสองค่า สร้าง "client" สำหรับคุยกับ Supabase ถ้าไม่มี (build แบบไม่มีการตั้งค่า) ได้ null แล้วแอปใช้ได้เฉพาะโหมดทดลอง

บรรทัด	ความหมาย
23–26	ตั้งค่าการเข้าสู่ระบบ: เก็บ session ไว้ (persistSession), ต่ออายุ token อัตโนมัติ (token ของ Supabase มีอายุสั้น ค่าปกติคือ 1 ชั่วโมง), และไม่ให้อิสรารอ่านลิงก์ในแถบที่อยู่เอง (AuthProvider จัดการเองเพื่อให้เว็บกับแอปทำงานเหมือนกัน)
31–36	ต่ออายุ token เฉพาะตอนแอปเปิดอยู่บนหน้าจอ พอแอปถูกพับไว้หยุดต่ออายุ (ประหยัดแบตเตอรี่)
38	บอกส่วนอื่นของแอปว่าใช้บัญชีจริงได้ไหม

การสมัครและเข้าสู่ระบบ

1. กรอกชื่อเล่น อีเมล รหัสผ่าน ≥ 8 ตัว [src/app/welcome.tsx](#)

↓ `supabase.auth.signUp`

2. Supabase สร้างผู้ใช้

trigger `handle_new_user` สร้างแถวโปรไฟล์ให้ทันที แล้วส่งอีเมลที่มีรหัส 6 หลักและลิงก์ยืนยัน

↓ `supabase.auth.verifyOtp` (กรอกรหัส) หรือแตะลิงก์ในอีเมล

3. ได้ session

แอปเก็บ token ไว้ในเครื่อง AppProvider เห็นเหตุการณ์ `SIGNED_IN` แล้วสร้าง `CloudRepo` โหลดข้อมูล

↓

4. โปรไฟล์ยังไม่ onboarded

`Stack.Protected` พาไปหน้าตั้งค่าเงิน กรอกรายละเอียดตอนนี้และเงินสำรอง แล้วเข้าหน้าหลัก

`src/app/welcome.tsx`

บรรทัด 148–172

```

148     const displayName = name.trim();
149     const { data, error } = await supabase!.auth.signUp({
150       email: address,
151       password,
152       options: { data: { display_name: displayName }, emailRedirectTo: authRedirectUrl() },
153     });
154     if (error) return setNotice({ text: authErrorMessage(error), tone: 'error' });
155     if (isExistingAccountSignUp(data.user)) {
156       setMode('signin');
157       setPassword('');
158       setNotice({
159         text: 'อีเมลนี้มีบัญชีอยู่แล้ว เข้าสู่ระบบด้วยรหัสผ่านเดิม หรือกด "ลืมรหัสผ่าน" เพื่อตั้งใหม่',
160         tone: 'info',
161         action: 'forgot',
162       });
163       return;
164     }
165     if (data.session) {
166       showAuthNotice({ kind: 'signed_up', name: displayName });
167       resetForm();
168       return;
169     }
170     setSent({ kind: 'confirm', email: address, name: displayName });
171     setResendIn(RESEND_WAIT_S);

```

บรรทัด	ความหมาย
148	ตัดช่องว่างหัวท้ายของชื่อเล่น
149–153	ส่งสมัคร: ส่งอีเมล รหัสผ่าน ชื่อเล่น (เก็บใน metadata ของผู้ใช้ ซึ่ง trigger จะคัดลอกไปใส่โปรไฟล์) และที่อยู่ลิงก์ในอีเมลจะพากลับมา
154	ถ้า error (เช่น รหัสผ่านสั้นเกินไป) แสดงข้อความภาษาไทยที่บอกวิธีแก้
155–164	กรณีพิเศษ: อีเมลนี้เคยสมัครแล้ว Supabase จะตอบเหมือนสำเร็จ (เพื่อไม่ให้คนนอกรู้ว่าอีเมลไหนมีบัญชี) แอปตรวจจบได้แล้วบอกให้ไปเข้าสู่ระบบหรือกดลิ้งรหัสผ่าน แทนที่จะขึ้น "สมัครสำเร็จ" หลอก ๆ
165–169	ถ้าโปรเจกต์ปิดการยืนยันอีเมลไว้ จะได้ session ทันที ขึ้นหน้าต่าง "สมัครบัญชีสำเร็จ"
170–171	ไม่ขึ้นไปหน้ากรอกรหัส 6 หลัก และเริ่มนับถอยหลังก่อนให้กดส่งรหัสใหม่ได้

Repo: ชั้นที่ซ่อนว่าข้อมูลอยู่ที่ไหน

หน้าจอไม่เคยคุยกับ Supabase หรือที่เก็บในเครื่องโดยตรง แต่คุยผ่าน **Repo** (ย่อจาก repository แปลว่าคลัง) ซึ่งมี 2 แบบที่หน้าตาเหมือนกันทุกประการ

src/data/repo.ts

บรรทัด 1–38

```

1  /**
2   * Data access. The screens never talk to Supabase or storage directly; they use
3   * a Repo. Two implementations:
4   *   - CloudRepo: Supabase (real accounts, data in PostgreSQL with RLS)
5   *   - DemoRepo: on-device only, starts with sample data (for trying the app
6   *                 without an account and as a fallback during a presentation)
7   */
8  import { Storage } from '../storage';
9  import { buildSampleTransactions, DEMO_OPENING_BALANCE_SATANG } from '../domain/sample';
10 import type { Profile, Transaction, TransactionInput } from '../domain/types';
11 import type { GoalInput, SavingsGoal } from '../domain/goals';
12 import { supabase } from '../supabase';
13
14 export class DuplicateSlipError extends Error {
15   constructor() {
16     super('รายการนี้มีอยู่แล้ว');
17     this.name = 'DuplicateSlipError';
18   }
19 }
20
21 export interface Repo {
22   mode: 'cloud' | 'demo';
23   getProfile(): Promise<Profile>;
24   updateProfile(patch: Partial<Omit<Profile, 'id'>>): Promise<Profile>;
25   /**
26    * Transactions of about the last 13 months, plus the net of confirmed money before them
27    * (it still counts in the balance, but is not needed as a list).
28    */
29   listTransactions(): Promise<{ txs: Transaction[]; carrySatang: number }>;
30   insert(input: TransactionInput): Promise<Transaction>;

```

```

31 update(id: string, patch: Partial<TransactionInput>): Promise<Transaction>;
32 remove(id: string): Promise<void>;
33 confirmMany(ids: string[]): Promise<void>;
34 listGoals(): Promise<SavingsGoal[]>;
35 insertGoal(input: GoalInput): Promise<SavingsGoal>;
36 updateGoal(id: string, patch: Partial<GoalInput>): Promise<SavingsGoal>;
37 removeGoal(id: string): Promise<void>;
38 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/repo.ts:1

บรรทัด	ความหมาย
1-7	คอมเมนต์: หน้าจอใช้ Repo เสมอ มี 2 ตัว คือ CloudRepo (Supabase, บัญชีจริง) และ DemoRepo (เก็บในเครื่อง มีข้อมูลตัวอย่าง ใช้ลองแอปหรือสำรองตอนพรีเซนต์)
14-19	error เฉพาะสำหรับสลิปซ้ำ (อธิบายในบท A5)
21	interface Repo = สัญญาว่า Repo ทุกตัวต้องทำสิ่งเหล่านี้ได้
22	บอกว่าเป็นแบบไหน
23-24	อ่านและแก้ไขไฟล์
25-29	โหลดรายการประมาณ 13 เดือนล่าสุด และ "ยอดยกมา" ของรายการที่เก่ากว่านั้น (ยังต้องนับในยอดคงเหลือ แต่ไม่ต้องโหลดมาเป็นรายการ)
30-33	เพิ่ม แก้ ลบ รายการ และยืนยันหลายรายการพร้อมกัน (สลิปรอยืนยัน)
34-37	คำสั่งของกระปุกออม (FR-6)

≈ ทำไมต้องมี 2 แบบ

เหมือนปลั๊กไฟ: หน้าจอคือเครื่องใช้ไฟฟ้า Repo คือเต้าเสียบ จะต่อกับไฟบ้าน (Supabase) หรือแบตเตอรี่สำรอง (เครื่อง) หน้าจอไม่ต้องรู้ เสียบแล้วใช้ได้เหมือนกัน ทีมเราจึงพรีเซนต์ได้แม้อินเทอร์เน็ตในห้องเรียนล่ม และ E2E ทดสอบได้โดยไม่แตะบัญชีจริง

แปลงชื่อระหว่างแอปกับฐานข้อมูล

src/data/repo.ts

บรรทัด 44-93

```

44 type TxRow = {
45   id: string;
46   kind: 'income' | 'expense';
47   amount_satang: number;
48   category_key: string;
49   title: string;
50   note: string | null;
51   occurred_at: string;
52   source: 'manual' | 'slip';
53   status: 'draft' | 'confirmed';
54   slip_ref: string | null;
55   slip_image_hash: string | null;
56   ocr_confidence: number | null;
57   review_flags: string[] | null;

```

```

58   created_at: string;
59 };
60
61 const fromRow = (r: TxRow): Transaction => ({
62   id: r.id,
63   kind: r.kind,
64   amountSatang: Number(r.amount_satang),
65   categoryKey: r.category_key,
66   title: r.title,
67   note: r.note,
68   occurredAt: r.occurred_at,
69   source: r.source,
70   status: r.status,
71   slipRef: r.slip_ref,
72   slipImageHash: r.slip_image_hash,
73   ocrConfidence: r.ocr_confidence === null ? null : Number(r.ocr_confidence),
74   reviewFlags: r.review_flags ?? [],
75   createdAt: r.created_at,
76 });
77
78 function toRow(p: Partial<TransactionInput>) {
79   const row: Record<string, unknown> = {};
80   if (p.kind !== undefined) row.kind = p.kind;
81   if (p.amountSatang !== undefined) row.amount_satang = p.amountSatang;
82   if (p.categoryKey !== undefined) row.category_key = p.categoryKey;
83   if (p.title !== undefined) row.title = p.title;
84   if (p.note !== undefined) row.note = p.note;
85   if (p.occurredAt !== undefined) row.occurred_at = p.occurredAt;
86   if (p.source !== undefined) row.source = p.source;
87   if (p.status !== undefined) row.status = p.status;
88   if (p.slipRef !== undefined) row.slip_ref = p.slipRef;
89   if (p.slipImageHash !== undefined) row.slip_image_hash = p.slipImageHash;
90   if (p.ocrConfidence !== undefined) row.ocr_confidence = p.ocrConfidence;
91   if (p.reviewFlags !== undefined) row.review_flags = p.reviewFlags;
92   return row;
93 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/repo.ts:44

บรรทัด	ความหมาย
44–59	TxRow = หน้าตาของแถวในฐานข้อมูล ใช้ชื่อแบบ snake_case (คั่นด้วยขีดล่าง) ตามธรรมเนียมของ SQL
61–76	fromRow: แถวฐานข้อมูล → Transaction ของแอป ใช้ Number(. . .) ครอบตัวเลข เพราะ bigint บางครั้งส่งมาเป็นข้อความ และ review_flags ที่เป็น null ให้เป็น array ว่าง
78–93	toRow: กลับทาง ใส่เฉพาะช่องที่มีค่า (ไม่ใช่ undefined) เพราะตอนแก้รายการ เราส่งแค่บางช่อง ถ้าใส่ทุกช่อง ช่องที่ไม่ได้ส่งจะถูกเขียนทับเป็นค่าว่าง

โหลดรายการจากคลาวด์

src/data/repo.ts

บรรทัด 166–216

```

166 export function createCloudRepo(userId: string): Repo {
167   const db = supabase!;
168   return {
169     mode: 'cloud',
170     async getProfile() {
171       const { data, error } = await db.from('profiles').select('*').eq('id', userId).single();

```

```

172     if (error) throw error;
173     return profileFromRow(data as ProfileRow);
174 },
175 async updateProfile(patch) {
176     const { data, error } = await db.from('profiles').update(profileToRow(patch)).eq('id',
userId).select().single();
177     if (error) throw error;
178     return profileFromRow(data as ProfileRow);
179 },
180 async listTransactions() {
181     // Up to ~13 months: enough for the 1-year range. Paged to stay under the API row limit.
182     const since = new Date(Date.now() - 400 * 24 * 3600 * 1000).toISOString();
183     const all: Transaction[] = [];
184     for (let from = 0; ; from += 1000) {
185         const { data, error } = await db
186             .from('transactions')
187             .select('*')
188             .gte('occurred_at', since)
189             .order('occurred_at', { ascending: false })
190             // A unique tiebreaker, so rows with the same time are never skipped or read twice
across pages.
191             .order('id', { ascending: true })
192             .range(from, from + 999);
193         if (error) throw error;
194         all.push...(data as TxRow[]).map(fromRow);
195         if (!data || data.length < 1000) break;
196     }
197     // Older confirmed money still counts in the balance: add it up (two columns only).
198     let carrySatang = 0;
199     for (let from = 0; ; from += 1000) {
200         const { data, error } = await db
201             .from('transactions')
202             .select('kind,amount_satang')
203             .eq('status', 'confirmed')
204             .lt('occurred_at', since)
205             .order('occurred_at', { ascending: false })
206             // A unique tiebreaker, so rows with the same time are never skipped or read twice
across pages.
207             .order('id', { ascending: true })
208             .range(from, from + 999);
209         if (error) throw error;
210         for (const r of (data ?? []) as Pick<TxRow, 'kind' | 'amount_satang'>[]) {
211             carrySatang += r.kind === 'income' ? Number(r.amount_satang) : -
Number(r.amount_satang);
212         }
213         if (!data || data.length < 1000) break;
214     }
215     return { txs: all, carrySatang };
216 },

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/repo.ts:166

บรรทัด	ความหมาย
166–169	สร้าง CloudRepo ของผู้ใช้คนนี้ supabase! (เครื่องหมายตกใจ) บอก TypeScript ว่า "มั่นใจว่าไม่ใช่ null"
170–174	อ่านโปรไฟล์: จากตาราง profiles เลือกทุกคอลัมน์ ที่ id เท่ากับผู้ใช้ เอาแถวเดียว (RLS ก็กันไว้อีกชั้นอยู่แล้ว)
175–179	แก้ไขโปรไฟล์แล้วขอแถวใหม่กลับมา

บรรทัด	ความหมาย
182	คำนวณวันที่ย้อนหลัง 400 วัน (ประมาณ 13 เดือน พอสำหรับช่วง "1 ปี" ในหน้าหลัก)
184–196	โหลดทีละหน้า หน้าละ 1,000 แถว (API จำกัดจำนวนแถวต่อครั้ง) เรียงจากใหม่ไปเก่า ถ้าได้น้อยกว่า 1,000 แปลว่าหมดแล้ว หยุด
190–191	เรียงด้วย id เป็นตัวตัดสินรอง ถ้ามีหลายรายการเวลาเดียวกัน จะไม่หลุดหรือซ้ำข้ามหน้า (บ๊ีกที่ทีมเจอและแก้ตอนตรวจทั้ง repo)
197–214	ขอยกมา: รายการที่ ยืนยันแล้ว และเก่ากว่า 400 วัน โหลดมาแค่ 2 คอลัมน์ (ประเภทและจำนวน) แล้วรวมเป็นตัวเลขเดียว รายรับบวก รายจ่ายลบ ยอดคงเหลือจึงยังถูกต้องแม้ผู้ใช้ใช้แอปมาหลายปี

โมดกทดลอง: DemoRepo

src/data/repo.ts

บรรทัด 264–305

```

264 // v2: rebalanced sample data (v1 could show a negative balance). Old v1 data is simply ignored.
265 const DEMO_KEY = 'mindpay.demo.v2';
266
267 interface DemoState {
268   profile: Profile;
269   txs: Transaction[];
270   goals?: SavingsGoal[];
271 }
272
273 const demoProfile: Profile = {
274   id: 'demo',
275   displayName: 'ผู้ทดลองใช้',
276   openingBalanceSatang: DEMO_OPENING_BALANCE_SATANG,
277   runwayFloorSatang: 50_000,
278   monthlyBudgetSatang: 1_000_000,
279   coachTone: 'friend',
280   onboarded: true,
281 };
282
283 const newId = () => `local-${Date.now().toString(36)}-${Math.random().toString(36).slice(2, 8)}`;
284
285 export async function resetDemoData() {
286   await Storage.removeItem(DEMO_KEY);
287 }
288
289 export function createDemoRepo(): Repo {
290   let cache: DemoState | null = null;
291
292   async function load(): Promise<DemoState> {
293     if (cache) return cache;
294     try {
295       const raw = await Storage.getItem(DEMO_KEY);
296       if (raw) cache = JSON.parse(raw) as DemoState;
297     } catch {
298       cache = null;
299     }
300     if (!cache) {
301       cache = { profile: demoProfile, txs: buildSampleTransactions() };
302       await save();
303     }
304     return cache;

```

บรรทัด 265: ข้อมูลโหมดทดลองเก็บในเครื่องด้วยชื่อ `mindpay.demo.v2` (v2 เพราะข้อมูลชุดแรกเคยทำให้ยอดติดลบ จึงเปลี่ยนชุดใหม่) บรรทัด 273–281 คือโปรไฟล์ของผู้ทดลองใช้: เงินตั้งต้น ฿3,000 เลื่อนสำรอง ฿500 จบ ฿10,000 บรรทัด 292–305: ครั้งแรกที่เปิดยังไม่มีข้อมูล จะสร้างรายการตัวอย่าง (`buildSampleTransactions` ใน [src/domain/sample.ts](#)) ประมาณ 75 วันย้อนหลังจากวันนี้ แล้วเก็บไว้ ข้อมูลตัวอย่างจึงสดใหม่เสมอไม่ว่าเปิดวันไหน

เปิดแอปแล้วเกิดอะไรขึ้น: AppProvider เลือกโหมด

src/data/AppProvider.tsx

บรรทัด 271–318

```

271 // Decide the starting mode once, then follow Supabase auth changes.
272 useEffect(() => {
273   let cancelled = false;
274   (async () => {
275     // An email link that opened the app wins over the saved mode.
276     if (await handleAuthLink(await openingLink())) return;
277     const mode = await Storage.getItem(MODE_KEY).catch(() => null);
278     if (mode === 'demo') {
279       if (!cancelled) await activateRepo(createDemoRepo(), 'demo');
280       return;
281     }
282     if (!supabase) {
283       if (!cancelled) setStatus('signedOut');
284       return;
285     }
286     const { data } = await supabase.auth.getSession();
287     if (cancelled) return;
288     const id = data.session?.user.id;
289     if (id) {
290       if (activeUserRef.current !== id) await activateRepo(createCloudRepo(id), id);
291     } else if (!activeUserRef.current) setStatus('signedOut');
292   })();
293
294   const sub = supabase?.auth.onAuthStateChange((event, session) => {
295     // Supabase warns against calling other supabase methods inside this callback
296     // (it can deadlock the auth lock), so the actual work is deferred.
297     setTimeout(() => {
298       // A password-reset code signs in with the PASSWORD_RECOVERY event instead of SIGNED_IN.
299       // Opened offline with an expired token, the session only comes back later as
300       // TOKEN_REFRESHED:
301       // that signs in too, but only while nothing is open (never over the demo or another
302       // account).
303       const signedIn =
304         event === 'SIGNED_IN' || event === 'PASSWORD_RECOVERY' || (event === 'TOKEN_REFRESHED'
305         && !activeUserRef.current);
306       if (signedIn && session && activeUserRef.current !== session.user.id) {
307         Storage.setItem(MODE_KEY, 'cloud').catch(() => {});
308         activateRepo(createCloudRepo(session.user.id), session.user.id);
309       }
310       if (event === 'SIGNED_OUT' && repoRef.current?.mode === 'cloud') clear();
311     }, 0);
312   });
313
314   const stopLinks = onIncomingLink((url) => {
315     handleAuthLink(url);
316   });

```

```

313     return () => {
314         cancelled = true;
315         sub?.data.subscription.unsubscribe();
316         stopLinks();
317     };
318 }, [activateRepo, clear, handleAuthLink]);

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/data/AppProvider.tsx:271`

บรรทัด	ความหมาย
272	ทำครั้งเดียวตอนแอปเริ่ม
275–276	ถ้าแอปถูกเปิดจากลิงก์ในอีเมล (ยืนยันสมัคร/ตั้งรหัสใหม่) จัดการลิงก์ก่อน ถ้าเข้าสู่ระบบสำเร็จก็จบ
277–281	อ่านโหมดที่จำไว้ในเครื่อง ถ้าเป็นโหมดทดลอง ใช้ DemoRepo
282–285	build ที่ไม่มี Supabase: ไปหน้าต้อนรับ
286–291	ถามว่ามี session ค้างอยู่ไหม (เคยเข้าสู่ระบบไว้) ถ้ามี ใช้ CloudRepo ของคนนั้น ไม่มีก็ไปหน้าต้อนรับ
294–309	ฟังเหตุการณ์ของระบบ Auth: เข้าสู่ระบบ (SIGNED_IN), กู้รหัสผ่าน (PASSWORD_RECOVERY) หรือ token กลับมา หลังออฟไลน์ → เปิดบัญชีนั้น · ออกจากระบบ (SIGNED_OUT) → ล้างข้อมูล ทั้งหมดห่อด้วย <code>setTimeout(..., 0)</code> เพราะ Supabase เตือนว่าเรียกคำสั่งอื่นในนี้ตรง ๆ อาจค้าง
310–312	ฟังลิงก์ที่เข้ามาระหว่างแอปเปิดอยู่
313–317	เก็บกวาดเมื่อปิด: เลิกฟังทุกอย่าง

`src/data/AppProvider.tsx`

บรรทัด 127–167

```

127     const loadAll = useCallback(async (r: Repo) => {
128         const gen = ++loadGen.current;
129         localChanges.current = new Map();
130         setLoadError(null);
131         try {
132             // Goals never block the app: if they cannot be loaded, the rest still works.
133             const [p, list, g] = await Promise.all([r.getProfile(), r.listTransactions(),
134             r.listGoals().catch(() => [] as SavingsGoal[])]);
135             if (gen !== loadGen.current || repoRef.current !== r) return;
136             // A slip saved by the automatic scan during the load is not lost when the list arrives.
137             const changes = localChanges.current;
138             const fresh = list.txs.filter((t) => !changes.has(t.id));
139             const kept = [...changes.values()].filter((t): t is Transaction => t !== null);
140             setProfile(p);
141             setTxs([...kept, ...fresh]);
142             setCarry(list.carrySatang);
143             setGoals(g);
144             setStatus('ready');
145         } catch (e) {
146             if (gen !== loadGen.current || repoRef.current !== r) return;
147             setLoadError(e instanceof Error ? e.message : 'โหลดข้อมูลไม่สำเร็จ');
148             setStatus('ready');
149         }
150     }, []);
151
152     const activateRepo = useCallback(
153         async (r: Repo, id: string) => {

```

```

153     if (activeUserRef.current !== id) {
154         // Another account (or leaving the demo): never show or use the previous one's data
           meanwhile.
155         setProfile(null);
156         setTxs([]);
157         setCarry(0);
158         setGoals([]);
159     }
160     activeUserRef.current = id;
161     repoRef.current = r;
162     setRepo(r);
163     setUserId(id);
164     await loadAll(r);
165 },
166 [loadAll],
167 );

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/AppProvider.tsx:127

บรรทัด	ความหมาย
127–128	loadAll: โหลดข้อมูลทั้งหมด แต่ละครั้งได้เลขรอบใหม่ (gen)
129–130	เริ่มจัดการเปลี่ยนแปลงในเครื่องใหม่ และล้าง error เก่า
133	โหลด 3 อย่างพร้อมกัน (un A5)
134	ถ้ามีการโหลดรอบใหม่กว่าเริ่มไปแล้ว หรือสลับบัญชีไปแล้ว ทั้งผลรอบนี้ (กันข้อมูลบัญชีเก่าโผล่ในบัญชีใหม่)
135–140	ถ้าระหว่างโหลด สแกนสลิปอัตโนมัติบันทึกรายการใหม่ไป เอารายการนั้นใส่กลับไว้ด้านบน ไม่ให้หายเพราะผลโหลดมากับ
139–143	ใส่ข้อมูลลง state ทั้งหมด แล้วบอกว่าพร้อม (หน้าจอวาดใหม่)
144–147	ถ้าโหลดพลาด เก็บข้อความ error ให้หน้าหลักแสดงปุ่ม "ลองอีกครั้ง"
151–167	activateRepo: เปลี่ยนไปใช้ Repo ของบัญชีใหม่ ถ้าเป็นคนละบัญชีกับเดิม ล้างข้อมูลเก่าออกก่อนทันที แล้วจํา Repo และโหลดข้อมูล

★ สรุป un A9

ฐานข้อมูลมี 4 ตาราง ทุกตารางเปิด RLS ให้แต่ละคนเห็นเฉพาะของตัวเอง เงินเก็บเป็นสตางค์และมี check กันค่าผิด สลิปซ้ำถูกกันด้วย unique index ระดับฐานข้อมูล แอปคุยกับฐานข้อมูลผ่าน Repo ซึ่งมีแบบคลาวด์และแบบในเครื่อง และ AppProvider เป็นคนเลือกว่าจะใช้แบบไหน

การเชื่อม AI: ทำไมต้องผ่านเซิร์ฟเวอร์ และทำงานยังไง

MindPay ใช้ AI 2 งาน คืออ่านสลิป (FR-4) และโคชน้องกล้า (FR-5) บทนี้อธิบายเส้นทางตั้งแต่แอปถามจนได้คำตอบ ทีละบรรทัด

AI ในแอปคืออะไร

AI ที่ใช้คือ โมเดลภาษาขนาดใหญ่ (Large Language Model, LLM) เช่น Claude ของบริษัท Anthropic และ Gemini ของ Google เราส่ง "ข้อความ" (และรูป) ไปทางอินเทอร์เน็ต มันตอบกลับเป็นข้อความ บริษัทเหล่านี้คิดเงินตามจำนวนที่ใช้ จึงต้องมี **API key** (กุญแจ) ไว้ระบุว่าใครเป็นคนจ่าย

คำ	ความหมาย
API	"ประตู" ให้โปรแกรมคุยกับโปรแกรม ส่งคำขอเป็น JSON ทาง HTTPS แล้วได้คำตอบเป็น JSON
API key	รหัสลับยาว ๆ ที่แนบไปกับคำขอ ใครได้กุญแจไป ก็ใช้ AI โดยให้เราจ่ายเงินได้
System prompt	"คำสั่งประจำตัว" ที่บอก AI ว่าต้องเป็นใคร ตอบยังไง ห้ามอะไร (ผู้ใช้ไม่เห็น)
Token	หน่วยนับข้อความของ AI (ประมาณคำหรือพยางค์) ใช้คิดราคาและจำกัดความยาวคำตอบ
JSON Schema	แบบฟอร์มที่บังคับให้ AI ตอบเป็นโครงสร้างตายตัว เช่น ต้องมีช่อง amount, dateIso
JSON	รูปแบบข้อมูลที่เขียนเหมือน object ของ JavaScript เช่น {"text": "สวัสดี"}

ทำไมแอปไม่เรียก AI เอง

! ห้ามใส่กุญแจ AI ไว้ในแอปเด็ดขาด

ไฟล์ APK หรือเว็บที่ผู้ใช้โหลดไป ใครก็แกะดูข้างในได้ ถ้ากุญแจ AI อยู่ในแอป คนแกะจะเอาไปใช้ฟรีโดยไม่ให้เราจ่าย (หรือทำให้กุญแจโดนแบน) ทีมเราจึงเก็บกุญแจไว้บนเซิร์ฟเวอร์ใน **Edge Functions** เท่านั้น แอปเรียก Edge Function แล้ว Edge Function ไปเรียก AI แทน

การมีเซิร์ฟเวอร์คั่นกลางให้ประโยชน์อีกหลายอย่าง

1. **ตรวจตัวตน:** เฉพาะผู้ใช้ที่เข้าสู่ระบบแล้วเท่านั้นที่ใช้ AI ได้
2. **จำกัดโควตา:** อ่านสลิปได้ 300 ใบ/วัน/คน ถามโคชได้ 60 ครั้ง/วัน/คน กันค่าใช้จ่ายบานปลาย
3. **เปลี่ยน AI ได้โดยไม่ต้องอัปเดตแอป:** สลับ Claude กับ Gemini แค่เปลี่ยน Secret บนเซิร์ฟเวอร์
4. **ทำความเข้าใจคำตอบ:** ตรวจสอบและแก้คำตอบของ AI ก่อนส่งถึงแอป (เช่น ปี พ.ศ. ที่ AI สัมแปลง)
5. **ความเป็นส่วนตัว:** ควบคุมได้ว่าส่งอะไรให้ AI บ้าง

เส้นทางของคำถาม 1 ขอลึงน้องกล้า

1. แอป `src/app/(tabs)/coach.tsx`

ผู้ใช้พิมพ์ "ซื้อรองเท้า 1,290 ได้ไหม" แอปคำนวณ "ข้อเท็จจริง" เอง (ยอดเงิน ค่าเฉลี่ย Runway ก่อนและหลังซื้อ) ด้วยโค้ดใน `src/domain`

↓ `askCoach()` ใน `src/services/coach.ts`

2. `supabase.functions.invoke('coach')`

ส่ง `{context, question}` พร้อม token การเข้าสู่ระบบของผู้ใช้ ทาง HTTPS



3. Edge Function `coach` `supabase/functions/coach/index.ts`

ตรวจสอบ token → ตรวจสอบขนาดคำขอ → ตรวจสอบโควตา → สร้างข้อความให้ AI (system prompt + DATA + QUESTION)

↓ คุกกี้แวลู (Secret)

4. Claude หรือ Gemini

เขียนคำตอบภาษาไทย ไม่เกิน 90 คำ ใช้เฉพาะตัวเลขที่ได้รับ

↑ คำตอบย้อนกลับทางเดิม

5. แอปแสดงและอ่านออกเสียง

น้องกล้าขยับปากพร้อมซับไต้เตล (FR-5)

ฝั่งแอป: `src/services/coach.ts`

`src/services/coach.ts`

บรรทัด 1-25

```
1 /** FR-5: ask น้องกล้า, the AI coach (Edge Function "coach"). */
2 import { FunctionsHttpError } from '@supabase/supabase-js';
3 import { supabase } from '../data/supabase';
4
5 export class CoachError extends Error {}
6
7 export async function askCoach(params: { context: unknown; question: string }): Promise<string> {
8   if (!supabase) throw new CoachError('น้องกล้าตอบคำถามได้เมื่อเข้าสู่ระบบด้วยบัญชีจริง ระหว่างนี้แตะการ์ด "สิ่งที่ควรรู้ตอนนี้" ให้กล้าเล่าให้ฟังได้');
9   const { data, error } = await supabase.functions.invoke('coach', { body: params });
10  if (error) {
11    if (error instanceof FunctionsHttpError) {
12      const body = await (error.context as Response).json().catch(() => null);
13      if (body?.error?.code === 'quota') throw new CoachError('วันนี้คุยกับน้องกล้าครบโควตาแล้ว พรุ่งนี้มาคุยกันใหม่');
14      if (body?.error?.code === 'not_configured') {
15        throw new CoachError('โค้ช AI ยังไม่พร้อม: ผู้ดูแลต้องใส่ GEMINI_API_KEY ใน Supabase → Edge Functions → Secrets (หรือ key ที่ใส่ไว้ใช้ไม่ได้) ระหว่างนี้ดูคำแนะนำด้านบนได้เลย');
16      }
17      if (body?.error?.code === 'busy') throw new CoachError('ตอนนี้มีคนคุยกับน้องกล้าเยอะ รอสักครู่แล้วถามใหม่');
18      if (body?.error?.code === 'unauthorized') throw new CoachError('เซสชันหมดอายุ กรุณาเข้าสู่ระบบอีกครั้ง');
19    }
20    throw new CoachError('ติดต่อน้องกล้าไม่ได้ตอนนี้ ลองใหม่อีกครั้ง');
21  }
22  const text = (data as { text?: string })?.text;
```

```

23 if (!text) throw new CoachError('น้องคล้าไม่ได้ตอบกลับ ลองถามใหม่อีกครั้งนะ');
24 return text;
25 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/services/coach.ts:1

บรรทัด	ความหมาย
2	นำชนิด error ของ Supabase ที่บอกว่า "เซิร์ฟเวอร์ตอบกลับมาว่าผิดพลาด" มาใช้
5	สร้าง error ชนิดของโค้ชเอง ข้อความของมันคือสิ่งที่จะแสดงให้ผู้ใช้เห็น
7	ฟังก์ชัน askCoach รับข้อมูลสรุป (context) และคำถาม คืนข้อความคำตอบ
8	ไม่มี Supabase (เช่น build ที่ไม่ได้ตั้งค่า) → บอกผู้ใช้ว่าต้องใช้บัญชีจริง
9	เรียก Edge Function ชื่อ coach ส่ง context และ question ไป ไสวสารีแบบ token การเข้าสู่ระบบให้เอง
10–21	ถ้าผิดพลาด อ่านรหัส error ที่เซิร์ฟเวอร์ส่งมา แล้วแปลงเป็นข้อความภาษาไทยที่เป็นมิตร: ครบโควตา (quota), ยังไม่ใส่ กุญแจ AI (not_configured), AI ไม่ว่าง (busy), หมดอายุการเข้าสู่ระบบ (unauthorized) อย่างอื่นบอกให้ลองใหม่
22–24	ดึงข้อความคำตอบ ถ้าไม่มีก็แจ้ง ถ้ามีส่งกลับไปให้น้องคล้า

ฝั่งเซิร์ฟเวอร์: Edge Function coach ที่ละบรรทัด

Edge Function เขียนด้วย TypeScript เหมือนแอป แต่รันบนเซิร์ฟเวอร์ของ Supabase ด้วยโปรแกรมชื่อ **Deno** (คล้าย Node.js) ฟังก์ชันหนึ่งตัวคือไฟล์เดอร์หนึ่งใน supabase/functions/ มีไฟล์ index.ts และไฟล์เดอร์ _shared เก็บโค้ดที่ใช้ร่วมกัน

supabase/functions/coach/index.ts

บรรทัด 1–26

```

1 // FR-5: the AI coach is น้องคล้า, the MindPay mascot. It explains the user's
2 // CONFIRMED numbers in one calm, friendly voice; the app reads the answer
3 // aloud, so it is written to be spoken. The app sends only totals by category
4 // (no slip images, no names of people, no account numbers).
5 import { aiProvider, BusyError, callAI, corsHeaders, envLimit, fail, json, NotConfiguredError,
6   requireUser, takeQuota } from '../_shared/common.ts';
7 import { plainText } from '../_shared/helpers.ts';
8
9 const DAILY_LIMIT = envLimit('COACH_DAILY_LIMIT', 60);
10 /** A question and a summary of totals: far below this. */
11 const MAX_BODY_BYTES = 64_000;
12
13 const SYSTEM = `You are น้องคล้า (Nong Kla), the mascot of MindPay, a Thai personal finance app for
14   students and first-jobbers.
15   You are a small golden money-tree sapling and the user's personal money coach.
16   Voice: calm, composed and warm (สุขุม เป็นมิตร), like a thoughtful friend who is good with money.
17   - Speak natural, everyday Thai. Call yourself "คล้า". Do not use ครับ orค่ะ; soft endings such as
18     "นะ" are fine.
19   - Your answer is read aloud by a text-to-speech voice and shown as subtitles, so write short
20     spoken sentences separated by spaces,
21     no emoji, no bullet points, no markdown, no lists, no headings.
22   Rules:
23   - Reply in Thai, at most 90 words.

```

- 22 - Use ONLY the numbers in the DATA block. Never invent amounts, dates, shops or trends. If the data cannot answer, say what is missing.
- 23 - Structure: what the numbers show -> why it matters -> one or two small suggestions -> leave the decision to the user.
- 24 - Never shame or scold ("ใช้เยอะเกินไป!" is not allowed). Be kind about mistakes.
- 25 - When the user asks "can I buy X", use the "purchaseCheck" numbers if present and state the runway before and after.
- 26 - You are not a licensed financial advisor: no investment, loan or crypto recommendations.`;

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `supabase/functions/coach/index.ts:1`

บรรทัด	ความหมาย
1-4	คอมเมนต์: โค้ชคือน้องกล้า อธิบายตัวเลขที่ยืนยันแล้ว คำตอบจะถูกอ่านออกเสียงจึงเขียนแบบภาษาพูด แอปส่งเฉพาะยอดรวมตามหมวด (ไม่มีรูปสลิป ชื่อคน หรือเลขบัญชี)
5-6	นำตัวช่วยจาก <code>_shared</code> มาใช้ (นามสกุล <code>.ts</code> ต้องเขียนเต็มใน Deno)
8	โควตาต่อวัน อ่านจาก <code>Secret COACH_DAILY_LIMIT</code> ถ้าไม่ได้ตั้งใช้ 60
10	ขนาดคำขอสองสูงสุด 64,000 ไบต์ (คำถามกับตัวเลขสรุปเล็กกว่านี้มาก กันคนส่งข้อมูลขยะก้อนใหญ่)
12-26	System prompt (เขียนเป็นภาษาอังกฤษเพราะ AI ทำตามคำสั่งภาษาอังกฤษได้แม่นยำกว่า แต่สั่งให้ตอบเป็นไทย) บอกว่า: เป็นน้องกล้า มาสคอตต้นกล้าสีทอง · น้ำเสียงสุ่ม เป็นมิตร เรียกตัวเองว่า "กล้า" ไม่ใช่ครับ/ค่ะ · คำตอบถูกอ่านออกเสียง จึงห้ามอีโมจิ ห้ามรายการ ห้าม markdown · ตอบไม่เกิน 90 คำ · ใช้เฉพาะตัวเลขในส่วน DATA ห้ามแต่งตัวเลขขึ้นมาเอง · โครงสร้างคำตอบ: ตัวเลขบอกอะไร → สำคัญยัง → คำแนะนำเล็ก ๆ 1-2 ข้อ → ให้ผู้ใช้ตัดสินใจเอง · ห้ามตำหนิ · ถ้าถาม "ซื้อได้ไหม" ให้ใช้ตัวเลข <code>purchaseCheck</code> บอก Runway ก่อนและหลัง · ไม่ใช่ที่ปรึกษาการเงินที่มีใบอนุญาต ห้ามแนะนำลงทุน กู้ยืม คริปโต

`supabase/functions/coach/index.ts`

บรรทัด 28-76

```

28 Deno.serve(async (req) => {
29   if (req.method === 'OPTIONS') return new Response('ok', { headers: corsHeaders });
30   if (req.method !== 'POST') return fail('method_not_allowed', 'Use POST', 405);
31
32   const userId = await requireUser(req);
33   if (!userId) return fail('unauthorized', 'Sign in again', 401);
34
35   if (Number(req.headers.get('content-length') ?? 0) > MAX_BODY_BYTES) return fail('too_large',
'Request is too large', 413);
36   let body: { context?: unknown; question?: string };
37   try {
38     body = await req.json();
39   } catch {
40     return fail('bad_request', 'Body must be JSON', 400);
41   }
42   if (!body || typeof body !== 'object') return fail('bad_request', 'Body must be a JSON object',
400);
43   const question = (body.question ?? '').toString().slice(0, 400).trim();
44   const data = JSON.stringify(body.context ?? {}).slice(0, 6000);
45
46   if (!aiProvider()) return fail('not_configured', 'AI key is not set on the server', 503);
47
48   try {
49     if (!(await takeQuota(userId, 'coach', DAILY_LIMIT))) {
50       return fail('quota', 'Daily coach limit reached, try again tomorrow', 429);
51     }
52     const text = await callAI({

```

```

53     task: 'coach',
54     system: SYSTEM,
55     maxTokens: 500,
56     content: [
57       {
58         type: 'text',
59         text: `DATA:\n${data}\n\nQUESTION:\n${question} || 'สรุปสถานะการเงินของเงินช่วงนี้ และบอกสิ่งที่คว
60       },
61     ],
62   });
63   return json({ text: plainText(text) });
64 } catch (e) {
65   if (e instanceof NotConfiguredError) {
66     console.error('coach: AI key missing or refused', e.message);
67     return fail('not_configured', 'AI key is not set on the server or was refused', 503);
68   }
69   if (e instanceof BusyError) {
70     console.warn('coach: AI busy', e.message);
71     return fail('busy', 'The AI service is busy, try again shortly', 503);
72   }
73   console.error('coach failed', e);
74   return fail('coach_error', 'The coach is unavailable right now', 502);
75 }
76 });

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → supabase/functions/coach/index.ts:28

บรรทัด	ความหมาย
28	Deno.serve = "เปิดร้าน" รอรับคำขอ ทุกคำขอที่เข้ามาจะเรียกฟังก์ชันนี้ 1 ครั้ง
29	คำขอแบบ OPTIONS คือเบราว์เซอร์ "ถามทาง" ก่อนส่งจริง (กฎ CORS ของเว็บ) ตอบ ok พร้อม header ที่อนุญาต
30	รับเฉพาะ POST อย่างอื่นตอบ 405
32–33	ตรวจตัวตน: อ่าน token จากคำขอแล้วถาม Supabase ว่าเป็นใคร ไม่ผ่าน → 401 (แอปจะบอก "เซสชันหมดอายุ")
35	คำขอใหญ่เกิน → 413 (ตรวจจากหัวคำขอ ก่อนอ่านเนื้อหา จึงไม่เปลืองหน่วยความจำ)
36–42	อ่านเนื้อหาเป็น JSON ถ้าอ่านไม่ได้หรือไม่ใช่ object → 400
43	คำถามตัดเหลือไม่เกิน 400 ตัวอักษร และตัดช่องว่างหัวท้าย
44	ข้อมูลสรุปแปลงเป็นข้อความ JSON ตัดเหลือไม่เกิน 6,000 ตัวอักษร (คุมค่าใช้จ่าย)
46	ไม่มีกุญแจ AI ใน Secrets → 503 not_configured
49–51	ตรวจโควตา และนับเพิ่ม 1 ในคำสั่งเดียว เกิน 60 ครั้งใน 24 ชั่วโมง → 429
52–62	เรียก AI ส่ง system prompt, จำกัดคำตอบ 500 token, และข้อความที่มี 2 ส่วน: DATA: ตามด้วยตัวเลข และ QUESTION: ตามด้วยคำถาม (ถ้าผู้ใช้ไม่ได้ถาม ใช้คำถามตั้งต้น "สรุปสถานะการเงินของเงินช่วงนี้...")
63	ล้าง markdown ที่ AI อาจใส่มา (plainText) แล้วตอบแอปเป็น {"text": "..."}
64–75	จัดการ error ตามชนิด: กุญแจผิด/ไม่มี → 503 not_configured, AI ไม่ว่าง → 503 busy, อื่น ๆ → 502 และบันทึก log ไว้ดูใน Dashboard (บันทึกเฉพาะชนิด error ไม่บันทึกคำถามหรือคำตอบของผู้ใช้)

ตัวอย่างกลาง: supabase/functions/_shared/common.ts

บรรทัด 17-83

```
17 /** Read one environment variable; a name the runtime refuses counts as "not set". */
18 const env = (name: string) => {
19   try {
20     return Deno.env.get(name);
21   } catch {
22     return undefined;
23   }
24 };
25 const geminiKey = () => readKey(env, GEMINI_KEY_NAMES);
26 const claudeKey = () => readKey(env, CLAUDE_KEY_NAMES);
27
28 export const corsHeaders = {
29   'Access-Control-Allow-Origin': '*',
30   'Access-Control-Allow-Headers': 'authorization, x-client-info, apikey, content-type',
31   'Access-Control-Allow-Methods': 'POST, OPTIONS',
32 };
33
34 export function json(body: unknown, status = 200) {
35   return new Response(JSON.stringify(body), {
36     status,
37     headers: { ...corsHeaders, 'Content-Type': 'application/json' },
38   });
39 }
40
41 /** Error body the app understands: { error: { code, message } } */
42 export function fail(code: string, message: string, status: number) {
43   return json({ error: { code, message } }, status);
44 }
45
46 /** Returns the signed-in user's id, or null when the request has no valid session. */
47 export async function requireUser(req: Request): Promise<string | null> {
48   const token = req.headers.get('Authorization')?.replace(/^Bearer\s+/i, '');
49   if (!token) return null;
50   const { data, error } = await admin().auth.getUser(token);
51   if (error || !data.user) return null;
52   return data.user.id;
53 }
54
55 function admin() {
56   return createClient(Deno.env.get('SUPABASE_URL')!, Deno.env.get('SUPABASE_SERVICE_ROLE_KEY')!,
57     {
58       auth: { persistSession: false },
59     });
60 }
61
62 /** Thrown when no AI key is set in the function secrets, or the key was refused. */
63 export class NotConfiguredError extends Error {}
64
65 /** The AI service is rate-limiting us (e.g. Gemini free tier) or overloaded: try again shortly. */
66 export class BusyError extends Error {}
67
68 /** A daily limit from the function settings; a missing or non-number setting uses the default. */
69 export function envLimit(name: string, fallback: number): number {
70   const n = Number(Deno.env.get(name));
```

```

70 return Number.isFinite(n) && n > 0 ? Math.floor(n) : fallback;
71 }
72
73 /**
74  * Daily cap per user so a leaked account or a bug cannot run up the AI bill.
75  * Returns false when the user is over the limit; otherwise records one use.
76  * Counting and recording happen in one database call under a per-user lock
77  * (take_ai_quota), so many requests at once cannot all slip under the limit.
78  */
79 export async function takeQuota(userId: string, kind: 'slip' | 'coach', dailyLimit: number) {
80   const { data, error } = await admin().rpc('take_ai_quota', { p_user: userId, p_kind: kind,
81     p_limit: dailyLimit });
82   if (error) throw error;
83   return data === true;
84 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → supabase/functions/_shared/common.ts:17

บรรทัด	ความหมาย
17–24	อ่านค่า Secret จากเชิร์ฟเวอร์ (Deno.env.get) ถ้าชื่อแปลจนอ่านไม่ได้ ถือว่าไม่ได้ตั้ง
25–26	หากุญแจ Gemini และ Claude จากรายชื่อที่ยอมรับ (ดู readKey ด้านล่าง)
28–32	header CORS: ยอมให้เว็บจากทุกที่เรียกได้ (ความปลอดภัยมาจาก token ไม่ใช่จากที่อยู่เว็บ)
34–39	json() สร้างคำตอบ HTTP ที่เป็น JSON พร้อม header
41–44	fail() สร้างคำตอบ error รูปแบบเดียวกันทุกครั้ง {"error": {"code", "message"}} ที่แอปอ่าน code ได้ (บรรทัด 13–18 ของ coach.ts ผังแอป)
46–53	requireUser: ตัดคำว่า "Bearer " ออกจาก header Authorization ได้ token แล้วถาม Auth ว่า token นี้เป็นของใคร คืน id ผู้ใช้ หรือ null
55–59	admin(): client ที่ใช้ service role key (ข้าม RLS ได้) มีเฉพาะบนเชิร์ฟเวอร์
61–65	error 2 ชนิด: ไม่ได้ตั้งกุญแจ/กุญแจใช้ไม่ได้ และ AI ไม่ว่าง (ติดลิมิตหรือโหลดเกิน)
67–71	อ่านโควตาจาก Secret ถ้าไม่ใช่ตัวเลขบวกใช้ค่าตั้งต้น
73–83	takeQuota: เรียกฟังก์ชัน SQL take_ai_quota ในฐานข้อมูล (rpc = เรียกฟังก์ชันระยะไกล) ซึ่งนับและบันทึกในชั้นตอนเดียว

ทำไมนับโควตาต้อง "ทำในชั้นตอนเดียว"

supabase/migrations/20261001000000_atomic_ai_quota.sql

บรรทัด 1–44

```

1 -- Atomic daily AI quota (audit 1 Oct 2569).
2 -- Before: the Edge Functions counted today's uses and inserted a new one in two
3 -- separate calls, so many requests sent at once could all pass the limit.
4 -- Now counting and recording happen in one call, under a lock per user and kind.
5 -- Called only by the Edge Functions with the service role.
6
7 create or replace function public.take_ai_quota(p_user uuid, p_kind text, p_limit integer)
8 returns boolean
9 language plpgsql
10 security definer
11 set search_path = ''

```

```

12 as $$
13 declare
14     used integer;
15 begin
16     if p_kind not in ('slip', 'coach') or p_limit is null or p_limit < 1 then
17         return false;
18     end if;
19     perform pg_advisory_xact_lock(hashtextextended(p_user::text || ':' || p_kind, 0));
20     select count(*) into used
21         from public.ai_usage
22         where user_id = p_user and kind = p_kind and created_at >= now() - interval '24 hours';
23     if used >= p_limit then
24         return false;
25     end if;
26     insert into public.ai_usage (user_id, kind) values (p_user, p_kind);
27     return true;
28 end;
29 $$;
30
31 revoke execute on function public.take_ai_quota(uuid, text, integer) from public, anon,
    authenticated;
32 grant execute on function public.take_ai_quota(uuid, text, integer) to service_role;
33
34 -- Amounts have the same upper bound as savings goals ($1,000,000,000), so a misread
35 -- amount can never overflow what the app handles.
36 alter table public.transactions
37     drop constraint if exists transactions_amount_max,
38     add constraint transactions_amount_max check (amount_satang <= 1000000000000);
39 alter table public.profiles
40     drop constraint if exists profiles_amounts_max,
41     add constraint profiles_amounts_max check (
42         abs(opening_balance_satang) <= 1000000000000
43         and (monthly_budget_satang is null or monthly_budget_satang <= 1000000000000)
44     );

```

► เปิดใน VS Code (เว็บ) [GitHub](#) VS Code ในเครื่อง: Ctrl+P → supabase/migrations/20261001000000_atomic_ai_quota.sql:1

บรรทัด	ความหมาย
1-5	คอมเมนต์เล่าบ๊วกเดิม: เดิมนับจำนวนครั้งวันนี้นับกับบันทึกครั้งใหม่เป็น 2 คำสั่งแยกกัน ถ้าส่งคำขอพร้อมกันหลายอัน ทุกอันนับได้ "ยังไม่เกิน" แล้วผ่านหมด (race condition) แก้ให้นับและบันทึกในคำสั่งเดียวภายใต้กฎแจจลือก
7-14	สร้างฟังก์ชัน SQL รับ id ผู้ใช้ ชนิตงาน (slip/coach) และเพดาน คินจริง/เท็จ รับด้วยสิทธิ์เจ้าของ (security definer)
16-18	ค่าที่ส่งมาผิด → ไม่อนุญาต
19	ลือก ต่อผู้ใช้ต่อชนิตงาน: คำขอที่ 2 ของคนเดียวกันต้องรอคำขอแรกเสร็จก่อน (ลือกปลดเองเมื่อจบคำสั่ง)
20-22	นับจำนวนการใช้ใน 24 ชั่วโมงล่าสุด
23-25	ถึงเพดานแล้ว → ไม่อนุญาต
26-27	ยังไม่ถึง → บันทึกการใช้ 1 ครั้ง แล้วอนุญาต
31-32	ห้ามผู้ใช้ทั่วไปเรียกฟังก์ชันนี้ตรง ๆ ให้เฉพาะเซิร์ฟเวอร์ (service_role)
34-44	(ไฟล์เดียวกัน) เพิ่มเพดานจำนวนเงิน \$1,000,000,000 ให้ตารางรายการและโปรไฟล์ ตรงกับ MAX_SATANG ในแอป

เลือก AI ตัวไหน: Claude หรือ Gemini

supabase/functions/_shared/helpers.ts

บรรทัด 1-45

```
1 // Pure helpers for the Edge Functions. No Deno APIs here, so the same file is
2 // unit-tested with vitest (see __tests__/helpers.test.ts).
3
4 export type AiProvider = 'claude' | 'gemini';
5
6 /**
7  * Secret names accepted for each service. The first is the documented one;
8  * the rest are Google's other standard name and common typing slips seen in
9  * the Supabase dashboard (a secret saved as "GEMINI API KEY", 28 ก.ย. 2569).
10 */
11 export const GEMINI_KEY_NAMES = [
12   'GEMINI_API_KEY',
13   'GOOGLE_API_KEY',
14   'GOOGLE_GENERATIVE_AI_API_KEY',
15   'GEMINI API KEY',
16   'GEMINI-API-KEY',
17   'GEMINI_KEY',
18 ];
19 export const CLAUDE_KEY_NAMES = ['ANTHROPIC_API_KEY', 'CLAUDE_API_KEY', 'ANTHROPIC API KEY'];
20
21 /**
22  * The first secret that is set, with spaces, line breaks or quotes from
23  * copy-paste removed. Returns null when none is set.
24 */
25 export function readKey(get: (name: string) => string | undefined | null, names: string[]):
  string | null {
26   for (const name of names) {
27     const raw = get(name);
28     const clean = raw?.trim().replace(/^[\"']|['\"]$/g, '').trim();
29     if (clean) return clean;
30   }
31   return null;
32 }
33
34 /**
35  * Which AI service answers: Claude when its key is set, otherwise Gemini.
36  * AI_PROVIDER=claude|gemini forces one. Returns null when no usable key is set.
37 */
38 export function pickProvider(env: { forced?: string | null; hasClaude: boolean; hasGemini:
  boolean }): AiProvider | null {
39   const forced = env.forced?.trim().toLowerCase();
40   if (forced === 'claude') return env.hasClaude ? 'claude' : null;
41   if (forced === 'gemini') return env.hasGemini ? 'gemini' : null;
42   if (env.hasClaude) return 'claude';
43   if (env.hasGemini) return 'gemini';
44   return null;
45 }
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → supabase/functions/_shared/helpers.ts:1

บรรทัด ความหมาย

1-2 โฟกัสนี้เป็นฟังก์ชันล้วน ไม่ใช้คำสั่งของ Deno จึงเอาไปเทสต์ด้วย vitest ได้เหมือนโค้ดในแอป

บรรทัด	ความหมาย
6–19	รายชื่อ Secret ที่ยอมรับ ตัวแรกคือชื่อที่ถูกต้อง ตัวอื่นเป็นชื่อสำรองและ ชื่อที่พิมพ์ผิดบ่อย คอมเมนต์เล่าว่าเคยมีคนตั้งชื่อ Secret เป็น "GEMINI API KEY" (มีเว้นวรรค) เมื่อ 28 ก.ย. 2569 ทำให้ AI ไม่ทำงาน ทีมจึงให้เซิร์ฟเวอร์รับชื่อเหล่านี้ด้วย
25–32	readKey: ใส่ชื่อที่ละตัว เจอตัวแรกที่มีค่า ตัดช่องว่าง การขึ้นบรรทัด และเครื่องหมายคำพูดที่ติดมาตอนถือปวงออก
38–45	pickProvider: ถ้าตั้ง AI_PROVIDER บังคับไว้ ใช้ตัวนั้น (ต้องมีคีย์แจด้วย) ไม่จันมีคีย์แจ Claude ใช้ Claude ก่อน ไม่มีก็ใช้ Gemini ไม่มีเลยคืน null

	Claude (Anthropic)	Gemini (Google)
Secret	ANTHROPIC_API_KEY	GEMINI_API_KEY
รุ่นที่ใช้ (ค่าเริ่มต้น)	claude-haiku-4-5 (เปลี่ยนได้ด้วย SLIP_MODEL, COACH_MODEL)	gemini-3.5-flash-lite แล้วลองรุ่นถัดไปเองถ้าใช้ไม่ได้
ค่าใช้จ่าย	ต้องเติมเครดิต อ่านสลิปใบละประมาณ 10 สตางค์	มีแบบฟรี (มีเพดานต่อนาที/ต่อวัน)
ข้อมูลของผู้ใช้	ไม่นำไปฝึกโมเดล	แบบฟรี Google อาจนำข้อมูลไปปรับปรุงโมเดล (README บอกไว้ตรง ๆ)

ส่งคำขอไปที่ Claude: callClaude

supabase/functions/_shared/common.ts

บรรทัด 114–200

```
114 /** Ask the configured AI and return its text reply. */
115 export async function callAI(req: AiRequest): Promise<string> {
116   return (await callAIWithModel(req)).text;
117 }
118
119 /**
120  * Longest wait for one call to the AI service, and for all tries of one request together
121  * (the platform stops a function after 150 s; both slip reads run at the same time).
122  */
123 const FETCH_TIMEOUT_MS = 45_000;
124 const REQUEST_DEADLINE_MS = 110_000;
125
126 /** Ask the configured AI; also says which model answered (the two slip reads compare it). */
127 export async function callAIWithModel(req: AiRequest): Promise<{ text: string; model: string }> {
128   const provider = aiProvider();
129   if (!provider) throw new NotConfiguredError('No AI key is set');
130   const started = Date.now();
131   const deadline = started + REQUEST_DEADLINE_MS;
132   const result = provider === 'claude' ? await callClaude(req, deadline) : await callGemini(req,
133     deadline);
134   // Metadata only: never log images, questions or replies.
135   console.log(JSON.stringify({ ai: provider, model: result.model, task: req.task, variant:
136     req.variant ?? 'primary', ms: Date.now() - started }));
137   return result;
138 }
139
140 /** fetch with a time limit; running out of time counts as the AI being busy (the app waits and
141   retries). */
142 async function timedFetch(url: string, init: RequestInit, deadline: number): Promise<Response> {
```

```

140   const left = deadline - Date.now();
141   if (left <= 1000) throw new BusyError('AI request deadline reached');
142   try {
143     return await fetch(url, { ...init, signal: AbortSignal.timeout(Math.min(FETCH_TIMEOUT_MS,
144   left)) });
145   } catch (e) {
146     const name = (e as Error)?.name;
147     if (name === 'TimeoutError' || name === 'AbortError') throw new BusyError(`AI request timed
148   out (${url.split('/')[2]})`);
149     throw e;
150   }
151 }
152
153 const sleep = (ms: number) => new Promise((resolve) => setTimeout(resolve, ms));
154
155 async function callClaude(req: AiRequest, deadline: number): Promise<{ text: string; model:
156   string }> {
157   const key = claudeKey();
158   const main = Deno.env.get(req.task === 'slip' ? 'SLIP_MODEL' : 'COACH_MODEL') ??
159   CLAUDE_DEFAULT_MODEL;
160   const model = req.variant === 'verify' ? (Deno.env.get('VERIFY_MODEL') ?? main) : main;
161   const body: Record<string, unknown> = {
162     model,
163     max_tokens: req.maxTokens,
164     system: req.system,
165     messages: [
166       {
167         role: 'user',
168         content: req.content.map((c) =>
169           c.type === 'text' ? c : { type: 'image', source: { type: 'base64', media_type:
170   c.mediaType, data: c.data } },
171       ),
172     ],
173   };
174   if (req.schema) {
175     body.output_config = { format: { type: 'json_schema', schema: req.schema } };
176   }
177   const res = await timedFetch(
178     'https://api.anthropic.com/v1/messages',
179     {
180       method: 'POST',
181       headers: {
182         'x-api-key': key,
183         'anthropic-version': '2023-06-01',
184         'content-type': 'application/json',
185       },
186       body: JSON.stringify(body),
187     },
188     deadline,
189   );
190   if (!res.ok) {
191     const detail = await res.text();
192     const msg = `Claude API ${res.status}: ${detail.slice(0, 300)}`;
193     if (isInvalidKey(res.status, detail)) throw new NotConfiguredError(msg);
194     if (res.status === 429 || res.status === 529) throw new BusyError(msg);
195     throw new Error(msg);
196   }
197   const data = await res.json();
198   const text = (data.content ?? []).
199     .filter((b: { type: string }) => b.type === 'text')

```

```

196     .map((b: { text: string }) => b.text)
197     .join('');
198     if (!text) throw new Error('Claude API returned no text');
199     return { text, model };
200 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → supabase/functions/_shared/common.ts:114

บรรทัด	ความหมาย
114–117	callAI เรียกตัวที่บอกชื่อรุ่นด้วย แล้วคืนแค่ข้อความ
119–124	เวลาสูงสุด: รอ AI ครั้งละไม่เกิน 45 วินาที และทั้งคำขอไม่เกิน 110 วินาที (Supabase จะตัดฟังก์ชันที่ 150 วินาที)
127–136	เลือก AI แล้วเรียก จับเวลา และเขียน log เฉพาะข้อมูลคำกับ (ตัวไหน รุ่นไหน งานอะไร ใช้กี่มิลลิวินาที) ไม่เคยบันทึกรูปคำถาม หรือคำตอบ
139–149	timedFetch: ส่งคำขอทางอินเทอร์เน็ตแบบมีเวลาจำกัด ถ้าหมดเวลาถือว่า AI ไม่ว่าง (แอปจะรอแล้วลองใหม่)
153–156	เลือกรุ่นของ Claude ตามงาน ส่วนการอ่านสลิปครั้งที่ 2 (verify) ใช้รุ่น VERIFY_MODEL ถ้าตั้งไว้
157–169	สร้างเนื้อหาคำขอตามรูปแบบของ Anthropic Messages API: รุ่น, จำนวน token สูงสุด, system prompt และข้อความของผู้ใช้ (รูปแปลงเป็นรูปแบบ base64 ที่ Claude รับ)
170–172	ถ้างานนี้ต้องการ JSON ตามแบบฟอร์ม (อ่านสลิป) ใส่ schema ไปด้วย Claude จะตอบตามแบบฟอร์มเป็น
173–185	ส่งไปที่ https://api.anthropic.com/v1/messages แบบถูกใจใน header x-api-key
186–192	ตอบกลับไม่สำเร็จ: คุกกี้แฉิด → NotConfigured, 429/529 (ส่งถี่เกิน/ระบบเต็ม) → Busy, อื่น ๆ → error ทั่วไป
193–199	สำเร็จ: เอาเฉพาะส่วนที่เป็นข้อความมาต่อกัน คืนพร้อมชื่อรุ่น

ฝั่ง Gemini (callGemini บรรทัด 202–268) หลักการเดียวกันแต่ซับซ้อนกว่า เพราะแบบฟรีติดลิมิตบ่อย: ไล่ลองหลายรุ่นตามลำดับ ถ้ารุ่นไหนใช้ไม่ได้กับคุกกี้แฉิดข้ามไปรุ่นถัดไป ถ้าทุกรุ่น "ไม่ว่าง" รอตามเวลาที่ Google บอก (ไม่เกิน 12 วินาที) แล้วลองทั้งหมดอีก 1 รอบ ก่อนจะบอกแอปว่า busy การตัดสินใจหลังแต่ละ error อยู่ในฟังก์ชัน geminiStep

supabase/functions/_shared/helpers.ts

บรรทัด 75–93

```

75 /**
76  * What to do after a failed Gemini call:
77  * - invalid_key: stop, the key is wrong (tell the app "not configured")
78  * - plain_schema: retry the same model with the schema described in the prompt
79  * - next_model: this model is not available to this key; try the next one
80  * - busy: rate limit or overload on this model. Each model has its own free
81  *   quota, so try the next model first and wait only when all are busy.
82  * - fail: anything else
83  */
84 export type GeminiStep = 'invalid_key' | 'plain_schema' | 'next_model' | 'busy' | 'fail';
85
86 export function geminiStep(status: number, body: string, usingJsonSchema: boolean): GeminiStep {
87   if (isInvalidKey(status, body)) return 'invalid_key';
88   if (status === 404) return 'next_model';
89   if (status === 429 && isZeroQuota(body)) return 'next_model';
90   if (status === 400 && usingJsonSchema) return 'plain_schema';
91   if (status === 429 || status === 500 || status === 503) return 'busy';
92   return 'fail';

```

บรรทัด	ความหมาย
87	กุญแจผิด (401/403 หรือข้อความ API key not valid) → หยุด บอกว่ายังไม่ได้ตั้งค่า
88	404 = ไม่มีรุ่นนี้ → ลองรุ่นถัดไป
89	429 ที่บอกว่าโควตาเป็น 0 (กุญแจนี้ไม่มีสิทธิ์ใช้รุ่นนี้เลย) → รุ่นถัดไป
90	400 ตอนใช้ JSON schema (รุ่นเก่าบางรุ่นไม่รองรับ) → ลองใหม่โดยเขียนแบบฟอร์มลงใน prompt แทน
91	429/500/503 → ไม่ว่าง ลองรุ่นอื่นก่อน แล้วค่อยรอ
92	อย่างอื่น → ล้มเหลว

ตั้งค่า AI (ผู้ดูแลทำครั้งเดียว)

1. สร้างกุญแจ: Gemini ที่ aistudio.google.com/apikey (ฟรี) หรือ Claude ที่ console.anthropic.com
2. เปิด Supabase Dashboard → **Edge Functions** → **Secrets** (ไม่ใช่ Vault และไม่ใช่หน้า API keys)
3. เพิ่ม Secret ชื่อ GEMINI_API_KEY หรือ ANTHROPIC_API_KEY ใส่ค่ากุญแจ
4. ไม่ต้อง deploy ใหม่ ฟังก์ชันอ่าน Secret ทุกครั้งที่มีการขอ
5. ถ้าแก้ไขโค้ดใน `supabase/functions` ต้อง deploy ด้วย `npx supabase functions deploy coach` (หรือ `parse-slip`)

หลักการใช้ AI อย่างรับผิดชอบที่ทีมเราใช้

หลักการ	ในโค้ด
AI ไม่คิดเลข โค้ดที่ทดสอบแล้วคิด AI แค่อธิบาย	แอปคำนวณ Runway และ purchaseCheck เองก่อนส่ง (FR-5, FR-6) system prompt ห้ามแต่งตัวเลข
AI ผิดได้ ต้องมีคนตรวจ	สลิปที่ไม่มั่นใจเป็นแบบร่าง อ่าน 2 ครั้งด้วย 2 รุ่น ไม่ตรงกันให้ผู้ใช้เช็ค (FR-4)
ส่งข้อมูลให้ AI น้อยที่สุด	โค้ชได้เฉพาะยอดรวม รูปที่ส่งอ่านคือรูปที่มี QR สลิปเท่านั้น และไม่เก็บรูปบนเซิร์ฟเวอร์
คุมค่าใช้จ่าย	โควตาต่อวันแบบ atomic, ตัดความยาวคำถาม, จำกัด token คำตอบ
บันทึกการใช้ AI ในการพัฒนา	docs/AI_USAGE_LOG.md บันทึกตาม SRS ว่าใช้ AI ช่วยอะไร แก้อะไร และสมาชิกตรวจอะไร

UX/UI: การออกแบบหน้าจอและ Design System

UI คือหน้าตา (สี ตัวอักษร ปุ่ม) UX คือประสบการณ์ (ใช้งานง่ายไหม รู้สึกยังไง) บทนี้อธิบายหลักการออกแบบของ MindPay และโค้ดที่บังคับให้ทุกหน้าทำตาม

หลักการออกแบบของ MindPay

หลักการ	ทำอย่างไรในแอป
ตัวเลขเดียว เข้าใจทันที	หน้าหลักมีการ์ดเขียวที่บอกยอดเงินและ "เงินพอใช้อีก 16 วัน" ตัวใหญ่ที่สุด ผู้ใช้ไม่ต้องคิดเลขเอง
ทำสิ่งที่ทำบ่อยให้ง่ายที่สุด	ปุ่มสแกนสลิปใหญ่สุดกลางแถบล่าง, สแกนอัตโนมัติตอนเปิดแอป, พูดจได้, ปุ่มลัด + ฿100/500/1,000/5,000 ตอนเพิ่มเงินเข้า
ไม่ดู ไม่ทำให้รู้สึกผิด	บอกล่าพุดว่า "ลองพักของที่ไม่จำเป็นสักหน่อยนะ ค่อย ๆ ไปด้วยกัน" แทน "ใช้เงินเยอะเกินไป!" (system prompt ของโค้ชห้ามตำหนิ)
บอกตรง ๆ ว่ามันใจแค่นี้	สลิปที่ไม่ชัดเป็น "แบบร่าง" พร้อมบอกว่าช่องไหนต้องตรวจ, Runway ใช้คำว่า "ถึงประมาณ"
อธิบายตัวเองได้	การ์ด "คำนวณยังงี้" ในหน้าเงินพอถึงแสดงสูตรที่ละบรรทัดเหมือนใบเสร็จ
ย้อนกลับได้	ลบรายการแล้วกด "เลิกทำ" ได้ (FR-1), สลิปที่เข้ายอดเองกด "ยังไม่รวมยอด" ได้ (FR-4)
ข้อความผิดพลาดต้องบอกวิธีแก้	"ใส่จำนวนเงินมากกว่า 0 เช่น 85 หรือ 85.50" แทน "Invalid input"
ทุกคนใช้ได้ (accessibility)	สีผ่านเกณฑ์อ่านง่าย, ปุ่มใหญ่พอแตะ, มีข้อความสำหรับโปรแกรมอ่านหน้าจอ, ปิดแอนิเมชันได้

Design System คืออะไร

Design System คือ "กฎกลาง + ชิ้นส่วนสำเร็จรูป" ที่ทุกหน้าใช้ร่วมกัน เหมือนชุดตัวต่อเลโก้ที่มีขนาดและสีมาตรฐาน หน้าจอทุกหน้าจึงดูเป็นแอปเดียวกัน และเปลี่ยนธีมสีได้ทั้งแอปในจุดเดียว ของ MindPay อยู่ใน

- `src/ui/theme.ts` ค่ากลาง (token): สี ฟอนต์ ขนาดตัวอักษร ระยะห่าง มุมโค้ง
- `src/ui/components.tsx`, `src/ui/inputs.tsx`, `src/ui/feedback.tsx` ชิ้นส่วน: ตัวหนังสือ ปุ่ม การ์ด ช่องกรอก ข้อความแจ้งเตือน
- `docs/DESIGN_SYSTEM.md` เอกสารกฎ **กฎข้อแรก: ห้ามหน้าจอไหนคิดสีหรือขนาดเอง**

สี: Green + Gold

ทิศทางแบรนด์คือ **สีเขียว** (เงิน การเติบโต ความมั่นคง) ใช้กับโครงสร้าง และ **สีทอง** ใช้ "น้อย ๆ" กับสิ่งที่สำคัญที่สุดบนหน้า เช่น ยอดเงิน ปุ่มหลัก ใบของต้นไม้เงิน คอมเมนต์บนสุดของ theme.ts เขียนว่า "Never paint a whole screen green"

`src/ui/theme.ts`

บรรทัด 1-21

1 /**

```

2  * MindPay design system tokens.
3  *
4  * Brand: Green (money, growth, stability) + Gold (accent for what matters most).
5  * Rule from the brand direction: green is for structure, gold is used sparingly
6  * for the single most important thing on a screen (the balance, the main action).
7  * Never paint a whole screen green.
8  */
9  import { useColorThemeKey, useSchemeChoice } from './themeMode';
10
11 export const palette = {
12   forest: '#0E3B2C', // hero surfaces, brand
13   forestDeep: '#082A1F',
14   leaf: '#1F7A52', // primary actions, positive
15   mint: '#DDEEE3',
16   gold: '#C8992A', // accent: balance, main CTA ring, tree leaves
17   goldBright: '#E2B64A',
18   goldSoft: '#F4E7C2',
19   clay: '#B4442F', // critical / expense emphasis
20   amber: '#B7791F', // watch
21 };

```

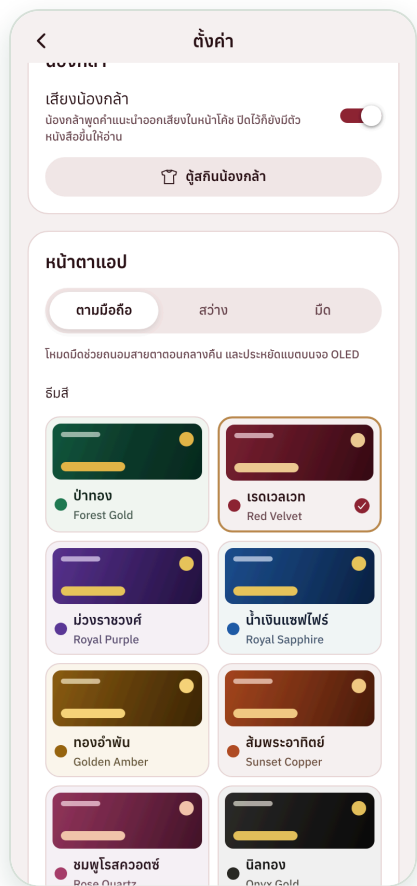
▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/ui/theme.ts:1

สีทั้งหมดถูกตั้งชื่อตาม **หน้าที่** (token) ไม่ใช่ตามสี เช่น ink = สีตัวหนังสือหลัก, primary = สีปุ่มหลัก หน้าจอเรียก theme.ink โดยไม่สนว่าตอนนี้เป็นธีมไหน

Token	ใช้กับ	ป่าทอง สว่าง	ป่าทอง มืด
bg	พื้นหลังหน้าจอ	#F2F5F1	#0A1410
surface	การ์ด ช่องกรอก	#FFFFFF	#111E18
ink	ตัวหนังสือหลัก	#11231B	#E7EFE9
inkSoft	ตัวหนังสือรอง คำอธิบาย	#4B5F55	#A2B6AA
inkFaint	placeholder ไอคอนเจียม ๆ (ห้ามใช้กับข้อความที่ต้องอ่าน)	#8A9A91	#6B8076
primary	ปุ่มหลัก สิ่ง que เลือกอยู่	#1F7A52	#3FA774
accent	ปุ่มทอง ขอบการ์ดที่ต้องสนใจ	#C8992A	#E2B64A
income / good / watch / critical	สีที่มีความหมาย: เงินเข้า, ดี, สะวัง, วิกฤต เหมือนกันทุกธีม ผู้ใช้จึงไม่สับสน	เขียว / เขียว / ส้มอำพัน / แดงอิฐ	

9 ธีม × สว่าง/มืด



ตั้งค่า → ธีมสี: เลือก 1 ใน 9 ธีม และโหมด ตามมือถือ / สว่าง / มืด

ผู้ใช้เลือกธีมได้ 9 แบบ: ป่าทอง (ค่าเริ่มต้น), เรดเวลเวท, ม่วงราชวงศ์, น้ำเงินแซฟไฟร์, ทองอำพัน, ส้มพระอาทิตย์, ชมพูโรสควอตซ์, นิลทอง และธีมเทศกาลฮาโลวีน แต่ละธีมมีชุดสีสว่างและมืด (COLOR_THEMES ใน theme.ts บรรทัด 93–204) เลือกแล้วจำไว้ในเครื่อง

ฟังก์ชัน buildTheme สร้างชุดสีเต็ม (ประมาณ 30 token) จากค่าพื้นฐานของแต่ละธีม และคำนวณสีบางตัวให้อ่านง่ายอัตโนมัติ เช่น สีตัวหนังสือบนปุ่มทอง (onAccent) เลือกระหว่างสีเข้มกับสีขาว ตัวที่อ่านออกบนพื้นทงนั้น

หน้าจอเรียกแค่ const theme = useTheme() ได้ชุดสีของธีมและโหมดที่ผู้ใช้เลือกตอนนี้ เปลี่ยนธีมแล้วทุกหน้าเปลี่ยนตามทันที

ความอ่านง่ายของสี (WCAG) และเทสต์ที่บังคับ

มาตรฐานสากล WCAG AA กำหนดว่าตัวหนังสือกับพื้นหลังต้องต่างกันอย่างน้อย 4.5 : 1 (1 = สีเดียวกัน, 21 = ดำบนขาว) คนสายตาไม่ดีหรือใช้มือถือกลางแดดจึงอ่านออก

คู่สี (ป่าทอง สว่าง)	อัตราส่วน	ผ่านไหม
ink uu bg	14.93	ผ่าน
inkSoft uu surface	6.85	ผ่าน
ตัวหนังสือขาว uu primary	5.30	ผ่าน
inkFaint uu surface	2.95	ไม่ผ่าน → จึงห้ามใช้กับข้อความ ใช้กับ placeholder และไอคอนตกแต่งเท่านั้น
ทอง accent เป็นตัวหนังสือบนพื้นขาว	2.61	ไม่ผ่าน → จึงมี token accentInk ที่เข้มกว่าไว้ใช้แทน

คำนวณด้วยสูตรเดียวกับฟังก์ชัน contrast() ในแอป

src/ui/theme.tsบรรทัด 261–279

```
261 function luminance(hex: string): number {
262   const n = parseInt(hex.replace('#', '').slice(0, 6), 16);
263   const ch = (v: number) => {
```

```

264     const c = v / 255;
265     return c <= 0.03928 ? c / 12.92 : ((c + 0.055) / 1.055) ** 2.4;
266 };
267 return 0.2126 * ch((n >> 16) & 255) + 0.7152 * ch((n >> 8) & 255) + 0.0722 * ch(n & 255);
268 }
269
270 /** WCAG contrast ratio between two hex colours (1 to 21). */
271 export function contrast(a: string, b: string): number {
272     const [hi, lo] = [luminance(a), luminance(b)].sort((x, y) => y - x);
273     return (hi + 0.05) / (lo + 0.05);
274 }
275
276 /** The first colour that reads on `bg` at 4.5:1 or better (the last one otherwise). */
277 export function readableOn(bg: string, ...colors: string[]): string {
278     return colors.find((c) => contrast(c, bg) >= 4.5) ?? colors[colors.length - 1];
279 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/ui/theme.ts:261

บรรทัด ความหมาย

- 261–268** luminance: ความสว่างที่ตาคนรับรู้ของสีหนึ่ง (0 = ดำ ถึง 1 = ขาว) แยกสีแดง เขียว น้ำเงิน แปลงค่า แล้วถ่วงน้ำหนัก เขียวหนักสุด (0.7152) เพราะตาคนไวต่อสีเขียวที่สุด
- 270–274** contrast: เอาสีที่สว่างกว่าเป็นตัวตั้ง (บวก 0.05) หาสีที่มืดกว่า (บวก 0.05) ตามสูตรของ WCAG
- 276–279** readableOn: ให้รายการสีมา ค้นสีแรกที่อ่านออกบนพื้นนั้น (≥ 4.5)

src/ui/__tests__/theme.test.ts

บรรทัด 34–58

```

34 for (const spec of COLOR_THEMES) {
35     for (const dark of [false, true]) {
36         it(`${spec.key} ${dark ? 'dark' : 'light'}: every text colour reads on the background it is
used on`, () => {
37             const t = buildTheme(spec.key, dark);
38             const pairs: [string, string, string][] = [
39                 ['inkSoft on bg', t.inkSoft, t.bg],
40                 ['inkSoft on surface', t.inkSoft, t.surface],
41                 ['onPrimary on primary', t.onPrimary, t.primary],
42                 ['onAccent on accent', t.onAccent, t.accent],
43                 ['accentInk on surface', t.accentInk, t.surface],
44                 ['accentInk on accentSoft', t.accentInk, t.accentSoft],
45                 ['good on goodSoft', t.good, t.goodSoft],
46                 ['watch on watchSoft', t.watch, t.watchSoft],
47                 ['critical on criticalSoft', t.critical, t.criticalSoft],
48                 ['good on bg', t.good, t.bg],
49                 ['watch on bg', t.watch, t.bg],
50                 ['critical on bg', t.critical, t.bg],
51                 ['heroInkSoft on hero', t.heroInkSoft, t.hero],
52                 ['heroInk on heroTop', t.heroInk, t.heroTop],
53             ];
54             const low = pairs.filter(([a, b]) => contrast(a, b) < 4.5).map(([n, a, b]) => `${n}
${contrast(a, b).toFixed(2)}`);
55             expect(low).toEqual([]);
56         });
57     }
58 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/ui/__tests__/theme.test.ts:34

เทสต์ TC-78 ตรวจสอบทุกธีม (9) × ทุกโหมด (สว่าง/มืด) = 18 ชุด แต่ละชุดตรวจ 14 คู่มือ (บรรทัด 38–53) ถ้าคู่มือต่ำกว่า 4.5 เทสต์ล้ม และบอกชื่อคู่มือพร้อมค่าที่ได้ (บรรทัด 54) ใครเพิ่มธีมใหม่แล้วเลือกสีอ่านยาก npm test จะไม่ผ่าน และ GitHub จะไม่ build แอปให้

ตัวอักษร

src/ui/theme.ts

บรรทัด 301–328

```
301 /**
302  * Two typefaces:
303  * - Noto Serif Thai for money and headings: the "international banking" feel
304  *   from the logo direction, and clear numerals.
305  * - Anuphan for everything else: a modern Thai sans that reads well small.
306  */
307 export const fonts = {
308   serif: 'NotoSerifThai_600SemiBold',
309   serifBold: 'NotoSerifThai_700Bold',
310   sans: 'Anuphan_400Regular',
311   sansMedium: 'Anuphan_500Medium',
312   sansSemi: 'Anuphan_600SemiBold',
313   sansBold: 'Anuphan_700Bold',
314 };
315
316 export const space = { xs: 4, sm: 8, md: 12, lg: 16, xl: 20, xxl: 28, xxxl: 40 };
317 export const radius = { sm: 8, md: 14, lg: 20, xl: 28, pill: 999 };
318
319 /** Type scale (size / line height). Thai needs generous line height for vowels above and below.
320  */
321 export const type = {
322   display: { fontSize: 38, lineHeight: 52 },
323   h1: { fontSize: 26, lineHeight: 38 },
324   h2: { fontSize: 20, lineHeight: 30 },
325   h3: { fontSize: 17, lineHeight: 26 },
326   body: { fontSize: 15, lineHeight: 24 },
327   small: { fontSize: 13, lineHeight: 20 },
328   micro: { fontSize: 11, lineHeight: 16 },
329 };
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/ui/theme.ts:301

- **Noto Serif Thai** (มีหัว) ใช้กับตัวเลขเงินและหัวข้อใหญ่ ให้ความรู้สึก "ธนาคารสากล" และตัวเลขอ่านชัด
- **Anuphan** (ไม่มีหัว กันสมัย) ใช้กับข้อความอื่นทั้งหมด อ่านง่ายแม้ตัวเล็ก
- บรรทัด 316–317: ระยะห่างมาตรฐาน 4/8/12/16/20/28/40 และมุมโค้ง 8/14/20/28/999 (999 = ปลายมนเป็นเม็ดยา)
- บรรทัด 319–328: ขนาดตัวอักษรคู่กับความสูงบรรทัดเสมอ ภาษาไทยมีสระบนล่างและวรรณยุกต์ ต้องมีความสูงบรรทัดประมาณ 1.4–1.6 เท่าของขนาดตัวอักษร ไม่ขึ้นสระจะชนบรรทัดข้างบน

ชิ้นส่วนสำเร็จรูป: ปุ่ม (Button)

src/ui/components.tsx

บรรทัด 261–338

```
261 export function Button({
262   label,
```

```

263   onPress,
264   kind = 'primary',
265   icon,
266   loading,
267   disabled,
268   style,
269   small,
270   shine,
271 }: {
272   label: string;
273   onPress: () => void;
274   /** onDark: outline button on the dark green hero surfaces */
275   kind?: 'primary' | 'gold' | 'ghost' | 'danger' | 'soft' | 'onDark';
276   icon?: IconName;
277   loading?: boolean;
278   disabled?: boolean;
279   style?: StyleProp<ViewStyle>;
280   small?: boolean;
281   /** A band of light sweeps across (for the one main action on a screen). */
282   shine?: boolean;
283 }) {
284   const theme = useTheme();
285   const press = usePressSpring(0.95);
286   const colors = {
287     primary: { bg: theme.primary, fg: theme.onPrimary, border: theme.primary },
288     gold: { bg: theme.accent, fg: theme.onAccent, border: theme.accent },
289     ghost: { bg: 'transparent', fg: theme.ink, border: theme.line },
290     danger: { bg: 'transparent', fg: theme.critical, border: theme.critical },
291     soft: { bg: theme.surfaceAlt, fg: theme.ink, border: theme.surfaceAlt },
292     onDark: { bg: 'rgba(244,241,230,0.10)', fg: '#F4F1E6', border: 'rgba(244,241,230,0.45)' },
293   }[kind];
294   const off = disabled || loading;
295   return (
296     <AnimatedPressable
297       onPress={() => {
298         Haptics.selectionAsync().catch(() => {});
299         onPress();
300       }}
301       onPressIn={off ? undefined : press.onPressIn}
302       onPressOut={press.onPressOut}
303       disabled={off}
304       // Small buttons are about 40pt tall: the touch area reaches 44pt and more (up and down
305       only,
306       // so buttons side by side never share a touch area).
307       hitSlop={small ? { top: 4, bottom: 4 } : undefined}
308       accessibilityRole="button"
309       aria-disabled={!off}
310       aria-busy={!loading}
311       style={[
312         {
313           backgroundColor: colors.bg,
314           borderColor: colors.border,
315           borderWidth: 1.5,
316           borderRadius: radius.pill,
317           paddingVertical: small ? 8 : 12,
318           paddingHorizontal: small ? 14 : 20,
319           flexDirection: 'row',
320           alignItems: 'center',
321           justifyContent: 'center',
322           gap: 8,
323           opacity: off ? 0.5 : 1,

```

```

323         overflow: shine ? 'hidden' : undefined,
324     },
325     style,
326     { transform: [{ scale: press.scale }] },
327   ]}
328   >
329   {shine && !off ? <Shine times={1} delay={900} color={kind === 'gold' ?
'rgba(255,255,255,0.45)' : 'rgba(255,255,255,0.28)'} /> : null}
330   {loading ? (
331     <ActivityIndicator color={colors.fg} />
332   ) : icon ? (
333     <Ionicons name={icon} size={small ? 16 : 19} color={colors.fg} />
334   ) : null}
335     <Text style={[small ? type.small : type.body, { fontFamily: fonts.sansSemi, color:
colors.fg }]}>{label}</Text>
336   </AnimatedPressable>
337 );
338 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/ui/components.tsx:261`

บรรทัด	ความหมาย
261–283	props ของปุ่ม: ข้อความ, สิ่งที่ทำเมื่อกด, ชนิด 6 แบบ (ค่าเริ่มต้น primary), ไอคอน, คำสั่งโหลด, ปิดใช้งาน, ขนาดเล็ก, แสงวิ่งผ่าน
284–285	สีของธีม และแอนิเมชันยุบตัวตอนกด (เหลือ 95%)
286–293	ตารางสีของปุ่มแต่ละชนิด: primary = พื้นสีขาวตัวขาว, gold = พื้นทอง, ghost = โปร่งมีขอบ, danger = ขอบแดง (ปุ่มลบ), soft = พื้นเทาอ่อน, onDark = โปร่งบนการ์ดเขียว ทุกสีมาจาก theme
294	ปุ่ม "ปิด" ถ้าถูกปิดใช้งานหรือคำสั่งโหลด (กันกดซ้ำ)
297–300	กดแล้วมือถือสั่นเบา ๆ (haptic) แล้วทำสิ่งที่ผู้เรียกกำหนด
304–306	ปุ่มเล็กสูงประมาณ 40 จุด ขยายพื้นที่แตะขึ้นลงอีก 4 จุด ให้ถึง 44 จุดตามแนวทางของ Apple/Google (นิ้วคนกว้างกว่าที่คิด) ขยายเฉพาะแนวตั้ง ปุ่มที่วางติดกันจึงไม่แย่งพื้นที่แตะ
307–309	บอกโปรแกรมอ่านหน้าจอว่าเป็นปุ่ม และสถานะปิด/คำสั่งโหลด
310–327	รูปร่าง: ขอบ 1.5, มุมโค้งแบบเบียด, ระยะในตามขนาด, ไอคอนกับข้อความเรียงแนวนอนตรงกลาง, จางลง 50% เมื่อปิดใช้งาน
329	แสงวิ่งผ่าน 1 ครั้ง (ใช้กับปุ่มหลักปุ่มเดียวของหน้า)
330–334	คำสั่งโหลดแสดงวงหมุนแทนไอคอน
335	ข้อความบนปุ่ม ขนาดตามปุ่ม ฟอนต์หนาถึงหนา

ชิ้นส่วน	ใช้เมื่อ
Screen	ทุกหน้า: พื้นหลัง ขอบ เลื่อนช่องที่คำสั่งพิมพ์ให้พื้นที่บอร์ด
Card	กลุ่มเนื้อหา มี 3 โท: ปกติ, พิเศษ (alt), ท่องอ่อน (accent)
Button / IconButton	การกระทำ 1 หน้ามีปุ่ม primary แค่ 1 ปุ่ม / ปุ่มไอคอนต้องมี label เสมอ

ชิ้นส่วน	ใช้เมื่อ
Segmented	สลับมุมมอง 2–4 แบบ เช่น วันนี้ / 7 วัน / 1 เดือน, รายจ่าย / รายรับ
Chip	ตัวเลือกหลายอัน เช่น หมวดหมู่, จำนวนเงินลด
Field / AmountField	ช่องกรอก ขอบเปลี่ยนสีตอนพิมพ์และตอนผิด บนเว็บตัวหนังสือ 16 เพื่อไม่ให้ iPhone ชุมหน้า
Badge	ป้ายสถานะสั้น ๆ เช่น "สบาย ๆ"
useToast	ยืนยันผล เช่น "บันทึกรายการแล้ว" มีปุ่ม "เลิกทำ" ได้
ConfirmSheet	ถามยืนยันก่อนทำสิ่งที่ย้อนไม่ได้ เช่น ลบ
EmptyState	ยังไม่มีข้อมูล: น้องกล้า + คำอธิบาย + ปุ่มให้เริ่ม

Accessibility: ทุกคนใช้ได้

เรื่อง	ทำยังไง
คนตาบอด/สายตาเลือนราง	ปุ่มทุกปุ่มมี accessibilityLabel ให้ TalkBack (Android) / VoiceOver (iPhone) อ่าน เช่นการ์ดยอดเงินอ่านว่า "ยอดคงเหลือ ฿11,359.00 เงินพอใช้อีก 16 วัน"
แยกสีไม่ออก (ตาบอดสี)	สถานะไม่ใช่สีอย่างเดียว มีข้อความกำกับเสมอ เช่น "ใกล้เส้นสำรอง"
เมารถจากการเคลื่อนไหว	เปิด "ลดการเคลื่อนไหว" (Reduce motion) ในมือถือ ทุกแอนิเมชันหยุดและแสดงผลทันที (useReduceMotion())
นิ้วใหญ่ มือสั่น	พื้นที่แตะอย่างน้อย 44 จุด
อ่านยาก	สีผ่าน WCAG AA ทุกธีม (TC-78)

แอนิเมชัน: สวยแต่ไม่กินแบตเตอรี่

แอปมีเอฟเฟกต์เยอะ (ส่วนต่าง ๆ ค่อย ๆ โผล่, ปุ่มแดง, ตันไม้เงินโยก, พลุตอนสำเร็จ) แต่มีกฎ 4 ข้อจาก README:

1. ขยับเฉพาะ transform (ย้าย หมุน ย่อขยาย) กับ opacity (ความจาง) และใช้ **native driver** ให้การขยับทำงานส่วน native ของมือถือ ไม่ผ่าน JavaScript จึงสิ้นเปลืองแบตเตอรี่ราคาประหยัด
2. ทุกเอฟเฟกต์หยุดเอง ไม่มีอะไรยับยั้งค้างตอนผู้ใช้ไม่ได้ทำอะไร (มี E2E ตรวจสอบว่าหน้าหลักนิ่งภายใน 9 วินาที)
3. เคารพ "ลดการเคลื่อนไหว"
4. ไม่ใช้ native module ใหม่ แอปที่ลงแล้วจึงได้เอฟเฟกต์ใหม่ทาง OTA โดยไม่ต้องลง APK ใหม่

เช็กลิสต์ก่อนเพิ่มหน้าจอใหม่ (จาก DESIGN_SYSTEM.md)

1. ไม่มีรหัสสี #... หรือ rgba (...) ในหน้าจอ ใช้ token หรือ alpha()
2. ตัวหนังสือใช้ <T> และเงินใช้ <Money>
3. ปุ่ม ชิป ช่องกรอก การ์ด มาจาก component ไม่สร้างใหม่เอง
4. ลองเปิดหน้านั้น 2–3 ธีม ทั้งสว่างและมืด

5. รับ npm test, npx tsc --noEmit, npx expo lint, npm run e2e:web

i ทบทวนศึกษา UX จากที่ไหน

บันทึกใน [docs/AI_USAGE_LOG.md](#) แถว 10 และ 19: ศึกษาแอปเหมียวจวด (ผู้ช่วยที่พูดกับผู้ใช้, "แค่เปิดแอปก็จดให้") และแนวการออกแบบของ Hugging Face (มาศคอตตัวเดียวหลายอารมณ์, สีอุ้งบนพื้นเรียบ, การ์ดโค้ง เส้นบาง) และหลัก Laws of UX เช่น aesthetic-usability effect (ของสวยทำให้รู้สึกว่ายาง่าย) และ peak-end rule (คนจำช่วงพีคและตอนจบ จึงมีพลุดอนทำสำเร็จ) แล้วออกแบบตัวละครน้องกล้าของ MindPay เอง ไม่ลอกของใคร

การทดสอบ: พิสูจน์ว่าระบบทำงานถูก

ทีมเราทำการทดสอบ 3 ระดับ: unit test 157 ข้อ, E2E ในเบราว์เซอร์ 145 ข้อ และแผนทดสอบบนมือถือจริง 12 ข้อ บทนี้สอนอ่านและเขียนเทสต์

ทำไมต้องเขียนเทสต์

เวลาแก้ไขโค้ดจุดหนึ่ง อาจทำให้อีกจุดพังโดยไม่รู้ตัว (เรียกว่า **regression**) ถ้าต้องกดทดสอบทุกหน้าเองทุกครั้งก็แย่ จะใช้เวลาหลายชั่วโมงและล้มบางจุดแน่นอน เทสต์อัตโนมัติคือ "โปรแกรมที่ตรวจโปรแกรม" รันเสร็จในไม่กี่วินาที และทีมตั้งไว้ว่าถ้าเทสต์ไม่ผ่าน GitHub จะไม่ปล่อยเว็บและ APK ให้ผู้ใช้

157

Unit test (Vitest)

ตรวจฟังก์ชันทีละตัว · รันไม่ถึง 2 วินาที

145

E2E (Playwright)

เปิดเว็บจริงแล้วกดเหมือนคน · ประมาณ 3 นาที

12

Manual test (MT-01-12)

คนทดสอบบนมือถือจริง

Unit test: ตรวจฟังก์ชันทีละตัว

ไฟล์เทสต์อยู่ในโฟลเดอร์ `__tests__` ชื่อลงท้าย `.test.ts` รันด้วยคำสั่ง `npm test` ใช้เครื่องมือ **Vitest** ตัวอย่างจากเทสต์ของ FR-6:

src/domain/__tests__/domain.test.ts

บรรทัด 35-58

```
35 // Sunday 27 Sep 2026, 14:00 in Bangkok
36 const NOW = new Date('2026-09-27T07:00:00.000Z');
37
38 let seq = 0;
39 function tx(p: Partial<Transaction> & { day: string; amount: number }): Transaction {
40   const { day, amount, ...rest } = p;
41   return {
42     id: `t${++seq}`,
43     kind: 'expense',
44     amountSatang: amount * 100,
45     categoryKey: 'food',
46     title: 'test',
47     note: null,
48     occurredAt: bkkToIso(day, '12:00'),
49     source: 'manual',
50     status: 'confirmed',
51     slipRef: null,
52     slipImageHash: null,
53     ocrConfidence: null,
54     reviewFlags: [],
55     createdAt: NOW.toISOString(),
56     ...rest,
57   };
58 }
```

▶ เปิดใน VS Code (เว็บ) GitHub

VS Code ในเครื่อง: Ctrl+P → src/domain/__tests__/domain.test.ts:35

บรรทัด	ความหมาย
35–36	ล็อก "วันนี้" ให้ตายตัว เป็นวันอาทิตย์ที่ 27 ก.ย. 2569 เวลา 14:00 น. ไม่ว่ารันเทสต์วันไหน ผลก็เหมือนเดิม (ถ้าใช้วันจริง เทสต์จะผ่านบ้างไม่ผ่านบ้างตามวันที่)
38–58	ฟังก์ชันช่วยสร้างรายการปลอม tx({ day, amount }) ใส่แค่วันกับจำนวนบาท ที่เหลือใช้ค่าตั้งต้น (รายจ่าย หมวดอาหาร ยืนยันแล้ว) ...rest ท้ายสุดให้เปลี่ยนช่องอื่นได้ เช่น tx({ day, amount, status: 'draft' })

src/domain/__tests__/domain.test.ts

บรรทัด 184–210

```

184  it('TC-14 computes days and depletion date', () => {
185    const r = computeRunway({ balanceSatang: 350_000, floorSatang: 50_000, averageSatang: 20_000,
now: NOW });
186    expect(r.days).toBe(15);
187    expect(r.depletionDay).toBe('2026-10-12');
188    expect(r.status).toBe('healthy');
189  });
190
191  it('TC-15 handles zero average without dividing by zero', () => {
192    const r = computeRunway({ balanceSatang: 350_000, floorSatang: 50_000, averageSatang: 0, now:
NOW });
193    expect(r.status).toBe('no_spending');
194    expect(r.days).toBeNull();
195    expect(r.depletionDay).toBeNull();
196  });
197
198  it('TC-16 flags balance at or below the low line', () => {
199    const r = computeRunway({ balanceSatang: 50_000, floorSatang: 50_000, averageSatang: 10_000,
now: NOW });
200    expect(r.status).toBe('below_floor');
201    expect(r.days).toBe(0);
202  });
203
204  it('TC-17 status thresholds', () => {
205    const at = (days: number) =>
206      computeRunway({ balanceSatang: 50_000 + days * 10_000, floorSatang: 50_000, averageSatang:
10_000, now: NOW }).status;
207    expect(at(6)).toBe('critical');
208    expect(at(7)).toBe('watch');
209    expect(at(14)).toBe('healthy');
210  });

```

▶ เปิดใน VS Code (เว็บ) GitHub

VS Code ในเครื่อง: Ctrl+P → src/domain/__tests__/domain.test.ts:184

บรรทัด	ความหมาย
184	it('ชื่อเทสต์', () => {...}) = เทสต์ 1 ข้อ ชื่อขึ้นต้นด้วยรหัส TC ที่ตรงกับเอกสาร Traceability
185	Arrange + Act: เตรียมข้อมูล (ยอด ฿3,500, เงินสำรอง ฿500, ใช้วันละ ฿200) แล้วเรียกฟังก์ชันจริง
186–188	Assert: expect(ค่าจริง).toBe(ค่าที่ต้องได้) ตรวจสอบว่าได้ 15 วัน, วันที่ 12 ต.ค. 2569 และสถานะ healthy ถ้าไม่ตรงข้อใดข้อหนึ่ง เทสต์ล้มพร้อมบอกว่าได้อะไรแทน
191–196	TC-15 ทดสอบ กรณีพิเศษ: ค่าเฉลี่ยเป็น 0 ต้องไม่หารด้วยศูนย์ ต้องได้ no_spending และวันเป็น null
198–202	TC-16 ทดสอบ ขอบ: ยอดเท่ากับเงินสำรองพอดี ต้องนับว่า "ต่ำกว่าเงินสำรอง"

บรรทัด	ความหมาย
204–210	TC-17 ทดสอบ ขอบของเกณฑ์ : 6 วัน critical, 7 วัน watch, 14 วัน healthy ถ้ามีคนเผลอแก้ < เป็น <= เทสต์นี้จะจับได้ทันที

★ เขียนเทสต์ที่ดี: ทดสอบขอบ ไม่ใช่แค่ค่ากลาง ๆ

บ๊ิกชอบซ่อนอยู่ตรงขอบ เช่น 0, ค่าเท่ากันพอดี, ค่าว่าง, วันแรกของเดือน, เกียงคืน, ข้อมูลภาษาไทยแปลก ๆ เทสต์ของทิมจึงมีทั้งกรณีปกติ กรณีพิเศษ และกรณีขอบ อาจารย์มักถามว่า "ทดสอบกรณีไหนบ้าง" ให้ตอบเป็น 3 กลุ่มนี้

ไฟล์ unit test หลัก (มีทั้งหมด 18 ไฟล์)

ไฟล์	ทดสอบ
domain.test.ts	เงิน, วันที่ไทย, ยอดคงเหลือ (FR-2), Runway (FR-6), สลิป (FR-4), คำแนะนำโค้ช (FR-5), หมวด
voice.test.ts	แยกประโยคพูดเป็นรายการ ตัวเลขคำไทย (FR-3)
slipAccuracy.test.ts, autoScan.test.ts	ความแม่นยำการอ่านสลิป และสแกนอัตโนมัติ (FR-4)
recurring.test.ts, recap.test.ts	รายการประจำเดือน (FR-1) และสรุปเดือนปฏิทิน (FR-2)
goals.test.ts	กระปุกออม (FR-6)
buddy.test.ts, klaTalk.test.ts	คำพูดน้องกล้า และการตัดซับไตเติลอ่านออกเสียง (FR-5)
auth.test.ts	ลิงก์อีเมลและข้อความ error ภาษาไทย
audit.test.ts	บ๊ิกที่เจอตอนตรวจทั้ง repo (1–3 ต.ค.) กันไม่ให้กลับมาอีก
ui/__tests__/theme.test.ts	สีทุกธีมอ่านง่าย (TC-78)
supabase/functions/_shared/__tests__/helpers.test.ts	ตัวช่วยของเซิร์ฟเวอร์: เลือก AI, ทำความสะอาดผลอ่านสลิป, รวมผลอ่าน 2 ครั้ง

E2E: ทดสอบทั้งแอปเหมือนคนใช้จริง

End-to-End (E2E) test เปิดเว็บเวอร์ชันจริงในเบราว์เซอร์อัตโนมัติ (เครื่องมือ **Playwright**) แล้วกด พิมพ์ เลื่อน ตามขั้นตอนเหมือนคน และตรวจว่าเห็นสิ่งที่ควรเห็น ไฟล์หลักคือ [e2e/web-auth.mjs](#) (ยาวกว่า 1,000 บรรทัด)

≈ ไม่แตะบัญชีจริง: เซิร์ฟเวอร์ปลอม

ถ้า E2E ใช้ Supabase จริง จะสร้างบัญชีขยะ ส่งอีเมลจริง และเปลืองโควตา AI ทีมเราจึงเขียน

`e2e/fake-backend.mjs` เป็น "Supabase ปลอม" ที่ตอบเหมือนของจริงแต่อยู่ในหน่วยความจำ ทุกคำขอที่ไม่ได้อยู่ของ Supabase จะถูกดักมาตอบด้วยตัวปลอมนี้ (`page.route`) เหมือนซ้อมละครกับนักแสดงแทน

`e2e/web-auth.mjs`

บรรทัด 29–60

```
29 // ----- helpers
30 const results = [];
31 function check(name, ok, detail = '') {
32   results.push({ name, ok, detail });
33   console.log(`${ok ? 'PASS' : 'FAIL'} ${name}${detail ? ` - ${detail}` : ''}`);
34   // In GitHub Actions a failed check also becomes an annotation on the run.
35   if (!ok && IN_CI) console.log(`::error title=${ENGINE} browser test::${name}${detail ? `
36   (${detail.replace(/\n/g, ' ').slice(0, 300)}):` : ''}`);
37 }
38 const browser = await (ENGINE === 'webkit' ? webkit : chromium).launch();
39 const phone = {
40   ...(ENGINE === 'webkit' ? { isMobile: true, hasTouch: true } : {}),
41   // E2E_SCHEME=dark runs everything with the phone in dark mode.
42   colorScheme: process.env.E2E_SCHEME === 'dark' ? 'dark' : 'light',
43 };
44 async function freshPage(tag) {
45   const ctx = await browser.newContext({ viewport: { width: 390, height: 844 },
46     deviceScaleFactor: 2, locale: 'th-TH', ...phone });
47   const page = await ctx.newPage();
48   page.on('pageerror', (e) => console.log(`[${tag}] pageerror`, e.message));
49   page.on('console', (m) => { if (m.type() === 'error') console.log(`[${tag}] console.error`,
50     m.text().slice(0, 200)); });
51   page.on('requestfailed', (r) => { if (!r.url().startsWith(SUPA)) console.log(`[${tag}] request
52     failed`, r.url().slice(0, 160)); });
53   await page.route(`${SUPA}/**`, supabase);
54   // The web QR reader downloads its decoder from a CDN: serve the same file from node_modules.
55   await routeQrDecoder(page);
56   return { ctx, page };
57 }
58 const shot = (page, name) => page.screenshot({ path: join(SHOTS, `${name}.png`) });
59 const visible = (page, text, timeout = 6000) => page.getByText(text, { exact: false
60   }).first().waitFor({ state: 'visible', timeout }).then(() => true, () => false);
61 const gone = (page, text, timeout = 6000) => page.getByText(text, { exact: false
62   }).first().waitFor({ state: 'hidden', timeout }).then(() => true, () => false);
63 const introGone = (page) => page.getByLabel('กำลังเปิด MindPay').waitFor({ state: 'detached',
64   timeout: 8000 }).catch(() => {});
65 const button = (page, name) => page.getByRole('button', { name, exact: false }).first();
66 /** Waits (up to `timeout`) until the element is visible. */
67 const shown = (locator, timeout = 6000) => locator.waitFor({ state: 'visible', timeout }).then(()
68   => true, () => false);
```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `e2e/web-auth.mjs:29`

บรรทัด ความหมาย

30–36 `check` (ชื่อ, ผ่านไหม, รายละเอียด) บันทึกผลแต่ละข้อ พิมพ์ PASS หรือ FAIL ถ้ารันบน GitHub และไม่ผ่าน จะขึ้นเป็นป้าย error บนหน้า Actions

บรรทัด	ความหมาย
37–42	เปิดเบราว์เซอร์ Chromium (หรือ WebKit แบบ Safari) โหมดสว่างหรือมืด
43–53	freshPage: เปิดแท็บใหม่ขนาดมือถือ (390×844) ภาษาไทย ดักคำขอไป Supabase ให้ตัวปลอมตอบ และพิมพ์ error ของหน้าเว็บถ้ามี
54	shot: ถ่ายภาพหน้าจอเก็บไว้ (ภาพหน้าจอหลายภาพในเล่มนี้มาจากตรงนี้)
55–56	visible / gone: รอจนเห็นข้อความ (หรือข้อความหายไป) ภายใน 6 วินาที
58	button: หาปุ่มจากชื่อที่โปรแกรมอ่านหน้าจออ่าน (accessibilityLabel) แอปที่ทำ accessibility ดีจึงเขียนทดสอบง่าย

ตัวอย่าง: ทดสอบแบบ "ลองปรับ" ของ FR-6

e2e/web-auth.mjs

บรรทัด 572–627

```

572 // 14. Runway: "can I buy this?" shows the days before/after and hands the question to the
    coach
573 {
574   const { ctx, page } = await freshPage('runway');
575   await page.goto(APP);
576   await visible(page, 'ลองใช้ด้วยข้อมูลตัวอย่าง', 8000);
577   await introGone(page);
578   await page.getByText('ลองใช้ด้วยข้อมูลตัวอย่าง').click();
579   await visible(page, 'ยอดคงเหลือ', 8000);
580   await page.goto(`${APP}runway`);
581   await introGone(page);
582   // What-if slider: drag the gold thumb to spend 30% less a day.
583   const slider = page.getByRole('slider', { name: 'ใช้น้อยลงวันละกี่เปอร์เซ็นต์' }).first();
584   await slider.scrollIntoViewIfNeeded();
585   // A raw mouse drag does not wait like click() does: make sure the opening animation has
    fully gone.
586   const introWait = Date.now();
587   await page.getByLabel('กำลังเปิด MindPay').waitFor({ state: 'detached', timeout: 20000
    }).catch(() => {});
588   const introMs = Date.now() - introWait;
589   // Wait until the page stops moving (cards slide in), so the drag starts on the thumb.
590   let box = await slider.boundingBox();
591   for (let i = 0; i < 12; i++) {
592     await page.waitForTimeout(250);
593     const again = await slider.boundingBox();
594     const still = box && again && Math.abs(again.y - box.y) < 0.5 && Math.abs(again.x - box.x)
    < 0.5;
595     box = again;
596     if (still) break;
597   }
598   const under = box
599     ? await page.evaluate(({ x, y }) => {
600       const el = document.elementFromPoint(x, y);
601       return el ? `${el.tagName} role=${el.getAttribute('role')}`
        label=${el.getAttribute('aria-label')}` : 'nothing';
602     }, { x: box.x + 15, y: box.y + box.height / 2 })
603     : 'no box';
604   if (box) {
605     // hover() waits until the slider itself receives the pointer (nothing on top of it).
606     await slider.hover({ position: { x: 15, y: box.height / 2 } }).catch(() => {});
607     await page.mouse.down();

```

```

608     await page.mouse.move(box.x + 15 + (box.width - 30) * 0.3, box.y + box.height / 2, { steps:
8 });
609     await page.mouse.move(box.x + 15 + (box.width - 30) * 0.6, box.y + box.height / 2, { steps:
8 });
610     await page.mouse.up();
611 }
612 const slid = (await slider.getAttribute('aria-valuenow').catch(() => null)) ?? '';
613 await page.waitForTimeout(900);
614 await shot(page, '28-runway-slider');
615 check('Runway: dragging the slider to 30% less a day shows the extra days', slid === '30' &&
(await visible(page, 'เพิ่มขึ้น'), `value=${slid} box=${JSON.stringify(box)} under=${under}
introWaitMs=${introMs}`);
616 // The arrow keys move it too (keyboard users on the web).
617 await slider.focus();
618 await page.keyboard.press('ArrowRight');
619 const keyed = (await slider.getAttribute('aria-valuenow').catch(() => null)) ?? '';
620 check('Runway: the slider also moves with the arrow keys', keyed === String(Number(slid || 0)
+ 5), `value=${keyed}`);
621 await page.keyboard.press('Home');
622 await page.fill('#runway-price', '2000');
623 check('Runway: buying ฿2,000 shows the days before and after', (await visible(page, 'ถ้าซื้อ
฿2,000')) && (await visible(page, 'ตอนนี้')));
624 await button(page, 'ถามโค้ชเรื่องนี้').click();
625 const asked = await page.locator('#coach-input').inputValue().catch(() => '');
626 check('Runway: "ask the coach" opens the coach with the question ready',
asked.includes('฿2,000'), asked);
627 await ctx.close();

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → e2e/web-auth.mjs:572

บรรทัด	ความหมาย
574–579	เปิดแท็บใหม่ ไปหน้าต้อนรับ รอแอนิเมชันเปิดแอปหาย และ "ลองใช้ด้วยข้อมูลตัวอย่าง" แล้วรอเห็นคำว่า "ยอดคงเหลือ"
580	ไปที่หน้าเงินพอถึง
583	หาแถบเลื่อนจากชื่อ "ใช้น้อยลงวันละกี่เปอร์เซ็นต์"
589–597	รอนหน้าจอหยุดขยับ (การ์ดค่อย ๆ เลื่อนเข้ามา) ไม่เงินจะลากพลาด
604–611	จำลองเมาส์: กดที่ปุ่มกลมของแถบ ลากไป 30% แล้ว 60% ของความกว้าง แล้วปล่อย
612–615	อ่านค่าปัจจุบันของแถบ ถ่ายภาพ และตรวจว่าได้ 30 และมีคำว่า "เพิ่มขึ้น"
617–620	ทดสอบคีย์บอร์ด: กดลูกศรขวา ค่าต้องเพิ่ม 5
622–623	พิมพ์ราคา 2000 ในเช็ก่อนจ่าย ต้องเห็น "ถ้าซื้อ ฿2,000" และ "ตอนนี้"
624–626	กด "ถามโค้ชเรื่องนี้" ช่องพิมพ์ในหน้าโค้ชต้องมีคำถามที่มี "฿2,000" รออยู่ (เชื่อม FR-6 กับ FR-5)

ผลล่าสุด: **145/145 ขั้วผ่าน** ทั้งโหมดสว่างและมืด (รันซ้ำตอนทำเล่มนี้ วันที่ 9 ต.ค. 2569 ก็ได้ 145/145 และ unit test 157/157) และมี workflow รันชุดเดียวกันบน WebKit (เอนจินของ Safari) แบบ iPhone โหมดมืดด้วย

Manual test: ทดสอบบนมือถือจริง

บางอย่างทดสอบอัตโนมัติไม่ได้ เช่น ไมโครโฟนจริง แกลเลอรีจริง สลิปจริงจากธนาคาร ทีมจึงมีแผน MT-01 ถึง MT-12 ใน [docs/TRACEABILITY.md](#) ให้สมาชิกทดสอบบนมือถือแล้วกรอกผล

ID	ขั้นตอน	ผลที่คาดหวัง
MT-02	จดรายจ่าย ฿85 หมวดอาหาร	ยอดคงเหลือลด ฿85, หมวดอาหารเพิ่ม, Runway เปลี่ยน
MT-03	ลบรายการ แล้วกด "เลิกทำ"	รายการกลับมา ยอดเท่าเดิม
MT-08	ถามโค้ช "ชื่อของ 500 บาทวันนี้ได้ไหม"	ตอบไทย อ้าง Runway ก่อน/หลัง ไม่ตำหนิ
MT-10	บัญชี A ไม่เห็นรายการของบัญชี B	RLS กันไว้
MT-11	บันทึกสลิปจริงลงแกลเลอรี แล้วเปิดแอป	ภายในไม่กี่วินาทีขึ้น "เพิ่ม 1 รายการจากสลิปใหม่"
MT-12	พูด "ข้าว 50 บาท น้ำ 15"	เห็น 2 รายการ บันทึกแล้วยอดลด ฿65

! ตรวจสอบนำเสนอ

ในเอกสาร Traceability ช่อง "ผล" ของ MT ทุกข้อยังเป็น ☐ (ยังไม่ได้กรอก) และในบันทึกการใช้ AI ช่อง "Human verification" ยังเป็น ☐ หลายข้อ ทีมควรทดสอบจริงและกรอกผลก่อนส่งงาน เพราะอาจารย์อาจถามว่า "ทดสอบบนมือถือจริงหรือยัง"

Traceability: ผูก Requirement → โค้ด → ทดสอบ

วิชา Software Engineering เน้น "Golden Thread" คือทุก requirement ต้องโยงไปหาโค้ดที่ทำ และเทสต์ที่พิสูจน์ได้ ทีมเราเก็บไว้ใน [docs/TRACEABILITY.md](#) เป็นตาราง 3 คอลัมน์: Acceptance criteria | Code | Test เช่น

Acceptance criteria (FR-6)	Code	Test
ค่าเฉลี่ยนับวันที่ไม่ได้ใช้เงินด้วย	averageDailyExpense	TC-12
ยอด ≤ เส้นสำรอง → สถานะ below_floor	computeRunway	TC-16

แต่ละเล่มในภาค ข มีตารางแบบนี้ครบทุกข้อของ FR นั้น

ตรวจสอบคุณภาพอื่น ๆ

คำสั่ง	ตรวจสอบอะไร
npm run typecheck	TypeScript ตรวจสอบนิพจน์ข้อมูลทั้งโปรเจกต์ (0 error)
npx expo lint	ESLint ตรวจสอบรูปแบบโค้ดและกฎของ React Hooks
Supabase Advisors	ตัวตรวจสอบความปลอดภัยและประสิทธิภาพของฐานข้อมูล (แจ้งเตือน 0 ปัญหาเรื่องตาราง)

คำสั่ง	ตรวจสอบอะไร
ผู้ตรวจอิสระ	ตอนตรวจทั้ง repo (1–3 ต.ค.) ให้ผู้ตรวจที่ไม่ได้เห็นงานเทียบพฤติกรรมก่อน/หลังแก้ (เช่น ประโยคพูด 260 ประโยค ชื่อผู้รับ 120 ชื่อ) เจอ regression 10 ข้อ แก้ครบพร้อมทดสอบ

Git และ GitHub: เก็บโค้ด ทำงานเป็นทีม และ build อัตโนมัติ

ทุกการแก้ไขโค้ดถูกบันทึกเป็นประวัติ และทุกครั้งที่เราส่งโค้ดขึ้น GitHub จะมีหุ่นยนต์ 4 ตัวช่วยทดสอบ สร้างเว็บ สร้าง APK และส่งอัปเดตให้ผู้ใช้

Git: ระบบบันทึกประวัติ

≈ เปรียบเทียบ

Git เหมือนปุ่ม "Save game" ของเกม ทุกครั้งที่ทำสำเร็จหนึ่งด่าน (แก้ฟิเจอร์เสร็จ) เราเซฟไว้พร้อมโน้ตว่าทำอะไร ถ้าพลาดภายหลังก็โหลดเซฟเก่ากลับมาได้ และดูย้อนได้ว่าใครเปลี่ยนอะไรเมื่อไร

คำ	ความหมาย
repository (repo)	โฟลเดอร์โปรเจกต์ที่ Git ดูแล รวมประวัติทั้งหมด (ซ่อนอยู่ในโฟลเดอร์ .git)
commit	"จุดเซฟ" 1 ครั้ง มีรหัส (เช่น 646b485) ข้อความอธิบาย ผู้ทำ และเวลา
branch	"เส้นทางคู่ขนาน" ของโค้ด แยกไปทำฟิเจอร์โดยไม่กระทบเส้นหลัก เส้นหลักของทีมชื่อ main
remote / origin	สำเนาของ repo บนเซิร์ฟเวอร์ (GitHub)
push	ส่ง commit ในเครื่องขึ้น GitHub
pull	ดึง commit ใหม่จาก GitHub ลงเครื่อง
clone	คัดลอก repo ทั้งหมดจาก GitHub ลงเครื่องครั้งแรก
pull request (PR)	คำขอรวม branch หนึ่งเข้า main ให้เพื่อนรีวิวก่อน
merge conflict	สองคนแก้บรรทัดเดียวกัน Git ไม่รู้จะเลือกของใคร ต้องตัดสินใจเอง
.gitignore	รายชื่อไฟล์ที่ไม่ต้องเก็บใน Git เช่น node_modules (ใหญ่และติดตั้งใหม่ได้), .env (อาจมีความลับ), dist (สร้างใหม่ได้)

ใช้ Git ประจำวัน

Terminal

วงจรทำงาน 1 รอบ

```

1 git pull                                # 1. ดึงของล่าสุดก่อนเริ่ม
2 git checkout -b fix-runway-text         # 2. แยก branch ใหม่ (ไม่บังคับ)
3 # ... แก้โค้ดใน VS Code ...
4 npm test                                # 3. รันเทสต์ให้ผ่านก่อน
5 git status                              # 4. ดูว่าแค่ไฟล์ไหนไปบ้าง
6 git add src/domain/runway.ts            # 5. เลือกไฟล์ที่จะเซฟ

```

```
7 git commit -m "Runway: say 'ประมาณ' in the date" # 6. เซฟพร้อมข้อความ
8 git push -u origin fix-runway-text # 7. ส่งขึ้น GitHub แล้วเปิด Pull Request
```

ใน VS Code ทำแบบเดียวกันได้ด้วยเมาส์ที่เก็บ **Source Control** (Ctrl+Shift+G): ไฟล์ที่แก้ไขขึ้นในรายการ กด + เพื่อเลือก
พิมพ์ข้อความ กด Commit แล้วกด Sync Changes

★ ข้อความ commit ที่ดี

บอกว่า "ทำอะไร และทำไม" สั้น ๆ เช่นของทีมเรา: *"Runway: arrow keys on the web; steadier drag test"*, *"Audit follow-up: fix regressions found by independent review"* อ่านประวัติแล้วรู้เรื่องโดยไม่ต้องเปิดโค้ด ดูประวัติทั้งหมดด้วย `git log --oneline`

ประวัติของ MindPay บน GitHub

repo อยู่ที่ github.com/2550expo-tech/Socrates-and-Skeletons - (สาธารณะ ใครก็อ่านได้) branch main มี **66 commit** ตั้งแต่ 27 ก.ย. ถึง 3 ต.ค. 2569

วันที่	commit	งานหลักของวันนั้น (จากข้อความ commit)
27 ก.ย.	20	MVP แรก FR-1, FR-2, FR-4, FR-5, FR-6 · build APK อัตโนมัติ · เว็บบน GitHub Pages · EAS Update · AI ใช้ Gemini หรือ Claude · อ่านสลิปใหม่อัตโนมัติตอนเปิดแอป
28 ก.ย.	22	ระบบสมัครแบบรหัสอีเมล, บัญชี, สแกนเองแยกตามช่วงเวลา, แก้ปัญหาคุกกี้ AI, อ่าน QR มุมขวาล่าง, เทสต์ในเบราว์เซอร์ (รวม Safari)
29 ก.ย.	19	พุดจด (FR-3), บัญชีพุดจด, ธีมสี 8 แบบ, เพิ่มเงินเข้า, รายการประจำ, กระปุกออม
30 ก.ย.	2	ธีมฮาโลวีน, เทสต์ความอ่านง่ายของสี
1-3 ต.ค.	3	ตรวจทั้ง repository แกะบีก จัด Design System ให้ตรงกัน

GitHub Actions: หุ่นยนต์ที่ทำงานทุกครั้ง push

CI/CD (Continuous Integration / Continuous Delivery) คือให้คอมพิวเตอร์ทดสอบและส่งของให้อัตโนมัติทุกครั้งโค้ดเปลี่ยน ของทีมเราเขียนเป็นไฟล์ YAML ใน `.github/workflows/` มี 4 ตัว

ไฟล์	ทำอะไร	ผลลัพธ์
<code>web.yml</code>	build เว็บ → unit test → E2E 145 ข้อ → ถ้าผ่านทั้งหมดจึงเผยแพร่	เว็บที่ 2550expo-tech.github.io/Socrates-and-Skeletons/
<code>android-apk.yml</code>	unit test → typecheck → สร้างโปรเจกต์ Android → build APK (15-25 นาที)	ไฟล์ <code>mind-pay.apk</code> ในหน้า Releases "latest-apk"
<code>eas-update.yml</code>	unit test → ส่งโค้ด JavaScript ใหม่ขึ้น EAS Update	แอปที่ลงแล้วได้โค้ดใหม่ตอนเปิดครั้งถัดไป

ไฟล์	ทำอะไร	ผลลัพธ์
e2e-webkit.yml	E2E ชัดเดียวกันบนแอนดรอยด์ Safari แบบ iPhone โหมดมืด	รายงานผล (ไม่บล็อกการเผยแพร่)

git push ขึ้น main

↓ GitHub เปิดเครื่องเสมือน Ubuntu ให้ฟรีตัวละ 1 เครื่อง ทำพร้อมกัน

web.yml

เทสต์ไม่ผ่าน = เว็บไม่เปลี่ยน ผู้ใช้ยังได้เวอร์ชันเดิมที่ต่ออยู่

android-apk.yml

เทสต์หรือ typecheck ไม่ผ่าน = ไม่มี APK ใหม่

eas-update.yml

ต้องมี secret EXPO_TOKEN และเทสต์ผ่าน

e2e-webkit.yml

ตรวจบน Safari เพิ่ม

อ่าน web.yml ที่ละบรรทัด

.github/workflows/web.yml

บรรทัด 1-76

```

1 # Publishes the web version of MindPay to GitHub Pages on every push to main:
2 # https://2550expo-tech.github.io/Socrates-and-Skeletons-/
3 # The web version shows the app at phone width, works with real accounts and
4 # demo data, and reads slips from photos the user uploads. Scanning the whole
5 # gallery is only in the phone app. On iPhone: open in Safari -> Share ->
6 # Add to Home Screen, and it opens full screen with the MindPay icon.
7 name: Deploy web app
8
9 on:
10   push:
11     branches: [main]
12   workflow_dispatch:
13
14 permissions:
15   contents: write
16
17 concurrency:
18   group: web-pages
19   cancel-in-progress: true
20
21 jobs:
22   web:
23     runs-on: ubuntu-latest
24     timeout-minutes: 20
25     steps:
26       - uses: actions/checkout@v4
27
28       - uses: actions/setup-node@v4
29         with:
30           node-version: 22
31           cache: npm
32
33       - run: npm ci
34
35       - name: App settings (public Supabase URL and publishable key)
36         run: cp .env.example .env

```

```

37
38   - name: Export web build
39     env:
40       EXPO_WEB_BASE_URL: /Socrates-and-Skeletons-
41     run: npx expo export --platform web --output-dir dist
42
43   - name: Home-screen icon and full-screen mode (iPhone "Add to Home Screen")
44     env:
45       EXPO_WEB_BASE_URL: /Socrates-and-Skeletons-
46     run: node scripts/web-home-screen.mjs dist
47
48   - name: Single-page app fallback for deep links
49     run: cp dist/index.html dist/404.html
50
51   - name: Unit tests
52     run: npm test
53
54   # Browser test of every sign-in path (sign up, email code, links, reset
55   # password, sign out) against a local stand-in for Supabase. If anything
56   # fails, the website is NOT published and the screenshots are attached.
57   - name: Install the test browser
58     run: npx playwright install --with-deps chromium
59
60   - name: Browser tests (sign-in flows)
61     run: node e2e/web-auth.mjs dist e2e-shots
62
63   - name: Keep screenshots when the browser tests fail
64     if: failure()
65     uses: actions/upload-artifact@v4
66     with:
67       name: e2e-screenshots
68       path: e2e-shots
69
70   - name: Publish to the gh-pages branch
71     uses: peaceiris/actions-gh-pages@v4
72     with:
73       github_token: ${ secrets.GITHUB_TOKEN }
74       publish_dir: ./dist
75       force_orphan: true
76       commit_message: 'Deploy web app'

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `.github/workflows/web.yml:1`

บรรทัด	ความหมาย
1-6	คอมเมนต์ (YAML ใช้ #) บอกว่าเผยแพร่เว็บไปที่ไหน และวิธีเพิ่มลงหน้าจอโฮมบน iPhone
7	ชื่อที่แสดงในแท็บ Actions ของ GitHub
9-12	เมื่อไรจะทำงาน: ทุกครั้งที่ push ขึ้น branch main และกดสั่งเองได้ (workflow_dispatch)
14-15	สคริปต์เขียนลง repo (ต้องใช้ตอนเผยแพร่ไป branch gh-pages)
17-19	ถ้า push ใหม่มาระหว่างที่ตัวเก่ายังทำงาน ยกเลิกตัวเก่า ทำแต่ตัวล่าสุด
21-24	งานชื่อ web รับบนเครื่อง Ubuntu ล่าสุด ไม่เกิน 20 นาที
26	ดีทิลโค้ดของ commit นี้ลงเครื่อง (uses = ใช้ action สำเร็จรูปที่คนอื่นเขียนไว้)

บรรทัด	ความหมาย
28–31	ติดตั้ง Node.js 22 และจำ cache ของ npm ให้ครั้งหน้าเร็วขึ้น
33	npm ci ติดตั้งแพ็คเกจตาม package-lock.json แบบเป๊ะ (ต่างจาก npm install ตรงที่ไม่เปลี่ยนเวอร์ชันเลย)
35–36	สร้าง .env จากตัวอย่าง (มีแค่กุญแจสาธารณะ)
38–41	build เว็บลงโพลเดอร์ dist พร้อมบอกว่าเว็บอยู่ใต้ /Socrates-and-Skeletons-
43–46	เพิ่มไอคอนและตั้งค่าให้ "เพิ่มไปยังหน้าจอล็อกอิน" แล้วเปิดเต็มจอเหมือนแอป
48–49	คัดลอกหน้าแรกเป็น 404.html เพราะ GitHub Pages ไม่รู้จักที่อยู่อย่าง /runway ถ้ามีคนเปิดตรง ๆ จะได้หน้า 404 ซึ่งก็คือแอปเราที่พาไปหน้านั้นให้
51–52	รัน unit test 157 ข้อ ไม่ผ่าน = หยุดทั้งงาน
57–61	ติดตั้งเบราวเซอร์ทดสอบ แล้วรัน E2E 145 ข้อ
63–68	if: failure() = เฉพาะตอนพัง: เก็บภาพหน้าจอจาก E2E ไว้ให้ดาวนโหลดดูว่าพังตรงไหน
70–76	ผ่านทุกด่านแล้วจึงเผยแพร่โพลเดอร์ dist ขึ้น branch gh-pages ซึ่ง GitHub Pages ใช้เป็นเว็บไซต์

อ่าน android-apk.yml (ส่วนสำคัญ)

.github/workflows/android-apk.yml

บรรทัด 12–90

```

12 on:
13   push:
14     branches: [main]
15     # Editing documents only does not need a new APK.
16     paths-ignore: ['**/*.md', 'docs/**']
17   workflow_dispatch:
18
19 permissions:
20   contents: write
21
22 # A build that already started runs to the end; newer pushes wait in line
23 # (only the newest waiting push is kept).
24 concurrency:
25   group: android-apk
26   cancel-in-progress: false
27
28 jobs:
29   apk:
30     runs-on: ubuntu-latest
31     timeout-minutes: 45
32     steps:
33       - uses: actions/checkout@v4
34
35       - uses: actions/setup-node@v4
36         with:
37           node-version: 22
38           cache: npm
39
40       - uses: actions/setup-java@v4
41         with:
42           distribution: temurin

```

```

43     java-version: 17
44
45     - name: Install dependencies
46       run: npm ci
47
48     - name: Unit tests
49       run: npm test
50
51     - name: Typecheck
52       run: npx tsc --noEmit
53
54     - name: App settings (public Supabase URL and publishable key)
55       run: cp .env.example .env
56
57     - name: Generate the native Android project
58       run: npx expo prebuild --platform android --no-install
59
60     - name: Give Gradle enough memory
61       # The default Metaspace ran out during Android lint and the build hung until the job
timeout.
62       run: |
63         sed -i '/^org.gradle.jvmargs=/d' android/gradle.properties
64         # The generated file has no trailing newline: start with one so the setting is not
glued onto the last line.
65         printf '\norg.gradle.jvmargs=-Xmx4g -XX:MaxMetaspaceSize=1g -Dfile.encoding=UTF-8\n' >>
        android/gradle.properties
66         tail -n 3 android/gradle.properties
67
68     - name: Build release APK
69       working-directory: android
70       # arm64 only: covers practically every Android phone from the last 7 years and builds
much faster.
71       # lintVitalAnalyzeRelease is skipped: it is a code-style check that is not needed to
produce the APK.
72       run: ./gradlew assembleRelease --no-daemon -PreactNativeArchitectures=arm64-v8a -x
        lintVitalAnalyzeRelease --stacktrace
73
74     - name: Rename APK
75       run: cp android/app/build/outputs/apk/release/app-release.apk mind-pay.apk
76
77     - uses: actions/upload-artifact@v4
78       with:
79         name: mind-pay-apk
80         path: mind-pay.apk
81
82     - name: Publish to the "latest-apk" release
83       env:
84         GH_TOKEN: ${GITHUB_TOKEN}
85       run: |
86         gh release delete latest-apk --yes --cleanup-tag || true
87         gh release create latest-apk mind-pay.apk \
88           --title "MindPay (latest APK)" \
89           --notes "Built from ${GITHUB_SHA::7} on $(date -u +%Y-%m-%d %H:%M UTC'). Open this
        page on an Android phone, tap mind-pay.apk, and install. Debug-signed build for testing." \
90           --prerelease

```

บรรทัด	ความหมาย
12–17	ทำงานเมื่อ push ขึ้น main ยกเว้นแค่เอกสาร (.md, docs/) ซึ่งไม่ต้อง build APK ใหม่
24–26	ตัวที่เริ่ม build แล้วให้ทำงานจบ ตัวใหม่ต่อคิว (build ใช้เวลานาน ไม่อยากทิ้งกลางทาง)
31	เวลาสูงสุด 45 นาที
40–43	ติดตั้ง Java 17 เพราะเครื่องมือ build ของ Android (Gradle) ต้องใช้
48–52	ด้านตรวจ: unit test และ typecheck ต้องผ่านก่อน
57–58	expo prebuild สร้างไฟล์เดอร์ android ทั้งโปรเจกต์จาก app.json (ทีมไม่เก็บไฟล์เดอร์นี้ใน Git สร้างใหม่ทุกครั้ง)
60–66	เพิ่มหน่วยความจำให้ Gradle (คอมเมนต์เล่าว่าเคยค้างจนหมดเวลาเพราะหน่วยความจำไม่พอ)
68–72	build APK แบบ release เฉพาะสถาปัตยกรรม arm64 (ครอบคลุมมือถือแทบทุกเครื่องใน 7 ปีหลัง และ build เร็วกว่า)
74–80	เปลี่ยนชื่อเป็น mind-pay.apk และเก็บไว้
82–90	ลบ release "latest-apk" เดิม แล้วสร้างใหม่พร้อมไฟล์ APK ใหม่ (ลิงก์ดาวน์โหลดจึงเหมือนเดิมตลอด) บอกว่า build จาก commit ไหน เมื่อไร

📌 APK แบบ debug-signed

APK นี้ "เซ็น" ด้วยกุญแจ debug ของ Android ใช้ทดสอบและพีเชนต์ได้ แต่ส่งขึ้น Play Store ไม่ได้ ถ้าจะลง Play Store ต้อง build ด้วย EAS Build แบบ production และมีบัญชีนักพัฒนา Google (README อธิบายไว้)

Secrets บน GitHub

ค่าลับที่ workflow ต้องใช้ เก็บใน Settings → Secrets and variables → Actions ของ repo ไม่ได้อยู่ในโค้ด

Secret	ใช้ที่	ได้มาจาก
GITHUB_TOKEN	web.yml, android-apk.yml	GitHub สร้างให้อัตโนมัติทุกครั้ง
EXPO_TOKEN	eas-update.yml	expo.dev → Account settings → Access tokens (ถ้าไม่ตั้ง workflow นี้ ข้ามไปเลย ๆ)

ส่วนกุญแจ AI ไม่ได้อยู่บน GitHub เลย อยู่ใน Supabase Secrets (un A10)

ไฟล์เอกสารใน repo

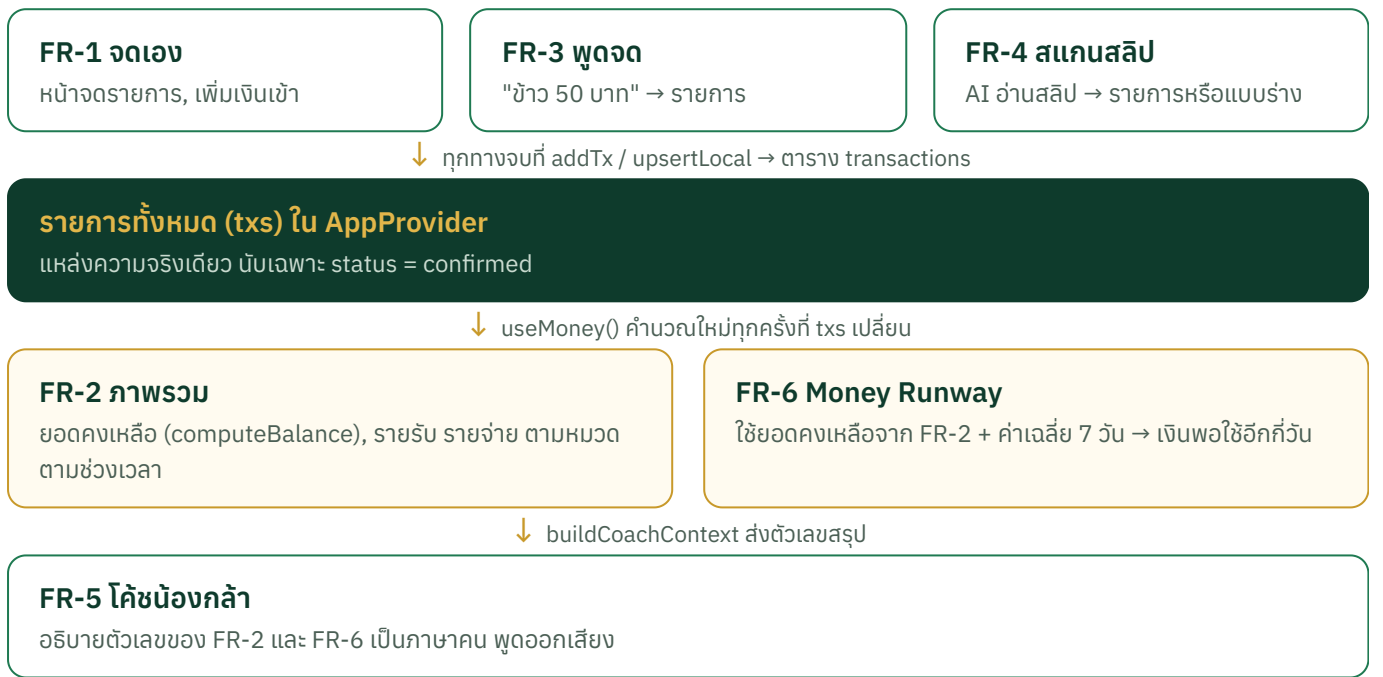
ไฟล์	มีอะไร
README.md	หน้าแรกของ repo: ลิงก์ลองใช้, FR, สถาปัตยกรรม, วิธีติดตั้ง, ตั้งค่า AI, ความปลอดภัย, ข้อจำกัด
docs/TRACEABILITY.md	FR → acceptance criteria → โค้ด → เทสต์ และแผนทดสอบมือ (MT)

ไฟล์	มีอะไร
docs/AI_USAGE_LOG.md	บันทึกการใช้ AI ตามที่ SRS กำหนด 20 แถว
docs/DESIGN_SYSTEM.md	กฎการออกแบบ (บท A11)
docs/SUPABASE_AUTH_SETUP.md	ตั้งค่าอีเมลยืนยันตัวตนของ Supabase
AGENTS.md / CLAUDE.md	คำแนะนำสำหรับผู้ช่วย AI ที่ช่วยเขียนโค้ด (ใช้ Expo 57, คำสั่งที่ควรรับ)

6 FR ทำงานร่วมกันยังไง และคำถามระดับระบบ

ไม่ว่ารับผิดชอบ FR ไหน อาจารย์อาจถามว่า "FR ของเธอเชื่อมกับของเพื่อนยังไง" หรือ "ทำไมเลือกเทคโนโลยีนี้" บทนี้เตรียมคำตอบไว้ให้

สายข้อมูลของทั้งระบบ



ใครใช้ของใคร

FR	ใช้ข้อมูล/ฟังก์ชันจาก	ส่งผลต่อ
FR-1	money.ts (แปลงจำนวนเงิน), categories.ts (หมวด), dates.ts	ทุกรายการที่จต/แก้/ลบ ทำให้ FR-2, FR-6, FR-5 คำนวณใหม่; หน้าจดยรายการยังเป็นหน้าตรวจสอบสลิปของ FR-4
FR-2	รายการจาก FR-1/3/4, ยอดตั้งต้นจากโปรไฟล์	ยอดคงเหลือเป็นตัวตั้งของ FR-6, ยอดตามหมวดส่งให้โค้ช FR-5
FR-3	categories.ts (เดาหมวด), money/dates, addTx ของ FR-1	สร้างรายการแบบยืนยันแล้ว เข้ายอดทันที
FR-4	Edge Function parse-slip + AI, หน้าจดยรายการของ FR-1 (ตรวจแบบร่าง), categories.ts	รายการจากสลิป: ชัด → เข้ายอด, ไม่ชัด → แบบร่าง (ไม่นับใน FR-2/FR-6 จนยืนยัน)
FR-5	ตัวเลขจาก FR-2 และ FR-6, Edge Function coach + AI	ไม่เปลี่ยนข้อมูล (อ่านอย่างเดียว) แต่ชวนผู้ใช้ไปหน้า Runway/สแกน
FR-6	computeBalance (FR-2), รายจ่ายที่ยืนยันแล้ว (FR-1/3/4), เส้นสำรองจากโปรไฟล์, กระปุกออม	ตัวเลข Runway บนหน้าหลัก, การ์ดเตือนของโค้ช, "เช็กก่อนจ่าย" ส่งคำถามให้ FR-5

ไฟล์ที่หลาย FR ใช้ร่วมกัน (แก้แล้วกระทบหลายคน)

ไฟล์	ใครใช้	ถ้าแก้ผิดจะเกิดอะไร
<code>src/domain/types.ts</code>	ทุก FR	TypeScript จะแจ้ง error ทั่วทั้งแอป
<code>src/domain/money.ts</code>	FR-1, 3, 4, 6	จำนวนเงินเพี้ยนทุกที่
<code>src/domain/dates.ts</code>	ทุก FR	รายการไปอยู่ผิดวัน ค่าเฉลี่ย 7 วันผิด
<code>src/domain/summary.ts</code>	FR-2, 5, 6	ยอดคงเหลือผิด → Runway และโค้ชผิดตาม
<code>src/domain/categories.ts</code>	FR-1, 2, 3, 4	หมวดผิด กราฟตามหมวดผิด
<code>src/data/AppProvider.tsx</code>	ทุกหน้า	ข้อมูลไม่โหลด/ไม่อัปเดต
<code>src/app/transaction.tsx</code>	FR-1 และ FR-4 (ตรวจสอบ)	จดยรายการหรือยืนยันสลิปไม่ได้

นี่คือเหตุผลที่ทีมมี unit test ครอบคลุมไฟล์เหล่านี้ทั้งหมด และให้ GitHub รับเทสต์ก่อนปล่อยทุกครั้ง

คำถามระดับระบบ ที่อาจารย์ถามใครก็ได้

- Q1** ทำไมเลือก Expo / React Native?

โค้ดชุดเดียวได้ทั้ง Android, iPhone และเว็บ ทีมเล็กเวลาน้อยจึงไม่ต้องเขียน 3 รอบ ภาษาเดียว (TypeScript) ทั้งแอปและเซิร์ฟเวอร์ มีเครื่องมือ build APK และส่งอัปเดตอัตโนมัติ
- Q2** ทำไมเลือก Supabase?

ได้ฐานข้อมูล PostgreSQL, ระบบสมัคร/เข้าสู่ระบบ, และที่รับโค้ดฝั่งเซิร์ฟเวอร์ในที่เดียว มีแพ็คเกจฟรี และ Row Level Security ทำให้แอปคุยกับฐานข้อมูลได้ตรง ๆ อย่างปลอดภัยโดยไม่ต้องเขียนเซิร์ฟเวอร์ API เอง
- Q3** ข้อมูลผู้ใช้ปลอดภัยยังไง?

ทุกตารางเปิด RLS แต่ละคนอ่านเขียนได้เฉพาะแถวของตัวเอง (ทดสอบ MT-10) คุกกี้ในแอปเป็นคุกกี้สาธารณะ คุกกี้แวลบและคุกกี้ AI อยู่บนเซิร์ฟเวอร์เท่านั้น รูปสลิปไม่ถูกเก็บบนเซิร์ฟเวอร์ โค้ชได้แค่ออกรวม
- Q4** ทำไมเก็บเงินเป็นสตางค์?

ทศนิยมในคอมพิวเตอร์ไม่เป๊ะ ($0.1 + 0.2 = 0.30000000000000004$) เก็บเป็นจำนวนเต็มสตางค์จึงบวกลบได้แม่นยำ 100% แปลงเป็นบาทตอนแสดงผลเท่านั้น
- Q5** ทำไมต้องจัดวันตามเวลาไทย?

ฐานข้อมูลเก็บเวลา UTC ซึ่งช้ากว่าไทย 7 ชั่วโมง ถ้าไม่แปลง รายการตอนเข้ามีดจะไปอยู่วันก่อนหน้า ยอด "วันนี้" และค่าเฉลี่ย 7 วันจะผิด ทุกจุดจึงใช้ bkkDayKey
- Q6** ยอดคงเหลือเก็บไว้ในฐานข้อมูลไหม?

ไม่ เก็บเฉพาะรายการ ตัวเลขทั้งหมด (ยอด ค่าเฉลี่ย Runway) คำนวณสดในเครื่องจากรายการทุกครั้งที่ข้อมูลเปลี่ยน ตัวเลขจึงไม่มีทางขัดกันเอง
- Q7** แบบร่าง (draft) คืออะไร ทำไมต้องมี?

รายการจากสลิปที่ AI อ่านไม่มั่นใจ รอผู้ใช้ตรวจสอบก่อน ยังไม่นับในยอดใด ๆ เพราะถ้า AI อ่านผิดแล้วนับทันที ทุกตัวเลขในแอปจะผิดตาม
- Q8** ใช้ AI ตรงไหน และป้องกัน AI ผิดยังไง?

2 ที่: อ่านสลิป (FR-4) และโค้ช (FR-5) ป้องกันด้วย: อ่าน 2 ครั้งด้วย 2 รุ่น, เหนือความมั่นใจ 80%, ตรวจสอบ QR บนสลิป, แบบร่างรอตรวจ, โค้ชห้ามแต่งตัวเลข ตัวเลขทั้งหมดคำนวณโดยโค้ดที่ทดสอบแล้ว

Q9	ทำไมไม่เรียก AI จากแอปตรง ๆ? กฎแฉ AI จะหลุดไปกับแอป Edge Function ถือกฎแฉไว้ ตรวจสอบว่าเข้าสู่ระบบจริง และจำกัดโควตาต่อวันต่อคน
Q10	ทดสอบยังไง? 3 ระดับ: unit test 157 ข้อ (ฟังก์ชันคำนวณ), E2E 145 ข้อ (เปิดเว็บจริงเหมือนคน ด้วยเซิร์ฟเวอร์ปลอม), manual test MT-01 ถึง MT-12 บนมือถือจริง ทุกครั้งที่ push GitHub รันเทสก่อนเผยแพร่
Q11	ถ้าอินเทอร์เน็ทหลุดระหว่างฟรีเซนต์? ใช้โหมดทดลอง ข้อมูลอยู่ในเครื่อง ทำงานได้ทุกอย่างยกเว้น AI
Q12	Requirement โยงกับโค้ดยังไง? เอกสาร docs/TRACEABILITY.md ผูกทุก acceptance criteria ของทุก FR กับไฟล์/ฟังก์ชันที่ทำ และรหัสเทส (TC/MT) ที่พิสูจน์
Q13	ใช้ AI ช่วยเขียนโค้ดไหม? ใช้ บันทึกไว้ใน docs/AI_USAGE_LOG.md ตามที่ SRS กำหนด ทุกแฉบอกจุดประสงค์ ผลลัพธ์ สิ่งที่ทรมานหรือตัดสินใจ และสิ่งที่สมาชิกต้องตรวจสอบ (ควรตอนตามจริงว่าตัวเองตรวจสอบอะไรไปแล้วบ้าง)
Q14	ถ้ามีผู้ใช้เยอะขึ้น ระบบรับไหวไหม? การคำนวณทำในเครื่องของแต่ละคน เซิร์ฟเวอร์แค่เก็บข้อมูล มี index ช่วยค้นเร็ว และโหลดเป็นหน้า หน้าละ 1,000 แถว ส่วนที่ต้องระวังคือค่าใช้จ่าย AI ซึ่งมีโควตาต่อวันคุมไว้ และควรเปลี่ยนจาก Gemini แบบฟรีเป็นแบบเสียเงินเมื่อมีผู้ใช้จริง
Q15	อะไรที่ยังไม่ได้ทำ (นอก MVP)? Accountability Mode (UC-7), ออฟไลน์เต็มระบบ (UC-8), เชื่อม Gmail/LINE, รวบรวมรับในอนาคตเข้า Money Runway และยังไม่ได้ทดสอบบนมือถือหลายรุ่น

คำศัพท์รวม

คำ	ความหมายแบบง่าย
Acceptance criteria	เงื่อนไขที่ต้องผ่าน จึงจะถือว่า requirement นั้นเสร็จ
API / API key	ประตูให้โปรแกรมคุยกัน / รหัสลับที่ใช้ผ่านประตู
APK	ไฟล์ติดตั้งแอป Android
async / await	เขียนงานที่ต้องรอ (เช่นอินเทอร์เน็ท) โดยหน้าจอไม่ค้าง
CI/CD	ให้คอมพิวเตอร์ทดสอบและส่งของให้อัตโนมัติทุกครั้งที่เกิดเปลี่ยน (GitHub Actions)
Commit / Push / Branch	จุดเซฟ / ส่งขึ้น GitHub / เส้นทางคู่ขนานของโค้ด
Component	ชิ้นส่วนหน้าจอใน React เป็นฟังก์ชันที่คืน JSX
Context / Provider	วิธีแชร์ข้อมูลให้ทุกหน้าใน React (AuthProvider)
Design System / Token	กฎและชิ้นส่วนกลางของการออกแบบ / ค่ากลางที่ตั้งชื่อตามหน้าที่ เช่น theme.ink
E2E test	ทดสอบทั้งแอปเหมือนคนกดจริงในเบราว์เซอร์
Edge Function	โค้ดฝั่งเซิร์ฟเวอร์บน Supabase (parse-slip, coach)
EAS Update / OTA	ส่งโค้ดใหม่ไปถึงแอปที่ลงแล้ว โดยไม่ต้องลง APK ใหม่

คำ	ความหมายแบบง่าย
Expo Router	ระบบเปลี่ยนหน้าที่ไฟล์ 1 ไฟล์ = หน้าจอ 1 หน้า
Hook	ฟังก์ชันพิเศษของ React ขึ้นต้นด้วย use เช่น useState
JSON	รูปแบบข้อมูลข้อความที่โปรแกรมใช้รับส่งกัน
JSX	หน้าที่ที่เขียนคล้าย HTML ในโค้ด React
Migration	ไฟล์ SQL ที่สร้าง/แก้ไขโครงสร้างฐานข้อมูล เรียงตามวันที่
Native module	ส่วนที่เขียนด้วยภาษาของมือถือ (เช่น ไมค์ แกลเลอรี) ต้องลง APK ใหม่ถึงจะมี
props / state	ค่าที่ส่งเข้า component / ความจำของ component
Pure function	ฟังก์ชันที่ใช้ค่าเดิมได้ผลเดิม ไม่ยุ่งกับข้างนอก ทดสอบง่าย
Regression	ของที่เคยทำงาน กลับพังหลังแก้ไขโค้ดส่วนอื่น
Repo (Repository)	1) โพลเดอร์โปรเจกต์ใน Git 2) ชั้น Repo ในแอปที่ซ่อนว่าข้อมูลเก็บที่ไหน
RLS	Row Level Security กฎในฐานข้อมูลให้แต่ละคนเห็นเฉพาะแถวของตัวเอง
Satang	หน่วยเงินในระบบ 100 สตางค์ = 1 บาท
Schema (JSON)	แบบฟอร์มบังคับโครงสร้างคำตอบของ AI
Session / Token	สถานะการเข้าสู่ระบบ / บัตรผ่านดิจิทัลที่แนบไปกับทุกคำขอ
SQL	ภาษาสั่งฐานข้อมูล
System prompt	คำสั่งประจำตัวที่บอก AI ว่าต้องเป็นใคร ตอบยังไ้
Traceability	การโยง requirement → การออกแบบ → โค้ด → เทสต์
TypeScript	JavaScript ที่มีชนิดข้อมูล ช่วยจับผิดตั้งแต่ตอนเขียน
Unit test	ทดสอบฟังก์ชันทีละตัว
WCAG	มาตรฐานความเข้าถึงได้ของเว็บ เช่น ความต่างสีตัวหนังสือ $\geq 4.5:1$

ภาค ข

FR-1 Transaction Management

จดยรายการ เพิ่ม แก้ จัดหมวด ลบ ภาคนี้เขียนในมุมมองของคนสร้าง FR-1 เอง ตั้งแต่ ปัญหาที่ยากแก่ หน้าจอ หลักการคำนวณ โค้ดจริงที่ละบรรทัด การทดสอบ ไปจนถึง คำถามที่อาจารย์น่าจะถาม

FR-1 ในหน้าเดียว

ถ้ามีคนถามว่า "FR-1 ของเธอทำอะไร" ให้ตอบจากหน้านี้

FR-1 **Transaction Management** คือ "สมุดบัญชี" ของ MindPay ผู้ใช้ **เพิ่ม แก้ไข จัดหมวด และลบ** รายการเงินของตัวเองได้ ทุกรายการที่บันทึกจะทำให้ยอดคงเหลือ (FR-2) และ Money Runway (FR-6) คำนวณใหม่ทันที นอกจากนี้หน้าจอรายการของฉันทยังถูกใช้เป็นหน้า "ตรวจสอบสลิป" ของ FR-4 ด้วย

1 รายการ = ประเภท (รายรับ/รายจ่าย) + จำนวนเงิน (สตางค์) + หมวด + ชื่อ + วันเวลา (+ บันทึก)

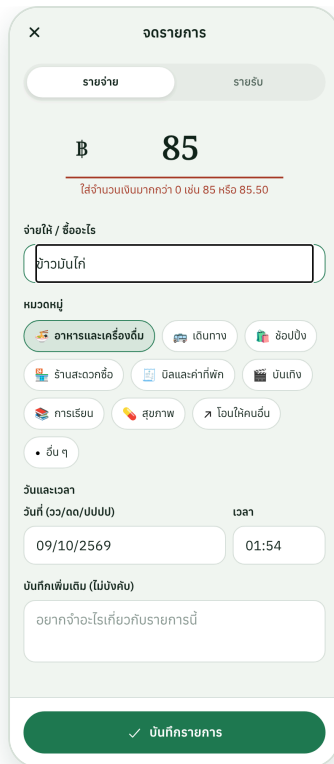
จำนวนเงินต้องมากกว่า 0 และทศนิยมไม่เกิน 2 ตำแหน่ง · หมวดแนะนำให้เองจากชื่อที่พิมพ์ · วันเวลาเป็นเวลาไทย · ลบแล้วกด "เลิกทำ" ได้ภายใน 5 วินาที

อยู่ตรงไหนในแอป

ที่	ทำอะไรได้
หน้าหลัก ปุ่ม "จดยรายการ"	จดยรายจ่าย (เช่น เงินสด) แบบฟอร์มเปิดเป็นรายจ่าย
หน้าหลัก ปุ่ม "เพิ่มเงินเข้า"	จดยรายรับ (เงินที่เข้ามาโดยไม่มีสลิป) มีปุ่มลด +฿100/500/1,000/5,000
แท็บ "รายการ"	ดูรายการทั้งหมดแยกตามวัน ค้นหา กรองรายรับ/รายจ่าย ปฏิทินการใช้จ่าย แตะรายการเพื่อแก้ไขหรือลบ
หน้าตรวจสอบสลิป (เปิดจากสลิปรอยืนยัน)	หน้าเดียวกับหน้าจดยรายการ แต่ไฮไลต์ช่องที่ AI ไม่แน่ใจ และปุ่ม "ยืนยันและรวมในยอดเงิน"

3 ข้อที่ต้องจำ

1. **เงินเก็บเป็นสตางค์** ผู้ใช้พิมพ์ "85.50" แอปแปลงเป็น 8,550 สตางค์ด้วย parseBahtToSatang ถ้าแปลงไม่ได้ (0, ตัวอักษร, ทศนิยม 3 ตำแหน่ง) จะไม่บันทึกและบอกวิธีแก้
2. **หมวดเดาให้เอง แต่ผู้ใช้มีสิทธิ์สุดท้าย** พิมพ์ "ข้าวมันไก่" แอปเลือกหมวด "อาหารและเครื่องดื่ม" ให้ แต่ถ้าผู้ใช้แตะเลือกหมวดเองแล้ว แอปจะไม่เปลี่ยนให้อีก
3. **ลบต้องยืนยัน และย้อนได้** ถ้ามกก่อนลบ ลบแล้วมีปุ่ม "เลิกทำ" 5 วินาที



✕ จดรายการ

รายจ่าย รายรับ

฿ 85

ใส่จำนวนเงินมากกว่า 0 เช่น 85 หรือ 85.50

จ่ายให้ / ชื่ออะไร

ข้าวมันไก่

หมวดหมู่

อาหารและเครื่องดื่ม เงินทาง ซอปปิง

ร้านสะดวกซื้อ บิลและค่าที่พัก บันทึกรายการ

การเรียน สุขภาพ โบนัส/โบนัส

อื่น ๆ

วันและเวลา

วันที่ (วว/ดด/ปปปป) เวลา

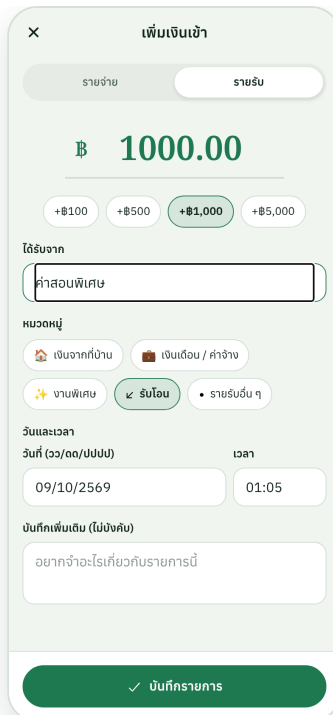
09/10/2569 01:54

บันทึกเพิ่มเติม (ไม่บังคับ)

อยากทำอะไรเกี่ยวกับรายการนี้

✓ บันทึกรายการ

หน้าจอลงรายการ: ฿85 ข้าวมันไก่
หมวดอาหารถูกเลือกให้เอง



✕ เพิ่มเงินเข้า

รายจ่าย รายรับ

฿ 1000.00

+฿100 +฿500 +฿1,000 +฿5,000

ได้รับจาก

ค่าสอนพิเศษ

หมวดหมู่

เงินจากที่บ้าน เงินเดือน / ค่าจ้าง

งานพิเศษ รับโอน รายรับอื่น ๆ

วันและเวลา

วันที่ (วว/ดด/ปปปป) เวลา

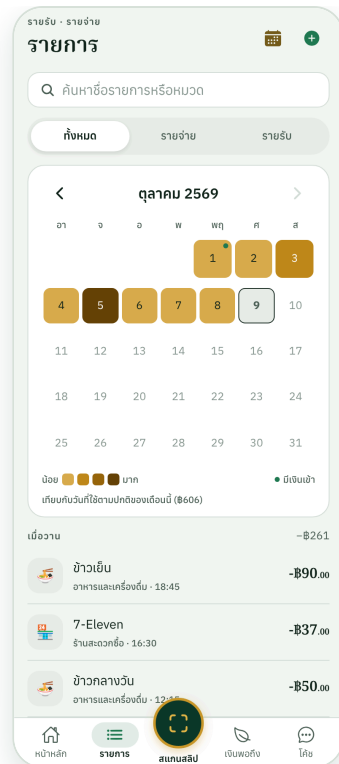
09/10/2569 01:05

บันทึกเพิ่มเติม (ไม่บังคับ)

อยากทำอะไรเกี่ยวกับรายการนี้

✓ บันทึกรายการ

เพิ่มเงินเข้า: เปิดเป็นรายรับ มีปุ่มลด
จำนวนเงิน



รายรับ · รายจ่าย

รายการ

ค้นหาชื่อรายการหรือหมวดหมู่

ทั้งหมด รายจ่าย รายรับ

< ตุลาคม 2569 >

อา จ อ พ พก ศ ส

1 2 3

4 5 6 7 8 9 10

11 12 13 14 15 16 17

18 19 20 21 22 23 24

25 26 27 28 29 30 31

น้อย มาก

เทียบกับวันที่เลือกตามปกติของเดือนนี้ (฿606)

เดือนรวม -฿261

ข้าวเย็น อาหารและเครื่องดื่ม · 18:45 -฿90.00

7-Eleven ร้านสะดวกซื้อ · 16:30 -฿37.00

ข้าวกลางวัน อาหารและเครื่องดื่ม · 12:00 -฿50.00

หน้าหลัก รายการ สแกนสลิป เงินพอเพียง ใต้

แท็บรายการ: ปฏิทินการใช้จ่าย และ
รายการแยกตามวัน

ปัญหาที่ฉันทายากแก้

ทุก FR ในแอปฟัง "รายการ" ถ้าจดยาก คนก็ไม่จด และตัวเลขทั้งแอปก็ผิด

ปัญหาที่ทีมพบตอนทำ SRS คือ ผู้ใช้ที่รับ-จ่ายผ่าน mobile banking บ่อยมักเลื่อนการจด เพราะต้องกรอกซ้ำ ข้อมูลจึงไม่ครบ และเห็นเพียงยอดคงเหลือ FR-4 (สแกนสลิป) และ FR-3 (พุดจด) ช่วยเรื่องนี้ แต่ **ยังมีเงินที่ไม่มีสลิปเสมอ**: เงินสด ค่าขนมที่พ่อแม่ให้มือ เงินที่เพื่อนคืน และสลิปที่ AI อ่านผิดก็ต้องมีที่ให้แก้

FR-1 จึงต้องเป็น **ทางที่เร็วที่สุดในการจดด้วยมือ และทางเดียวในการแก้ทุกอย่าง** เป้าหมาย UX ในเอกสาร Traceability คือ "จดย่อยจ่ายเงินสด 1 รายการ ≤ 5 แตะ, ≤ 20 วินาที"

เป้าหมายที่ฉันทตั้งไว้ตอนออกแบบ

เป้าหมาย	ทำยังไงในแอป
จดเร็ว	ปุ่ม "จดย่อยรายการ" และ "เพิ่มเงินเข้า" บนหน้าหลัก, ช่องเงินตัวใหญ่ตรงกลาง แป็นตัวเลขเด็งขึ้นมเอง, วันเวลาเป็น "ตอนนี้" ไว้ให้แล้ว, หมวดเดาให้จากชื่อ
ไม่ให้ข้อมูลผิดเข้าระบบ	ตรวจจำนวนเงินก่อนบันทึก, วันที่ต้องไม่เกินวันนี้, ฐานข้อมูลตรวจซ้ำอีกชั้น (amount > 0, ชื่อ ≤ 120 ตัว)
แก้ได้ทุกช่อง	แตะรายการไหนก็เปิดฟอร์มเดิมพร้อมค่าเก่า แก้แล้วบันทึก
ลบแล้วไม่ตกใจ	ถามยืนยัน บอกผลกระทบ ("ยอดคงเหลือและ Money Runway จะคำนวณใหม่ทันที") และเลิกทำได้
หาเจอง่าย	แยกตามวัน (วันนี้ เมื่อวาน ...), ค้นหาจากชื่อ หมวด หรือบันทึก, กรองรายรับ/รายจ่าย, ปฏิทินแตะดูทีละวัน
เป็นส่วนตัว	ผู้ใช้เห็นเฉพาะรายการของตัวเอง (RLS ฐานข้อมูล)

Requirement และเงื่อนไขผ่าน

จาก docs/TRACEABILITY.md: Acceptance criteria → โค้ด → เทสต์

i Requirement FR-1

"ผู้ใช้เพิ่ม แก้ไข จัดหมวด และลบธุรกรรมของตนได้"

Acceptance criteria	โค้ด	เทสต์
จำนวนเงินต้องมากกว่า 0 และมีทศนิยมไม่เกิน 2 ตำแหน่ง	money.ts parseBahtToSatang	TC-01
เก็บเงินเป็นสตางค์ ผลรวมไม่เพี้ยน	money.ts	TC-02
เลือกหมวดได้ และแนะนำหมวดจากชื่อร้าน	categories.ts	TC-25
เพิ่มเงินเข้าเองได้ง่าย: ปุ่ม "เพิ่มเงินเข้า" เปิดฟอร์มเป็นรายรับพร้อมจำนวนลัด (+฿100/500/1,000/5,000) เพราะเงินเข้ามักไม่มีสลิป (เพิ่ม 29 ก.ย. ตามที่ผู้ใช้ขอ)	index.tsx (ActionTile), transaction.tsx (kind=income)	E2E money
รายการประจำรายเดือน (ค่าหอ ค่าเน็ต ค่าขนม): หากจากรายการเอง บอกวันครบกำหนดถัดไป	recurring.ts	TC-72
ลบต้องยืนยันก่อน และกดเลิกทำได้	transaction.tsx	MT-03, E2E money
ผู้ใช้เห็นเฉพาะรายการของตัวเอง	supabase/migrations RLS	MT-10
(ตรวจ repo 1-3 ต.ค.) ช่องเงินรับเลขไทย ทศนิยมแบบลูกน้ำ คอมมา คั่นหลักพัน เพดาน ฿1,000,000,000 · วันที่/เวลาที่พิมพ์บนเว็บต้องถูกต้องก่อนบันทึก · ปิดหน้าต่างบันทึกแล้วไม่ย้อนเกิน	money.ts, transaction.tsx, ui/nav.ts	TC-79, E2E

รายการ 1 รายการ มีอะไรบ้าง

ก่อนดูหน้าจอ ต้องรู้ว่าแอปเก็บอะไรไว้ในรายการหนึ่ง

ช่อง (ในแอป)	คอลัมน์ (ในฐานข้อมูล)	ความหมาย	FR-1 ตั้งค่าอย่างไร
id	id	รหัสไม่ซ้ำ	ฐานข้อมูลสร้างให้
kind	kind	'income' หรือ 'expense'	ปุ่มสลับ รายจ่าย/รายรับ
amountSatang	amount_satang	จำนวนเงิน (บวกเสมอ)	ช่องเงินตัวใหญ่ แปลงเป็นสตางค์
categoryKey	category_key	รหัสหมวด เช่น 'food'	ชิปหมวด (เอาให้จากชื่อ)
title	title	ชื่อ ≤ 120 ตัวอักษร	ช่อง "จ่ายให้ / ชื่ออะไร" หรือ "ได้รับจาก"
note	note	บันทึกเพิ่มเติม ≤ 500 ตัว หรือ null	ช่องบันทึก (ไม่บังคับ)
occurredAt	occurred_at	เวลาที่เกิด (เก็บแบบ UTC)	วันและเวลา (แสดงแบบไทย)
source	source	'manual' หรือ 'slip'	'manual' เสมอ
status	status	'draft' หรือ 'confirmed'	'confirmed' (นับในยอดทันที)
slipRef, slipImageHash, ocrConfidence	-	ข้อมูลจากสลิป (FR-4)	null
reviewFlags	review_flags	ช่องที่ AI ไม่แน่ใจ	[] (ว่าง)

หมวดหมู่ 15 หมวด

ทีมเลือกใช้รายการหมวดตายตัวสำหรับ MVP เพื่อให้รายงานเทียบกันได้ทุกเดือน (คอมเมนต์ใน

`src/domain/categories.ts`)

รายจ่าย (10)	ตัวอย่างคำที่ใช้เดา	รายรับ (5)	ตัวอย่างคำที่ใช้เดา
 อาหารและเครื่องดื่ม	ร้าน ข้าว กาแฟ ชานม kfc grabfood	 เงินจากที่บ้าน	พ่อ แม่ ค่าขนม
 เดินทาง	bts mrt grab วิน ปตท ค่ารถ		
 ช้อปปิ้ง	shopee lazada central uniqlo	 เงินเดือน / ค่าจ้าง	เงินเดือน salary ค่าจ้าง
 ร้านสะดวกซื้อ	7-eleven เซเว่น family mart lawson		

รายจ่าย (10)	ตัวอย่างคำที่ใช้เดา
 บิลและค่าที่พัก	ค่าไฟ ประปา หอพัก ค่าเช่า ais
 บันเทิง	major netflix spotify เกม
 การเรียน	มหาวิทยาลัย หนังสือ ถ่ายเอกสาร
 สุขภาพ	โรงพยาบาล คลินิก ร้านยา ฟิตเนส
➤ โอนให้คนอื่น	นาย นาง น.ส. mr. ms.
• อื่น ๆ	(ค่าเริ่มต้นของรายจ่าย)

รายรับ (5)	ตัวอย่างคำที่ใช้เดา
✨ งานพิเศษ	freelance ฟรีแลนซ์ พาร์กไทม์
✓ รับโอน	(ค่าเริ่มต้นของรายรับ)
• รายรับอื่น ๆ	-

บทที่ 5

ทิวทัศน์หน้าจอ: จดรายการและเพิ่มเงินเข้า

ไฟล์ src/app/transaction.tsx หน้าเดียวทำได้ 4 อย่าง: จดใหม่ เพิ่มเงินเข้า แก้ไข และตรวจสอบสลิป

รายการ

รายการ

฿ 0.00

ใส่จำนวนเงินมากกว่า 0 เช่น 85 หรือ 85.50

จ่ายให้ / ชื่ออะไร

เช่น ข้าวมันไก่

หมวดหมู่

อาหารและเครื่องดื่ม เดินทาง ช้อปปิ้ง

ร้านสะดวกซื้อ บิลและค่าที่พัก บันทึงบิล

การเรียนรู้ สุขภาพ โบนัส

อื่น ๆ

วันและเวลา

วันที่ (วว/ดด/ปปปป) เวลา

09/10/2569 01:54

บันทึกเพิ่มเติม (ไม่บังคับ)

อย่ากรอกข้อมูลเกี่ยวกับรายการนี้

บันทึกรายการ

กดบันทึกโดยไม่ใส่เงิน: ขึ้น "ใส่จำนวนเงินมากกว่า 0 เช่น 85 หรือ 85.50" ในช่อง และไม่บันทึก

- 1 **หัวข้อ:** ปุ่มปิด (×) ชื่อหน้าเปลี่ยนตามโหมด ("รายการ" / "เพิ่มเงินเข้า" / "แก้ไขรายการ" / "ตรวจสอบสลิป") และปุ่มถึงขยะ (เฉพาะตอนแก้ไข)
- 2 **สลับ รายการ / รายการ** เปลี่ยนแล้วรายการหมวดเปลี่ยนตาม
- 3 **ช่องจำนวนเงินตัวใหญ่** ฟอนต์มีหัวแบบเดียวกับยอดเงินบนหน้าหลัก เป็นพิมพ์ตัวเลขมีจุดทศนิยม เป็นสีเขียวเมื่อเป็นรายการรับ
- 4 **"จ่ายให้ / ชื่ออะไร"** (หรือ "ได้รับจาก" ถ้าเป็นรายการรับ) ระหว่างพิมพ์แอปเด้งหมวดให้
- 5 **หมวดหมู่** ซิป 10 หมวด (รายการ) หรือ 5 หมวด (รายการ) และเลือกเอง
- 6 **วันและเวลา** ค่าเริ่มต้นคือตอนนี้ บนเว็บพิมพ์ "วว/ดด/ปปปป" แบบ พ.ศ. บน Android เปิดปฏิทินของเครื่อง บน iPhone ใช้ตัวเลือกแบบ compact
- 7 **บันทึกเพิ่มเติม** (ไม่บังคับ)
- 8 **ปุ่ม "บันทึกรายการ"** ติดด้านล่าง

! สิ่งที่น่าสนใจเห็นตอนทดสอบ

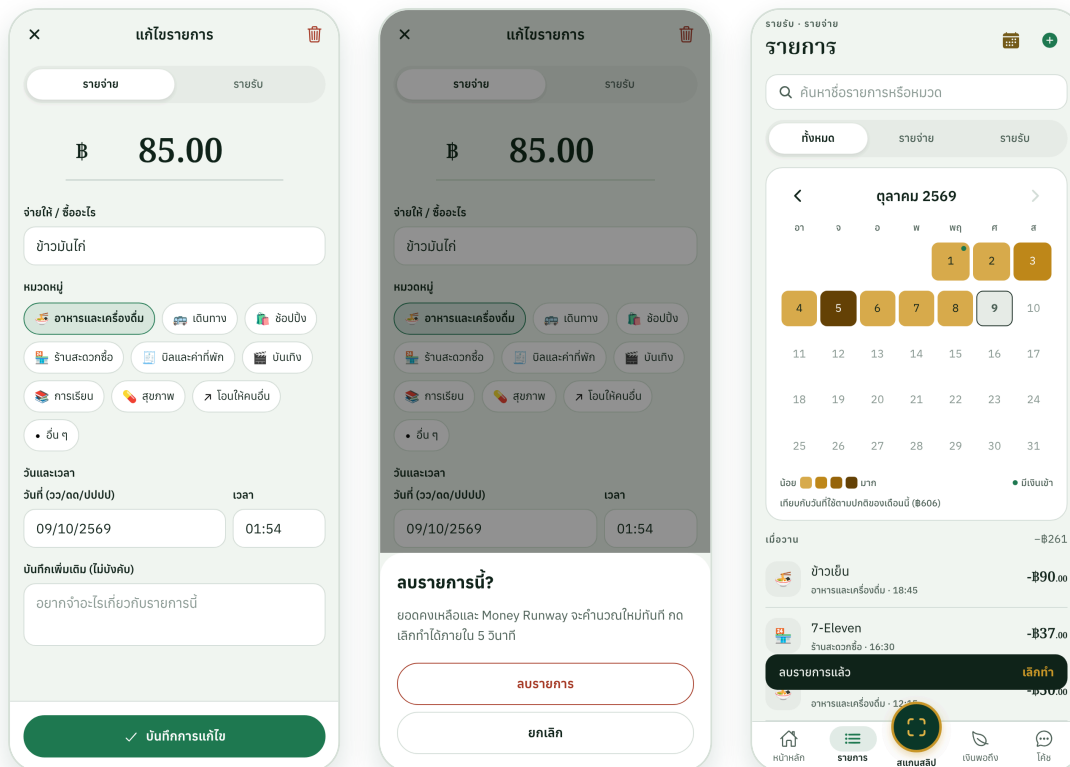
หลังขึ้นข้อความเตือนจำนวนเงินแล้ว ถ้าพิมพ์ "85" ลงไป ข้อความเตือนยังค้างอยู่ (ดูภาพข้างล่าง) จนกว่าจะกดบันทึกอีกครั้ง เพราะโค้ดล้างข้อความเตือนตอนกดบันทึกเท่านั้น (บรรทัด 94) ไม่ได้ล้างตอนพิมพ์ ไม่มีผลต่อความถูกต้องของข้อมูล แต่เป็นจุดที่ปรับ UX ได้ (แก้ง่าย: ล้าง amountError ใน onChangeText ของช่องเงิน)



"เพิ่มเงินเข้า" ต่างจาก "จดยรายการ" ยังไง

เป็นหน้าเดียวกัน แต่หน้าหลักส่งค่า kind=income มาด้วย ฟอรั่มจึงเปิดเป็นรายรับ หัวข้อเป็น "เพิ่มเงินเข้า" และมีชิปจำนวนลด +฿100, +฿500, +฿1,000, +฿5,000 (เฉพาะตอนสร้างรายรับใหม่) เหตุผล: ผู้ใช้ขอเมื่อ 29 ก.ย. ว่า "เงินเข้ามักไม่มีสลิป" (เช่น ค่าขนมอบเงินสด) จึงต้องจดเองได้เร็วที่สุด บันทึกแล้วขึ้น "เพิ่มเงินเข้าแล้ว"

แก้ไข ลบ และเลิกทำ



แตะรายการในแท็บรายการ: เปิดหน้า "แก้ไขรายการ" พร้อมค่าเดิมและปุ่มถึงขยับสีแดง

กดถึงขยับ: แผ่นถ้ามยืนยัน "ลบรายการนี้?" บอกผลกระทบก่อนลบ

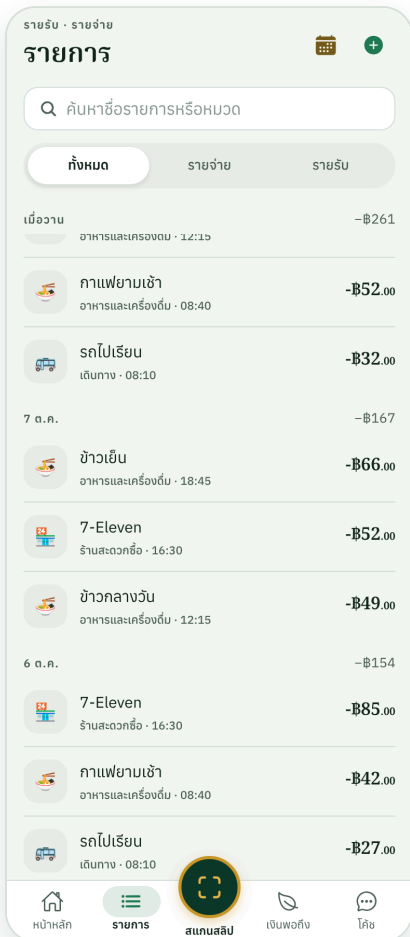
ลบแล้ว: "ลบรายการแล้ว" พร้อมปุ่ม "เลิกทำ" อยู่ 5 วินาที

- ข้อความยืนยันการลบบอกผลกระทบชัด: "ยอดคงเหลือและ Money Runway จะคำนวณใหม่ทันที กดเลิกทำได้ภายใน 5 วินาที" ถ้าเป็นแบบร่างจากสลิปจะบอกว่า "ถ้าสแกนรูปเดิมอีกครั้งจะอ่านใหม่ได้"
- ปุ่มยืนยันเป็นแบบ **danger** (ขอบแดง) ตาม Design System สำหรับการกระทำที่ย้อนไม่ได้
- **เลิกทำ** ทำงานโดยบันทึกรายการเดิมกลับเข้าไปใหม่ (ทุกช่องเหมือนเดิม แต่ได้ id ใหม่) toast ที่มีปุ่มจะค้าง 5 วินาที ส่วน toast ธรรมดาค้าง 2.8 วินาที

บทที่ 6

ทิวทัศน์หน้าจอ: แท็บ "รายการ"

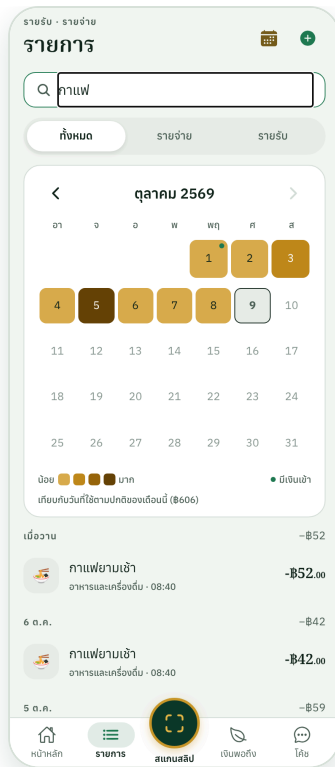
ไฟล์ src/app/(tabs)/transactions.tsx ดูรายการทั้งหมด หาให้เจอ แล้วแตะเพื่อแก้ไข



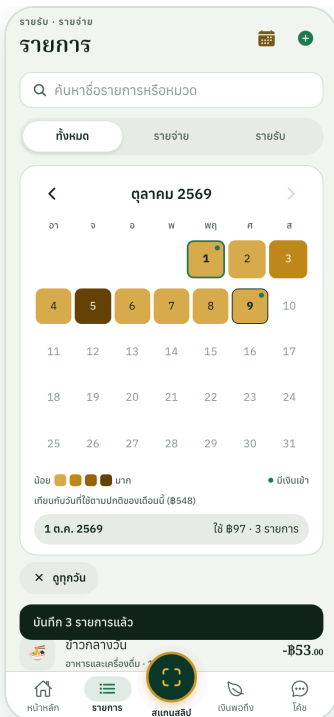
รายการแยกตามวัน: หัวแต่ละวันบอกยอดสุทธิของวันนั้น

- 1 หัวเก็บ "รายการรับ · รายการจ่าย / รายการ" ปุ่มปฏิทิน (ซ่อน/แสดง) และปุ่ม + จัดรายการ
- 2 ช่องค้นหา ค้นจากชื่อรายการ ชื่อหมวด หรือบันทึก (ไม่สนตัวพิมพ์เล็กใหญ่)
- 3 ตัวกรอง ทั้งหมด / รายการจ่าย / รายการรับ
- 4 การ์ดสลิปปรอยืนยัน (ถ้ามี) พาไปหน้าตรวจสอบสลิปของ FR-4
- 5 ปฏิทินการใช้จ่าย (ของ FR-2) แตะวันไหนเห็นเฉพาะรายการวันนั้น และปุ่ม "ดูทุกวัน"
- 6 หัวของแต่ละวัน "วันนี้", "เมื่อวาน", "6 ต.ค." พร้อมยอดสุทธิของวัน เช่น -฿261 หัวนี้ติดค้างด้านบนตอนเลื่อน
- 7 แถวรายการ ไอคอนหมวด ชื่อ หมวด · เวลา · "จากสลิป" และจำนวนเงิน (รายการสีเขียวมี +, รายการจ่ายมี -) จุดทองที่มุมไอคอน = รายการที่จดจากสลิปวันนี้

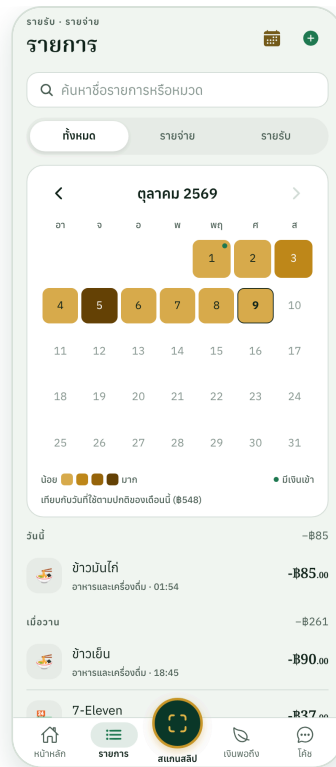
แท็บนี้แสดงเฉพาะรายการที่ **ยืนยันแล้ว** แบบร่างอยู่ในหน้า "สลิปปรอ ยืนยัน" แยกต่างหาก



ค้นหา "กาแฟ" เหลือเฉพาะ กาแฟยามเช้า



แต่วันในปฏิทิน เห็นเฉพาะรายการของวันนั้น



รายการที่เพิ่งจดอยู่บนสุดของ "วันนี้"

หลักการทำงาน: ตรวจจำนวนเงิน เดทหมวด และวัน เวลา

กฎ 3 ข้อที่ทำให้ข้อมูลที่เข้าระบบถูกต้องและจดง่าย

1. ตรวจจำนวนเงิน (parseBahtToSatang)

ฟังก์ชันนี้อธิบายทีละบรรทัดแล้วในบท A5 สรุปกฎที่ FR-1 ใช้ (ทุกค่ารับกับโค้ดจริงแล้ว):

ผู้ใช้พิมพ์	ได้	ผล
85	8,500 สตางค์	บันทึกได้ ฿85.00
85.5	8,550	฿85.50
1,000	100,000	฿1,000.00
12.	1,200	฿12.00 (จุดทศนิยมไม่เป็นไร)
0 / abc / 85.555 / -5	null	ไม่บันทึก ขึ้น "ใส่จำนวนเงินมากกว่า 0 เช่น 85 หรือ 85.50"

i ช่องเงินตัวใหญ่กรองตัวอักษรตั้งแต่พิมพ์

AmountField (ใน [src/ui/inputs.tsx](#)) ลบทุกอย่างที่ไม่ใช่ 0-9 จุด หรือคอมมา ตั้งแต่ตอนพิมพ์ (บรรทัด 189) ผู้ใช้จึงพิมพ์ตัวอักษรไม่ได้เลย ผลข้างเคียงคือเลขไทย (๘๕) ก็ถูกลบในช่องนี้ด้วย แม้ parseBahtToSatang จะรองรับเลขไทย (ซึ่งใช้ได้ทั้งในช่องเงินของหน้าตั้งค่า)

2. เดทหมวดจากชื่อ (suggestCategory)

ระหว่างผู้ใช้พิมพ์ชื่อ แอปเรียก suggestCategory ทุกตัวอักษร จนกว่าผู้ใช้จะแตะเลือกหมวดเอง กติกา:

- แปลงเป็นตัวพิมพ์เล็ก แล้วหาคำสำคัญ (keyword) ของทุกหมวดในข้อความ
- คำที่เจอก่อน (อยู่ซ้ายสุด) ชนะ เช่น "KFC Central" เจอ kfc (อาหาร) ก่อน central (ช้อปปิ้ง) → อาหาร
- ถ้าเริ่มที่ตำแหน่งเดียวกัน คำที่ยาวกว่าชนะ เช่น "ร้านยา" ตรงทั้ง "ร้าน" (อาหาร) และ "ร้านยา" (สุขภาพ) → สุขภาพ
- คำภาษาอังกฤษต้องอยู่ ต้นคำ: "ais" ไม่นับใน "PAISAN" แต่ "GRABPAY" ยังนับเป็น grab
- ถ้าข้อความขึ้นต้นด้วยคำนำหน้าชื่อคน (นาย นาง น.ส. Mr. Ms.) คำสำคัญที่สั้นกว่า 5 ตัวอักษรไม่นับ: "นาย วินัย" ไม่เป็น "วิน" (เดินทาง)
- ร้านที่เจาะจงชนะกฎ "ชื่อคน": "ร้านข้าว นายสมชาย" → อาหาร ไม่ใช่โอนให้คนอื่น
- ไม่เจออะไรเลย: ถ้าเป็นชื่อคน → "โอนให้คนอื่น" ไม่ขึ้น → "อื่น ๆ" (รายรับ → "รับโอน")

ชื่อที่พิมพ์ (รายจ่าย)	หมวดที่ได้ (รับจริง)	เพราะ
ข้าวมันไก่	อาหารและเครื่องดื่ม	คำว่า "ข้าว"
7-ELEVEN สาขา 1234	ร้านสะดวกซื้อ	"7-eleven"
ร้านข้าว นายสมชาย	อาหารและเครื่องดื่ม	ร้านเราจะจนจะกดซื้อคน
นาย สมชาย ใจดี	โอนให้คนอื่น	ขึ้นต้นด้วย "นาย"
สมชาย	อื่น ๆ	ไม่มีคำนำหน้า ไม่มีคำสำคัญ ("ซา" ไม่อยู่ในรายการ เพื่อไม่ให้ "สมชาย" กลายเป็นอาหาร)
ร้านยา	สุขภาพ	คำยาวกว่าชนะ
KFC Central	อาหารและเครื่องดื่ม	คำที่อยู่ซ้ายกว่าชนะ
mini big c	ร้านสะดวกซื้อ	"mini big c" ยาวกว่า "big c"
GRABPAY	เดินทาง	"grab" อยู่ต้นคำ
PAISAN	อื่น ๆ	"ais" ไม่อยู่ต้นคำ
นาย วินัย	โอนให้คนอื่น	"วิน" สั้นเกินหลังคำนำหน้า
ค่าหอพัก / ค่าไฟ	บิลและค่าที่พักร	"หอพัก" / "ค่าไฟ"
ถ่ายเอกสาร	การเรียน	"ถ่ายเอกสาร"

ชื่อที่พิมพ์ (รายรับ)	หมวดที่ได้
ค่าขนมจากแม่	เงินจากที่บ้าน
เงินเดือนกัญยา	เงินเดือน / ค่าจ้าง
ฟรีแลนซ์	งานพิเศษ
เพื่อนคืนเงิน	รับโอน (ค่าเริ่มต้น)

★ บัญชีจริงที่ทดสอบได้ (TC-25)

บันทึกการใช้ AI แถว 3 เล่าว่าตอนแรกมีคำสำคัญ "ซา" ในหมวดอาหาร ทำให้ชื่อคน "สมชาย" ถูกจัดเป็นอาหาร เทสต์ TC-25 จับได้ ทีมจึงเปลี่ยนเป็นคำที่เจาะจงขึ้น เช่น "ซานม", "ซาไข่มุก" ภาษาไทยไม่มีช่องว่างระหว่างคำ การหาคำย่อจึงต้องระวังแบบนี้

3. วันและเวลาแบบไทย

- ค่าเริ่มต้นคือ "ตอนนี้" ตามเวลาไทย (bkkDayKey, bkkTime)
- เลือกวันในอนาคตไม่ได้ (Android/iOS ตั้ง maximumDate เป็นวันนี้ ส่วนเว็บขึ้น "วันนี้หรือก่อนหน้าเท่านั้น")
- บนเว็บ วันที่พิมพ์ต้องอ่านได้ (เช่น 27/09/2569) และเวลาต้องเป็น HH:MM ถ้าผิด ปุ่มบันทึกไม่ทำงานและบอกตัวอย่างที่ถูกใส่ช่อง (เพิ่มตอนตรวจ repo 1–3 ท.ค.)
- ตอนบันทึก รวมวัน + เวลาไทยเป็นเวลาสากลด้วย bkkToIso แล้วค่อยเก็บ

บทที่ 8

เบื้องหลัง: บันทึก แก้ ลบ ไปถึงฐานข้อมูลยังงี้

เส้นทางเดียวกันทุกครั้ง: หน้าจอ → AppProvider → Repo → Supabase แล้วตัวเลขทั้งแอปคำนวณใหม่

หน้าจอรายการ [src/app/transaction.tsx](#)

save(): ตรวจสอบเงินและวันที่ → รวมเป็น input → addTx (ใหม่) หรือ updateTx (แก้) · doDelete(): removeTx แล้วแสดง toast ที่มี "เลิกทำ"



AppProvider [src/data/AppProvider.tsx](#)

addTx / updateTx / removeTx / confirmTx ส่งต่อให้ Repo แล้วอัปเดต txs ใน state กันที่สำเร็จ (และจดไว้ว่าเปลี่ยนในเครื่อง กันการไหลย้อนกลับ)



Repo [src/data/repo.ts](#)

CloudRepo: insert/update/delete ผ่าน supabase-js · DemoRepo: แก้ข้อมูลในเครื่องแล้วบันทึก · ทั่วทั้งสลิปซ้ำ (DuplicateSlipError)



ตาราง transactions (Supabase)

check: amount_satang > 0 และ ≤ 100,000,000,000, kind/status/source ต้องเป็นค่าที่กำหนด, title ≤ 120, note ≤ 500 · RLS: แลวต้องเป็นของผู้ใช้ (user_id = auth.uid()) · trigger อัปเดต updated_at

↓ txs เปลี่ยน

useMoney คำนวณใหม่

ยอดคงเหลือ (FR-2), Runway (FR-6), แบบร่าง (FR-4) · หน้าหลัก แท็บรายการ ปฏิทิน และโค้ช อัปเดตเอง

★ ป้องกัน 2 ชั้น

แอปตรวจสอบข้อมูลก่อนส่ง (จำนวนเงิน วันที่ ความยาว) และฐานข้อมูลตรวจซ้ำด้วย check constraint ถ้ามีใครส่งคำขอตรงไปที่ API โดยไม่ผ่านแอป (เช่น แอ็กเคอร์) ก็ยังใส่จำนวนเงินติดลบหรือเพิ่มรายการให้คนอื่นไม่ได้

แผนที่โค้ดของ FR-1

แตะชื่อไฟล์เพื่อเปิดใน VS Code

ไฟล์	หน้าที่ใน FR-1
<code>src/app/transaction.tsx</code>	หน้าจอ/แก้ไข/ลบ/ตรวจสอบสลิป 1 รายการ (364 บรรทัด)
<code>src/app/(tabs)/transactions.tsx</code>	แท็บรายการ: ค้นหา กรอง แยกตามวัน ปฏิทิน
<code>src/ui/TxRow.tsx</code>	แถวรายการ 1 แถว (ใช้ทั้งแท็บรายการ หน้าหลัก หน้าสลิปรอยืนยัน หน้าสแกน)
<code>src/domain/categories.ts</code>	หมวด 15 หมวด และการเดาหมวด
<code>src/domain/money.ts</code>	แปลง/แสดงจำนวนเงิน (บท A5)
<code>src/domain/dates.ts</code>	วันเวลาไทย, parseSlipDate/parseSlipTime สำหรับวันที่พิมพ์บนเว็บ
<code>src/domain/summary.ts</code>	groupByDay (แยกตามวันพร้อมยอดสุทธิ), computeBalance
<code>src/domain/recurring.ts</code>	หารายการประจำเดือน
<code>src/data/AppProvider.tsx</code>	addTx, updateTx, removeTx, confirmTx
<code>src/data/repo.ts</code>	insert, update, remove, confirmMany (คลาวด์และโหมดทดลอง)
<code>src/ui/inputs.tsx</code>	Field, AmountField
<code>src/ui/feedback.tsx</code>	toast พร้อม "เลิกทำ" และ ConfirmSheet
<code>src/ui/nav.ts</code>	goBack, useLeaveWhenDone (ปิดหน้าต่างหลังบันทึกอย่างปลอดภัย)
<code>src/app/(tabs)/index.tsx</code>	ปุ่ม "จดรายการ" และ "เพิ่มเงินเข้า" บนหน้าหลัก
<code>supabase/migrations/20260927000000_init.sql</code>	ตาราง transactions และ RLS
<code>src/domain/__tests__/domain.test.ts</code> , <code>src/domain/__tests__/audit.test.ts</code> , <code>src/domain/__tests__/recurring.test.ts</code>	TC-01, TC-02, TC-25, TC-79, TC-72

[e2e/web-auth.mjs](#)E2E "Money: add, edit, delete and
undo" (บรรทัด 435–493)

อ่านโค้ด: หน้าจอดรายการ (transaction.tsx) ตอนที่ 1

— สมอง

ไฟล์ src/app/transaction.tsx มี 364 บรรทัด บทนี้อ่านครั้งแรก (บรรทัด 1–178) คือข้อมูลที่หน้าจำไว้ และสิ่งที่เกิดขึ้นเมื่อกดปุ่มต่าง ๆ

ไฟล์นี้เป็น **ไฟล์ใหญ่ที่สุดของ FR-1** และเป็นหน้าเดียวที่ใช้ทำงาน 4 อย่าง: **จดย่อยจ่ายใหม่, เพิ่มเงินเข้า** (รายรับใหม่), **แก้ไขรายการเดิม** และ **ตรวจแบบร่างจากสลิป** (ที่ FR-4 สร้างไว้) หน้าเดียวรู้ได้ว่าต้องทำงานแบบไหนจาก "พารามิเตอร์ใน URL" ที่ส่งมาตอนเปิดหน้า

เปิดหน้าด้วย	ได้หน้าแบบ	หัวหน้าจอ
/transaction	จดย่อยจ่ายใหม่	จดย่อยรายการ
/transaction?kind=income	เพิ่มเงินเข้า	เพิ่มเงินเข้า
/transaction?id=abc (รายการยืนยันแล้ว)	แก้ไข/ลบ	แก้ไขรายการ
/transaction?id=abc (รายการ status = draft)	ตรวจสลิป	ตรวจสลิป

≈ นึกภาพหน้าจอเป็นแบบฟอร์มกระดาษ + พนักงาน

ครั้งแรกของไฟล์ (บทนี้) เหมือน **พนักงานที่ถือแบบฟอร์ม**: จำว่าช่องไหนเขียนอะไรไว้ (state), รู้ว่าถ้ามีคนเปลี่ยนประเภทต้องเปลี่ยนหมวดตาม (changeKind) และรู้ขั้นตอนตอนยื่นแบบฟอร์ม (save) ครั้งหลัง (บทถัดไป) คือ **ตัวแบบฟอร์มที่ผู้ใช้มองเห็น (JSX)**

ส่วนที่ 1: คอมเมนต์หัวไฟล์และ import (บรรทัด 1–22)

src/app/transaction.tsx

บรรทัด 1–22

```

1  /**
2   * FR-1 add / edit / categorize / delete one transaction.
3   * Also used to review a slip draft: unsure fields are highlighted and the main
4   * button confirms the draft so it starts counting in totals.
5   */
6  import DateTimePicker, { DateTimePickerAndroid } from '@react-native-community/datetimepicker';
7  import { useLocalSearchParams } from 'expo-router';
8  import { goBack, useLeaveWhenDone } from '../ui/nav';
9  import { useMemo, useState } from 'react';
10 import { KeyboardAvoidingView, Platform, Pressable, ScrollView, View } from 'react-native';
11 import { SafeAreaView } from 'react-native-safe-area-context';
12 import { useApp } from '../data/AppProvider';
13 import { categoriesFor, defaultCategory, suggestCategory } from '../domain/categories';
14 import { bkkDayKey, bkkTime, bkkToIso, formatThaiDay, parseSlipDate, parseSlipTime } from
    '../domain/dates';
15 import { formatBaht, parseBahtToSatang, satangToInput } from '../domain/money';
16 import { ALT_PREFIX, amountChoices, FLAG_LABEL, type ReviewFlag } from '../domain/slip';
17 import type { TxKind } from '../domain/types';

```

```

18 import { Button, Card, Chip, IconButton, Ionicons, Row, Segmented, T } from '../ui/components';
19 import { ConfirmSheet, useToast } from '../ui/feedback';
20 import { AmountField, Field } from '../ui/inputs';
21 import { fonts, radius, space, useTheme } from '../ui/theme';
22 import { HeaderDecor } from '../ui/halloween';

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/transaction.tsx:1

บรรทัด	ความหมาย
1-5	คอมเมนต์หลายบรรทัด (/** ... */) บอกว่าไฟล์นี้คือ FR-1: เพิ่ม แก้ จัดหมวด ลบ รายการ 1 รายการ และยังใช้ตรวจสอบร่างจากสลิป: ช่องที่ AI ไม่แน่ใจจะถูกไฮไลต์ และปุ่มหลักจะ "ยืนยัน" แบบร่างให้เริ่มนับในยอดเงิน
6	ตัวเลือกวันที่/เวลาของมือถือ (DateTimePicker ใช้บน iPhone, DateTimePickerAndroid เปิดหน้าต่างเลือกบน Android) มาจากไลบรารีภายนอก
7	useLocalSearchParams ของ Expo Router อ่านค่าหลัง ? ใน URL เช่น id และ kind
8	ฟังก์ชันออกจากหน้า 2 ตัวจากไฟล์ src/ui/nav.ts (อธิบายท้ายบท)
9	hook ของ React: useState (ความจำของหน้า) และ useMemo (จำผลคำนวณ)
10	ชิ้นส่วนพื้นฐานของ React Native: กล่องเลื่อนหลักยิบอร์ด, Platform (รู้ว่ามีบน web/ios/android), ปุ่มกดได้, กล่องเลื่อน, กล่อง View
11	SafeAreaView กันเนื้อหาไปชนรอยมากหรือแถบล่างของมือถือ
12	useApp ประมวลผลข้อมูลกลางของแอป (รายการทั้งหมด และฟังก์ชันเพิ่ม/แก้/ลบ)
13	3 ฟังก์ชันจากไฟล์หมวด: หมวดของแต่ละประเภท, หมวดตั้งต้น, แดหมวดจากข้อความ
14	ฟังก์ชันวันเวลาแบบไทย 6 ตัว: แปลงเป็น "วันไทย", แปลงเป็น "เวลาไทย", รวมวัน+เวลากลับเป็นเวลามาตรฐาน, แสดงวันแบบไทย, อ่านวันที่ที่พิมพ์ (27/09/2569), อ่านเวลาที่พิมพ์ (08:30)
15	ฟังก์ชันเงิน: แสดงเป็น "฿85.00", แปลงข้อความเป็นสตริง, แปลงสตริงกลับเป็นข้อความในช่องกรอก ("85.00")
16	ของจาก FR-4 (สลิป): คำนำหน้าบรรทัด "AI อ่านได้ 2 แบบ:", ฟังก์ชันดึงยอดเงิน 2 แบบจากบันทึก, ชื่อภาษาไทยของช่องที่ต้องตรวจ, ชนิด ReviewFlag (amount / date / counterparty / direction)
17	ชนิด TxKind = 'expense' หรือ 'income' (import type = เอาแค่ชนิด)
18	ชิ้นส่วนหน้าต่างจาก Design System ของเรา: ปุ่ม, การ์ด, ชิป, ปุ่มไอคอน, ไอคอน, แถวแนวนอน, ปุ่มสลับสองทาง, ข้อความ
19	หน้าต่างยืนยัน (ConfirmSheet) และ toast (ข้อความแจ้งเตือนสั้น ๆ ด้านล่าง)
20	ช่องกรอกเงินตัวใหญ่ (AmountField) และช่องกรอกทั่วไป (Field)
21	ค่าการออกแบบ: ฟอนต์, ความโค้งมน, ระยะห่าง และ useTheme (สีตามธีมที่ผู้ใช้เลือก)
22	ของตกแต่งหัวหน้าจอ (แสดงเฉพาะช่วงธันวาคม)

★ อ่าน import ให้เป็น = เห็นแผนที่ไฟล์

แค่ดู import 17 บรรทัดนี้ เราก็รู้แล้วว่าหน้านี้พึ่งพาอะไร: ข้อมูลกลาง (data), สูตรล้วน (domain: หมวด วันที่ เงิน สลิป) และหน้าตา (ui) ตรงกับ 3 ชั้นของสถาปัตยกรรมในบท A2 ถ้าอยากรู้ว่า suggestCategory ทำงานยังไง ก็ตามไปที่ [src/domain/categories.ts](#) ตามบรรทัด 13 (ใน VS Code กด Ctrl ค้างแล้วคลิกชื่อฟังก์ชัน = กระโดดไปที่นิยาม)

ส่วนที่ 2: หน้าเปิดขึ้นมา รู้ได้ยังไงว่าเป็นหน้าแบบไหน (บรรทัด 24–32)

src/app/transaction.tsx

บรรทัด 24–32

```
24 export default function TransactionForm() {
25   const theme = useTheme();
26   const toast = useToast();
27   const leaveWhenDone = useLeaveWhenDone();
28   const { id, kind: kindParam } = useLocalSearchParams<{ id?: string; kind?: string }>();
29   const { txs, addTx, updateTx, removeTx } = useApp();
30   const existing = useMemo(() => txs.find((t) => t.id === id), [txs, id]);
31   const isDraft = existing?.status === 'draft';
32   const flags = (existing?.reviewFlags ?? []) as ReviewFlag[];
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/transaction.tsx:24

บรรทัด	ความหมาย
24	ประกาศ component ชื่อ TransactionForm และ export default เพื่อให้ Expo Router รู้ว่าไฟล์ src/app/transaction.tsx = หน้า /transaction
25	ขอสีของธีมปัจจุบัน (ป่า, Red Velvet, ...) เก็บไว้ในตัวแปร theme
26	ขอฟังก์ชันแสดง toast
27	ขอฟังก์ชัน "ออกจากหน้าหลังทำเสร็จ" (ปลอดภัยกว่าการสั่งย้อนกลับตรง ๆ)
28	อ่าน URL: เอา id และ kind ออกมา (ตั้งชื่อใหม่ว่า kindParam เพราะชื่อ kind จะใช้กับ state ข้างล่าง) ทั้งคู่อาจไม่มี จึงมี ? ในชนิด
29	ดึงจากข้อมูลกลาง: รายการทั้งหมด txs และ 3 ฟังก์ชัน เพิ่ม/แก้/ลบ
30	หารายการเดิม: ค้นใน txs ตัวที่ id ตรงกับ URL ถ้าไม่มี id (จดใหม่) ผลคือ undefined ใช้ useMemo ให้ค้นใหม่เฉพาะเมื่อ txs หรือ id เปลี่ยน
31	เป็นแบบร่างไหม? existing?.status แปลว่า "ถ้ามี existing ค่อยดู status" ถ้าไม่มีได้ undefined ซึ่งไม่เท่ากับ 'draft' → false
32	รายการช่องที่ AI ขอให้ตรวจ (ถ้าไม่มีใช้ [] ว่า) as ReviewFlag[] บอก TypeScript ว่าเป็นรายการ ReviewFlag

ส่วนที่ 3: ความจำของแบบฟอร์ม — state ทั้ง 14 ตัว (บรรทัด 34–49)

src/app/transaction.tsx

บรรทัด 34–49

```
34 // "เพิ่มเงินเข้า" opens the form as income (money that comes in without a slip).
```

```

35 const startKind: TxKind = existing?.kind ?? (kindParam === 'income' ? 'income' : 'expense');
36 const [kind, setKind] = useState<TxKind>(startKind);
37 const [amount, setAmount] = useState(existing ? satangToInput(existing.amountSatang) : '');
38 const [title, setTitle] = useState(existing?.title ?? '');
39 const [category, setCategory] = useState(existing?.categoryKey ?? defaultCategory(startKind));
40 const [categoryTouched, setCategoryTouched] = useState(!existing);
41 const [day, setDay] = useState(existing ? bkkDayKey(existing.occurredAt) : bkkDayKey(new
Date()));
42 const [time, setTime] = useState(existing ? bkkTime(existing.occurredAt) : bkkTime(new
Date()));
43 const [note, setNote] = useState(existing?.note ?? '');
44 const [busy, setBusy] = useState(false);
45 const [askDelete, setAskDelete] = useState(false);
46 const [amountError, setAmountError] = useState<string | null>(null);
47 // Web only: the browser build has no native date picker, so dates are typed.
48 const [dayText, setDayText] = useState(() => { const [y, m, d] = day.split('-'); return
`${d}/${m}/${Number(y) + 543}`; });
49 const [timeText, setTimeText] = useState(time);

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/transaction.tsx:34

ทุกบรรทัดที่ขึ้นต้นด้วย const [x, setX] = useState(...) คือ "ช่องความจำ" 1 ช่อง: x คือค่าปัจจุบัน และ setX คือฟังก์ชันเปลี่ยนค่า (เปลี่ยนแล้ว React วาดหน้าใหม่เอง) ค่าในวงเล็บคือค่าเริ่มต้น สังเกตว่า ทุกช่องเริ่มจากรายการเดิมถ้ามี ไม่มีก็ใช้ค่าว่าง/ค่าตั้งต้น

บรรทัด	ความหมาย
34–35	ประเภทเริ่มต้น: ถ้าแก้ไขเดิมใช้ประเภทเดิม (?? = "ถ้าซ้ายไม่มีค่า ใช้ขวา") ถ้าเป็นของใหม่ ดู URL: kind=income → รายรับ ไม่จั้นรายจ่าย (คอมเมนต์บอกว่าปุ่ม "เพิ่มเงินเข้า" เปิดหน้านี้แบบรายรับ สำหรับเงินที่เข้ามาโดยไม่มีสลิป)
36	state ประเภทที่เลือกอยู่ (ผู้ใช้สลับได้ด้วยปุ่ม รายจ่าย/รายรับ)
37	state จำนวนเงินเป็นข้อความ (ไม่ใช่ตัวเลข เพราะผู้ใช้อาจพิมพ์ค้างไว้ "85." ระหว่างพิมพ์) ของเดิม 8,500 สตางค์ → "85.00"
38	state ชื่อรายการ ("จ่ายให้ / ชื่ออะไร")
39	state หมวด: ของเดิมใช้หมวดเดิม ของใหม่ใช้หมวดตั้งต้น (รายจ่าย = other_expense "อื่น ๆ", รายรับ = transfer_in "รับโอน")
40	state "ผู้ใช้เลือกหมวดเองหรือยัง" (!!existing แปลงเป็น true/false: ของเดิม = true ถือว่าเลือกแล้ว จะได้ไม่เด้งหมวดกัน) ใช้ในบรรทัด 70
41	state วัน รูปแบบ YYYY-MM-DD ใช้ปี ค.ศ. (ไม่ใช่ พ.ศ.) เช่น "2026-10-08" (bkkDayKey ของเวลาเดิม หรือของตอนนี้)
42	state เวลา รูปแบบ "HH:MM" เช่น "08:30"
43	state บันทึกเพิ่มเติม
44	state "กำลังบันทึกอยู่" ใช้ล็อกปุ่มไม่ให้กดซ้ำ และแสดงวงหมุนบนปุ่ม
45	state "เปิดหน้าต่างถามยืนยันการลบอยู่ไหม"
46	state ข้อความผิดพลาดของจำนวนเงิน (null = ไม่มีข้อผิดพลาด)

บรรทัด	ความหมาย
47–48	เฉพาะเว็บ: เบราว์เซอร์ไม่มีตัวเลือกวันที่แบบมือถือ จึงให้พิมพ์ ค่าเริ่มต้นแปลง "2026-10-08" → แยกด้วย split(' - ') ได้ y, m, d → ประกอบเป็น "08/10/2569" (ปี + 543 = พ.ศ.) useState(() => ...) แบบส่งฟังก์ชันแปลงว่า คำนวณค่าเริ่มต้นครั้งเดียวตอนเปิดหน้า
49	ข้อความเวลาที่พิมพ์บนเว็บ เริ่มจากเวลาใน state

! ทำไมเว็บต้องมีทั้ง day และ dayText?

dayText คือ **สิ่งที่ผู้ใช้พิมพ์จริง** (อาจพิมพ์ค้างครึ่ง ๆ เช่น "08/1") ส่วน day คือ **วันที่ที่ถูกต้องล่าสุด** ที่ใช้บันทึก ถ้ามีตัวเดียว ทุกครั้งที่พิมพ์ไม่ครบระบบจะลบสิ่งที่พิมพ์ทิ้ง ผู้ใช้จะพิมพ์ไม่ได้เลย

ส่วนที่ 4: ตรวจสอบวันที่ที่พิมพ์ และค่าที่คำนวณจาก state (บรรทัด 50–60)

src/app/transaction.tsx

บรรทัด 50–60

```

50 // What is wrong with the typed date/time (web); saving waits until both are fine.
51 const typedDay = parseSlipDate(dayText);
52 const dayError =
53   Platform.OS !== 'web' ? null : !typedDay ? 'เช่น 27/09/2569' : typedDay > bkkDayKey(new Date())
? 'วันนี้หรือก่อนหน้าเท่านั้น' : null;
54 const timeError = Platform.OS !== 'web' || parseSlipTime(timeText) ? null : 'เช่น 08:30';
55
56 const cats = categoriesFor(kind);
57 const pickerDate = new Date(bkkToIso(day, time));
58 // The two AI reads disagreed: show both, and offer the amounts as buttons.
59 const altLine = isDraft ? (existing?.note ?? '').split('\n').find((l) =>
  l.startsWith(ALT_PREFIX)) : undefined;
60 const choices = isDraft && flags.includes('amount') ? amountChoices(existing?.note) : [];

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/transaction.tsx:50

บรรทัด	ความหมาย
50	คอมเมนต์: ช่องวัน/เวลาที่พิมพ์ผิดตรงไหน การบันทึกจะรอจนถูกต้องทั้งคู่
51	แปลงข้อความวันที่ที่พิมพ์เป็น "YYYY-MM-DD" ถ้าอ่านไม่ได้ได้ null
52–53	ข้อผิดพลาดของวันที่ อ่านแบบ "ถ้า...ไม่จริงถ้า..." (ternary ต่อกัน): ไม่ใช่เว็บ → ไม่มีข้อผิดพลาด (มือถือเลือกจากปฏิทิน ผิดไม่ได้) / อ่านไม่ได้ → "เช่น 27/09/2569" / วันอนาคต → "วันนี้หรือก่อนหน้าเท่านั้น" / นอกนั้น → null ไม่มีข้อผิดพลาด (เปรียบเทียบข้อความ "2026-10-09" > "2026-10-08" ได้เพราะรูปแบบเรียงจากปีไปวัน)
54	ข้อผิดพลาดของเวลา: ไม่ใช่เว็บ หรืออ่านได้ → null ไม่จริง "เช่น 08:30"
56	หมวดที่จะแสดงเป็นชิป ตามประเภทที่เลือก (รายจ่าย 10 หมวด / รายรับ 5 หมวด)
57	รวมวัน+เวลาไทยเป็น Date ให้ตัวเลือกวันที่ของมือถือรู้ว่าตอนนี้เลือกอยู่ที่ไหน
58–59	(FR-4) ถ้าเป็นแบบร่าง หาบรรทัดในบันทึกที่ขึ้นต้นด้วย "AI อ่านได้ 2 แบบ:" เพื่อแสดงให้ผู้ผู้ใช้เห็น
60	(FR-4) ถ้าเป็นแบบร่างและยอดเงินถูกต้องตรงตรวจ ถึงยอด 2 แบบมาทำเป็นปุ่มให้เลือก ไม่ขึ้นรายการว่าง

i ทำไมไม่เก็บ dayError ไว้ใน state?

เพราะ **คำนวณได้จาก state ที่มีอยู่แล้ว** (dayText) ทุกครั้งที่หน้าवादใหม่ บรรทัด 51–54 ก็คำนวณใหม่ให้ตรงเสมอ ถ้าเก็บไว้ใน state ต้องคอยอัปเดตสองที่ สัมเมื่อไหร่ก็แสดงผิด หลักนี้เรียกว่า "อย่าเก็บสิ่งที่คำนวณได้" (derived state) ส่วน amountError เก็บเป็น state เพราะเราอยากให้ขึ้น **หลังกดบันทึก** เท่านั้น ไม่ใช่ทุกตัวอักษรที่พิมพ์

ส่วนที่ 5: เปลี่ยนประเภท เปลี่ยนชื่อ และตัวเลือกว่าที่ Android (บรรทัด 62–85)

src/app/transaction.tsx

บรรทัด 62–85

```
62 function changeKind(k: TxKind) {
63   setKind(k);
64   if (!categoriesFor(k).some((c) => c.key === category)) setCategory(suggestCategory(title,
65   k));
66 }
67 function changeTitle(s: string) {
68   setTitle(s);
69   // Suggest a category while the user types, until they pick one themselves.
70   if (!categoryTouched) setCategory(suggestCategory(s, kind));
71 }
72
73 function openAndroidPicker(mode: 'date' | 'time') {
74   DateTimePickerAndroid.open({
75     value: pickerDate,
76     mode,
77     is24Hour: true,
78     maximumDate: mode === 'date' ? new Date() : undefined,
79     onChange: (e, d) => {
80       if (e.type !== 'set' || !d) return;
81       if (mode === 'date') setDay(bkkDayKey(d));
82       else setTime(bkkTime(d));
83     },
84   });
85 }
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/transaction.tsx:62

บรรทัด	ความหมาย
62–63	changeKind ถูกเรียกเมื่อกดปุ่ม รายจ่าย/รายรับ: เก็บประเภทใหม่
64	ถ้าหมวดที่เลือกอยู่ ไม่อยู่ใน หมวดของประเภทใหม่ (เช่น "อาหาร" แต่สลับเป็นรายรับ) ให้เดาหมวดใหม่จากชื่อ some = "มีสักตัวไหมที่..." และ ! = กลับค่า → ป้องกันรายรับที่มีหมวด "อาหาร" ซึ่งไม่มีความหมาย
67–68	changeTitle ถูกเรียกทุกครั้งที่พิมพ์ชื่อ: เก็บข้อความใหม่
69–70	เดาหมวดระหว่างพิมพ์ แต่เฉพาะเมื่อผู้ใช้อยังไม่ได้กดเลือกหมวดเอง (ถ้าเลือกเองแล้ว ระบบเคารพการเลือกนั้น ไม่เดากับ) พิมพ์ "Starbucks" → หมวดเปลี่ยนเป็น อาหารและเครื่องดื่ม ทันที
73–74	openAndroidPicker เปิดหน้าต่างเลือกว่าวันหรือเวลาของ Android (mode = 'date' หรือ 'time')
75	ค่าที่เลือกอยู่ตอนเปิด
76	เลือกว่าวัน หรือเลือกเวลา

บรรทัด	ความหมาย
77	นาฬิกาแบบ 24 ชั่วโมง
78	เลือกวันอนาคตไม่ได้ (วันสูงสุด = วันนี้) สำหรับโหมดเวลาไม่จำกัด (undefined)
79–80	เมื่อผู้ใช้ปิดหน้าต่าง: ถ้ากดยกเลิก (type ไม่ใช่ 'set') หรือไม่มีวันที่ → ไม่ทำอะไร (return ออกเลย)
81–82	ถ้าเลือกวัน เก็บเป็น "วันไทย" ถ้าเลือกเวลา เก็บเป็น "เวลาไทย"
83–85	ปัดวงเล็บของ onChange, ของ object ที่ส่งให้ open และของฟังก์ชัน

“ ถ้าอาจารย์ถาม: เดามวดตอนไหน และหยุดเดาตอนไหน

"ระบบเดามวดทุกครั้งที่ใช้พิมพ์ชื่อรายการ ผ่านฟังก์ชัน suggestCategory แต่ทันทีที่ผู้ใช้กดเลือกหมวดเอง ตัวแปร categoryTouched จะเป็น true และระบบจะหยุดเดา เพื่อไม่ให้ทับสิ่งที่ผู้ใช้ตั้งใจเลือก และรายการเดิมที่เปิดมาแก้ถือว่าเลือกแล้วตั้งแต่ต้น"

ส่วนที่ 6: save — หัวใจของการบันทึก (บรรทัด 87–130)

ฟังก์ชันนี้รับค่า confirm (true/false) ปุ่ม "บันทึกรายการ" และ "ยืนยันและรวมในยอดเงิน" ส่ง true ส่วนปุ่ม "บันทึกแบบร่างไว้ก่อน" ของหน้าตรวจสอบส่ง false

src/app/transaction.tsx

บรรทัด 87–130

```

87  async function save(confirm: boolean) {
88    if (busy) return;
89    const satang = parseBahtToSatang(amount);
90    if (satang === null) {
91      setAmountError('ใส่จำนวนเงินมากกว่า 0 เช่น 85 หรือ 85.50');
92      return;
93    }
94    setAmountError(null);
95    if (dayError || timeError) return;
96    const input = {
97      kind,
98      amountSatang: satang,
99      categoryKey: category,
100     title: title.trim(),
101     note: note.trim() || null,
102     occurredAt: bkkToIso(day, time),
103   };
104   setBusy(true);
105   try {
106     if (existing) {
107       await updateTx(existing.id, {
108         ...input,
109         ...(confirm ? { status: 'confirmed' as const, reviewFlags: [] } : {}),
110       });
111       toast({ message: confirm && isDraft ? 'ยืนยันรายการแล้ว รวมในยอดเงินแล้ว' : 'บันทึกการแก้ไขแล้ว' });
112     } else {
113       await addTx({
114         ...input,
115         source: 'manual',
116         status: 'confirmed',

```

```

117         slipRef: null,
118         slipImageHash: null,
119         ocrConfidence: null,
120         reviewFlags: [],
121     });
122     toast({ message: kind === 'income' ? 'เพิ่มเงินเข้าแล้ว' : 'บันทึกรายการแล้ว' });
123 }
124 leaveWhenDone();
125 } catch (e) {
126     toast({ message: e instanceof Error && e.name === 'DuplicateSlipError' ? 'มีรายการจากสลิปนี้อยู่
แล้ว' : 'บันทึกไม่สำเร็จ ลองอีกครั้ง', tone: 'error' });
127 } finally {
128     setBusy(false);
129 }
130 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/app/transaction.tsx:87`

บรรทัด	ความหมาย
87	async = ฟังก์ชันนี้มีการรอ (await) งานที่ใช้เวลา เช่น ส่งข้อมูลไปเซิร์ฟเวอร์
88	กันกดซ้ำ: ถ้ากำลังบันทึกอยู่ ออกเลย (กดรัว ๆ 3 ครั้ง ก็ได้รายการเดียว)
89	แปลงข้อความเงินเป็นสตริง "85.50" → 8550, "abc" หรือ "0" → null
90–93	ถ้าแปลงไม่ได้: แสดงข้อความ "ใส่จำนวนเงินมากกว่า 0 เช่น 85 หรือ 85.50" ในช่องเงิน แล้ว หยุด (ไม่บันทึก)
94	เงินถูกแล้ว ล้างข้อความผิด
95	ถ้าวันหรือเวลาที่พิมพ์ (เว็บ) ผิด หยุด (ข้อความผิดแสดงอยู่ในช่องแล้ว)
96–103	รวบรวมข้อมูลเป็นก้อนเดียว (object ชื่อ input): ประเภท, จำนวนสตริง, หมวด, ชื่อ (ตัดช่องว่างหัวท้ายด้วย trim()), บันทึก (ถ้าตัดช่องว่างแล้วว่าง ใช้ null = "ไม่มี" แทนข้อความว่าง), เวลาที่เกิดขึ้น (รวมวัน+เวลาไทยเป็นเวลามาตรฐาน ISO)
104	ตั้งสถานะกำลังบันทึก (ปุ่มขึ้นวงหมุน)
105	try = "ลองทำ ถ้าพังให้ไปที่ catch"
106–110	กรณีแก้ไขเดิม: เรียก updateTx ด้วย id เดิม ... input (spread) = ทางทุกช่องของ input ออกมา และถ้า confirm เพิ่ม status = 'confirmed' และล้างตรงตรวจ (as const บอกว่าเป็นคำว่า 'confirmed' ตรง ๆ ไม่ใช่ข้อความอะไรก็ได้) ถ้าไม่ confirm ทาง {} ว่าง = ไม่เปลี่ยนสถานะ
111	toast: ถ้ายืนยันแบบร่าง → "ยืนยันรายการแล้ว รวมในยอดเงินแล้ว" ไม่จิ้น → "บันทึกการแก้ไขแล้ว"
112–121	กรณีรายการใหม่: addTx ด้วย input บวกช่องที่เหลือ: แหล่งที่มา 'manual' (จดเอง), สถานะ 'confirmed' (จดเองบันทึก ไม่ต้องรอยืนยัน), ไม่มีเลขอ้างอิงสลิป, ไม่มีรหัสรูปสลิป, ไม่มีความมั่นใจ OCR, ไม่ตรงตรวจ
122	toast: รายรับ → "เพิ่มเงินเข้าแล้ว", รายจ่าย → "บันทึกรายการแล้ว"
124	บันทึกสำเร็จ ปิดหน้าต่างกลับไปหน้าก่อน (ถ้าหน้านี้ยังเปิดอยู่)
125–126	ถ้าพัง: ถ้าเป็นข้อผิดพลาด DuplicateSlipError (สลิปนี้มีรายการแล้ว) → "มีรายการจากสลิปนี้อยู่แล้ว" ไม่จิ้น (เช่น เน็ตหลุด) → "บันทึกไม่สำเร็จ ลองอีกครั้ง" โทนสีแดง และ หน้ายังเปิดอยู่ ข้อมูลที่พิมพ์ไม่หาย
127–129	finally = ทำเสมอไม่ว่าสำเร็จหรือพัง: ปลดล็อกปุ่ม

กดบันทึก

busy? → ออก



ตรวจเงิน

แปลงไม่ได้ → แสดงข้อผิดพลาด หยุด



ตรวจวัน/เวลา (เว็บ)

ผิดพลาด → หยุด



รวม input

kind, amountSatang, categoryKey, title, note, occurredAt



มี existing?

มี → updateTx (+ ยืนยันถ้า confirm) · ไม่มี → addTx (manual, confirmed)



สำเร็จ → toast + ปิดหน้า · พัง → toast สีแดง หน้าค้างไว้

! สิ่งที่ต้องเจอต่อนอ่านโค้ดละเอียด

บรรทัด 94 เป็นที่เดียวที่ล้างข้อความผิดพลาดของเงิน (นอกจากตอนกดซิปเงิน) แปลว่าถ้าผู้ใช้กดบันทึกตอนช่องว่าง เห็นข้อความแดง แล้วพิมพ์ 85 ข้อความแดงจะ **ยังค้างอยู่** จนกดบันทึกอีกครั้ง ไม่ใช่บั๊กที่ทำให้ข้อมูลผิดพลาด แต่ปรับ UX ได้ด้วยการล้างข้อผิดพลาดใน onChangeText ของ AmountField (บรรทัด 241) จับบันทึกไว้ในหัวข้อ "ข้อจำกัด" ท้ายเล่ม

ส่วนที่ 7: doDelete — ลบแล้วเลิกทำได้ (บรรทัด 132–155)

src/app/transaction.tsx

บรรทัด 132–155

```
132  async function doDelete() {
133    if (!existing) return;
134    setAskDelete(false);
135    setBusy(true);
136    try {
137      const removed = await removeTx(existing.id);
138      leaveWhenDone();
139      toast({
140        message: 'ลบรายการแล้ว',
141        action: removed
142          ? {
143            label: 'เลิกทำ',
144            onPress: () => {
145              const { id: _id, createdAt: _c, ...rest } = removed;
146              addTx(rest).catch(() => toast({ message: 'กู้คืนไม่สำเร็จ', tone: 'error' }));
147            },
148          }
149      });
150    } catch {
151      // ...
152    }
153  }
154}
```

```

149      : undefined,
150    });
151  } catch {
152    toast({ message: 'ลบไม่สำเร็จ ลองอีกครั้ง', tone: 'error' });
153    setBusy(false);
154  }
155 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/transaction.tsx:132

บรรทัด	ความหมาย
132–133	ฟังก์ชันลบ ถ้าไม่มีรายการเดิม (หน้าจาดใหม่) ไม่มีอะไรให้ลบ ออกเลย
134	ปิดหน้าต่างถามยืนยัน
135	ล๊อคปุ่ม
137	ลบ และรับ "รายการที่ถูกลบ" กลับมา (removeTx คืนสำเนาของรายการที่ลบ ไว้ใช้คู่คืน)
138	ปิดหน้าต่างเด้ง (รายการไม่มีแล้ว ไม่มีอะไรให้เด้ง)
139–140	toast "ลบรายการแล้ว"
141–143	ถ้าได้รายการที่ลบกลับมา ใส่ปุ่ม "เลิกทำ" ใน toast (toast ที่มีปุ่มค้างนาน 5 วินาที ปกติ 2.8 วินาที)
144–145	เมื่อกดเลิกทำ: แยก id และ createdAt ออก ที่เหลือเก็บใน rest (destructuring: id: __id = เอา id มาใส่ตัวแปรชื่อ __id ชัดล้างหมายถึง "ไม่ได้ใช้") เพราะรายการใหม่ต้องได้ id และเวลาสร้างใหม่จากฐานข้อมูล
146	เพิ่มรายการกลับด้วยข้อมูลเดิมทุกช่อง ถ้าไม่สำเร็จ toast "คู่คืนไม่สำเร็จ" (.catch = จับข้อผิดพลาดของงาน async แบบไม่ ต้องใช้ try)
149	ถ้าไม่ได้รายการกลับมา ไม่มีปุ่มเลิกทำ
151–153	ลบไม่สำเร็จ: toast สีแดง "ลบไม่สำเร็จ ลองอีกครั้ง" และปลดล๊อคปุ่ม (หน้ายังเปิดอยู่ ลองใหม่ได้)

≈ เลิกทำ = จัดใหม่ด้วยข้อมูลเดิม

ฐานข้อมูลลบจริงไปแล้ว ไม่มี "ถังขยะ" การเลิกทำจึงเหมือน ถ่ายเอกสารใบเสร็จไว้ก่อนฉีกทิ้ง แล้วเอาสำเนาไปยื่นใหม่ ผลคือยอดเงินกลับมาเหมือนเดิมทุกบาท แต่รายการได้ id ใหม่ (ไม่มีผลกับผู้ใช้ เพราะไม่มีหน้าจอไหนแสดง id)

ส่วนที่ 8: ตัวช่วยวาด — flagged และ pill (บรรทัด 157–178)

src/app/transaction.tsx

บรรทัด 157–178

```

157 const flagged = (f: ReviewFlag) => isDraft && flags.includes(f);
158 const pill = (label: string, onPress: () => void, warn: boolean, ally: string) => (
159   <Pressable
160     onPress={onPress}
161     accessibilityRole="button"
162     accessibilityLabel={ally}
163     style={{
164       flex: 1,
165       flexDirection: 'row',
166       alignItems: 'center',

```

```

167         gap: 8,
168         borderWidth: 1.5,
169         borderColor: warn ? theme.watch : theme.line,
170         borderRadius: radius.md,
171         paddingVertical: 12,
172         paddingHorizontal: 14,
173         backgroundColor: theme.surface,
174     }}
175 >
176     <T v="body">{label}</T>
177 </Pressable>
178 );

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/app/transaction.tsx:157`

บรรทัด	ความหมาย
157	<code>flagged('amount')</code> ตอบ true เมื่อเป็นแบบร่างและ AI ขอให้ตรวจสอบช่องนั้น ใช้ไฮไลต์ช่องในส่วน JSX
158	<code>pill</code> ฟังก์ชันเล็กที่ "คืนหน้าตา" (JSX) ของปุ่มแคปซูลบน Android รับ ข้อความ, สิ่งที่ทำเมื่อกด, ต้องเตือนไหม, ข้อความสำหรับโปรแกรมอ่านหน้าจอ
159–162	ปุ่มกดได้ บอกโปรแกรมอ่านหน้าจอว่าเป็น "ปุ่ม" และอ่านชื่อว่าจะไร (เช่น "เลือกวันที่") — การเข้าถึง (accessibility)
163–174	หน้าตา: ยึดเต็มพื้นที่, เรียงแนวนอน, จัดกลางแนวตั้ง, ช่องว่าง 8, เส้นขอบ 1.5 สีเหลืองเตือนถ้า <code>warn</code> ไม่ขึ้นสีเส้นปกติ, มุมโค้งขนาดกลาง, ระยะในขอบ, พื้นสีการ์ด
176	ข้อความบนปุ่ม เช่น <code>"^{new} 8 ต.ค. 2569"</code>

ตัวช่วยจากไฟล์อื่น: `nav.ts` (ทั้งไฟล์)

`src/ui/nav.ts`

บรรทัด 1–23

```

1  /**
2   * Leave the current screen. On the web a screen can be opened directly (a
3   * reload, a bookmark, a link from an email) with nothing behind it; then
4   * "back" would do nothing, so go to the home screen instead.
5   */
6  import { router, useNavigation } from 'expo-router';
7
8  export function goBack() {
9      if (router.canGoBack()) router.back();
10     else router.replace('/');
11 }
12
13 /**
14  * Leave after something async finished (a save): only if this screen is still the one on show.
15  * The user may have closed it already while it was saving; going back again would also close
16  * the screen under it.
17  */
18 export function useLeaveWhenDone(): () => void {
19     const navigation = useNavigation();
20     return () => {
21         if (navigation.isFocused()) goBack();
22     };
23 }

```

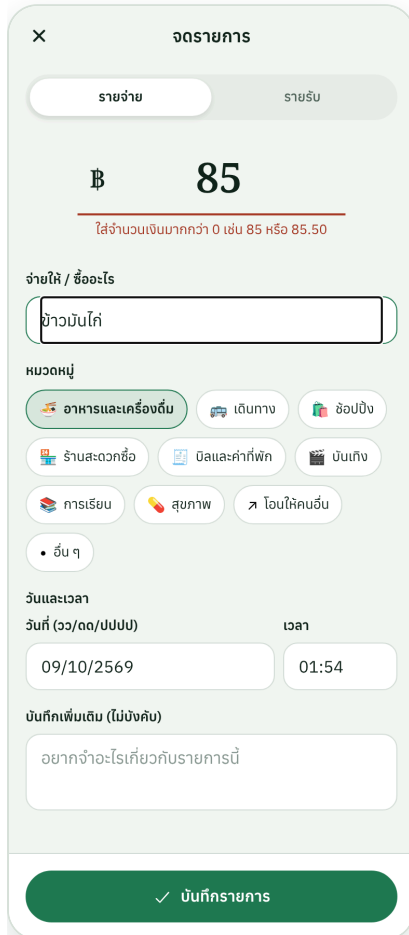
► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/ui/nav.ts:1`

บรรทัด	ความหมาย
1–5	บนเว็บ หน้าอาจถูกเปิดตรง ๆ (รีโหลด, บุกมาร์ก, ลิงก์จากอีเมล) โดยไม่มีหน้าก่อนหน้า กด "ย้อนกลับ" จะไม่เกิดอะไร จึงไปหน้าหลักแทน
8–11	goBack: ถ้าย้อนได้ → ย้อน ไม่จั้น → แทนที่ด้วยหน้าหลัก "/"
13–17	คอมเมนต์: ออกจากหน้าหลังงาน async เสร็จ เฉพาะเมื่อนี้ยังแสดงอยู่ เพราะผู้ใช้อาจปิดหน้าไปแล้วระหว่างรอบันทึก ถ้าย้อนอีกจะปิดหน้าข้างใต้ไปด้วย (เช่น เด้งออกจากแท็บที่ไม่ได้ตั้งใจ)
18–22	useLeaveWhenDone เป็น hook (ชื่อขึ้นต้นด้วย use) จับ navigation ของหน้าไว้ คำนึงฟังก์ชันที่เช็ค isFocused() ก่อนเรียก goBack

อ่านโค้ด: หน้าจอดรายการ ตอนที่ 2 — หน้าตา (JSX)

บรรทัด 180–364 คือสิ่งที่ผู้ใช้เห็น อ่านจากบนลงล่างได้ตรงกับหน้าจอจริง



หน้าจอดรายการที่กรอกแล้ว เทียบกับโค้ดใน
บทนี้ได้ทีละส่วน: หัวหน้าจอ → ปุ่มสลับ →
ช่องเงิน → ชื่อ → หมวด → วันเวลา → บันทึก
→ ปุ่มล่าง

ส่วนที่ 9: โครงหน้าและหัวหน้าจอ (บรรทัด 180–193)

src/app/transaction.tsx

บรรทัด 180–193

```
180   return (
181     <SafeAreaView edges={['top', 'bottom']} style={{ flex: 1, backgroundColor: theme.bg }}>
182       <KeyboardAvoidingView style={{ flex: 1 }} behavior={Platform.OS === 'ios' ? 'padding' :
undefined}>
183         <Row justify="space-between" style={{ paddingHorizontal: space.sm, paddingTop: space.sm
}}>
184           <IconButton icon="close" label="ปิด" onPress={() => goBack()} />
185           <T v="h3">{!existing ? (kind === 'income' ? 'เพิ่มเงินเข้า' : 'จดยรายการ') : isDraft ? 'ตรวจ
สลิป' : 'แก้ไขรายการ'}</T>
186           {existing ? (
187             <IconButton icon="trash-outline" label="ลบรายการ" color={theme.critical} onPress={()
=> setAskDelete(true)} />
188           ) : (
```

```

189         <HeaderDecor />
190     )}
191 </Row>
192
193     <ScrollView contentContainerStyle={{ padding: space.lg, gap: space.lg, paddingBottom:
space.xxxl }} keyboardShouldPersistTaps="handled">

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/transaction.tsx:180

บรรทัด	ความหมาย
180	return (= "หน้าตาของหน้านี้คือ..." ทุกอย่างหลังจากนี้เป็น JSX
181	กล่องนอกสุด กั้นขอบบนและล่างของมือถือ เต็มจอ (flex: 1) พื้นสีส้ม
182	บน iPhone ดันเนื้อหาขึ้นเมื่อคีย์บอร์ดเด้ง (padding) Android จัดการเองอยู่แล้วจึงไม่ใส่
183	แถวหัวหน้าจอ: 3 ชั้น กระจายซ้าย-กลาง-ขวา
184	ซ้าย: ปุ่ม × ชื่อ "ปิด" กดแล้วย้อนกลับ (ไม่บันทึก)
185	กลาง: ชื่อหน้าแบบ 4 ทาง ไม่มีของเดิม → (รายรับ? "เพิ่มเงินเข้า" : "จดรายการ") มีของเดิม → (แบบร่าง? "ตรวจสอบ" : "แก้ไขรายการ")
186–190	ขวา: มีของเดิม → ปุ่มถึงขยะสีแดง "ลบรายการ" (กดแล้วเปิดหน้าต่างถาม ไม่ลบทันที) ไม่มี → ของตกแต่ง
193	เริ่มส่วนเลื่อนได้ ระยะขอบใหญ่ ระยะห่างระหว่างชั้นใหญ่ keyboardShouldPersistTaps="handled" = แต่ละขั้นตอนคีย์บอร์ดเปิดอยู่ ก็กดติดในครั้งเดียว

ส่วนที่ 10: การ์ดตรวจสอบ (บรรทัด 194–230) — เชื่อม FR-4

src/app/transaction.tsx

บรรทัด 194–230

```

194     {isDraft ? (
195         <Card tone="accent">
196             <Row gap={space.sm} align="flex-start">
197                 <Icons name="receipt-outline" size={20} color={theme.accentInk} />
198                 <View style={{ flex: 1, gap: 4 }}>
199                     <T v="body" style={{ fontFamily: fonts.sansSemi }}>
200                         {flags.length === 0 ? 'อ่านสลิปได้ครบ ตรวจอีกครั้งแล้วกดยืนยัน' : `ช่วยตรวจ:
${flags.map((f) => FLAG_LABEL[f]).join(', ')}`}
201                     </T>
202                     <T v="small">
203                         ยังไม่รวมใบยอดเงินจนกว่าจะยืนยัน
204                         {existing?.ocrConfidence !== null ? ` · ความมั่นใจต่ำสุด
${Math.round(existing.ocrConfidence * 100)}%` : ''}
205                     </T>
206                     {altLine ? (
207                         <T v="small" color={theme.ink}>
208                             {altLine}
209                         </T>
210                     ) : null}
211                     {choices.length > 1 ? (
212                         <Row gap={space.sm} style={{ flexWrap: 'wrap', marginTop: 4 }}>
213                             <T v="small">ดูที่สลิปแล้วเลือก:</T>
214                             {choices.map((c) => (
215                                 <Chip
216                                     key={c}

```

```

217             label={formatBaht(c)}
218             selected={parseBahtToSatang(amount) === c}
219             onPress={() => {
220                 setAmount(satangToInput(c));
221                 setAmountError(null);
222             }}
223         />
224     )}
225 </Row>
226     ) : null}
227 </View>
228 </Row>
229 </Card>
230 ) : null}

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/transaction.tsx:194

บรรทัด	ความหมาย
194–195	แสดงเฉพาะแบบร่าง: การ์ดโทบนัน
196–197	แถว: ไอคอนใบเสร็จ
199–201	ข้อความหลัก: ไม่มีธง → "อ่านสลิปได้ครบ ตรวจสอบอีกครั้งแล้วกดยืนยัน" มีธง → "ช่วยตรวจสอบ: ยอดเงิน, วันที่" (แปลงธงเป็นชื่อไทยด้วย FLAG_LABEL แล้วเชื่อมด้วยจุลภาค)
202–205	"ยังไม่รวมในยอดเงินจนกว่าจะยืนยัน" และถ้ามีความมั่นใจ ต่อท้าย " · ความมั่นใจต่ำสุด 72%" (0.72×100 ปดเศษ)
206–210	ถ้า AI สองรอบอ่านไม่ตรงกัน แสดงบรรทัด "AI อ่านได้ 2 แบบ: ..."
211–226	ถ้ามียอดให้เลือกว่ามากกว่า 1 ยอด: "ดูที่สลิปแล้วเลือก:" ตามด้วยชิปยอดเงิน ชิปที่ตรงกับช่องเงินตอนนี้ถูกเลือก กดชิป → ใส่ยอดนั้นในช่องเงินและล้างข้อผิดพลาด

ส่วนนี้เป็นของ FR-4 แต่ **อยู่ในไฟล์ FR-1** เพราะการตรวจแบบร่าง = การแก้ไขรายการ 1 รายการ ใช้ช่องกรอกชุดเดียวกัน ไม่ต้องสร้างหน้าใหม่ (ดูเล่ม FR-4 สำหรับที่มาของธงและยอด 2 แบบ)

ส่วนที่ 11: ปุ่มสลับ ช่องเงิน และชิปเงินเข้า (บรรทัด 232–256)

src/app/transaction.tsx

บรรทัด 232–256

```

232     <Segmented<TxKind>
233       options={[
234         { key: 'expense', label: 'รายจ่าย' },
235         { key: 'income', label: 'รายรับ' },
236       ]}
237       value={kind}
238       onChange={changeKind}
239     />
240
241     <AmountField id="tx-amount" value={amount} onChangeText={setAmount} error={amountError}
242     flagged={flagged('amount')} color={kind === 'income' ? theme.income : theme.ink} />
243     {!existing && kind === 'income' ? (
244       <Row gap={space.sm} style={{ flexWrap: 'wrap', justifyContent: 'center', marginTop: -
245       space.sm }}>
246         {[10_000, 50_000, 100_000, 500_000].map((q) => (
247           <Chip
248             key={q}

```

```

247         label={`+${formatBaht(q, { decimals: false })}`}
248         selected={parseBahtToSatang(amount) === q}
249         onPress={() => {
250             setAmount(satangToInput(q));
251             setAmountError(null);
252         }}
253     />
254 ))}
255 </Row>
256 ) : null}

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/transaction.tsx:232

บรรทัด	ความหมาย
232–239	ปุ่มสลับ 2 ทาง (Segmented) "รายจ่าย" "รายรับ" <TxKind> บอกว่าค่าเป็น TxKind ค่าปัจจุบัน = kind เปลี่ยนแล้ว เรียก changeKind (ส่วนที่ 5)
241	ช่องเงินตัวใหญ่: id สำหรับทดสอบ E2E, ค่า = amount, พิมพ์แล้ว setAmount, แสดงข้อผิดพลาด, ไฮไลต์ถ้า AI ขอตรวจสอบ, สีเขียวรายรับ / สีหมึกปกติรายจ่าย
242	เฉพาะ เพิ่มเงินเข้าใหม่ (ไม่ใช้การแก้ไข และเป็นรายรับ)
243–244	แถวชิป 4 ยอด: 10,000 / 50,000 / 100,000 / 500,000 สตางค์ = ฿100 / ฿500 / ฿1,000 / ฿5,000 (เลขมี _ คั่นให้อ่านง่าย ภาษาไม่สนใจขีดล่าง)
247	ป้าย "+฿100" (ไม่แสดงทศนิยม)
248	ชิปที่ตรงกับยอดในช่องจะถูกเป็น "ถูกเลือก"
249–252	กดชิป → ใส่ "100.00" ในช่องเงิน และล้างข้อผิดพลาด

ส่วนที่ 12: ชื่อรายการและหมวด (บรรทัด 258–284)

src/app/transaction.tsx

บรรทัด 258–284

```

258     <Field
259       id="tx-title"
260       label={kind === 'expense' ? 'จ่ายให้ / ชื่ออะไร' : 'ได้รับจาก'}
261       value={title}
262       onChangeText={changeTitle}
263       placeholder={kind === 'expense' ? 'เช่น ข้าวมันไก่' : 'เช่น ค่าขนมจากที่บ้าน'}
264       maxLength={120}
265       error={flagged('counterparty') ? 'ตรวจสอบชื่อให้ตรงกับสลิป' : null}
266     />
267
268     <View style={{ gap: space.sm }}>
269       <T v="small" color={theme.ink} style={{ fontFamily: fonts.sansSemi }}>หมวดหมู่</T>
270       <Row gap={space.sm} style={{ flexWrap: 'wrap' }}>
271         {cats.map((c) => (
272           <Chip
273             key={c.key}
274             label={c.label}
275             glyph={c.glyph}
276             selected={category === c.key}
277             onPress={() => {
278               setCategory(c.key);

```

```

279         setCategoryTouched(true);
280     }}
281     />
282     )})
283     </Row>
284     </View>

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/transaction.tsx:258

บรรทัด	ความหมาย
258–259	ช่องกรอกชื่อ id "tx-title"
260	ป้ายเปลี่ยนตามประเภท: รายจ่าย "จ่ายให้ / ชื่ออะไร" รายรับ "ได้รับจาก"
261–262	ค่า = title พิมพ์แล้วเรียก changeTitle (ซึ่งเดาหมวดด้วย)
263	ตัวอย่างจาก ๆ ในช่องว่าง: "เช่น ข้าวมันไก่" / "เช่น ค่าขนนมจากที่บ้าน"
264	ยาวสุด 120 ตัวอักษร ตรงกับ check ในฐานข้อมูล (title ≤ 120)
265	ถ้า AI ไม่พอใจชื่อผู้รับ แสดง "ตรวจสอบชื่อให้ตรงกับสลิป"
268–269	หัวข้อ "หมวดหมู่" ตัวหนาเล็ก
270–271	แถวที่ขึ้นบรรทัดใหม่ได้ (wrap) วนสร้างชิปหมวดทีละหมวดด้วย map
273	key = รหัสหมวด ให้ React แยกชิปแต่ละอันได้
274–276	ชื่อหมวด, ไอคอน (🍷) และถูกเลือกถ้าตรงกับ category
277–280	กดชิป → เลือกหมวดนั้น และ จำว่าผู้ใช้เลือกเองแล้ว (หยุดเดา)

ส่วนที่ 13: วันและเวลา 3 แบบตามแพลตฟอร์ม (บรรทัด 286–336)

src/app/transaction.tsx

บรรทัด 286–336

```

286     <View style={{ gap: 6 }}>
287       <T v="small" color={theme.ink} style={{ fontFamily: fonts.sansSemi }}>วันและเวลา</T>
288       {Platform.OS === 'web' ? (
289         <Row gap={space.sm} align="flex-start">
290           <View style={{ flex: 1 }}>
291             <Field
292               id="tx-day"
293               label="วันที่ (วว/ดด/ปปปป)"
294               value={dayText}
295               onChangeText={(s) => {
296                 setDayText(s);
297                 const parsed = parseSlipDate(s);
298                 if (parsed && parsed <= bkkDayKey(new Date())) setDay(parsed);
299               }}
300               error={dayError}
301             />
302           </View>
303           <View style={{ width: 110 }}>
304             <Field
305               id="tx-time"
306               label="เวลา"

```

```

307         value={timeText}
308         onChangeText={(s) => {
309             setTimeText(s);
310             const t = parseSlipTime(s);
311             if (t) setTime(t);
312         }}
313         error={timeError}
314     />
315 </View>
316 </Row>
317 ) : Platform.OS === 'android' ? (
318     <Row gap={space.sm}>
319         {pill(`📅 ${formatThaiDay(day)}`, () => openAndroidPicker('date'),
flagged('date'), 'เลือกวันที่')}
320         {pill(`🕒 ${time}`, () => openAndroidPicker('time'), false, 'เลือกเวลา')}
321     </Row>
322 ) : (
323     <Row gap={space.sm}>
324         <DateTimePicker
325             value={pickerDate}
326             mode="date"
327             display="compact"
328             maximumDate={new Date()}
329             locale="th-TH"
330             onChange={(_, d) => d && setDay(bkkDayKey(d))}
331         />
332         <DateTimePicker value={pickerDate} mode="time" display="compact" locale="th-TH"
onChange={(_, d) => d && setTime(bkkTime(d))} />
333     </Row>
334 )}
335 {flagged('date') ? <T v="small" color={theme.watch}>อ่านวันที่บนสลิปไม่ชัด ตรวจสอบให้ตรงก่อน
ยืนยัน</T> : null}
336 </View>

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/transaction.tsx:286

บรรทัด	ความหมาย
286–287	หัวข้อ "วันและเวลา"
288–289	แบบที่ 1 เว็บ: แถวสองช่อง
290–294	ช่องวันที่ (กว้างยึดเต็ม) ป้าย "วันที่ (วว/ดด/ปปปป)" ค่า dayText
295–299	พิมพ์แล้ว: เก็บข้อความ ลองอ่านเป็นวันที่ ถ้าอ่านได้ และไม่ใช้วันอนาคต จึงเปลี่ยน day จริง
300	แสดงข้อผิดพลาดจากบรรทัด 52–53
303–314	ช่องเวลา กว้าง 110 ป้าย "เวลา" พิมพ์แล้วอ่านเป็นเวลา ถ้าถูกเปลี่ยน time และแสดงข้อผิดพลาดจากบรรทัด 54
317–321	แบบที่ 2 Android: ปุ่มแคปซูล 2 ปุ่ม "📅 วันที่" (เตือนสีเหลืองถ้า AI ไม่แน่ใจวันที่) และ "🕒 เวลา" กดแล้วเปิดหน้าต่างเลือกของ Android (ส่วนที่ 5)
322–333	แบบที่ 3 iPhone: ตัวเลือกวันแบบกะทัดรัด ห้ามเกินวันนี้ ภาษาไทย และตัวเลือกเวลา เลือกแล้วเก็บเป็นวัน/เวลาไทย (d && setDay(...) = ทำเฉพาะเมื่อมี d)
335	ถ้า AI อ่านวันที่บนสลิปไม่ชัด: ข้อความเตือนสีเหลือง

i ทำไมต้อง 3 แบบ?

เพราะแต่ละแพลตฟอร์มมีตัวเลือกวันที่ไม่เหมือนกัน: iPhone ฝังในหน้าได้, Android ต้องเปิดเป็นหน้าต่าง, เว็บไม่มีตัวเลือกของไลบรารีนี้เลย Platform.OS ทำให้โค้ดชุดเดียวรันได้ทั้ง 3 ที่ (บน A7) และทั้ง 3 แบบมีกฎเดียวกัน: ห้ามวนอนาคต

ส่วนที่ 14: บันทึกเพิ่มเติม ปุ่มล่าง และหน้าต่างยืนยันลบ (บรรทัด 338–364)

src/app/transaction.tsx

บรรทัด 338–364

```
338     <Field id="tx-note" label="บันทึกเพิ่มเติม (ไม่บังคับ)" value={note} onChangeText={setNote}
multiline maxLength={500} placeholder="อยากทำอะไรเกี่ยวกับรายการนี้" />
339   </ScrollView>
340
341   <View style={{ padding: space.lg, gap: space.sm, borderTopWidth: 1, borderTopColor:
theme.line, backgroundColor: theme.surface }}>
342     {isDraft ? (
343       <>
344         <Button label="ยืนยันและรวมในยอดเงิน" kind="gold" icon="checkmark-circle" onPress={() =>
save(true)} loading={busy} />
345         <Button label="บันทึกแบบร่างไว้ก่อน" kind="ghost" onPress={() => save(false)} disabled=
{busy} />
346       </>
347     ) : (
348       <Button label={existing ? 'บันทึกการแก้ไข' : 'บันทึกรายการ'} icon="checkmark" onPress={() =>
save(true)} loading={busy} />
349     )}
350   </View>
351 </KeyboardAvoidingView>
352
353   <ConfirmSheet
354     visible={askDelete}
355     title="ลบรายการนี้?"
356     body={isDraft ? 'แบบร่างจากสลิปนี้จะถูกลบ ถ้าสแกนรูปเดิมอีกครั้งจะอ่านใหม่ได้' : 'ยอดคงเหลือและ Money Runway
จะคำนวณใหม่ทันที กดเลิกทำได้ภายใน 5 วินาที'}
357     confirmLabel="ลบรายการ"
358     destructive
359     onConfirm={doDelete}
360     onCancel={() => setAskDelete(false)}
361   />
362 </SafeAreaView>
363 );
364 }
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/transaction.tsx:338

บรรทัด	ความหมาย
338	ช่องบันทึกหลายบรรทัด ไม่บังคับ ยาวสุด 500 ตัวอักษร (ตรงกับ check note ≤ 500 ในฐานข้อมูล)
339	ปิดส่วนเลื่อนได้
341	แถบปุ่มล่าง อยู่นอกส่วนเลื่อน จึงติดอยู่ล่างจอเสมอ มีเส้นคั่นด้านบน

บรรทัด	ความหมาย
342–346	แบบร่าง: ปุ่มทอง "ยืนยันและรวมในยอดเงิน" (save(true)) และปุ่มโปร่ง "บันทึกแบบร่างไว้ก่อน" (save(false)) <> . . . </> คือกล่องล่องหน (Fragment) ห่อ 2 ปุ่ม
347–349	ไม่ใช่แบบร่าง: ปุ่มเดียว "บันทึกการแก้ไข" หรือ "บันทึกรายการ" แสดงวงหมุนระหว่าง busy
353–354	หน้าต่างยืนยัน แสดงเมื่อ askDelete เป็น true
355	หัวข้อ "ลบบรายการนี้?"
356	เนื้อหา: แบบร่าง → "แบบร่างจากสลิปนี้จะถูกลบ ถ้าสแกนรูปเดิมอีกครั้งจะอ่านใหม่ได้" ปกติ → "ยอดคงเหลือและ Money Runway จะคำนวณใหม่ทันที กดเลิกทำได้ภายใน 5 วินาที" (บอกผลกระทบ ก่อน ผู้ใช้ตัดสินใจ)
357–358	ปุ่มยืนยัน "ลบบรายการ" สีแดง (destructive = การกระทำที่ทำลายข้อมูล)
359–360	ยืนยัน → doDelete, ยกเลิก → ปิดหน้าต่าง
362–364	ปิดกล่องนอกสุด, ปิด return, ปิดฟังก์ชัน

★ ลองเองใน VS Code

เปิด `src/app/transaction.tsx:356` แล้วลองเปลี่ยนข้อความในหน้าต่างลบ รับ npm run web เปิดรายการใดก็ได้ กดถึงขยะ จะเห็นข้อความใหม่ทันที (อย่าลืมเปลี่ยนกลับ หรือ git checkout -- src/app/transaction.tsx)

บทที่ 12

อ่านโค้ด: src/domain/categories.ts ทั้งไฟล์

111 บรรทัด หมวด 15 หมวด และ 'สมอง' ที่เดาหมวดจากชื่อร้าน

ส่วนที่ 1: รูปร่างของหมวด (บรรทัด 1–14)

src/domain/categories.ts

บรรทัด 1–14

```
1 /**
2  * Categories (FR-1 "จัดหมวด"). Fixed list for the MVP so reports stay comparable.
3  * `keywords` let the app suggest a category from a slip's counterparty name.
4  */
5 import type { TxKind } from '../types';
6
7 export interface Category {
8   key: string;
9   kind: TxKind;
10  label: string;
11  /** Single glyph used as the category icon in lists and charts. */
12  glyph: string;
13  keywords: string[];
14 }
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/categories.ts:1

บรรทัด	ความหมาย
1–4	คอมเมนต์: หมวดของ FR-1 "จัดหมวด" เป็นรายการตายตัวใน MVP เพื่อให้รายงานเปรียบเทียบกันได้ (ถ้าผู้ใช้สร้างหมวดเองได้ "อาหาร" กับ "กิน" จะแยกกัน กราฟของ FR-2 จะเพี้ยน) และ keywords ใช้เดาหมวดจากชื่อผู้รับบนสลิป
5	นำชนิด TxKind มาใช้
7	interface = แบบพิมพ์เขียวของ object หมวด 1 หมวด ต้องมี 5 ช่อง
8	key: รหัสภาษาอังกฤษที่เก็บในฐานข้อมูล (category_key) เช่น 'food'
9	kind: เป็นหมวดของรายจ่ายหรือรายรับ
10	label: ชื่อไทยที่ผู้ใช้เห็น
11–12	glyph: อีโมจิ 1 ตัวใช้เป็นไอคอนในรายการและกราฟ
13	keywords: คำที่ถ้าเจอในชื่อ แปลว่าน่าจะเป็นหมวดนี้

i ทำไมเก็บ key ไม่เก็บชื่อไทย?

ฐานข้อมูลเก็บ 'food' ไม่ใช่ "อาหารและเครื่องดื่ม" ถ้าวันหนึ่งอยากเปลี่ยนชื่อไทยเป็น "อาหาร/เครื่องดื่ม" แค่แก้บรรทัด 17 รายการค่าทุกรายการเปลี่ยนชื่อตามเอง ไม่ต้องแก้ข้อมูลในฐานข้อมูลเลย

ส่วนที่ 2: รายการหมวดทั้ง 15 หมวด (บรรทัด 16–42)

src/domain/categories.ts

บรรทัด 16–42

```
16 export const CATEGORIES: Category[] = [
17   { key: 'food', kind: 'expense', label: 'อาหารและเครื่องดื่ม', glyph: '🍽️',
18     keywords: ['ร้าน', 'ข้าว', 'กาแฟ', 'cafe', 'café', 'coffee', 'อาหาร', 'ก๋วยเตี๋ยว', 'ชาวม', 'ชาไข่มุก',
19       'mk restaurant', 'สุกี้', 'kfc', 'mcdonald', 'starbucks', 'amazon', 'food', 'grabfood', 'lineman',
20       'foodpanda', 'บุฟเฟ่ต์'] },
21   { key: 'transport', kind: 'expense', label: 'เดินทาง', glyph: '🚗',
22     keywords: ['bts', 'mrt', 'grab', 'bolt', 'taxi', 'แท็กซี่', 'วิน', 'ปตท', 'ptt', 'shell',
23       'bangchak', 'บงจาก', 'ค่ารถ', 'ทางด่วน', 'easy pass'] },
24   { key: 'shopping', kind: 'expense', label: ' ช้อปปิ้ง', glyph: '🛒',
25     keywords: ['shopee', 'lazada', 'central', 'uniqlo', 'lotus', 'big c', 'bigc', 'makro',
26       'watsons', 'boots', 'ikea', 'tiktok shop'] },
27   { key: 'convenience', kind: 'expense', label: 'ร้านสะดวกซื้อ', glyph: '🏪',
28     keywords: ['7-eleven', '7eleven', 'เซเว่น', 'cp all', 'family mart', 'familymart', 'lawson',
29       'tops daily', 'mini big c'] },
30   { key: 'bills', kind: 'expense', label: 'บิลและค่าที่พัก', glyph: '📄',
31     keywords: ['ค่าไฟ', 'การไฟฟ้า', 'ประปา', 'true', 'ais', 'dtac', '3bb', 'หอพัก', 'ค่าเช่า', 'rent',
32       'internet'] },
33   { key: 'fun', kind: 'expense', label: 'บันเทิง', glyph: '🎬',
34     keywords: ['major', 'sf cinema', 'netflix', 'spotify', 'youtube', 'steam', 'game', 'เกม',
35       'คอนเสิร์ต', 'ticket'] },
36   { key: 'study', kind: 'expense', label: 'การเรียนรู้', glyph: '📖',
37     keywords: ['มหาวิทยาลัย', 'university', 'book', 'หนังสือ', 'se-ed', 'ศึกษา', 'ค่าทอม', 'ถ่ายเอกสาร']
38   },
39   { key: 'health', kind: 'expense', label: 'สุขภาพ', glyph: '💊',
40     keywords: ['โรงพยาบาล', 'hospital', 'คลินิก', 'clinic', 'ร้านยา', 'pharmacy', 'fitness', 'ฟิตเนส']
41   },
42   { key: 'transfer_out', kind: 'expense', label: 'โอนให้คนอื่น', glyph: '➡️',
43     keywords: ['นาย', 'นาง', 'น.ส.', 'นางสาว', 'mr.', 'ms.', 'mrs.'] },
44   { key: 'other_expense', kind: 'expense', label: 'อื่น ๆ', glyph: '•', keywords: [] },
45   { key: 'allowance', kind: 'income', label: 'เงินจากที่บ้าน', glyph: '🏠', keywords: ['พ่อ', 'แม่', 'ค่า
46     บนม'] },
47   { key: 'salary', kind: 'income', label: 'เงินเดือน / ค่าจ้าง', glyph: '💰', keywords: ['เงินเดือน',
48     'salary', 'payroll', 'ค่าจ้าง'] },
49   { key: 'part_time', kind: 'income', label: 'งานพิเศษ', glyph: '✨', keywords: ['freelance', 'ฟรี
50     แลนซ์', 'พาร์ทไทม์'] },
51   { key: 'transfer_in', kind: 'income', label: 'รับโอน', glyph: '⬅️', keywords: [] },
52   { key: 'other_income', kind: 'income', label: 'รายรับอื่น ๆ', glyph: '•', keywords: [] },
53 ];
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/categories.ts:16

บรรทัด	ความหมาย
16	ค่าคงที่ CATEGORIES เป็นรายการ (array) ของ Category
17–18	food 🍽️ อาหารและเครื่องดื่ม: ร้าน ข้าว กาแฟ ก๋วยเตี๋ยว ชาวม แบนด์ (KFC, Starbucks, Amazon = คาเฟ่เมซอน) และแอปส่งอาหาร
19–20	transport 🚗 เดินทาง: BTS MRT Grab Bolt แท็กซี่ วิน ปีมน้ำมัน ทางด่วน
21–22	shopping 🛒 ช้อปปิ้ง: Shopee Lazada Central Lotus ฯลฯ
23–24	convenience 🏪 ร้านสะดวกซื้อ: 7-Eleven, CP ALL, FamilyMart, Lawson, mini Big C

บรรทัด	ความหมาย
25–26	bills 📄 บิลและค่าที่หัก: ค่าไฟ ประปา ค่ามือถือ เน็ต หอพัก ค่าเช่า
27–28	fun 🎬 บันเทิง: โรงหนัง Netflix Spotify เกม คอนเสิร์ต
29–30	study 📖 การเรียน: มหาวิทยาลัย หนังสือ ค่าเทอม ค่าเอกสาร
31–32	health 💊 สุขภาพ: โรงพยาบาล คลินิก ร้านยา ฟิตเนส
33–34	transfer_out ➡ โอนให้คนอื่น: คำนำหน้าชื่อคน (นาย นาง น.ส. Mr.) — ใช้แยกต่างหาก ดูบรรทัด 95 และ 106–108
35	other_expense • อื่น ๆ (ไม่มีคำสำคัญ = หมวดสำรองของรายจ่าย)
37	allowance 🏠 เงินจากที่บ้าน: พ่อ แม่ ค่าขนม
38	salary 💼 เงินเดือน / ค่าจ้าง
39	part_time ✨ งานพิเศษ: freelance ฟรีแลนซ์ พาร์ทไทม์
40	transfer_in ⚡ รับโอน (หมวดตั้งต้นของรายรับ)
41	other_income • รายรับอื่น ๆ
42	ปิดรายการ (;)

ส่วนที่ 3: ฟังก์ชันช่วยหาหมวด (บรรทัด 44–56)

src/domain/categories.ts

บรรทัด 44–56

```

44 const BY_KEY = new Map(CATEGORIES.map((c) => [c.key, c]));
45
46 export function getCategory(key: string): Category {
47   return BY_KEY.get(key) ?? BY_KEY.get('other_expense')!;
48 }
49
50 export function categoriesFor(kind: TxKind): Category[] {
51   return CATEGORIES.filter((c) => c.kind === kind);
52 }
53
54 export function defaultCategory(kind: TxKind): string {
55   return kind === 'income' ? 'transfer_in' : 'other_expense';
56 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/categories.ts:44

บรรทัด	ความหมาย
44	สร้าง Map (สมุดค้นหา) จาก key → หมวด ทำครั้งเดียวตอนโหลดไฟล์ คันทันทีหลังเร็วมาก ไม่ต้องวนหาทีละตัว
46–48	getCategory('food') ค้นหาหมวด ถ้าไม่รู้จัก key (เช่น ข้อมูลเก่าที่หมวดถูกลบไปแล้ว) คืน "อื่น ๆ" แทน แอปจะไม่พัง (! ท้าย = ยืนยันว่า other_expense มีแน่นอน)
50–52	categoriesFor('income') กรองเหลือเฉพาะหมวดของประเภทนั้น (filter = เก็บเฉพาะตัวที่ตรงเงื่อนไข)
54–56	หมวดตั้งต้น: รายรับ → รับโอน, รายจ่าย → อื่น ๆ

ส่วนที่ 4: ตรวจสอบว่าเป็นชื่อคนใหม่ และหาคำสำคัญ (บรรทัด 58–78)

src/domain/categories.ts

บรรทัด 58–78

```
58 /** A slip counterparty that is a person ("นาย สมชาย", "MR. JOHN"). */
59 const PERSON_TITLE = /\s*(?:นาย|นางสาว|นาง|น.ส.|ด.ช.|ด.ญ.|mr|mrs|ms|miss)
    (?:\s|$)/i;
60
61 /**
62  * Where a keyword matches and how long it is (null = no match).
63  * - Latin keywords must start a word: "ais" is not in "PAISAN", "rent" is not in "LAURENT";
    brands
64  *   written together still match ("GRABPAY", "SHOPEEPAY", "TRUEMOVE", "CENTRALWORLD").
65  * - After a person title, short keywords are skipped: "วิน" is not in "นาย วินัย", "ais" not in
66  *   "MS AISHA". (Thai has no spaces between words, so keywords are substrings; a unit test,
67  *   TC-25, guards against short words like "ชา" matching the name "สมชาย".)
68  */
69 function findKeyword(text: string, keyword: string, person: boolean): { at: number; len: number }
    | null {
70   if (person && keyword.length < 5) return null;
71   if (/^[a-z0-9 .&'"]+$/.test(keyword)) {
72     const re = new RegExp(`^[^a-z](${keyword.replace(/[\.*+?^$()|[\]\\\]/g, '\\$&')})`);
73     const m = re.exec(text);
74     return m ? { at: m.index + m[1].length, len: keyword.length } : null;
75   }
76   const at = text.indexOf(keyword);
77   return at < 0 ? null : { at, len: keyword.length };
78 }
```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/categories.ts:58

บรรทัด	ความหมาย
58–59	PERSON_TITLE คือ regular expression (รูปแบบข้อความ) อ่านว่า: ต้นข้อความ (^) อาจมีช่องว่าง (\s*) แล้วตามด้วยคำนำหน้าคำใดคำหนึ่ง (นาย นางสาว นาง น.ส. ด.ช. ด.ญ. mr mrs ms miss) แล้วต้องตามด้วยช่องว่างหรือจบข้อความ i ท้าย = ไม่สนตัวใหญ่ตัวเล็ก
61–68	คอมเมนต์อธิบาย 2 กฎสำคัญ: (1) คำภาษาอังกฤษต้อง ขึ้นต้นคำ "ais" จะไม่ไปเจอใน "PAISAN", "rent" ไม่เจอใน "LAURENT" แต่ชื่อแบรนด์ที่เขียนติดกันยังเจอ (GRABPAY) (2) ถ้าเป็นชื่อคน ข้ามคำสั้น: "วิน" ไม่ไปเจอใน "นาย วินัย" ภาษาไทยไม่มีช่องว่างระหว่างคำ จึงต้องค้นแบบข้อความย่อย และมีเทสต์ TC-25 กันคำสั้นอย่าง "ชา" ไปเจอใน "สมชาย"
69	ฟังก์ชันรับ ข้อความ, คำสำคัญ, เป็นชื่อคนใหม่ คืนตำแหน่งที่เจอและความยาว หรือ null
70	กฎ (2): เป็นชื่อคนและคำสั้นกว่า 5 ตัวอักษร → ถือว่าไม่เจอ
71	ถ้าคำสำคัญเป็นภาษาอังกฤษ/ตัวเลขล้วน...
72	...สร้างรูปแบบ "ต้นข้อความ หรือหลังตัวที่ไม่ใช่ a-z แล้วตามด้วยคำนั้น" (replace ยาว ๆ คือการ escape อักขระพิเศษอย่าง . ใน "mr." ให้เป็นจุดธรรมดา)
73–74	ค้นหา เจอ → คืนตำแหน่ง (เลื่อนข้ามตัวอักษรนำหน้า 1 ตัวถ้ามี) และความยาว ไม่เจอ → null
76–77	ภาษาไทย: ค้นแบบข้อความย่อยธรรมดาด้วย indexOf (ได้ -1 ถ้าไม่เจอ)

ส่วนที่ 5: suggestCategory – เดาหมวด (บรรทัด 80–111)

src/domain/categories.ts

บรรทัด 80–111

```
80 /**
81  * Suggest a category from free text (a slip's counterparty or a title). The keyword that comes
82  * first wins ("KFC Central" is food), and the longer one where two start together ("ร้านยา" is
83  * health, not the "ร้าน" of food; "mini big c" is convenience, not the "big c" of shopping).
84  * Specific shops win over the generic "person" rule, so "ร้านข้าว นายสมชาย" becomes food, not a
85  * transfer to a person.
86  */
87 export function suggestCategory(text: string | null | undefined, kind: TxKind): string {
88   if (!text) return defaultCategory(kind);
89   const t = text.toLowerCase();
90   const person = PERSON_TITLE.test(t);
91   let best: string | null = null;
92   let bestAt = Infinity;
93   let bestLen = 0;
94   for (const c of categoriesFor(kind)) {
95     if (c.key === 'transfer_out') continue;
96     for (const k of c.keywords) {
97       const m = findKeyword(t, k, person);
98       if (m && (m.at < bestAt || (m.at === bestAt && m.len > bestLen))) {
99         best = c.key;
100        bestAt = m.at;
101        bestLen = m.len;
102      }
103    }
104  }
105  if (best) return best;
106  if (kind === 'expense') {
107    const persons = BY_KEY.get('transfer_out')!;
108    if (person || persons.keywords.some((k) => t.includes(k))) return persons.key;
109  }
110  return defaultCategory(kind);
111 }
```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/categories.ts:80

บรรทัด	ความหมาย
80–86	คอมเมนต์ 3 บรรทัด: คำที่ มาก่อนชนะ ("KFC Central" = อาหาร), ถ้าเริ่มที่เดียวกัน คำยาวกว่าชนะ ("ร้านยา" = สุขภาพ ไม่ใช่ "ร้าน" ของอาหาร), และ ร้านค้าเฉพาะชนะกฎชื้อคน ("ร้านข้าว นายสมชาย" = อาหาร)
87–88	ไม่มีข้อความ → หมวดตั้งต้น
89	แปลงเป็นตัวเล็กทั้งหมด ("KFC" → "kfc") จะได้เทียบกับคำสำคัญที่เขียนตัวเล็กได้
90	ขึ้นต้นด้วยคำนำหน้าชื่อคนไหม
91–93	ตัวแปรเก็บ "ผู้ชนะตอนนี้": หมวด, ตำแหน่ง (เริ่มที่อนันต์ = ยังไม่มีใคร), ความยาว
94–95	วนทุกหมวดของประเภทนี้ ยกเว้น transfer_out (กฎชื้อคนเก็บไว้ทีหลัง ให้ร้านค้าได้ก่อน)
96–97	วนทุกคำสำคัญของหมวด ลองหา
98	เจอ และ (มาก่อนผู้ชนะเดิม หรือ เริ่มที่เดียวกันแต่ยาวกว่า) → ...

บรรทัด	ความหมาย
99-101	...เป็นผู้ชนะคนใหม่
105	มีผู้ชนะ → ตอบหมวดนั้น
106-108	ไม่มีร้านไหนตรง และเป็นรายจ่าย: ถ้าเป็นชื้อคน (หรือมีคำนำหน้าคนอยู่ตรงไหนก็ได้) → โอนให้คนอื่น
110	นอกนั้น → หมวดตั้งต้น

ผลจากการรันฟังก์ชันจริง (คัดลอกไฟล์ไปรันด้วย Node):

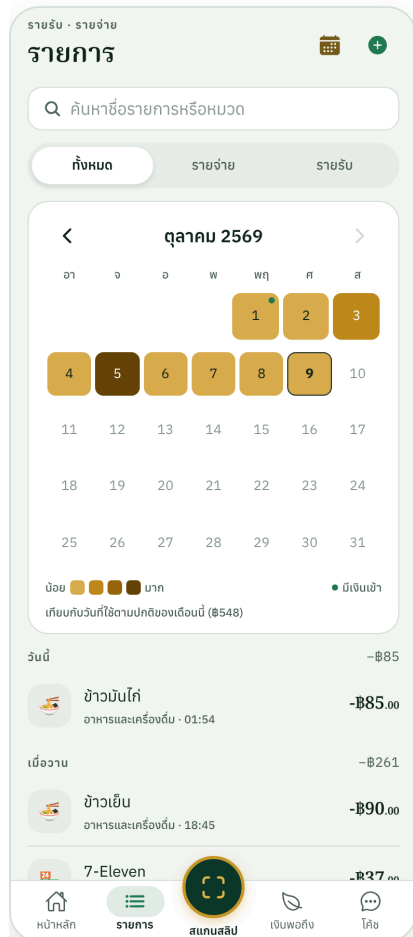
ข้อความ	ถ้าเป็นรายจ่าย	ถ้าเป็นรายรับ	เพราะ
KFC Central	food	transfer_in	"kfc" ที่ตำแหน่ง 0 มาก่อน "central" · รายรับไม่มีคำนี้
ร้านยา	health	transfer_in	"ร้านยา" กับ "ร้าน" เริ่มที่ 0 เท่ากัน ยาวกว่าชนะ
mini big c	convenience	transfer_in	"mini big c" ยาวกว่า "big c" (และ "big c" เริ่มทีหลังด้วย)
GRABPAY	transport	transfer_in	"grab" ขึ้นต้นคำ ติดกันก็ได้
PAISAN	other_expense	transfer_in	"ais" อยู่กลางคำ ไม่นับ
LAURENT	other_expense	transfer_in	"rent" อยู่กลางคำ ไม่นับ
นาย วินัย	transfer_out	transfer_in	ชื้อคน ข้ามคำสั้น "วิน" → กฎชื้อคน
MS AISHA	transfer_out	transfer_in	ชื้อคน ข้าม "ais"
ร้านข้าว นายสมชาย	food	transfer_in	ร้านค้าชนะกฎชื้อคน
ค่าขนมจากแม่	other_expense	allowance	คำ "ค่าขนม" อยู่ในหมวดรายรับเท่านั้น
บริษัท ABC	other_expense	transfer_in	ไม่ตรงคำไหนเลย

“ อธิบายให้เพื่อนฟังใน 3 ประโยค

"เราไม่ได้ใช้ AI เดาหมวด แต่ใช้รายการคำสำคัญที่เขียนเอง เพราะเร็ว ฟรี ทำงานออฟไลน์ และทดสอบได้แน่นอน คำที่เจอก่อนชนะ ถ้าเจอที่เดียวกันคำยาวชนะ และชื้อคนจะเป็น 'โอนให้คนอื่น' ก็ต่อเมื่อไม่มีชื้อร้านอยู่เลย"

อ่านโค้ด: แท็บรายการ (transactions.tsx)

ไฟล์ src/app/(tabs)/transactions.tsx 167 บรรทัด: แสดงทุกรายการแยกตามวัน ค้นหา กรอง และปฏิทิน



แท็บรายการ: หัวแท็บ, ช่องค้นหา, ปุ่มกรอง
3 ทาง, ปฏิทิน (ปิด/เปิดได้), หัวข้อวันพร้อม
ยอดสุทธิ และแถวรายการ

ส่วนที่ 1: import และชนิดตัวกรอง (บรรทัด 1–24)

src/app/(tabs)/transactions.tsx

บรรทัด 1–24

```
1 /**
2  * FR-1 Transaction Management: every transaction, grouped by day, with search
3  * and a filter. Tap a row to edit or delete it.
4  */
5 import { router } from 'expo-router';
6 import { useMemo, useState } from 'react';
7 import { RefreshControl, SectionList, TextInput, View } from 'react-native';
8 import { SafeAreaView } from 'react-native-safe-area-context';
9 import { useApp, useMoney } from '../../data/AppProvider';
10 import { getCategory } from '../../domain/categories';
11 import { bkkDayKey, previousMonth, relativeDayLabel } from '../../domain/dates';
12 import { formatBaht } from '../../domain/money';
13 import { recapMonths, spendCalendar } from '../../domain/recap';
14 import { groupByDay } from '../../domain/summary';
```

```

15 import { Button, Card, Divider, EmptyState, IconButton, Ionicons, Row, Segmented, T } from
    './../ui/components';
16 import { Buddy } from './../ui/Buddy';
17 import { Reveal } from './../ui/effects';
18 import { SpendCalendar } from './../ui/SpendCalendar';
19 import { TxRow } from './../ui/TxRow';
20 import { inputBox, inputText } from './../ui/inputs';
21 import { space, useTheme } from './../ui/theme';
22 import { TitleDecor } from './../ui/halloween';
23
24 type Filter = 'all' | 'expense' | 'income';

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/(tabs)/transactions.tsx:1

บรรทัด	ความหมาย
1-4	คอมเมนต์: FR-1 Transaction Management ทุกรายการแยกตามวัน มีค้นหาและตัวกรอง และแถวเพื่อแก้ไขหรือลบ
5	router ใช้สั่งเปิดหน้าอื่น (หน้าจดยรายการ, หน้าสลิปรอยืนยัน)
6	hook จำผลคำนวณ และความจำของหน้า
7	ชั้นส่วน React Native: ดึงลงเพื่อรีเฟรช, SectionList (รายการยาวที่แบ่งเป็นหมวด ๆ มีหัวข้อ), ช่องพิมพ์, กล้อง
8	กันรอยบาก
9	ข้อมูลกลาง useApp (รายการ, รีเฟรช, วันนี้) และ useMoney (ตัวเลขที่คำนวณแล้ว รวมถึงแบบร่าง)
10-14	สูตรล้วน: หาหมวด, วันไทย/เดือนก่อน/ป้าย "วันนี้ เมื่อวาน", แสดงเงิน, ปฏิทินรายเดือน (ของ FR-2), แยกตามวัน
15-22	ชั้นส่วนหน้าตา: ปุ่ม การ์ด เส้นค้น หน้าว่าง ปุ่มไอคอน ไอคอน แถว ปุ่มสลับ ข้อความ, ตัวละครกล้า (Buddy), เอฟเฟกต์ ค่อย ๆ โผล่, ปฏิทิน, แถวรายการ TxRow, หน้าตาช่องกรอก, ค่าการออกแบบ, ของตกแต่งหัวข้อ
24	ชนิดตัวกรองมี 3 ค่า: ทั้งหมด / รายจ่าย / รายรับ

ส่วนที่ 2: ความจำของหน้าและเดือนในปฏิทิน (บรรทัด 26-44)

src/app/(tabs)/transactions.tsx

บรรทัด 26-44

```

26 export default function Transactions() {
27   const theme = useTheme();
28   const { txs, refresh, refreshing, today } = useApp();
29   const { drafts } = useMoney();
30   const [filter, setFilter] = useState<Filter>('all');
31   const [query, setQuery] = useState('');
32   const [searching, setSearching] = useState(false);
33   const [showCalendar, setShowCalendar] = useState(true);
34   const [month, setMonth] = useState(today.slice(0, 7));
35   const [day, setDay] = useState<string | null>(null);
36   const cal = useMemo(() => spendCalendar(txs, month, today), [txs, month, today]);
37   // Months that can be shown: back to the oldest with records, never past this month.
38   const oldest = useMemo(() => recapMonths(txs).at(-1) ?? today.slice(0, 7), [txs, today]);
39   const nextMonth = (m: string) => {
40     const [y, mm] = m.split('-').map(Number);
41     return mm === 12 ? `${y + 1}-01` : `${y}-${String(mm + 1).padStart(2, '0')}`;
42   };
43   const canPrev = month > oldest;

```

44 `const canNext = month < today.slice(0, 7);`

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/app/(tabs)/transactions.tsx:26`

บรรทัด	ความหมาย
26	component ของแท็บ (export default ให้ Expo Router ใช้)
27	สืของธัม
28	จากข้อมูลกลาง: รายการทั้งหมด, ฟังก์ชันโหลดใหม่, คำสั่งโหลดอยู่ใหม่, วันนี้ (เช่น "2026-10-08")
29	แบบร่างจากสลิปที่รอยืนยัน
30	ตัวกรอง เริ่มที่ "ทั้งหมด"
31	คำค้นหา เริ่มว่าง
32	คำสั่งพิมพ์ค้นหาอยู่ใหม่ (ใช้เปลี่ยนสืชอบช่องค้นหา)
33	แสดงปฏิทินใหม่ เริ่มที่แสดง
34	เดือนที่ปฏิทินแสดง เริ่มที่เดือนนี้ (<code>slice(0, 7)</code> ตัด "2026-10-08" เหลือ "2026-10")
35	วันที่แตะในปฏิทิน (<code>null</code> = ทุกวัน)
36	ข้อมูลปฏิทินของเดือนนั้น (ยอดจ่ายแต่ละวัน) คำนวณใหม่เมื่อรายการ เดือน หรือวันนี้เปลี่ยน
37–38	เดือนเก่าสุดที่ย้อนไปได้ = เดือนเก่าสุดที่มีรายการ (<code>.at(-1)</code> = ตัวสุดท้าย) ถ้าไม่มีเลยใช้เดือนนี้
39–42	ฟังก์ชันหาเดือนถัดไป: "2026-12" → "2027-01" นอกนั้นบวกเดือน 1 และเติม 0 ข้างหน้าให้ครบ 2 หลัก ("2026-09" → "2026-10")
43	ปุ่มเดือนก่อนกดได้ถ้ายังไม่ถึงเดือนเก่าสุด
44	ปุ่มเดือนถัดไปกดได้ถ้ายังไม่ถึงเดือนนี้ (ไม่ให้อู่นาคต)

ส่วนที่ 3: กรอง ค้นหา และแยกตามวัน — หัวใจของแท็บ (บรรทัด 46–54)

`src/app/(tabs)/transactions.tsx`

บรรทัด 46–54

```
46 const sections = useMemo(() => {
47   const q = query.trim().toLowerCase();
48   const list = txs
49     .filter((t) => t.status === 'confirmed')
50     .filter((t) => filter === 'all' || t.kind === filter)
51     .filter((t) => !day || bkkDayKey(t.occurredAt) === day)
52     .filter((t) => !q || t.title.toLowerCase().includes(q) ||
getCategory(t.categoryKey).label.includes(q) || (t.note ?? '').toLowerCase().includes(q));
53   return groupByDay(list).map((g) => ({ title: g.day, net: g.netSatang, data: g.items }));
54 }, [txs, filter, query, day]);
```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/app/(tabs)/transactions.tsx:46`

บรรทัด	ความหมาย
46	sections = รายการที่จะแสดง แบ่งเป็นกลุ่มตามวัน คำนวณใหม่เมื่อ รายการ ตัวกรอง คำค้น หรือวันที่เลือกเปลี่ยน (บรรทัด 54)
47	คำค้นตัดช่องว่างหัวท้ายและทำเป็นตัวเล็ก (" KFC " → "kfc")
48–49	กรองชั้นที่ 1: เฉพาะรายการที่ยืนยันแล้ว (แบบร่างไปอยู่ในหน้าสลิปรอยืนยันแทน)
50	กรองชั้นที่ 2: ถ้าเลือก "ทั้งหมด" ผ่านหมด ไม่จำเป็นต้องประเภทตรง
51	กรองชั้นที่ 3: ถ้าแต่ละวันในปฏิทิน ต้องเป็นวันนั้น (ตามเวลาไทย)
52	กรองชั้นที่ 4 ค้นหา: ไม่มีคำค้น → ผ่าน มี → ต้องเจอใน ชื่อรายการ หรือ ชื่อหมวดไทย หรือ บันทึก อย่างใดอย่างหนึ่ง (= "หรือ")
53	ส่งที่เหลือให้ groupByDay แยกตามวัน แล้วแปลงเป็นรูปที่ SelectList ต้องการ: title (วัน), net (ยอดสุทธิของวัน), data (รายการของวัน)

≈ กรองเป็นชั้น ๆ เหมือนตะแกรงร่อนทราย

รายการทั้งหมดถูกเทผ่านตะแกรง 4 ชั้น ชั้นแรกร่อนแบบร่างออก ชั้นสองร่อนประเภทที่ไม่ต้องการออก ชั้นสามร่อนวันอื่นออก ชั้นสี่ร่อนรายการที่ไม่มีคำค้นออก ที่เหลือคือสิ่งที่ผู้ใช้เห็น ลำดับชั้นสลับกันได้ ผลเหมือนกัน เพราะทุกชั้นแค่ "ตัดออก"

ตัวช่วยจาก FR-2: groupByDay (summary.ts บรรทัด 110–126)

src/domain/summary.ts

บรรทัด 110–126

```

110 /** Group transactions by Bangkok day, newest day first, newest item first within a day. */
111 export function groupByDay(txs: Transaction[]): { day: string; items: Transaction[]; netSatang:
    number }[] {
112   const map = new Map<string, Transaction[]>();
113   for (const t of [...txs].sort((a, b) => b.occurredAt.localeCompare(a.occurredAt))) {
114     const day = bkkDayKey(t.occurredAt);
115     const list = map.get(day) ?? [];
116     list.push(t);
117     map.set(day, list);
118   }
119   return [...map.entries()].map(([day, items]) => ({
120     day,
121     items,
122     netSatang: items
123       .filter((t) => t.status === 'confirmed')
124       .reduce((s, t) => s + (t.kind === 'income' ? t.amountSatang : -t.amountSatang), 0),
125   }));
126 }
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/summary.ts:110

บรรทัด	ความหมาย
110	คอมเมนต์: แยกรายการตามวันไทย วันใหม่สุดก่อน และในวันเดียวกันรายการใหม่สุดก่อน

บรรทัด	ความหมาย
111	รับรายการ ค็นรายการของกลุ่ม แต่ละกลุ่มมี วัน, รายการ, ยอดสุทธิ
112	Map จาก "วัน" → รายการของวันนั้น
113	[...txs] คัดลอกก่อน (ไม่ไปเรียงของเดิม) แล้ว เรียงเวลาจากใหม่ไปเก่า (เทียบข้อความ ISO ได้เพราะเรียงจากปิลงไปวินาที) แล้ววนทีละรายการ
114	หาว่าเป็นวันไหนตามเวลาไทย
115–117	หยิบรายการของวันนั้น (ไม่มีใช้ []) ใส่เพิ่ม แล้วเก็บกลับ
119–121	แปลง Map เป็นรายการกลุ่ม (Map จำลำดับที่ใส่ จึงเป็นวันใหม่สุดก่อนอยู่แล้ว)
122–124	ยอดสุทธิของวัน = รับ – จ่าย นับเฉพาะยืนยันแล้ว (reduce = ทบยอดทีละรายการ เริ่มที่ 0)

ตัวอย่างที่รันจริง (วันนี้ = 8 ต.ค. 2569 เวลาไทย):

รายการ (เวลาที่เก็บ UTC)	เวลาไทย	อยู่กลุ่ม
ข้าวมันไก่ ฿65 · 2026-10-08T01:30Z	8 ต.ค. 08:30	วันนี้
ค่านม +฿500 · 2026-10-08T05:00Z	8 ต.ค. 12:00	วันนี้
ชานม ฿30 · 2026-10-07T17:10Z	8 ต.ค. 00:10	วันนี้ (ถึงจะเป็นวันที่ 7 ในเวลา UTC)
กาแฟ ฿45 · 2026-10-07T16:30Z	7 ต.ค. 23:30	เมื่อวาน

ผล: กลุ่ม "วันนี้" เรียงเป็น ค่านม, ข้าวมันไก่, ชานม ยอดสุทธิ +฿405 (500 – 65 – 30) และกลุ่ม "เมื่อวาน" มีกาแฟ ยอด – ฿45 นี่คือเหตุผลที่ต้องใช้ **วันไทย** ไม่ใช่วัน UTC (un A8)

ส่วนที่ 4: หั้วเก็บ ช่องค้นหา ตัวกรอง และการดสลิปรยืนยัน (บรรทัด 56–112)

src/app/(tabs)/transactions.tsx

บรรทัด 56–112

```

56   return (
57     <SafeAreaView edges={['top']} style={{ flex: 1, backgroundColor: theme.bg }}>
58       <View style={{ paddingHorizontal: space.lg, gap: space.md, paddingTop: space.sm,
paddingBottom: space.sm }}>
59         <Row justify="space-between">
60           <Row gap={space.sm} align="flex-end">
61             <View style={{ gap: 2 }}>
62               <T v="label">รายรับ · รายจ่าย</T>
63               <T v="h1">รายการ</T>
64             </View>
65             <TitleDecor />
66           </Row>
67           <Row gap={space.xs}>
68             <IconButton
69               icon={showCalendar ? 'calendar' : 'calendar-outline'}
70               label={showCalendar ? 'ซ่อนปฏิทิน' : 'ดูปฏิทินการใช้จ่าย'}
71               color={showCalendar ? theme.accentInk : theme.ink}
72               onPress={() => {
73                 setShowCalendar((v) => !v);

```

```

74         setDay(null);
75     }}
76     />
77     <IconButton icon="add-circle" label="จดรายการ" color={theme.primary} onPress={() =>
router.push('/transaction')} />
78     </Row>
79     </Row>
80     <View style={[inputBox(theme, { focused: searching }), { gap: space.sm }]}>
81     <Ionicons name="search" size={18} color={theme.inkSoft} />
82     <TextInput
83         nativeID="tx-search"
84         value={query}
85         onChangeText={setQuery}
86         onFocus={() => setSearching(true)}
87         onBlur={() => setSearching(false)}
88         placeholder="ค้นหาชื่อรายการหรือหมวด"
89         placeholderTextColor={theme.inkFaint}
90         accessibilityLabel="ค้นหารายการ"
91         style={[inputText(theme), { paddingVertical: 10 }]}
92     />
93     </View>
94     <Segmented<Filter>
95         options={[
96             { key: 'all', label: 'ทั้งหมด' },
97             { key: 'expense', label: 'รายจ่าย' },
98             { key: 'income', label: 'รายรับ' },
99         ]}
100         value={filter}
101         onChange={setFilter}
102     />
103     {drafts.length > 0 ? (
104         <Card onPress={() => router.push('/drafts')} style={{ paddingVertical: space.md,
borderColor: theme.accent, borderWidth: 1.5 }}>
105             <Row gap={space.sm}>
106                 <Ionicons name="receipt-outline" size={20} color={theme.accentInk} />
107                 <Text v="body" style={{ flex: 1 }}>สลิปพร้อมยืนยัน {drafts.length} รายการ</Text>
108                 <Ionicons name="chevron-forward" size={18} color={theme.inkFaint} />
109             </Row>
110         </Card>
111     ) : null}
112     </View>

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/(tabs)/transactions.tsx:56

บรรทัด	ความหมาย
57	กล่องนอกสุด กันขอบบน (ขอบล่างมีแถบเก็บอยู่แล้ว)
58	ส่วนหัวที่ไม่เลื่อน (อยู่นอก SectionList)
59–66	ซ้าย: ป้ายเล็ก "รายรับ · รายจ่าย" หัวข้อใหญ่ "รายการ" และของตกแต่ง
67–76	ขวา ปุ่มปฏิทิน: ไอคอนกบเมื่อเปิด/โปร่งเมื่อปิด ป้ายสำหรับโปรแกรมอ่านจอเปลี่ยนตาม ("ซ่อนปฏิทิน"/"ดูปฏิทินการใช้จ่าย") กดแล้วสลับ (v => !v) และล้างวันที่เลือก
77	ปุ่ม ⊕ "จดรายการ" เปิดหน้า /transaction
80	กรอบช่องค้นหา ใช้หน้าต่างเดียวกับช่องกรอกทุกช่องในแอป ขอบเปลี่ยนสีตอนพิมพ์

บรรทัด	ความหมาย
81	ไอคอนแว่นขยาย
82–92	ช่องพิมพ์: id "tx-search" (ใช้ในเทสต์), ค่า query, พิมพ์แล้ว setQuery (ผลค้นหาเปลี่ยนกันก็เพราะ sections ขึ้นกับ query), โฟกัส/หลุดโฟกัสเปลี่ยนสีขอบ, ข้อความจาก "ค้นหาชื่อรายการหรือหมวด", ป้ายอ่านจ่อ "ค้นหารายการ"
94–102	ปุ่มสลับ 3 ทาง "ทั้งหมด รายจ่าย รายรับ"
103–111	ถ้ามีแบบร่าง: การ์ดขอบสีเน้น "สลิปรอยืนยัน N รายการ >" กดแล้วไปหน้า /drafts (ของ FR-4)

ส่วนที่ 5: รายการแบบมีหัวข้อวัน และหน้าว่าง 3 แบบ (บรรทัด 114–167)

src/app/(tabs)/transactions.tsx

บรรทัด 114–167

```

114     <SectionList
115       extraData={today}
116       sections={sections}
117       keyExtractor={(t) => t.id}
118       stickySectionHeadersEnabled
119       refreshControl={<RefreshControl refreshing={refreshing} onRefresh={refresh} tintColor=
{theme.primary} />}
120       contentContainerStyle={{ paddingHorizontal: space.lg, paddingBottom: 120 }}
121       ListHeaderComponent={
122         showCalendar ? (
123           <View style={{ gap: space.sm, paddingTop: space.xs }}>
124             <SpendCalendar
125               cal={cal}
126               selected={day}
127               onSelect={setDay}
128               onPrev={canPrev ? () => { setMonth(previousMonth(month)); setDay(null); } :
undefined}
129               onNext={canNext ? () => { setMonth(nextMonth(month)); setDay(null); } :
undefined}
130             />
131             {day ? <Button label="ดูทุกวัน" kind="soft" small icon="close" onPress={() =>
setDay(null)} style={{ alignSelf: 'flex-start' }} /> : null}
132           </View>
133         ) : null
134       }
135       renderSectionHeader={({ section }) => (
136         <Row justify="space-between" style={{ backgroundColor: theme.bg, paddingTop: space.md,
paddingBottom: space.xs }}>
137           <T v="label" color={theme.inkSoft}>{relativeDayLabel(section.title)}</T>
138           <T v="small" style={{ fontVariant: ['tabular-nums'] }}>{formatBaht(section.net, {
sign: true, decimals: false })}</T>
139         </Row>
140       )}
141       renderItem={({ item, index }) => (
142         <Reveal index={Math.min(index, 6)} from={8}>
143           {index > 0 ? <Divider /> : null}
144           <TxRow tx={item} />
145         </Reveal>
146       )}
147       ListEmptyComponent={
148         day ? (
149           <EmptyState icon="calendar-outline" art={<Buddy mood="calm" size={84} />} title=
`${relativeDayLabel(day)} ไม่มีรายการ` body="แต่ละวันอื่นในปฏิทิน หรือกด ดูทุกวัน" />

```

```

150         ) : query || filter !== 'all' ? (
151             <EmptyState icon="search" art={<Buddy mood="thinking" size={84} />} title="ไม่พบรายการ"
body="ลองเปลี่ยนคำค้นหรือตัวกรองดูนะ" />
152         ) : (
153             <EmptyState
154                 icon="wallet-outline"
155                 art={<Buddy mood="sleepy" size={84} />}
156                 title="ยังไม่มีรายการที่ยืนยัน"
157                 body="สแกนสลิปจากแอปเลอริ หรือจดย่อยจ่ายเงินสดด้วยตัวเอง"
158                 action="จดย่อยรายการแรก"
159                 onAction={() => router.push('/transaction')}
160             />
161         )
162     }
163     ListFooterComponent={sections.length > 0 ? <Button label="จดย่อยรายการใหม่" kind="soft"
icon="add" onPress={() => router.push('/transaction')} style={{ marginTop: space.xl }} /> : null}
164 />
165 </SafeAreaView>
166 );
167 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/(tabs)/transactions.tsx:114

บรรทัด	ความหมาย
114	SectionList วาดเฉพาะแถวที่อยู่บนจอ (รายการ 1,000 แถวก็ไม่น้อย)
115	วาดใหม่เมื่อข้ามเที่ยงคืนด้วย ("วันนี้" กลายเป็น "เมื่อวาน")
116	ข้อมูลกลุ่มจากส่วนที่ 3
117	ใช้ id เป็นกุญแจของแต่ละแถว
118	หัวข้อวันติดขอบบนขณะเลื่อน (sticky)
119	ดึงลงเพื่อโหลดใหม่จากเซิร์ฟเวอร์
120	ระยะขอบ และเว้นล่าง 120 ไม่ให้แถวสุดท้ายโดนแถบเก็บบัง
121–134	ส่วนหัวของรายการ: ถ้าเปิดปฏิทิน แสดง SpendCalendar (แต่ละวัน = setDay, ปุ่ม < > เปลี่ยนเดือนและล่างวัน ถ้ากดไม่ได้ส่ง undefined = ไม่มีปุ่ม) และถ้าเลือกวันอยู่มีปุ่ม "ดูทุกวัน"
135–140	หัวข้อวัน: ซ้าย "วันนี้"/"เมื่อวาน"/"30 ก.ย." ขวา ยอดสุทธิแบบมีเครื่องหมาย เช่น "+฿405" หรือ "-฿45" (ตัวเลขความกว้างเท่ากัน tabular-nums จะได้ตรงกัน)
141–146	แต่ละแถว: ค่อย ๆ โฟล (หน้าจอไม่เกิน 6 ลำดับ แถวล่าง ๆ จะไม่รอนาน) มีเส้นคั่นยกเว้นแถวแรกของวัน แล้ววาด TxRow
147–149	หน้าว่างแบบที่ 1: เลือกวันที่ไม่มีรายการ → "8 ต.ค. ไม่มีรายการ" คล้ายหน้าสงบ
150–151	แบบที่ 2: มีคำค้นหรือตัวกรองแต่ไม่เจอ → "ไม่พบรายการ" คล้ายหน้าครุ่นคิด
152–160	แบบที่ 3: ไม่มีรายการเลย → "ยังไม่มีรายการที่ยืนยัน" คล้ายหน้าว่าง พร้อมปุ่ม "จดย่อยรายการแรก"
163	ท้ายรายการ (ถ้ามีรายการ) ปุ่ม "จดย่อยรายการใหม่"

★ **หลัก UX: หน้าว่างต้องบอกว่าทำอะไรต่อ**

หน้าว่างไม่ได้แสดงแค่ "ไม่มีข้อมูล" แต่บอก **สาเหตุ** (ไม่มีจริง / กรองจนไม่เหลือ / วันนั้นไม่มี) และ **ทางออก** (จดยรายการแรก / เปลี่ยนคำค้น / แต่วันอื่น) นี่คือหลัก "ช่วยผู้ใช้ไปต่อ" ในบท A11

อ่านโค้ด: แฉรายการ TxRow.tsx ทั้งไฟล์

83 บรรทัด ใช้ซ้ำ 4 ที่: แท็บรายการ หน้าหลัก หน้าสลิปรอยืนยัน และหน้าสแกน แท็บที่เดียวเปลี่ยนทุกที่

src/ui/TxRow.tsx

บรรทัด 1-35

```

1 import { router } from 'expo-router';
2 import { Pressable, View } from 'react-native';
3 import { isNewFromSlip } from '../domain/buddy';
4 import { getCategory } from '../domain/categories';
5 import { bkkTime } from '../domain/dates';
6 import { FLAG_LABEL, type ReviewFlag } from '../domain/slip';
7 import type { Transaction } from '../domain/types';
8 import { Badge, Money, Row, T } from '../components';
9 import { radius, space, useTheme } from '../theme';
10
11 /** One transaction in a list. Tapping opens it for editing (FR-1). */
12 export function TxRow({ tx, showTime = true }: { tx: Transaction; showTime?: boolean }) {
13   const theme = useTheme();
14   const cat = getCategory(tx.categoryKey);
15   const draft = tx.status === 'draft';
16   const fresh = isNewFromSlip(tx);
17   return (
18     <Pressable
19       onPress={() => router.push({ pathname: '/transaction', params: { id: tx.id } })}
20       accessibilityRole="button"
21       accessibilityLabel={` ${tx.title} ${tx.kind === 'income' ? 'รายรับ' : 'รายจ่าย'}
22       ${tx.amountSatang / 100} บาท${draft ? ' รอยืนยัน' : ''}${fresh ? ' ใหม่วันนี้' : ''}`}
23       style={({ pressed }) => ({ paddingVertical: space.md, opacity: pressed ? 0.6 : 1 })}
24     >
25       <Row gap={space.md}>
26         <View
27           style={{
28             width: 42,
29             height: 42,
30             borderRadius: radius.md,
31             backgroundColor: draft ? theme.accentSoft : theme.surfaceAlt,
32             alignItems: 'center',
33             justifyContent: 'center',
34           }}
35         >
36           <T v="h3">{cat.glyph}</T>

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/ui/TxRow.tsx:1

บรรทัด ความหมาย

- | | |
|--------------|---|
| 1-9 | import: router, กล่องกดได้, ตรวจสอบว่าเป็นสลิปใหม่วันนี้ไหม, หาหมวด, เวลาไทย, ชื่อธงตรวจ, ชนิด Transaction, ชิ้นส่วนป้าย/เงิน/แถว/ข้อความ, ค่าการออกแบบ |
| 11-12 | component รับ tx (รายการ) และ showTime (แสดงเวลาไหม ค่าเริ่มต้น true) — ข้อมูลที่ส่งเข้า component เรียกว่า props |
| 13-16 | เตรียม: สี, หมวดของรายการ, เป็นแบบร่างไหม, เป็นสลิปที่แอปจดเองวันนี้ไหม |

บรรทัด	ความหมาย
18–19	ทั้งแถวกดได้ กดแล้วเปิด /transaction พร้อม id → หน้าแก้ไข (นี่คือการเชื่อมแท็บรายการกับหน้าแก้ไข)
20–21	ป้ายสำหรับโปรแกรมอ่านจอ อ่านเป็นประโยคเดียว เช่น "ข้าวมันไก่ รายจ่าย 65 บาท" ต่อท้าย "รอยืนยัน" / "ใหม่วันนี้" ถ้าใช่
22	ตอนกดค้าง จางลงเหลือ 60% ให้รู้ว่ากดติด
24–34	กล่องไอคอน 42×42 มุมโค้ง พื้นสีเน้นอ่อนถ้าเป็นแบบร่าง ไม่จັນสีพื้นรอง
35	ไอโมจิของหมวด เช่น 🍽️

src/ui/TxRow.tsx

บรรทัด 36–83

```

36      {fresh ? (
37        <View
38          accessibilityLabel="ใหม่วันนี้"
39          style={{
40            position: 'absolute',
41            top: -3,
42            right: -3,
43            width: 12,
44            height: 12,
45            borderRadius: 6,
46            backgroundColor: theme.accent,
47            borderWidth: 2,
48            borderColor: theme.surface,
49          }}
50        />
51      ) : null}
52    </View>
53    <View style={{ flex: 1, gap: 2 }}>
54      <T v="body" numberOfLines={1}>
55        {tx.title || cat.label}
56      </T>
57      <Row gap={6}>
58        <T v="micro" numberOfLines={1}>
59          {cat.label}
60          {showTime ? ` · ${bkkTime(tx.occurredAt)} ` : ''}
61          {tx.source === 'slip' ? (fresh ? ' · จดจากสลิปวันนี้' : ' · จากสลิป') : ''}
62        </T>
63      </Row>
64      {draft ? (
65        <Row gap={4} style={{ flexWrap: 'wrap', marginTop: 2 }}>
66          {tx.reviewFlags.length === 0 ? (
67            <Badge label="พร้อมยืนยัน" tone="good" />
68          ) : (
69            tx.reviewFlags.map((f) => <Badge key={f} label={`ตรวจ${FLAG_LABEL[f as ReviewFlag]
70              ?? f}`} tone="watch" />)}
71          </Row>
72        ) : null}
73    </View>
74    <Money
75      satang={tx.kind === 'income' ? tx.amountSatang : -tx.amountSatang}
76      size="h3"
77      sign
78      color={draft ? theme.inkSoft : tx.kind === 'income' ? theme.income : theme.expense}

```

```

79     />
80   </Row>
81 </Pressable>
82 );
83 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/ui/TxRow.tsx:36`

บรรทัด	ความหมาย
36–51	ถ้าเป็นสลิปใหม่วันนี้: จุดกลมสีเน้น 12×12 ที่มุมขวาบนของไอคอน (<code>position: 'absolute'</code> ลอยทับ เลื่อนออกไป 3 จุด)
53–56	คอลัมน์กลางยึดเต็ม: บรรทัดแรกชื่อรายการ (ถ้าไม่มีชื่อใช้ชื่อหมวด) ยาวเกินตัดเป็น "..." ในบรรทัดเดียว
57–63	บรรทัดเล็ก: "อาหารและเครื่องดื่ม · 08:30" และถ้ามาจากสลิป ต่อ " · จากสลิป" (หรือ " · จดจากสลิปวันนี้")
64–72	ถ้าเป็นแบบร่าง: ป้ายเขียว "พร้อมยืนยัน" ถ้าไม่มีธง หรือป้ายเหลือง "ตรวจยอดเงิน" "ตรวจวันที่" ตามธง
74–79	จำนวนเงินด้านขวา: รายรับเป็นบวก รายจ่ายติดลบ แสดงเครื่องหมายเสมอ (+฿500.00 / -฿65.00) สี: แบบร่าง = เทา จาง (ยังไม่นับ), รายรับ = สีรายรับ, รายจ่าย = สีรายจ่าย

1 ทำไมแยก TxRow ออกเป็นไฟล์ของตัวเอง

เพราะแถวรายการหน้าตาเดียวกันถูกใช้หลายหน้า ถ้าเขียนซ้ำ 4 ที่ วันหนึ่งแก้สักรายจ่ายแล้วลืมแก้อีก 3 ที่ แอปจะดูไม่เป็นหนึ่งเดียว การแยกเป็น component คือหลัก **DRY (Don't Repeat Yourself)** = อย่าเขียนซ้ำ และเป็นเหตุผลที่ React ให้แบ่งหน้าจอเป็น component (un A6)

อ่านโค้ด: ข้อมูล 1 รายการหน้าตาเป็นยังไง (types.ts)

ก่อนตามข้อมูลไปพื้นฐานข้อมูล ต้องรู้ก่อนว่า 'รายการ 1 รายการ' มีช่องอะไรบ้าง

src/domain/types.ts

บรรทัด 1-39

```
1 /**
2  * Core data types for MindPay.
3  *
4  * Money is always stored as an integer number of satang (1 baht = 100 satang)
5  * so that adding many amounts never produces floating-point errors
6  * (0.1 + 0.2 !== 0.3 in JavaScript).
7  */
8
9 export type TxKind = 'income' | 'expense';
10
11 /** draft = not counted in any total until the user confirms it (Charter: slip results stay
12  drafts). */
13
14 export type TxStatus = 'draft' | 'confirmed';
15
16 export type TxSource = 'manual' | 'slip';
17
18 export interface Transaction {
19   id: string;
20   kind: TxKind;
21   amountSatang: number;
22   categoryKey: string;
23   title: string;
24   note: string | null;
25   /** ISO 8601 instant, e.g. "2026-09-27T01:15:00.000Z" */
26   occurredAt: string;
27   source: TxSource;
28   status: TxStatus;
29   /** Transaction reference read from the slip (QR or text). Used for duplicate detection. */
30   slipRef: string | null;
31   /** SHA-256 of the processed slip image. Second line of duplicate detection. */
32   slipImageHash: string | null;
33   /** Lowest field confidence (0..1) reported by the slip reader, null for manual entries. */
34   ocrConfidence: number | null;
35   /** Fields the slip reader was unsure about, shown to the user during review. */
36   reviewFlags: string[];
37   createdAt: string;
38 }
39
40 /** Everything the user can type or change in the form. */
41 export type TransactionInput = Omit<Transaction, 'id' | 'createdAt'>;
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/types.ts:1

บรรทัด ความหมาย

- 1-7 คอมเมนต์: เงินเก็บเป็น **จำนวนเต็มสตางค์** เสมอ (1 บาท = 100 สตางค์) เพราะทศนิยมในคอมพิวเตอร์บวกกันแล้วเพี้ยน (0.1 + 0.2 ไม่เท่ากับ 0.3 ใน JavaScript)
- 9 ประเภท: รายรับ หรือ รายจ่าย

บรรทัด	ความหมาย
11–12	สถานะ: draft = ยังไม่นับในยอดใด ๆ จนกว่าผู้ใช้จะยืนยัน (ตามข้อตกลงใน Charter ว่าผลจากสลิปต้องเป็นแบบร่างก่อน) / confirmed = นับแล้ว
14	ที่มา: manual (จดเอง FR-1 และพูด FR-3) หรือ slip (FR-4)
16	แบบพิมพ์เขียวของรายการ 1 รายการ
17	id รหัสไม่ซ้ำ (ฐานข้อมูลสร้างให้)
18–19	ประเภท และจำนวนเงินเป็นสตางค์ (บวกเสมอ ก็ศทางดูจากประเภท)
20–22	รหัสหมวด, ชื่อ, บันทึกลับ (null = ไม่มี)
23–24	เวลาที่เกิดขึ้น เป็นข้อความรูปแบบ ISO 8601 เวลา UTC เช่น "2026-09-27T01:15:00.000Z" (= 08:15 เวลาไทย)
25–26	ที่มา และสถานะ
27–30	(FR-4) เลขอ้างอิงจากสลิป และรหัส SHA-256 ของรูป ใช้กันสลิปซ้ำ 2 ชั้น
31–32	(FR-4) ความมั่นใจต่ำสุด 0–1 ของ AI ที่อ่านสลิป จดเองเป็น null
33–34	(FR-4) ช่องที่ AI ไม่แน่ใจ
35	เวลาที่สร้างแถวในฐานข้อมูล
38–39	TransactionInput = Transaction ที่ ตัด id และ createdAt ออก (omit) เพราะสองช่องนี้ฐานข้อมูลสร้างให้ ผู้ใช้ส่งเองไม่ได้ — นี่คือนิดที่หน้าจอรายการส่งให้ addTx

อ่านโค้ด: AppProvider — ศูนย์กลางที่หน้าจอเรียก

หน้าจอไม่คุยกับฐานข้อมูลเอง แต่เรียก addTx / updateTx / removeTx / confirmTx ของ AppProvider

ตัวช่วยที่ใช้ร่วมกัน (บรรทัด 118–125 และ 320–323)

src/data/AppProvider.tsx

บรรทัด 118–125

```

118  /** Each load gets a number; an older load that finishes late is ignored (it may be another
    account's). */
119  const loadGen = useRef(0);
120  /** Changes made on this phone while a load is running (null = deleted), put back over what it
    returns. */
121  const localChanges = useRef(new Map<string, Transaction | null>());
122
123  const noteLocal = (id: string, tx: Transaction | null) => {
124    localChanges.current.set(id, tx);
125  };

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/AppProvider.tsx:118

บรรทัด ความหมาย

118–119 (เกี่ยวกับการโหลด) แต่ละรอบการโหลดได้หมายเลข ถ้ารอบเก่าเสร็จช้า ข้อมูลจะถูกทิ้ง (อาจเป็นของอีกบัญชี)

120–121 localChanges สมุดจดการเปลี่ยนแปลงที่ทำในเครื่องนี้ ระหว่างที่กำลังโหลดข้อมูลจากเซิร์ฟเวอร์ (null = ถูกลบ) เมื่อโหลดเสร็จจะเอามากับผลโหลดอีกที

123–125 noteLocal จดลงสมุดนั้น: id นี่ตอนนี้เป็นรายการนี้ (หรือถูกลบ)

≈ ทำไมต้องมีสมุด localChanges

สมมติถึงจอลงเพื่อโหลดใหม่ แล้วระหว่างรอ 2 วินาที ผู้ใช้ลบรายการหนึ่ง ข้อมูลที่โหลดมา (ซึ่งถ่ายรูปไว้ ก่อน ลบ) ยังมีรายการนั้นอยู่ ถ้าเอามาแทนตรง ๆ รายการที่ลบจะ "ฟื้นคืนชีพ" บนจอ สมุดนี้ทำให้แอปจำได้ว่า "รายการนี้ฉันลบไปแล้วนะ" แล้วตัดออกจากผลโหลด

src/data/AppProvider.tsx

บรรทัด 320–323

```

320  const need = () => {
321    if (!repoRef.current) throw new Error('ยังไม่ได้เข้าสู่ระบบ');
322    return repoRef.current;
323  };

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/AppProvider.tsx:320

บรรทัด ความหมาย

320–323 need() คืน Repo ปัจจุบัน (คลาวด์หรือทดลอง) ถ้ายังไม่ได้เข้าสู่ระบบ โยนข้อผิดพลาด "ยังไม่ได้เข้าสู่ระบบ" — ทุกฟังก์ชันข้างล่างเริ่มด้วยการเรียก need()

ฟังก์ชันจัดการรายการ 4 ตัว (บรรทัด 378–427)

src/data/AppProvider.tsx

บรรทัด 378–427

```
378 // After each save the result is shown only if the same account is still open
379 // (an account switch during the save must not mix the two).
380 async saveProfile(patch) {
381   const r = need();
382   const p = await r.updateProfile(patch);
383   if (repoRef.current === r) setProfile(p);
384 },
385 async addTx(input) {
386   const r = need();
387   const tx = await r.insert(input);
388   if (repoRef.current !== r) return tx;
389   noteLocal(tx.id, tx);
390   setTxs((list) => [tx, ...list]);
391   return tx;
392 },
393 async updateTx(id, patch) {
394   const r = need();
395   const tx = await r.update(id, patch);
396   if (repoRef.current !== r) return tx;
397   noteLocal(id, tx);
398   setTxs((list) => list.map((t) => (t.id === id ? tx : t)));
399   return tx;
400 },
401 async removeTx(id) {
402   const removed = txs.find((t) => t.id === id);
403   const r = need();
404   await r.remove(id);
405   if (repoRef.current !== r) return removed;
406   noteLocal(id, null);
407   setTxs((list) => list.filter((t) => t.id !== id));
408   return removed;
409 },
410 async confirmTxs(ids) {
411   const r = need();
412   await r.confirmMany(ids);
413   if (repoRef.current !== r) return;
414   const set = new Set(ids);
415   setTxs((list) =>
416     list.map((t) => {
417       if (!set.has(t.id)) return t;
418       const next = { ...t, status: 'confirmed' as const, reviewFlags: [] };
419       noteLocal(t.id, next);
420       return next;
421     }),
422   );
423 },
424 upsertLocal(tx) {
425   noteLocal(tx.id, tx);
426   setTxs((list) => [tx, ...list.filter((t) => t.id !== tx.id)]);
427 },
```

▶ เปิดใน VS Code (เต็ม) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/AppProvider.tsx:378

บรรทัด	ความหมาย
378–379	คอมเมนต์สำคัญ: หลังบันทึกแต่ละครั้ง จะแสดงผลก็ต่อเมื่อ ยังเป็นบัญชีเดิม (ถ้าสลับบัญชีระหว่างบันทึก ต้องไม่เอาข้อมูลสองบัญชีมาปนกัน)
380–384	saveProfile (ของการตั้งค่า ไม่ใช่ FR-1)
385–386	addTx รับ input จำ Repo ณ ตอนเริ่มไว้ในตัวแปร r
387	สั่ง Repo เพิ่ม รอจนเสร็จ ได้รายการที่มี id และ createdAt แล้ว
388	ถ้าระหว่างรอ Repo เปลี่ยนไปแล้ว (ออกจากระบบ/สลับบัญชี) ค้นผลแต่ ไม่แตะหน้าจอ
389	จดลงสมุด localChanges
390	ใส่รายการใหม่ไว้หน้าสุดของรายการทั้งหมด (setTxs(list => [tx, ...list]) = รับรายการเดิม ค้นรายการใหม่ที่มี tx นำหน้า) → React วาดทุกหน้าที่ใช้ txs ใหม่
391	ค้นรายการที่เพิ่มให้ผู้เรียก
393–400	updateTx เหมือนกัน แต่ใช้ map แทนรายการที่ id ตรงด้วยตัวใหม่ ตัวอื่นคงเดิม
401–402	removeTx ก่อนลบ หาเก็บสำเนารายการไว้ก่อน (ใช้ทำ "เลิกทำ")
403–404	สั่ง Repo au
405	บัญชีเปลี่ยนระหว่างลบ → ค้นสำเนาแต่ไม่แตะหน้าจอ
406	จดว่า id นี้ถูกลบ (null)
407	กรองรายการนั้นออก (filter เก็บเฉพาะ id ที่ไม่ตรง)
408	ค้นสำเนาให้หน้าจอรายการทำปุ่มเลิกทำ
410–423	confirmTxs (ใช้ใน FR-4 "ยืนยันทั้งหมด") ยืนยันหลายรายการพร้อมกัน: สั่ง Repo, ทำ Set ของ id ไว้เช็คเร็ว, ทุกรายการที่อยู่ใน Set เปลี่ยนเป็น confirmed และล้างธง ({ ...t, status: ... } = คัดลอกแล้วเปลี่ยนบางช่อง)
424–427	upsertLocal: ใส่/แทนรายการในจอโดยไม่ผ่าน Repo (ใช้กับสลิปที่ประมวลผลบนเซิร์ฟเวอร์ ซึ่งบันทึกลงฐานข้อมูลแล้ว)

i ทำไมไม่โหลดใหม่ทั้งหมดหลังบันทึก

หลังเพิ่มรายการสำเร็จ แอป **ไม่** ขอรายการทั้งหมดจากเซิร์ฟเวอร์ใหม่ แต่ใส่รายการที่ได้กลับมาลงในรายการเดิมเลยเร็วกว่า (ไม่ต้องรอตาวนโหลดหลายร้อยแถว) และประหยัดเน็ต รายการที่ได้กลับมาเป็นของจริงจากฐานข้อมูล (มี id จริง) จึงเชื่อถือได้

อ่านโค้ด: Repo — ประตูดิยาสู่ฐานข้อมูล (repo.ts)

ไฟล์ src/data/repo.ts 389 บรรทัด อ่านเฉพาะส่วนของ FR-1: สัญญา (interface) การแปลงชื่อช่อง คลาวด์ และ โหมดทดลอง

ส่วนที่ 1: สัญญาของ Repo (บรรทัด 1–38)

src/data/repo.ts

บรรทัด 1–38

```

1  /**
2   * Data access. The screens never talk to Supabase or storage directly; they use
3   * a Repo. Two implementations:
4   *   - CloudRepo: Supabase (real accounts, data in PostgreSQL with RLS)
5   *   - DemoRepo: on-device only, starts with sample data (for trying the app
6   *                 without an account and as a fallback during a presentation)
7   */
8  import { Storage } from '../storage';
9  import { buildSampleTransactions, DEMO_OPENING_BALANCE_SATANG } from '../domain/sample';
10 import type { Profile, Transaction, TransactionInput } from '../domain/types';
11 import type { GoalInput, SavingsGoal } from '../domain/goals';
12 import { supabase } from '../supabase';
13
14 export class DuplicateSlipError extends Error {
15   constructor() {
16     super('รายการนี้มีอยู่แล้ว');
17     this.name = 'DuplicateSlipError';
18   }
19 }
20
21 export interface Repo {
22   mode: 'cloud' | 'demo';
23   getProfile(): Promise<Profile>;
24   updateProfile(patch: Partial<Omit<Profile, 'id'>>): Promise<Profile>;
25   /**
26    * Transactions of about the last 13 months, plus the net of confirmed money before them
27    * (it still counts in the balance, but is not needed as a list).
28    */
29   listTransactions(): Promise<{ txs: Transaction[]; carrySatang: number }>;
30   insert(input: TransactionInput): Promise<Transaction>;
31   update(id: string, patch: Partial<TransactionInput>): Promise<Transaction>;
32   remove(id: string): Promise<void>;
33   confirmMany(ids: string[]): Promise<void>;
34   listGoals(): Promise<SavingsGoal[]>;
35   insertGoal(input: GoalInput): Promise<SavingsGoal>;
36   updateGoal(id: string, patch: Partial<GoalInput>): Promise<SavingsGoal>;
37   removeGoal(id: string): Promise<void>;
38 }

```

▶ เปิดใน VS Code (เร็ว) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/repo.ts:1

บรรทัด	ความหมาย
1-7	คอมเมนต์: หน้าจจะไม่คุยกับ Supabase หรือที่เก็บข้อมูลโดยตรง แต่ใช้ Repo ซึ่งมี 2 แบบ: CloudRepo (บัญชีจริง ข้อมูลใน PostgreSQL มี RLS) และ DemoRepo (ในเครื่องอย่างเดียว เริ่มด้วยข้อมูลตัวอย่าง สำหรับลองใช้โดยไม่ต้องมีบัญชี และสำรองตอนนำเสนอถ้าเน็ตล่ม)
8-12	import: ที่เก็บในเครื่อง, ข้อมูลตัวอย่าง, ชนิดต่าง ๆ, ตัวเชื่อม Supabase
14-19	DuplicateSlipError ข้อผิดพลาดพิเศษ "รายการนี้มีอยู่แล้ว" ตั้งชื่อ (name) ไว้ หน้าจอรายการบรรทัด 126 ใช้ชื่อนี้เลือกข้อความ
21-22	interface Repo = สัญญา ว่า Repo ทุกแบบต้องมีอะไร: โหมด 'cloud' หรือ 'demo'
23-24	อ่าน/แก้ไขข้อมูลผู้ใช้
25-29	อ่านรายการประมาณ 13 เดือนล่าสุด พร้อมยอดสุทธิของเงินที่ต่ำกว่านั้น (ยังนับในยอดคงเหลือ แต่ไม่ต้องโหลดมาเป็นรายการ)
30-33	4 คำสั่งของ FR-1: เพิ่ม (คืนรายการที่มี id), แก้บางช่อง (Partial = ทุกช่องไม่บังคับ), ลบ, ยืนยันหลายรายการ ทุกตัวคืน Promise = "ผลจะมาถึงหลัง ต้อง await"
34-37	คำสั่งของกระปุกออม (FR-6)

≈ interface = สัญญาจ้าง

AppProvider ไม่สนว่าใครทำงานให้ จะเป็นพนักงานคลาวด์หรือพนักงานในเครื่อง ขอแค่ทำได้ตามสัญญา (มี insert, update, remove, ...) ถ้าวันหนึ่งเปลี่ยนจาก Supabase เป็นฐานข้อมูลอื่น เขียน Repo แบบใหม่ตามสัญญาเดิม หน้าจอทุกหน้าไม่ต้องแก้ไขเลย

ส่วนที่ 2: แปลงชื่อช่องระหว่างแอปกับฐานข้อมูล (บรรทัด 40-93)

src/data/repo.ts

บรรทัด 40-93

```

40 // -----
41 // Row <-> model mapping (database uses snake_case)
42 // -----
43
44 type TxRow = {
45   id: string;
46   kind: 'income' | 'expense';
47   amount_satang: number;
48   category_key: string;
49   title: string;
50   note: string | null;
51   occurred_at: string;
52   source: 'manual' | 'slip';
53   status: 'draft' | 'confirmed';
54   slip_ref: string | null;
55   slip_image_hash: string | null;
56   ocr_confidence: number | null;
57   review_flags: string[] | null;
58   created_at: string;
59 };
60

```

```

61 const fromRow = (r: TxRow): Transaction => ({
62   id: r.id,
63   kind: r.kind,
64   amountSatang: Number(r.amount_satang),
65   categoryKey: r.category_key,
66   title: r.title,
67   note: r.note,
68   occurredAt: r.occurred_at,
69   source: r.source,
70   status: r.status,
71   slipRef: r.slip_ref,
72   slipImageHash: r.slip_image_hash,
73   ocrConfidence: r.ocr_confidence === null ? null : Number(r.ocr_confidence),
74   reviewFlags: r.review_flags ?? [],
75   createdAt: r.created_at,
76 });
77
78 function toRow(p: Partial<TransactionInput>) {
79   const row: Record<string, unknown> = {};
80   if (p.kind !== undefined) row.kind = p.kind;
81   if (p.amountSatang !== undefined) row.amount_satang = p.amountSatang;
82   if (p.categoryKey !== undefined) row.category_key = p.categoryKey;
83   if (p.title !== undefined) row.title = p.title;
84   if (p.note !== undefined) row.note = p.note;
85   if (p.occurredAt !== undefined) row.occurred_at = p.occurredAt;
86   if (p.source !== undefined) row.source = p.source;
87   if (p.status !== undefined) row.status = p.status;
88   if (p.slipRef !== undefined) row.slip_ref = p.slipRef;
89   if (p.slipImageHash !== undefined) row.slip_image_hash = p.slipImageHash;
90   if (p.ocrConfidence !== undefined) row.ocr_confidence = p.ocrConfidence;
91   if (p.reviewFlags !== undefined) row.review_flags = p.reviewFlags;
92   return row;
93 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/data/repo.ts:40`

บรรทัด	ความหมาย
40–42	คอมเมนต์: ฐานข้อมูลตั้งชื่อแบบ snake_case (amount_satang) ส่วน TypeScript ใช้ camelCase (amountSatang) จึงต้องแปลง
44–59	TxRow หน้าตาของ 1 แถวในตาราง transactions ตามที่ Supabase ส่งมา (ชื่อเดียวกับคอลัมน์ใน SQL)
61–76	fromRow แยกฐานข้อมูล → รายการของแอป ทีละช่อง สังเกต: Number (. . .) ครอบคลุม amount_satang เพราะคอลัมน์ bigint อาจมาเป็นข้อความ, ความมั่นใจ null คงเป็น null, ธงที่เป็น null ใช้ [] แทน
78–79	toRow ทางกลับ: รับข้อมูลบางช่อง (Partial) เริ่มจาก object ว่าง
80–91	ใส่เฉพาะช่องที่ ส่งมาจริง (!== undefined) ตอนแก้ไขจึงส่งไปเฉพาะช่องที่เปลี่ยน ช่องอื่นในฐานข้อมูลไม่ถูกแตะ (ระวัง: null ยังถูกส่ง เพราะ null คือ "ตั้งเป็นไม่มี" ไม่ใช่ "ไม่ได้ส่ง")
92	คืน object ที่พร้อมส่ง

`src/data/repo.ts`

บรรทัด 159–160

```

159 /** Postgres unique violation = the same slip was saved before (see unique indexes in the
    migration). */

```

```
160 const isUniqueViolation = (e: { code?: string } | null) => e?.code === '23505';
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/repo.ts:159

บรรทัด ความหมาย

159–160 Postgres ตอบรหัสข้อผิดพลาด **23505** เมื่อข้อมูลชน unique index = สลিপเดิมถูกบันทึกไปแล้ว ฟังก์ชันนี้ตรวจสอบหาค่านั้น (e?.code ปลอดภัยแม้ e เป็น null)

ส่วนที่ 3: CloudRepo — คำสั่งของ FR-1 ที่คุยกับ Supabase (บรรทัด 217–237)

src/data/repo.ts

บรรทัด 217–237

```
217    async insert(input) {
218      const { data, error } = await
db.from('transactions').insert(toRow(input)).select().single();
219      if (isUniqueViolation(error)) throw new DuplicateSlipError();
220      if (error) throw error;
221      return fromRow(data as TxRow);
222    },
223    async update(id, patch) {
224      const { data, error } = await db.from('transactions').update(toRow(patch)).eq('id',
id).select().single();
225      if (isUniqueViolation(error)) throw new DuplicateSlipError();
226      if (error) throw error;
227      return fromRow(data as TxRow);
228    },
229    async remove(id) {
230      const { error } = await db.from('transactions').delete().eq('id', id);
231      if (error) throw error;
232    },
233    async confirmMany(ids) {
234      if (ids.length === 0) return;
235      const { error } = await db.from('transactions').update({ status: 'confirmed', review_flags:
[] }).in('id', ids);
236      if (error) throw error;
237    },
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/repo.ts:217

บรรทัด ความหมาย

217–218 insert: db.from('transactions') = ตาราง transactions, .insert(toRow(input)) = เพิ่มแถว, .select().single() = ขอแถวที่เพิ่มกลับมา 1 แถว (จะได้ id และ created_at ที่ฐานข้อมูลสร้าง) ผลมาเป็น { data, error } (ไม่โยนข้อผิดพลาดเอง)

219 ชน unique → โยน DuplicateSlipError

220 ข้อผิดพลาดอื่น (เน็ต, check ไม่ผ่าน, RLS ไม่ให้) → โยนต่อ

221 สำเร็จ → แปลงเป็นรายการของแถว

223–228 update: .update(...) เฉพาะช่องที่ส่ง .eq('id', id) = เฉพาะแถวที่ id ตรง แล้วขอแถวใหม่กลับ

229–232 remove: .delete().eq('id', id) ลบแถวที่ id ตรง

บรรทัด ความหมาย

233–237 **confirmMany:** ไม่มี id ไม่ทำอะไร ไม่ขึ้นอัปเดตทุกแถวที่ id อยู่ในรายการ (`.in('id', ids)`) ในคำสั่งเดียว ให้ `status = confirmed` และล้างธง

! ไม่มี user_id ในคำสั่งเลย แล้วรู้ได้ยังไงว่าของใคร?

สังเกตว่า insert ไม่ได้ส่ง user_id และ update/delete ไม่ได้กรอง user_id เลย เพราะ ฐานข้อมูลทำให้เอง: คอลัมน์ user_id มีค่าตั้งต้น `auth.uid()` (ผู้ใช้ที่ล็อกอินอยู่) และ RLS ทำให้คำสั่งเห็นเฉพาะแถวของตัวเอง ถ้าใครส่ง id ของรายการคนอื่นมาลบ ฐานข้อมูลจะ "มองไม่เห็น" แถวนั้น ลบได้ 0 แถว (ดูหัวข้อตารางข้างล่าง)

ส่วนที่ 4: DemoRepo – โหมดทดลองในเครื่อง (บรรทัด 283 และ 313–362)

src/data/repo.ts

บรรทัด 283–283

```
283 const newId = () => `local-${Date.now().toString(36)}-${Math.random().toString(36).slice(2, 8)}`;
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/data/repo.ts:283

บรรทัด ความหมาย

283 สร้าง id ในเครื่อง: "local-" + เวลาปัจจุบันเลขฐาน 36 + สุ่ม 6 ตัว เช่น "local-mg0x1k2p-4f9a1c" (ไม่มีฐานข้อมูล สร้างให้ จึงต้องสร้างเอง)

src/data/repo.ts

บรรทัด 313–362

```
313 const dupe = (s: DemoState, input: Partial<TransactionInput>, exceptId?: string) =>
314   s.txs.some(
315     (t) =>
316       t.id !== exceptId &&
317       ((input.slipRef && t.slipRef === input.slipRef) || (input.slipImageHash &&
318         t.slipImageHash === input.slipImageHash)),
319   );
320 return {
321   mode: 'demo',
322   async getProfile() {
323     return (await load()).profile;
324   },
325   async updateProfile(patch) {
326     const s = await load();
327     s.profile = { ...s.profile, ...patch };
328     await save();
329     return s.profile;
330   },
331   async listTransactions() {
332     // Everything is on the phone: nothing is carried over.
333     return { txs: [...(await load()).txs], carrySatang: 0 };
334   },
335   async insert(input) {
336     const s = await load();
337     if (dupe(s, input)) throw new DuplicateSlipError();
338     const tx: Transaction = { ...input, id: newId(), createdAt: new Date().toISOString() };
```

```

339     s.txs.push(tx);
340     await save();
341     return tx;
342 },
343 async update(id, patch) {
344     const s = await load();
345     if (dupe(s, patch, id)) throw new DuplicateSlipError();
346     const i = s.txs.findIndex((t) => t.id === id);
347     if (i < 0) throw new Error('ไม่พบรายการ');
348     s.txs[i] = { ...s.txs[i], ...patch };
349     await save();
350     return s.txs[i];
351 },
352 async remove(id) {
353     const s = await load();
354     s.txs = s.txs.filter((t) => t.id !== id);
355     await save();
356 },
357 async confirmMany(ids) {
358     const s = await load();
359     const set = new Set(ids);
360     s.txs = s.txs.map((t) => (set.has(t.id) ? { ...t, status: 'confirmed', reviewFlags: [] } :
t));
361     await save();
362 },

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/data/repo.ts:313`

บรรทัด	ความหมาย
313–318	dupe เลียนแบบ unique index ของฐานข้อมูล: มีรายการอื่น (ไม่นับตัวเอง exceptId) ที่เลขอ้างอิงสลิปตรงกัน หรือรหัสรูปตรงกันใหม่ (ตรวจเฉพาะเมื่อ input มีค่านั้น รายการตนเองไม่มี จึงไม่ซ้ำ)
320–321	คืน object ที่ทำตามสัญญา Repo โหมด 'demo'
331–334	อ่านรายการ: คัดลอกทั้งหมดในเครื่อง ไม่มียอดยกมา (ทุกอย่างอยู่ในเครื่องแล้ว)
335–342	insert: โหลด, ซ้ำ → โยน DuplicateSlipError, สร้างรายการใหม่จาก input + id ใหม่ + เวลาสร้างตอนนี้, ใส่ในรายการ, บันทึกลงเครื่อง, คืนรายการ
343–351	update: ตรวจซ้ำ (ไม่นับตัวเอง), หาตำแหน่ง, ไม่เจอ → "ไม่พบรายการ", เจอ → รวมของเดิมกับช่องที่เปลี่ยน ({ ...เดิม, ...patch } ตัวหลังทับตัวหน้า), บันทึก
352–356	remove: กรองออก แล้วบันทึก
357–362	confirmMany: ทุกรายการที่ id อยู่ในชุด เปลี่ยนเป็น confirmed ล้างธง แล้วบันทึก

★ ทำไมโหมดทดลองต้องทำตัวเหมือนฐานข้อมูลจริง

เพราะเทสต์ E2E และการนำเสนอใช้โหมดนี้ ถ้าโหมดทดลองยอมให้สลิปซ้ำแต่ของจริงไม่ยอม เราจะทดสอบผ่านแต่ใช้จริงพัง ทีมจึงเลียนแบบกฎสำคัญ (กันซ้ำ, id ใหม่) ให้เหมือนกัน

อ่านโค้ด: ตาราง transactions ในฐานข้อมูล (SQL)

ไฟล์ supabase/migrations/20260927000000_init.sql บรรทัด 24-53 และกฎความปลอดภัย

supabase/migrations/20260927000000_init.sql

บรรทัด 24-53

```

24 -- -----
25 -- transactions (FR-1). Money is stored in satang (integer) to avoid rounding errors.
26 -- -----
27 create table if not exists public.transactions (
28   id uuid primary key default gen_random_uuid(),
29   user_id uuid not null default auth.uid() references auth.users (id) on delete cascade,
30   kind text not null check (kind in ('income', 'expense')),
31   amount_satang bigint not null check (amount_satang > 0),
32   category_key text not null,
33   title text not null default '' check (char_length(title) <= 120),
34   note text check (note is null or char_length(note) <= 500),
35   occurred_at timestamptz not null,
36   source text not null default 'manual' check (source in ('manual', 'slip')),
37   status text not null default 'confirmed' check (status in ('draft', 'confirmed')),
38   slip_ref text, -- FR-4 duplicate key #1 (QR / printed reference)
39   slip_image_hash text, -- FR-4 duplicate key #2 (SHA-256 of the image)
40   ocr_confidence numeric(3, 2) check (ocr_confidence is null or (ocr_confidence >= 0 and
    ocr_confidence <= 1)),
41   review_flags text[] not null default '{}',
42   created_at timestamptz not null default now(),
43   updated_at timestamptz not null default now()
44 );
45
46 create index if not exists transactions_user_time_idx on public.transactions (user_id,
    occurred_at desc);
47
48 -- The database itself refuses a second copy of the same slip, even if two
49 -- phones scan at the same moment (FR-4 "ตรวจรายการซ้ำ").
50 create unique index if not exists transactions_user_slip_ref_idx
51 on public.transactions (user_id, slip_ref) where slip_ref is not null;
52 create unique index if not exists transactions_user_slip_hash_idx
53 on public.transactions (user_id, slip_image_hash) where slip_image_hash is not null;

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → supabase/migrations/20260927000000_init.sql:24

บรรทัด	ความหมาย
24-26	คอมเมนต์ (SQL ใช้ --): ตารางของ FR-1 เงินเก็บเป็นสตาจจำนวนเต็มกันปัดเศษผิด
27	สร้างตาราง ถ้ายังไม่มี
28	id ชนิด uuid (รหัสสุ่ม 36 ตัวอักษร) เป็นคีย์หลัก ฐานข้อมูลสุ่มให้เอง
29	user_id ห้ามว่าง ค่าตั้งต้น = auth.uid() (คนที่ล็อกอินส่งคำขอ) อ้างถึงตารางผู้ใช้ ถ้าลบบัญชี รายการทั้งหมดถูกลบตาม (on delete cascade)
30	ประเภท ต้องเป็น 'income' หรือ 'expense' เท่านั้น (check)

บรรทัด	ความหมาย
31	จำนวนสตริง bigint (จำนวนเต็มขนาดใหญ่) ต้องมากกว่า 0
32	รหัสหมวด ห้ามว่าง
33	ชื่อ คำตั้งต้นว่าง ยาวไม่เกิน 120
34	บันทึก null ได้ ถ้ามียาวไม่เกิน 500
35	เวลาที่เกิดขึ้น timestamptz (เวลาพร้อมเขตเวลา) ห้ามว่าง
36–37	ที่มา (ตั้งต้น manual) และสถานะ (ตั้งต้น confirmed) จำกัดค่าที่อนุญาต
38–41	ช่องของ FR-4: เลขอ้างอิงสลิป, รหัสรูป, ความมั่นใจ (ทศนิยม 2 ตำแหน่ง ระหว่าง 0–1), ธง (รายการข้อความ ตั้งต้นว่าง)
42–43	เวลาสร้างและเวลาแก้ล่าสุด ตั้งต้นเป็นตอนนี้
46	index (สารบัญ) ตามผู้ใช้และเวลาจากใหม่ไปเก่า ทำให้ "ดึงรายการของเงินเรียงตามเวลา" เร็วแม้มีหลายแสนแถว
48–53	unique index 2 ตัว: ผู้ใช้คนเดียวกันมีเลขอ้างอิงสลิปซ้ำไม่ได้ และรหัสรูปซ้ำไม่ได้ (เฉพาะแถวที่มีค่า) — ฐานข้อมูลเองปฏิเสธสลิปซ้ำ แม้สองเครื่องสแกนพร้อมกัน

supabase/migrations/20261001000000_atomic_ai_quota.sql

บรรทัด 34–38

```

34 -- Amounts have the same upper bound as savings goals ($1,000,000,000), so a misread
35 -- amount can never overflow what the app handles.
36 alter table public.transactions
37   drop constraint if exists transactions_amount_max,
38   add constraint transactions_amount_max check (amount_satang <= 100000000000);

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#) VS Code ในเครื่อง: Ctrl+P → supabase/migrations/20261001000000_atomic_ai_quota.sql:34

ภายหลังทีมเพิ่ม **เพดาน** จำนวนเงิน: ไม่เกิน 100,000,000,000 สตางค์ (\$1,000 ล้าน) เท่ากับ MAX_SATANG ใน [src/domain/money.ts:6](#) ทั้งแอปและฐานข้อมูลใช้ตัวเลขเดียวกัน ตัวเลขที่ AI อ่านผิดจนใหญ่เกินจะไม่ทำให้ระบบพัง

กฎความปลอดภัย RLS ของตารางนี้

supabase/migrations/20260927000100_harden.sql

บรรทัด 23–25

```

23 drop policy if exists "own transactions" on public.transactions;
24 create policy "own transactions" on public.transactions
25   for all using ((select auth.uid()) = user_id) with check ((select auth.uid()) = user_id);

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#) VS Code ในเครื่อง: Ctrl+P → supabase/migrations/20260927000100_harden.sql:23

บรรทัด	ความหมาย
23	ลบกฎชื่อเดิมก่อน (ถ้ามี) เพื่อสร้างใหม่
24	สร้างกฎ "own transactions" บนตาราง transactions

บรรทัด ความหมาย

- 25** for all = ทุกคำสั่ง (อ่าน เพิ่ม แก้ ลบ) using = แถวที่ **มองเห็น/แตะได้** ต้องมี user_id = ผู้ใช้ที่ล็อกอิน, with check = แถวที่ **เขียนลงไป** ต้องเป็นของตัวเองด้วย (ใส่ user_id คนอื่นไม่ได้) คสอบ (select auth.uid()) ให้คำนวณครั้งเดียวต่อคำสั่ง เร็วกว่าคำนวณทุกแถว

supabase/migrations/20260927000000_init.sql

บรรทัด 92-104

```
92 create or replace function public.touch_updated_at() returns trigger
93 language plpgsql as $$
94 begin
95     new.updated_at := now();
96     return new;
97 end $$;
98
99 drop trigger if exists profiles_touch on public.profiles;
100 create trigger profiles_touch before update on public.profiles
101     for each row execute function public.touch_updated_at();
102 drop trigger if exists transactions_touch on public.transactions;
103 create trigger transactions_touch before update on public.transactions
104     for each row execute function public.touch_updated_at();
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → supabase/migrations/20260927000000_init.sql:92

บรรทัด ความหมาย

- 92-97** ฟังก์ชันในฐานข้อมูล (ภาษา plpgsql): ตั้ง updated_at ของแถวใหม่เป็นตอนนี้ แล้วคืนแถว
- 99-101** trigger ของตาราง profiles: ก่อนอัปเดตทุกแถว เรียกฟังก์ชันนั้น
- 102-104** trigger เดียวกันของ transactions: ทุกครั้งที่แก้ไขรายการ updated_at เปลี่ยนเอง แอปไม่ต้องส่ง

“ ถ้าอาจารย์ถาม: ถ้าแอปมีบั๊กส่งเงินติดลบไป จะเกิดอะไร

"ฐานข้อมูลจะปฏิเสธ เพราะมี check constraint amount_satang > 0 คำสั่ง insert จะได้ error กลับมา Repo โยนข้อผิดพลาด หน้าจอรายการจับได้ใน catch แล้วแสดง 'บันทึกไม่สำเร็จ ลองอีกครั้ง' ข้อมูลผิดจึงเข้าฐานข้อมูลไม่ได้ ถึงแอปจะพลาดก็ตาม"

บทที่ 19

อ่านโค้ด: ชิ้นส่วนหน้าต่างที่ FR-1 ใช้ (inputs.tsx, feedback.tsx)

ช่องกรอกเงิน ช่องกรอกทั่วไป toast และหน้าต่างยืนยันลบ

ช่องกรอกทั่วไป Field (inputs.tsx บรรทัด 6–96)

src/ui/inputs.tsx

บรรทัด 6–28

```
6 /**
7  * The one look of every text box in the app (design system): surface fill, 1.5pt border in
8  * `line`, `primary` while typing, `critical` when something is wrong; radius md.
9  */
10 export function inputBox(theme: Theme, state: { focused?: boolean; error?: boolean } = {}):
    TextStyle {
11   return {
12     flexDirection: 'row',
13     alignItems: 'center',
14     borderWidth: 1.5,
15     borderColor: state.error ? theme.critical : state.focused ? theme.primary : theme.line,
16     borderRadius: radius.md,
17     backgroundColor: theme.surface,
18     paddingHorizontal: 14,
19   };
20 }
21
22 /**
23  * The text inside an input box: body size, and 16 on the web, where iPhone Safari zooms the
24  * page into any field with smaller text.
25  */
26 export function inputText(theme: Theme): TextStyle {
27   return { flex: 1, paddingVertical: 12, fontFamily: fonts.sans, fontSize: Platform.OS === 'web'
28     ? 16 : 15, color: theme.ink };
29 }
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/ui/inputs.tsx:6

บรรทัด	ความหมาย
6–9	คอมเมนต์: หน้าตาเดียวของทุกช่องกรอกในแอป (Design System): พื้นสีการ์ด ขอบ 1.5 สีเส้นปกติ, สีหลักตอนพิมพ์, สีแดงเมื่อผิด มุมโค้งขนาดกลาง
10–20	inputBox คืนสไตล์กรอบ สีขอบเลือกตามลำดับ: ผิด → แดง, กำลังพิมพ์ → สีหลัก, ปกติ → สีเส้น
22–28	inputText สไตล์ตัวอักษรในช่อง ขนาด 15 แต่บนเว็บ 16 เพราะ Safari บน iPhone จะซูมทั้งหน้าเมื่อแตะช่องที่ตัวอักษรเล็กกว่า 16 (รายละเอียดเล็ก ๆ ที่ทำให้ใช้งานบนเว็บไม่หลุดหจิด)

src/ui/inputs.tsx

บรรทัด 30–96

```
30 export function Field({
31   id,
```

```

32   label,
33   value,
34   onChangeText,
35   placeholder,
36   keyboardType,
37   multiline,
38   hint,
39   error,
40   maxLength,
41   right,
42   inputProps,
43 }: {
44   id: string;
45   label: string;
46   value: string;
47   onChangeText: (s: string) => void;
48   placeholder?: string;
49   keyboardType?: KeyboardTypeOptions;
50   multiline?: boolean;
51   hint?: string;
52   error?: string | null;
53   maxLength?: number;
54   right?: ReactNode;
55   /** Extra TextInput settings: autoComplete, secureTextEntry, onSubmitEditing, ... */
56   inputProps?: Omit<TextInputProps, 'value' | 'onChangeText' | 'style'>;
57 }) {
58   const theme = useTheme();
59   const [focused, setFocused] = useState(false);
60   return (
61     <View style={{ gap: 6 }}>
62       <T v="small" color={theme.ink} style={{ fontFamily: fonts.sansSemi }}>
63         {label}
64       </T>
65       <View style={inputBox(theme, { focused, error: !!error })}>
66         <TextInput
67           {...inputProps}
68           onFocus={e => {
69             setFocused(true);
70             inputProps?.onFocus?.(e);
71           }}
72           onBlur={e => {
73             setFocused(false);
74             inputProps?.onBlur?.(e);
75           }}
76           nativeID={id}
77           accessibilityLabel={label}
78           value={value}
79           onChangeText={onChangeText}
80           placeholder={placeholder}
81           placeholderTextColor={theme.inkFaint}
82           keyboardType={keyboardType}
83           multiline={multiline}
84           maxLength={maxLength}
85           style={[inputText(theme), { minHeight: multiline ? 72 : undefined, textAlignVertical:
multiline ? 'top' : 'center' }]}
86         />
87         {right}
88       </View>
89       {error ? (
90         <T v="small" color={theme.critical}>{error}</T>
91       ) : hint ? (

```

```

92     <T v="micro">{hint}</T>
93   ) : null}
94 </View>
95 );
96 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/ui/inputs.tsx:30

บรรทัด	ความหมาย
30–57	รายการ props ที่ Field รับ: id, ป้าย, ค่า, ฟังก์ชันเมื่อพิมพ์ และตัวเลือกไม่บังคับ (ข้อความจาก, ชนิดคีย์บอร์ด, หลายบรรทัด, คำแนะนำ, ข้อผิดพลาด, ความยาวสูงสุด, ของทางขวา, ค่าเพิ่มเติมของ TextInput) บรรทัด 30–42 คือชื่อ บรรทัด 43–57 คือชนิดของแต่ละตัว
58–59	สี และ state "กำลังพิมพ์ใหม่" ของช่องนี้เอง
61–64	ป้ายเหนือช่อง (ตัวหนาเล็ก)
65	กรอบ สีตามสถานะ (! !error แปลงข้อผิดพลาดเป็น true/false)
66–75	ช่องพิมพ์จริง ใส่ค่าเพิ่มเติมก่อน แล้วดับไฟก์ส/หลอดไฟก์ส: เปลี่ยน state ของตัวเอง และเรียกของผู้ใช้ต่อ (? . () = เรียกถ้ามี)
76–77	id (ให้เกสต์หาเจอ) และป้ายสำหรับโปรแกรมอ่านจอ = ป้ายที่เห็น
78–84	ส่งค่าต่าง ๆ ต่อให้ TextInput
85	สไตร์ ถ้าหลายบรรทัดสูงอย่างน้อย 72 และข้อความชิดบน
87	ของทางขวา (เช่น ปุ่มตาของช่องรหัสผ่าน)
89–93	ใต้ช่อง: มีข้อผิดพลาด → ข้อความแดง ไม่มีแต่มีคำแนะนำ → ข้อความเล็ก ไม่มีทั้งคู่ → ไม่แสดง

ช่องกรอกเงิน AmountField (inputs.tsx บรรทัด 156–209)

src/ui/inputs.tsx

บรรทัด 156–209

```

156 /** Large amount entry set in the serif face, like the balance on the home screen. */
157 export function AmountField({
158   id,
159   value,
160   onChangeText,
161   color,
162   error,
163   flagged,
164 }: {
165   id: string;
166   value: string;
167   onChangeText: (s: string) => void;
168   color?: string;
169   error?: string | null;
170   flagged?: boolean;
171 }) {
172   const theme = useTheme();
173   return (
174     <View style={{ alignItems: 'center', gap: space.xs, paddingVertical: space.md }}>
175       <View

```

```

176     style={{
177       flexDirection: 'row',
178       alignItems: 'center',
179       borderBottomWidth: 2,
180       borderBottomColor: error ? theme.critical : flagged ? theme.watch : theme.line,
181       paddingHorizontal: space.md,
182     }}
183   >
184     <T v="h1" color={color ?? theme.ink}>฿</T>
185     <TextInput
186       nativeID={id}
187       accessibilityLabel="จำนวนเงิน (บาท)"
188       value={value}
189       onChangeText={(s) => onChangeText(s.replace(/^[^0-9.]/g, ''))}
190       placeholder="0.00"
191       placeholderTextColor={theme.inkFaint}
192       keyboardType="decimal-pad"
193       style={{
194         fontFamily: fonts.serif,
195         fontSize: 40,
196         lineHeight: 54,
197         width: 220,
198         maxWidth: '100%',
199         textAlign: 'center',
200         color: color ?? theme.ink,
201         paddingVertical: 4,
202         fontVariant: ['tabular-nums'],
203       }}
204     />
205   </View>
206   {error ? <T v="small" color={theme.critical}>{error}</T> : null}
207 </View>
208 );
209 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/ui/inputs.tsx:156

บรรทัด	ความหมาย
156	คอมเมนต์: ช่องเงินตัวใหญ่ ฟอนต์ serif เหมือนยอดเงินบนหน้าหลัก
157–171	props: id, ค่า, ฟังก์ชันเมื่อพิมพ์, สี, ข้อผิดพลาด, ถูกตั้งธงใหม่
174	จัดกลางทั้งหมด
175–183	มีแค่ เส้นใต้ (ไม่ใช่กรอบ) สี: ผิด → แดง, AI ขอตระจ → เหลือง, ปกติ → สีเส้น
184	สัญลักษณ์ ฿ ตัวใหญ่ หน้าตัวเลข
186–187	id และป้ายอ่านจ่อ "จำนวนเงิน (บาท)"
189	กรองตัวอักษร : ลบทุกตัวที่ไม่ใช่ 0–9 จุด หรือจุลภาค ทั้งก่อนส่งต่อ (พิมพ์ "฿85บาท" ได้ "85") [^...] = "ไม่ใช่ตัวเหล่านี้", g = ทุกตัว
190–191	ข้อความจาง "0.00"
192	มือถือเปิดคีย์บอร์ดตัวเลขมีจุดทศนิยม
193–203	ตัวใหญ่ 40 ฟอนต์ serif กว้าง 220 จัดกลาง ตัวเลขความกว้างเท่ากัน

! ข้อสังเกตจากบรรทัด 189

parseBahtToSatang ใน money.ts รองรับเลขไทย (๘๕) แต่ช่องนี้รับเลขไทยทั้งก่อนจะถึง parse (เลขไทยไม่อยู่ใน 0–9) ผลคือพิมพ์ "๘๕" ในช่องเงินแล้วช่องว่างเปล่า ไม่ใช่ปัญหาใหญ่ (คีย์บอร์ดตัวเลขของมือถือพิมพ์เลขอารบิกอยู่แล้ว) แต่ถ้าจะรองรับ แก้ regex เป็น `/[^\d-๐-๙. , ]/g`

toast และ ConfirmSheet (feedback.tsx)

src/ui/feedback.tsx

บรรทัด 12–52

```

12 interface ToastOptions {
13   message: string;
14   tone?: 'info' | 'error';
15   action?: { label: string; onPress: () => void };
16 }
17
18 const ToastContext = createContext<(t: ToastOptions) => void>(() => {});
19
20 export function useToast() {
21   return useContext(ToastContext);
22 }
23
24 export function ToastProvider({ children }: { children: ReactNode }) {
25   const theme = useTheme();
26   // The message sits on the theme's ink; its button uses the theme's accent (light mode) or the
27   // light-mode main colour (dark mode), falling back to plain text colour if that would be
28   // faint.
29   const light = buildTheme(theme.colorTheme, false);
30   const actionColor = readableOn(theme.ink, theme.dark ? light.primary : theme.accent, theme.bg);
31   const insets = useSafeAreaInsets();
32   const [toast, setToast] = useState<ToastOptions | null>(null);
33   const [anim] = useState(() => new Animated.Value(0));
34   const timer = useRef<ReturnType<typeof setTimeout> | null>(null);
35
36   const show = useCallback(
37     (t: ToastOptions) => {
38       if (timer.current) clearTimeout(timer.current);
39       setToast(t);
40       // iOS VoiceOver ignores live regions: say the message out loud (Android and the web use
41       // the live region).
42       if (Platform.OS === 'ios') AccessibilityInfo.announceForAccessibility(t.message);
43       if (t.tone === 'error')
44         Haptics.notificationAsync(Haptics.NotificationFeedbackType.Error).catch(() => {});
45       Animated.spring(anim, { toValue: 1, useNativeDriver: true }).start();
46       timer.current = setTimeout(() => {
47         Animated.timing(anim, { toValue: 0, duration: 200, useNativeDriver: true }).start(() =>
48           setToast(null));
49       }, t.action ? 5000 : 2800);
50     },
51     [anim],
52   );

```

```

49
50 useEffect(() => () => {
51   if (timer.current) clearTimeout(timer.current);
52 }, []);

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/ui/feedback.tsx:12`

บรรทัด	ความหมาย
12–16	ตัวเลือกของ toast: ข้อความ, โทน (ปกติ/ผิด), ปุ่มการกระทำ (ป้าย + สิ่งที่ทำ) ไม่บังคับ
18	Context = ท่อส่งข้อมูลข้ามหน้า ค่าเริ่มต้นเป็นฟังก์ชันว่าง (ถ้าใช้ออก Provider จะไม่พัง แต่ไม่แสดง)
20–22	<code>useToast()</code> ที่หน้าจาดรายการเรียก ได้ฟังก์ชัน <code>show</code> กลับมา
24	<code>ToastProvider</code> ครอบทั้งแอป (ใน <code>_layout.tsx</code>) จึงแสดง toast ได้จากทุกหน้า แม้หน้าที่สั่งจะปิดไปแล้ว (หน้าจาดรายการ ปิดก่อน toast "ลบรายการแล้ว" ยังอยู่)
26–29	เลือกสีปุ่มการกระทำให้อ่านออกบนพื้น toast ทุกธีม
30–33	ระยะขอบล่าง, toast ที่แสดงอยู่, ค่าภาพเคลื่อนไหว, ตัวจับเวลา
35–37	show : ยกเลิกตัวจับเวลาเก่า (toast ใหม่แทนอันเก่า)
38	ตั้ง toast ใหม่
39–40	บน iPhone สั่งให้ VoiceOver อ่านข้อความออกเสียง (การเข้าถึง)
41	ถ้าเป็นข้อผิด สั้นเครื่องแบบเตือน
42	แจ้งเตือนแบบสปริง
43–45	ตั้งเวลาซ่อน: มีปุ่ม (เช่น เลิกทำ) 5 วินาที ไม่มี 2.8 วินาที ซ่อนด้วยการจางลง 0.2 วินาที แล้วล้าง toast
50–52	ถ้า Provider ถูกถอด ยกเลิกตัวจับเวลา (กันเรียก <code>setState</code> หลังถอด)

`src/ui/feedback.tsx`

บรรทัด 54–101

```

54 return (
55   <ToastContext.Provider value={show}>
56     {children}
57     {toast ? (
58       <Animated.View
59         accessibilityLiveRegion={toast.tone === 'error' ? 'assertive' : 'polite'}
60         style={{
61           position: 'absolute',
62           left: space.lg,
63           right: space.lg,
64           bottom: insets.bottom + 96,
65           opacity: anim,
66           transform: [{ translateY: anim.interpolate({ inputRange: [0, 1], outputRange: [20, 0]
67         }) }],
68       >
69         <View
70           style={{
71             // The colour theme's own ink, so the message matches every theme (not only the
72             green one).

```

```

72         backgroundColor: toast.tone === 'error' ? theme.critical : theme.ink,
73         borderRadius: radius.md,
74         paddingVertical: 12,
75         paddingHorizontal: 16,
76         flexDirection: 'row',
77         alignItems: 'center',
78         gap: 12,
79     }}
80     >
81     <Text style={{ flex: 1, fontFamily: fonts.sansMedium, fontSize: 14, color: toast.tone
=== 'error' ? readableOn(theme.critical, '#FFFFFF', theme.bg) : theme.bg }}>
82         {toast.message}
83     </Text>
84     {toast.action ? (
85         <Pressable
86             onPress={() => {
87                 toast.action?.onPress();
88                 setToast(null);
89             }}
90             hitSlop={10}
91             accessibilityRole="button"
92         >
93             <Text style={{ fontFamily: fonts.sansBold, fontSize: 14, color: actionColor }}>
{toast.action.label}</Text>
94         </Pressable>
95     ) : null}
96 </View>
97 </Animated.View>
98 ) : null}
99 </ToastContext.Provider>
100 );
101 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → `src/ui/feedback.tsx:54`

บรรทัด	ความหมาย
55–56	ส่ง show ลงท่อ และวาดแอปทั้งหมด (children) ตามปกติ
57–59	ถ้ามี toast: กล้องเคลื่อนไหว บอกโปรแกรมอ่านจอกว่าเป็น "ข้อความสด" (ผิด = แทรกทันที, ปกติ = รอจังหวะ)
60–67	ลอยเหนือแถบเก็บ 96 จุด จางเข้าและเลื่อนขึ้น 20 จุดตามคำภาพเคลื่อนไหว
69–80	กล้องพื้นสีหมึกของริบ (ผิด = แดง) มุมโค้ง เรียงแนวนอน
81–83	ข้อความ
84–95	ถ้ามีปุ่ม: กดแล้วทำการกระทำ (เช่น เพิ่มรายการกลับ) และซ่อน toast ทันที พื้นที่กดขยายรอบ 10 จุด (hitSlop) ให้กดง่าย

`src/ui/feedback.tsx`

บรรทัด 103–144

```

103 export function ConfirmSheet({
104     visible,
105     title,
106     body,
107     confirmLabel,
108     destructive,
109     onConfirm,

```

```

110   onCancel,
111 }: {
112   visible: boolean;
113   title: string;
114   body: string;
115   confirmLabel: string;
116   destructive?: boolean;
117   onConfirm: () => void;
118   onCancel: () => void;
119 }) {
120   const theme = useTheme();
121   const insets = useSafeAreaInsets();
122   return (
123     <Modal visible={visible} transparent animationType="fade" onRequestClose={onCancel}>
124       <Pressable style={{ flex: 1, backgroundColor: theme.overlay }} onPress={onCancel}
accessibilityRole="button" accessibilityLabel="ปิด" />
125       <View
126         style={{
127           backgroundColor: theme.surface,
128           borderTopLeftRadius: radius.xl,
129           borderTopRightRadius: radius.xl,
130           padding: space.xl,
131           paddingBottom: insets.bottom + space.xl,
132           gap: space.md,
133         }}
134       >
135         <T v="h2">{title}</T>
136         <T v="body" color={theme.inkSoft}>{body}</T>
137         <View style={{ gap: space.sm, marginTop: space.sm }}>
138           <Button label={confirmLabel} kind={destructive ? 'danger' : 'primary'} onPress=
{onConfirm} />
139           <Button label="ยกเลิก" kind="ghost" onPress={onCancel} />
140         </View>
141       </View>
142     </Modal>
143   );
144 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/ui/feedback.tsx:103

บรรทัด	ความหมาย
103–119	ConfirmSheet รับ: แสดงไหม, หัวข้อ, เนื้อหา, ป้ายปุ่มยืนยัน, เป็นการทำลายไหม, ทำอะไรเมื่อยืนยัน/ยกเลิก
123	Modal หน้าต่างทึบทั้งจอ พื้นโปร่ง จางเข้า ปุ่มย้อนกลับของ Android = ยกเลิก
124	พื้นหลังมืดครึ่งจอ แต่แล้ว = ยกเลิก
125–134	แผ่นล่างนุ่มบนโค้ง
135–136	หัวข้อ และเนื้อหา
138	ปุ่มยืนยัน ถ้า destructive ใช้แบบ danger (แดง)
139	ปุ่ม "ยกเลิก" แบบโปร่ง

✕

แก้ไขรายการ

🗑

รายการจ่าย

รายการรับ

฿ 85.00

จ่ายให้ / ชื่ออะไร

ข้าวปั้นไก่

หมวดหมู่

🍽️ อาหารและเครื่องดื่ม

🚌 เดินทาง

🏠 ออปปิ้ง

🏪 ร้านสะดวกซื้อ

💰 บิลและค่าที่พัก

🏠 บ้านเช่า

📖 การเรียน

📷 สุขภาพ

👤 โอนให้คนอื่น

• อื่น ๆ

วันและเวลา

วันที่ (วว/ดด/ปปปป)

เวลา

09/10/2569

01:54

ลบบรรณการนี้?

ยอดคงเหลือและ Money Runway จะคำนวณใหม่ทันที กด เลิกทำได้ภายใน 5 วินาที

ลบบรรณการ

ยกเลิก

ConfirmSheet ตอนลบบรรณการ: หัวข้อ
เนื้อหาที่บอกผลกระทบ ปุ่มแดง และปุ่ม
ยกเลิก

อ่านโค้ด: หน้าจอดรายการถูกเปิดจากไหน

ปุ่มบนหน้าหลัก การลงทะเบียนหน้า และการส่งพารามิเตอร์

ปุ่มบนหน้าหลัก (index.tsx บรรทัด 471–482)

src/app/(tabs)/index.tsx

บรรทัด 471–482

```

471     <Reveal index={3}>
472       <View style={{ gap: space.sm }}>
473         <Row gap={space.sm} align="stretch">
474           <ActionTile icon="scan-outline" label="สแกนสลิป" sub="อ่านจากรูปในเครื่อง" tone="primary"
onPress={() => router.push('/scan')} />
475           <ActionTile icon="mic" label="จดด้วยเสียง" sub="พูดว่า "ข้าว 50 บาท"" tone="gold" pulse
onPress={() => router.push('/voice')} />
476         </Row>
477         <Row gap={space.sm} align="stretch">
478           <ActionTile icon="remove" label="จดยาการ" sub="รายจ่ายเงินสด ฯลฯ" tone="plain" onPress=
{() => router.push('/transaction')} />
479           <ActionTile icon="add" label="เพิ่มเงินเข้า" sub="เงินโอน เงินสด ค่าขนม" tone="income" onPress=
{() => router.push({ pathname: '/transaction', params: { kind: 'income' } })} />
480         </Row>
481       </View>
482     </Reveal>

```

▶ เปิดใน VS Code (เว็บ) GitHub

VS Code ในเครื่อง: Ctrl+P → src/app/(tabs)/index.tsx:471

บรรทัด	ความหมาย
471–472	กลุ่มปุ่ม 4 ปุ่มค่อย ๆ โผล่เป็นลำดับที่ 3 ของหน้า
473–476	แถบบน: "สแกนสลิป" (FR-4 ไปหน้า /scan) และ "จดด้วยเสียง" (FR-3 ไปหน้า /voice สี่ทอง มีชิพจอต)
478	"จดยาการ" ไอคอนลบ คำอธิบาย "รายจ่ายเงินสด ฯลฯ" → router.push('/transaction') เปิดหน้าจดยาการแบบรายจ่าย
479	"เพิ่มเงินเข้า" ไอคอนบวก สี่รายรับ คำอธิบาย "เงินโอน เงินสด ค่าขนม" → push แบบมีพารามิเตอร์ { kind: 'income' } ได้ URL /transaction?kind=income หน้าจดยาการบรรทัด 35 อ่านค่านี้แล้วตั้งเป็นรายรับ

ลงทะเบียนหน้าใน _layout.tsx (บรรทัด 97–126)

src/app/_layout.tsx

บรรทัด 97–126

```

97 function Screens({ signedIn, needsOnboarding, inApp }: { signedIn: boolean; needsOnboarding:
boolean; inApp: boolean }) {
98   const theme = useTheme();
99   return (
100     <>
101       {/* Screens slide in from the side; scanning and adding a transaction rise from the bottom.
*/}
102       <Stack screenOptions={{ headerShown: false, contentStyle: { backgroundColor: theme.bg },
animation: 'slide_from_right' }}>
103         <Stack.Protected guard={!signedIn}>

```

```

104     <Stack.Screen name="welcome" options={{ animation: 'fade' }} />
105   </Stack.Protected>
106   <Stack.Protected guard={needsOnboarding}>
107     <Stack.Screen name="onboarding" options={{ animation: 'fade' }} />
108   </Stack.Protected>
109   <Stack.Protected guard={inApp}>
110     <Stack.Screen name="(tabs)" options={{ animation: 'fade' }} />
111     <Stack.Screen name="transaction" options={{ presentation: 'modal', animation:
'slide_from_bottom' }} />
112     <Stack.Screen name="scan" options={{ presentation: 'fullScreenModal', animation:
'slide_from_bottom' }} />
113     <Stack.Screen name="voice" options={{ presentation: 'modal', animation:
'slide_from_bottom' }} />
114     <Stack.Screen name="achievements" />
115     <Stack.Screen name="skins" />
116     <Stack.Screen name="recap" options={{ presentation: 'fullScreenModal', animation:
'fade' }} />
117     <Stack.Screen name="goals" />
118     <Stack.Screen name="whatsnew" options={{ presentation: 'modal', animation:
'slide_from_bottom' }} />
119     <Stack.Screen name="goal" options={{ presentation: 'modal', animation:
'slide_from_bottom' }} />
120     <Stack.Screen name="drafts" />
121     <Stack.Screen name="settings" />
122   </Stack.Protected>
123 </Stack>
124 </>
125 );
126 }

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/app/_layout.tsx:97

บรรทัด	ความหมาย
97	component Screens รับสถานะ 3 อย่าง: ล็อกอินแล้วไหม, ต้องตั้งค่าเริ่มต้นไหม, เข้าแอปได้แล้วไหม
98–100	สไลม์ และกล่องล่องหน (Fragment)
101	คอมเมนต์: หน้าทัวไปเลื่อนเข้าจากด้านข้าง ส่วนหน้าสแกนและหน้าจดยรายการ เลื่อนขึ้นจากด้านล่าง
102	Stack = กองหน้าจอซ้อนกันเหมือนกองไฟ (เปิดหน้าใหม่ = วางทับ, ย้อนกลับ = หยิบใบบนออก) ไม่แสดงแถบหัวมาตรฐาน (แอปวาดหัวเอง) พื้นสไลม์ เลื่อนเข้าจากขวาเป็นค่าตั้งต้น
103–105	Stack.Protected = "หน้าที่เปิดได้เฉพาะเมื่อเงื่อนไขเป็นจริง": หน้า welcome เปิดได้เฉพาะตอน ยังไม่ล็อกอิน
106–108	หน้าตั้งค่าเริ่มต้น เปิดได้เฉพาะตอนที่ต้องตั้งค่า
109–110	หน้าที่ต้อง เข้าแอปแล้ว เท่านั้น เริ่มจากกลุ่มแท็บ (tabs) — ถ้าใครพิมพ์ URL /transaction ตรง ๆ ตอนยังไม่ล็อกอิน จะเข้าไม่ได้
111	หน้า "transaction" (= ไฟล์ src/app/transaction.tsx) เปิดแบบ modal (แผ่นที่ทับหน้าเดิม) เลื่อนขึ้นจากล่าง ผู้ใช้จึงรู้สึก "กรอกเสร็จแล้วปิดกลับไปหน้าเดิม" ไม่ใช่ไปหน้าใหม่
112–121	หน้าอื่น ๆ: สแกน (เต็มจอ), เสียง (modal), ความสำเร็จ, สกิน, สรุปเดือน (เต็มจอ จางเข้า), กระจุกออม, มีอะไรใหม่, กระจุก 1 ใบ, สลิปรียืนยัน, ตั้งค่า
122–126	ปิดกลุ่ม ปิด Stack ปิด Fragment และปิดฟังก์ชัน

หน้าหลัก กด "เพิ่มเงินเข้า"

`router.push({ pathname: '/transaction', params: { kind: 'income' } })`



Expo Router

หาไฟล์ `src/app/transaction.tsx` เปิดแบบ modal เลื่อนขึ้นจากล่าง



TransactionForm

`useLocalSearchParams()` ได้ `kind = 'income' → startKind = 'income' → หัวหน้าจอ "เพิ่มเงินเข้า" ซป +฿100 ... +฿5,000`

อ่านโค้ด: รายการประจำเดือน (recurring.ts ทั้งไฟล์)

148 บรรทัด หาค่าหอ ค่าเน็ต ค่าขนม ที่มาวันใกล้เคียงทุกเดือน จากรายการที่จดไว้เอง (TC-72)

! บอกตรง ๆ ก่อนอ่าน

ฟังก์ชันในไฟล์นี้ ผ่านเทสต์ TC-72 ครบ แต่ยังไม่เห็นหน้าจอไหนเรียกใช้ (ค้นทั้งโปรเจกต์แล้ว ไม่มี import จาก recurring นอกจากไฟล์เทสต์) ตรรกะพร้อมแล้ว รอนำไปแสดงเป็นการ์ด "ค่าหอพัก ฿3,500 อีก 2 วัน" ถ้าอาจารย์ถาม ให้ตอบตามจริงแบบนี้

ส่วนที่ 1: รูปร่างของผลลัพธ์และค่าคงที่ (บรรทัด 1-38)

src/domain/recurring.ts

บรรทัด 1-38

```
1 /**
2  * Monthly regulars ("รายการประจำ"): money that goes out or comes in around the
3  * same day every month (rent, phone bill, allowance, pay). Found from the
4  * transactions themselves, so the app can say "ค่าหอพัก ฿3,500 อีก 2 วัน".
5  * Pure; unit tests in __tests__/recurring.test.ts (TC-72).
6  */
7 import { addDays, bkkDayKey, daysBetween, daysInMonth } from './dates';
8 import type { Transaction, TxKind } from './types';
9
10 export interface Recurring {
11   key: string;
12   title: string;
13   kind: TxKind;
14   categoryKey: string;
15   /** The usual amount (median). */
16   amountSatang: number;
17   /** Usual day of the month. */
18   dayOfMonth: number;
19   /** Months it was seen in. */
20   months: number;
21   lastDay: string;
22   /** When it is expected next ("YYYY-MM-DD"). */
23   nextDay: string;
24   /** Days from today to nextDay (0 = today, negative = a few days late). */
25   daysLeft: number;
26 }
27
28 const LOOKBACK_DAYS = 120;
29 const DAY_TOLERANCE = 3;
30 const AMOUNT_TOLERANCE = 0.2;
31
32 const normalize = (title: string) => title.trim().toLowerCase().replace(/\s+/g, ' ');
33
34 function median(values: number[]): number {
35   const s = [...values].sort((a, b) => a - b);
36   const mid = Math.floor(s.length / 2);
37   return s.length % 2 ? s[mid] : Math.round((s[mid - 1] + s[mid]) / 2);
```

บรรทัด	ความหมาย
1-6	คอมเมนต์: รายการประจำ = เงินที่ออกหรือเข้ารายวันเดิมทุกเดือน (ค่าเช่า ค่าโทรศัพท์ ค่าขนม เงินเดือน) หากจากรายการเอง เพื่อให้แอปบอกได้ว่า "ค่าหอพัก \$3,500 อีก 2 วัน" เป็นฟังก์ชันล้น มีเทสต์ TC-72
7-8	ใช้ฟังก์ชันวันที่ 4 ตัว และชนิดข้อมูล
10-26	ผลลัพธ์ 1 รายการประจำ: ฤดูแฉ, ชื่อ, ประเภท, หมวด, ยอดปกติ (ค่ามัธยฐาน) , วันที่ปกติของเดือน, เห็นมาที่เดือน, วันล่าสุด, ครั้งถัดไป , อีกกี่วัน (0 = วันนี้, ติดลบ = เลยมานแล้ว)
28	ดูย้อนหลัง 120 วัน (ประมาณ 4 เดือน)
29	วันคลาดเคลื่อนได้ ± 3 วัน
30	ยอดคลาดเคลื่อนได้ $\pm 20\%$
32	ทำชื่อให้เป็นมาตรฐาน: ตัดช่องว่างหัวท้าย ตัวเล็ก และช่องว่างหลายตัวเหลือตัวเดียว ("ค่าหอพัก " กับ "ค่าหอพัก" = ชื่อเดียวกัน)
34-38	ค่ามัธยฐาน (median) : เรียงจากน้อยไปมาก หีบตัวกลาง ถ้าจำนวนคู่ใช้ค่าเฉลี่ยของสองตัวกลาง (พิเศษ) ใช้มัธยฐานแทนค่าเฉลี่ย เพราะไม่โดนค่าผิดปกติครั้งเดียวดึง

ส่วนที่ 2: วันในเดือนแบบ "วงกลม" (บรรทัด 40-81)

src/domain/recurring.ts

บรรทัด 40-81

```

40 /** Distance between two days of the month, wrapping around the month end (31st and 1st are 1
    apart). */
41 function domDistance(a: number, b: number): number {
42     const d = Math.abs(a - b);
43     return Math.min(d, 31 - d);
44 }
45
46 /**
47  * The usual day of the month for payments that may wrap around the month end (a median on a
48  * circle): the day closest to all payments (2 Jun, 1 Jul, 31 Jul, 31 Aug -> the 1st; one early
49  * payment on the 30th does not move the 1st), then the one whose farthest payment is nearest,
50  * then the day of the latest payment (20th and 21st -> the 21st if that was last).
51  */
52 function usualDay(doms: number[]): number {
53     const latest = doms[doms.length - 1];
54     let best = 1;
55     let bestScore: [number, number, number] = [Infinity, Infinity, Infinity];
56     for (let c = 1; c <= 31; c++) {
57         const ds = doms.map((d) => domDistance(d, c));
58         const score: [number, number, number] = [ds.reduce((a, b) => a + b, 0), Math.max(...ds),
59             domDistance(latest, c)];
60         const better = score[0] - bestScore[0] || score[1] - bestScore[1] || score[2] - bestScore[2];
61         if (better < 0) {
62             best = c;
63             bestScore = score;
64         }
65     }
66 }

```

```

65     return best;
66 }
67
68 /** The usual day in a given month ("2026-02" and day 31 -> the 28th). */
69 function dayInMonth(month: string, dom: number): string {
70     return `${month}-${String(Math.min(dom, daysInMonth(month))).padStart(2, '0')}`;
71 }
72
73 const nextMonthKey = (month: string) => {
74     const [y, m] = month.split('-').map(Number);
75     return m === 12 ? `${y + 1}-01` : `${y}-${String(m + 1).padStart(2, '0')}`;
76 };
77
78 const prevMonthKey = (month: string) => {
79     const [y, m] = month.split('-').map(Number);
80     return m === 1 ? `${y - 1}-12` : `${y}-${String(m - 1).padStart(2, '0')}`;
81 };

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/recurring.ts:40

บรรทัด	ความหมาย
40–44	domDistance ระยะห่างของวันในเดือน แบบวนรอบ : วันที่ 31 กับวันที่ 1 ห่างกัน 1 (ไม่ใช่ 30) เพราะจ่ายค่าหาวันที่ 31 หรือวันที่ 1 ก็คือ "ช่วงต้นเดือน" เหมือนกัน
46–51	คอมเมนต์: หา "วันปกติ" ที่ใกล้ทุกครั้งที่ย้ายมากที่สุด เช่น 2 มิ.ย., 1 ก.ค., 31 ก.ค., 31 ส.ค. → วันที่ 1 ถ้าเสมอ ใช้ตัวที่ครั้งไกลสุดใกล้ที่สุด ถ้ายังเสมอใช้ตัวที่ใกล้ครั้งล่าสุด
52–55	usualDay: จำครั้งสุดท้าย เริ่มคำตอบที่ 1 คะแนนดีสุดเริ่มที่อนันต์
56	ลองทุกวัน 1–31 เป็นผู้สมัคร
57	ระยะจากผู้สมัครถึงทุกครั้งที่ย้าย
58	คะแนน 3 ชั้น: [ผลรวมระยะ, ระยะไกลสุด, ระยะถึงครั้งสุดท้าย] (น้อย = ดี)
59	เทียบทีละชั้น: ถ้าชั้นแรกเท่ากัน (ลบกันได้ 0 ซึ่งถือเป็นเท็จ) จะไปดูชั้นถัดไป
60–63	ดีกว่า (ดีดลบ) → เป็นคำตอบใหม่
68–71	วันปกติในเดือนที่กำหนด แต่ไม่เกินจำนวนวันของเดือน ("2026-02" กับวันที่ 31 → 28 ก.พ.)
73–81	เดือนถัดไป และเดือนก่อนหน้า (ข้ามปีได้: ธ.ค. → ม.ค. และ ม.ค. → ธ.ค.)

ส่วนที่ 3: findRecurring – รายการประจำ (บรรทัด 83–135)

src/domain/recurring.ts

บรรทัด 83–135

```

83 export function findRecurring(txs: Transaction[], today: string): Recurring[] {
84     const from = addDays(today, -LOOKBACK_DAYS);
85     const groups = new Map<string, Transaction[]>();
86     for (const t of txs) {
87         if (t.status !== 'confirmed') continue;
88         const day = bkkDayKey(t.occurredAt);
89         if (day < from || day > today) continue;
90         const key = `${t.kind}|${normalize(t.title)}`;
91         if (!normalize(t.title)) continue;

```

```

92     groups.set(key, [...(groups.get(key) ?? []), t]);
93 }
94
95 const out: Recurring[] = [];
96 for (const [key, list] of groups) {
97     const sorted = [...list].sort((a, b) => a.occurredAt.localeCompare(b.occurredAt));
98     const days = sorted.map((t) => bkkDayKey(t.occurredAt));
99     const months = new Set(days.map((d) => d.slice(0, 7)));
100    // Monthly things: seen in at least two months, about once a month (not a daily coffee).
101    if (months.size < 2 || sorted.length > months.size * 1.5) continue;
102    const doms = days.map((d) => Number(d.slice(8, 10)));
103    const dom = usualDay(doms);
104    if (doms.some((d) => domDistance(d, dom) > DAY_TOLERANCE)) continue;
105    const amount = median(sorted.map((t) => t.amountSatang));
106    if (sorted.some((t) => Math.abs(t.amountSatang - amount) > amount * AMOUNT_TOLERANCE))
107        continue;
108    const lastDay = days[days.length - 1];
109    // Still going: seen within the last ~45 days.
110    if (daysBetween(lastDay, today) > 45) continue;
111    // Next time: the first usual day (from last month on, so rent due on the 30th that is two
112    // days
113    // late on the 2nd still shows) that was not paid yet (no payment within a week of it) and is
114    // not more than a few days gone.
115    const month = today.slice(0, 7);
116    const later = nextMonthKey(month);
117    const candidates = [prevMonthKey(month), month, later, nextMonthKey(later)].map((m) =>
118    dayInMonth(m, dom));
119    const next =
120        candidates.find((c) => Math.abs(daysBetween(c, lastDay)) > 7 && daysBetween(c, today) <=
121        DAY_TOLERANCE) ??
122        candidates[candidates.length - 1];
123    const last = sorted[sorted.length - 1];
124    out.push({
125        key,
126        title: last.title.trim(),
127        kind: last.kind,
128        categoryKey: last.categoryKey,
129        amountSatang: amount,
130        dayOfMonth: dom,
131        months: months.size,
132        lastDay,
133        nextDay: next,
134        daysLeft: daysBetween(today, next),
135    });
136 }
137 return out.sort((a, b) => a.daysLeft - b.daysLeft || b.amountSatang - a.amountSatang);
138 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/recurring.ts:83

บรรทัด	ความหมาย
83–84	รับรายการทั้งหมดและวันนี้ เริ่มนับจาก 120 วันก่อน
85–93	จัดกลุ่ม: ข้ามแบบร่าง, ข้ามที่อยู่นอกช่วง, กระจายกลุ่ม = "ประเภท ชื่อมาตรฐาน" (เช่น "expense ค่าหอพัก") ข้ามรายการไม่มีชื่อ แล้วใส่ในกลุ่ม
96–99	แต่ละกลุ่ม: เรียงตามเวลา, ได้รายการวัน, นับว่าอยู่ที่เดือน (Set ของ "YYYY-MM")

บรรทัด	ความหมาย
100–101	เงื่อนไข 1: ต้องเห็นอย่างน้อย 2 เดือน และไม่ถี่เกิน 1.5 ครั้งต่อเดือน (กาแฟทุกวันไม่ใช่รายการประจำ)
102–104	เงื่อนไข 2: หาวันปกติ ทุกครั้งต้องห่างจากวันปกติไม่เกิน 3 วัน
105–106	เงื่อนไข 3: ยอดปกติ = มัธยฐาน ทุกครั้งต้องต่างจากยอดปกติไม่เกิน 20%
107–109	เงื่อนไข 4: ยังไม่เลิก: ครั้งล่าสุดไม่เกิน 45 วันก่อน
111–116	หาครั้งถัดไป: ผู้สมัคร 4 ตัว = วันปกติในเดือนก่อน, เดือนนี้, เดือนหน้า, เดือนถัดไปอีก
117–119	เลือกตัวแรกที่ยังไม่ได้จ่าย (ห่างจากครั้งล่าสุดเกิน 7 วัน) และ ยังไม่เลยมาเกิน 3 วัน ถ้าไม่มีเลยใช้ตัวสุดท้าย
120–132	สร้างผลลัพธ์ ใช้ชื่อ/ประเภท/หมวดจากครั้งล่าสุด และ daysLeft = จากวันนี้ถึงครั้งถัดไป
134	เรียง: ใกล้ถึงก่อน ถ้าเท่ากันยอดมากก่อน

ส่วนที่ 4: ตัวช่วยแสดงผล (บรรทัด 137–148)

src/domain/recurring.ts	บรรทัด 137–148
<pre> 137 /** Regulars due soon (from a few days late up to `withinDays` ahead). */ 138 export function upcomingRecurring(list: Recurring[], withinDays = 7): Recurring[] { 139 return list.filter((r) => r.daysLeft >= -DAY_TOLERANCE && r.daysLeft <= withinDays); 140 } 141 142 /** "วันนี้", "พรุ่งนี้", "อีก 3 วัน", "เลยมา 2 วัน". */ 143 export function dueLabel(daysLeft: number): string { 144 if (daysLeft === 0) return 'วันนี้'; 145 if (daysLeft === 1) return 'พรุ่งนี้'; 146 if (daysLeft > 1) return `อีก \${daysLeft} วัน`; 147 return `เลยมา \${-daysLeft} วัน`; 148 } </pre>	
▶ เปิดใน VS Code (เว็บ) GitHub	VS Code ในเครื่อง: Ctrl+P → src/domain/recurring.ts:137

บรรทัด	ความหมาย
137–140	upcomingRecurring: เฉพาะที่ใกล้ถึง (เลยมาไม่เกิน 3 วัน ถึงอีกไม่เกิน 7 วัน)
142–148	dueLabel: 0 → "วันนี้", 1 → "พรุ่งนี้", มากกว่า 1 → "อีก 3 วัน", ติดลบ → "เลยมา 2 วัน"

ตัวอย่างที่รันจริง วันนี้ = 8 ต.ค. 2569:

ข้อมูล	ผล
Netflix ฿169 วันที่ 15 ส.ค. และ 15 ก.ย.	ประจำ · วันที่ 15 · ครั้งถัดไป 15 ต.ค. · "อีก 7 วัน"
ค่าหอพัก ฿3,500 วันที่ 30 ส.ค. และ 30 ก.ย.	ประจำ · วันที่ 30 · ครั้งถัดไป 30 ต.ค. · "อีก 22 วัน"
กาแฟ ฿60 วันที่ 3 ส.ค., 1 ก.ย., 2 ก.ย.	ถูกนับเป็นประจำด้วย (3 ครั้งใน 2 เดือน ไม่เกิน 1.5×2) · วันที่ 2 · ครั้งถัดไป 2 พ.ย.

แถวสุดท้ายแสดง **ข้อจำกัด**: ข้อมูลน้อย ๆ ที่บังเอิญตรงเงื่อนไขก็ถูกนับได้ เมื่อมีข้อมูลหลายเดือนขึ้น (กาแพวันละแก้ว)
เงื่อนไขความถี่จะตัดออกเอง

การทดสอบ FR-1

ฉันพิสูจน์ยังงใจว่าเงินถูก หมวดถูก และหน้าจอบันทึกร/แก้/ลบได้จริง

157/157

unit test ทั้งโปรเจกต์ผ่าน (รันซ้ำ 9 ๓.ค. 2569)

TC-01, 02, 25, 72,

79

unit test ของ FR-1 โดยตรง

7 ข้อ

E2E "Money" ในเบราว์เซอร์ (เพิ่ม แก้ ลบ เลิกทำ เงินเข้า)

Unit test: เงินและหมวด (domain.test.ts)

src/domain/__tests__/domain.test.ts

บรรทัด 70-88

```
70 describe('money', () => {
71   it('TC-01 parses amounts as printed on slips', () => {
72     expect(parseBahtToSatang('1,234.50')).toBe(123450);
73     expect(parseBahtToSatang('฿85')).toBe(8500);
74     expect(parseBahtToSatang('85.00 บาท')).toBe(8500);
75     expect(parseBahtToSatang('0.1')).toBe(10);
76     expect(parseBahtToSatang('abc')).toBeNull();
77     expect(parseBahtToSatang('-5')).toBeNull();
78     expect(parseBahtToSatang('0')).toBeNull();
79     expect(parseBahtToSatang('1.234')).toBeNull();
80   });
81
82   it('TC-02 avoids floating point drift', () => {
83     const sum = [10, 20].map((b) => parseBahtToSatang(b / 100)).reduce((a, b) => a + b, 0);
84     expect(sum).toBe(30);
85     expect(formatBaht(123450)).toBe('฿1,234.50');
86     expect(formatBaht(-8500, { sign: true })).toBe('-฿85.00');
87   });
88 });
```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/__tests__/domain.test.ts:70

บรรทัด	ความหมาย
70	describe = กลุ่มเทสต์ชื่อ "money"
71	it = เทสต์ 1 ข้อ TC-01 "อ่านจำนวนเงินแบบที่พิมพ์บนสลิป"
72	"1,234.50" ต้องได้ 123,450 สตางค์ (expect(...).toBe(...) = "คาดว่า...ต้องเท่ากับ...")
73-74	มีสัญลักษณ์ ฿ หรือคำว่า บาท ก็อ่านได้ = 8,500
75	"0.1" = 10 สตางค์ (ไม่ใช่ 1)
76-79	ต้อง ปฏิเสธ (null): ตัวอักษร, ติดลบ, ศูนย์, ทศนิยม 3 ตำแหน่ง
82-84	TC-02: ฿0.10 + ฿0.20 ต้องได้ 30 สตางค์พอดี (ถ้าใช้ทศนิยมจะได้ 0.30000000000000004)

บรรทัด	ความหมาย
85-86	แสดงผล: 123,450 → "฿1,234.50" และ -8,500 แบบมีเครื่องหมาย → "-฿85.00"

src/domain/__tests__/domain.test.ts	บรรทัด 296-301
<pre> 296 it('TC-25 suggests categories from the payee', () => { 297 expect(suggestCategory('7-ELEVEN สาขา 1234', 'expense')).toBe('convenience'); 298 expect(suggestCategory('ร้านข้าว นายสมชาย', 'expense')).toBe('food'); 299 expect(suggestCategory('นาย สมชาย ใจดี', 'expense')).toBe('transfer_out'); 300 expect(suggestCategory('Shopee', 'expense')).toBe('shopping'); 301 }); </pre>	
▶ เปิดใน VS Code (เว็บ) GitHub VS Code ในเครื่อง: Ctrl+P → src/domain/__tests__/domain.test.ts:296	

บรรทัด	ความหมาย
296	TC-25 "แนะนำหมวดจากชื่อผู้รับ"
297	"7-ELEVEN สาขา 1234" → ร้านสะดวกซื้อ
298	"ร้านข้าว นายสมชาย" → อาหาร (ร้านค้าชณะกฎซื้อคน และ "ชา" ใน "สมชาย" ไม่ทำให้เป็นหมวดอื่น)
299	"นาย สมชาย ใจดี" → โอนให้คนอื่น
300	"Shopee" → ช้อปปิ้ง

Unit test: เงินที่ผู้ใช้พิมพ์ (audit.test.ts TC-79)

src/domain/__tests__/audit.test.ts	บรรทัด 41-70
<pre> 41 describe('TC-79 money typed by the user', () => { 42 it('reads a decimal comma, Thai digits and grouped thousands, and refuses the rest', () => { 43 expect(parseBahtToSatang('85,50')).toBe(8550); 44 expect(parseBahtToSatang('12,5')).toBe(1250); 45 expect(parseBahtToSatang('1,234.50')).toBe(123450); 46 expect(parseBahtToSatang('1,234,567')).toBe(123456700); 47 expect(parseBahtToSatang('๘๕')).toBe(8500); 48 expect(parseBahtToSatang('12.')).toBe(1200); 49 expect(parseBahtToSatang('1,23,4')).toBeNull(); 50 expect(parseBahtToSatang('12.345')).toBeNull(); 51 }); 52 53 it('caps huge amounts and never turns a tiny one into zero', () => { 54 expect(parseBahtToSatang('999999999999999')).toBeNull(); 55 expect(parseBahtToSatang(String(MAX_SATANG / 100))).toBe(MAX_SATANG); 56 expect(parseBahtToSatang(String(MAX_SATANG / 100 + 1))).toBeNull(); 57 expect(parseBahtToSatang(0.004)).toBeNull(); 58 }); 59 60 it('takes zero only when asked (an opening balance may be ฿0.00)', () => { 61 expect(parseBahtToSatang('0.00')).toBeNull(); 62 expect(parseBahtToSatang('0.00', { allowZero: true })).toBe(0); 63 expect(parseBahtToSatang('0', { allowZero: true })).toBe(0); 64 }); 65 66 it('rounds to the nearest baht when showing whole baht', () => { </pre>	

```

67     expect(formatBaht(9999, { decimals: false })).toBe('฿100');
68     expect(formatBaht(9949, { decimals: false })).toBe('฿99');
69   });
70 });

```

▶ เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/__tests__/audit.test.ts:41

บรรทัด	ใส่	ต้องได้	เพราะ
43– 44	"85,50" / "12,5"	8,550 / 1,250	มือถือบางภาษาใช้จุลภาคเป็นจุดทศนิยม
45– 46	"1,234.50" / "1,234,567"	123,450 / 123,456,700	จุลภาคคั่นหลักพัน
47	"๘๕"	8,500	เลขไทย
48	"12."	1,200	จุดค่างท้าย (พิมพ์ยังไม่เสร็จ) ก็อ่านได้
49– 50	"1,23,4" / "12.345"	null	จุลภาคผิดที่ / ทศนิยมเกิน 2
54– 56	เลขยาวมาก / เพดานพอดี / เกินเพดาน ฿1	null / MAX / null	ไม่เกิน ฿1,000 ล้าน
57	0.004 บาท	null	ไม่ปัดเศษเล็ก ๆ ให้กลายเป็น 0 แล้วบันทึก
61– 63	"0.00"	null แต่ถ้า allowZero ได้ 0	รายการต้องมากกว่า 0 แต่ยอดตั้งต้นเป็น 0 ได้
67– 68	9,999 / 9,949 สตางค์ แบบไม่มีทศนิยม	"฿100" / "฿99"	ปัดเป็นบาทที่ใกล้ที่สุด

Unit test: รายการประจำ (recurring.test.ts TC-72)

src/domain/__tests__/recurring.test.ts

บรรทัด 31–75

```

31 describe('Monthly regulars', () => {
32   it('TC-72 finds rent, bills and allowance that come around the same day each month, and when
   they are due next', () => {
33     const txs = [
34       tx('2026-07-05', 'ค่าหอพัก', 3500),
35       tx('2026-08-04', 'ค่าหอพัก', 3500),
36       tx('2026-09-05', 'ค่าหอพัก', 3500),
37       tx('2026-08-01', 'ค่าขนมจากที่บ้าน', 10000, { kind: 'income', categoryKey: 'allowance' }),
38       tx('2026-09-01', 'ค่าขนมจากที่บ้าน', 10000, { kind: 'income', categoryKey: 'allowance' }),
39       tx('2026-08-20', 'ค่าเน็ต AIS', 599),
40       tx('2026-09-21', 'ค่าเน็ต AIS', 620),
41       // Not monthly: a daily lunch, a one-off, a price that changes a lot, a day that moves
   around.
42       ...['2026-09-01', '2026-09-02', '2026-09-03', '2026-08-01', '2026-08-02'].map((d) => tx(d,
   'ข้าวกลางวัน', 60, { categoryKey: 'food' })),
43       tx('2026-09-10', 'Shopee', 590, { categoryKey: 'shopping' }),
44       tx('2026-08-10', 'เติมน้ำมัน', 300, { categoryKey: 'transport' }),
45       tx('2026-09-10', 'เติมน้ำมัน', 900, { categoryKey: 'transport' }),
46       tx('2026-08-03', 'ตัดผม', 150, { categoryKey: 'other_expense' }),
47       tx('2026-09-18', 'ตัดผม', 150, { categoryKey: 'other_expense' }),

```

```

48     // A draft does not count.
49     tx('2026-08-12', 'ค่าโสหุศล', 300, { status: 'draft' }),
50     tx('2026-09-12', 'ค่าโสหุศล', 300, { status: 'draft' }),
51 ];
52 const found = findRecurring(txs, '2026-09-29');
53 expect(found.map((r) => r.title)).toEqual(['ค่าขนมจากที่บ้าน', 'ค่าหอพัก', 'ค่าเน็ต AIS']);
54 const [allowance, rent, net] = found;
55 expect(allowance).toMatchObject({ kind: 'income', amountSatang: 1_000_000, dayOfMonth: 1,
56   nextDay: '2026-10-01', daysLeft: 2 });
57 expect(rent).toMatchObject({ kind: 'expense', amountSatang: 350_000, dayOfMonth: 5, months:
58   3, nextDay: '2026-10-05', daysLeft: 6 });
59 expect(net).toMatchObject({ amountSatang: 60_950, nextDay: '2026-10-21' });
60
61 // Due within a week (or a few days late).
62 expect(upcomingRecurring(found).map((r) => r.title)).toEqual(['ค่าขนมจากที่บ้าน', 'ค่าหอพัก']);
63 // Two days late this month: still shown as due.
64 const late = findRecurring(txs, '2026-10-07').find((r) => r.title.startsWith('ค่าหอ'));
65 expect(late).toMatchObject({ nextDay: '2026-10-05', daysLeft: -2 });
66 // Paid early for next month: the one after that is next.
67 const early = findRecurring([...txs, tx('2026-09-30', 'ค่าขนมจากที่บ้าน', 10000, { kind: 'income',
68   categoryKey: 'allowance' })], '2026-09-30');
69 expect(early.find((r) => r.kind === 'income')?.nextDay).toBe('2026-11-01');
70 // A regular that stopped (nothing for over 45 days) is dropped.
71 expect(findRecurring(txs, '2026-12-31')).toEqual([]);
72
73 expect(dueLabel(0)).toBe('วันนี้');
74 expect(dueLabel(1)).toBe('พรุ่งนี้');
75 expect(dueLabel(4)).toBe('อีก 4 วัน');
76 expect(dueLabel(-2)).toBe('เลยมา 2 วัน');
77 });
78 });

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → src/domain/__tests__/recurring.test.ts:31

บรรทัด	ความหมาย
33–40	ข้อมูลที่ ควรเจอ : ค่าหอ 3 เดือน (วันที่ 4–5, มีช่องว่างท้ายชื่อครั้งหนึ่ง), ค่าขนม 2 เดือน (รายรับ), ค่าเน็ต 599 และ 620
41–47	ข้อมูลที่ ไม่ควรเจอ : ขาวกลางวันทุกวัน (ถี่เกินไป), Shopee ครั้งเดียว, น้ำมัน 300 กับ 900 (ยอดต่างกัน 20%), ตัดผม 3 ส.ค. กับ 18 ก.ย. (วันไม่ตรง)
48–50	แบบร่างไม่นับ
52–53	วันนี้ 29 ก.ย. ต้องเจอ 3 อย่างเรียงตามไคล้ถึง: ค่าขนม, ค่าหอ, ค่าเน็ต
55–57	ค่าขนม ฿10,000 วันที่ 1 ครั้งถัดไป 1 ต.ค. อีก 2 วัน · ค่าหอ ฿3,500 วันที่ 5 เห็น 3 เดือน อีก 6 วัน · ค่าเน็ตยอดมาตรฐาน ฿609.50 ครั้งถัดไป 21 ต.ค.
60	ไคล้ถึงใน 7 วัน: ค่าขนม และค่าหอ
62–63	7 ต.ค. ค่าหอยังไม่จ่าย → ยังแสดง "เลยมา 2 วัน"
65–66	จ่ายค่าขนมล่วงหน้า 30 ก.ย. → ครั้งถัดไปข้ามเป็น 1 พ.ย.
68	31 ธ.ค. ไม่มีอะไรเกิน 45 วันแล้ว → ว่าง
70–73	ป้าย: วันนี้ / พรุ่งนี้ / อีก 4 วัน / เลยมา 2 วัน

E2E: เพิ่ม แก้ ลบ เลิกทำ ในเบราร์วเซอร์จริง

e2e/web-auth.mjs

บรรทัด 435–493

```
435 // 12. Money: add, edit, delete and undo; the balance follows every change
436 {
437   const { ctx, page } = await freshPage('money');
438   await page.goto(APP);
439   await visible(page, 'ลองใช้ด้วยข้อมูลตัวอย่าง', 8000);
440   await introGone(page);
441   await page.getByText('ลองใช้ด้วยข้อมูลตัวอย่าง').click();
442   await visible(page, 'ยอดคงเหลือ', 8000);
443   const balance = async () => {
444     await page.waitForTimeout(900); // let the count-up finish
445     const label = await page.locator('[aria-label^="ยอดคงเหลือ"]').first().getAttribute('aria-label');
446     return Math.round(Number(label.match(/฿([\d,]+\.\d{2})/)[1].replace(/,/g, '')) * 100);
447   };
448   const start = await balance();
449   await button(page, 'จดรายการ').click();
450   await visible(page, 'จ่ายให้ / ชื่ออะไร');
451   await button(page, 'บันทึกรายการ').click();
452   check('Add: an empty amount is not saved', await visible(page, 'จ่ายให้ / ชื่ออะไร'));
453   await page.fill('#tx-amount', '123');
454   await page.fill('#tx-title', 'ทดสอบกาแฟ');
455   await button(page, 'บันทึกรายการ').click();
456   check('Add: saved with a confirmation', await visible(page, 'บันทึกรายการแล้ว'));
457   await visible(page, 'ยอดคงเหลือ', 8000);
458   check('Add: the balance goes down by ฿123', (await balance()) === start - 12_300);
459   await page.goto(`${APP}transactions`);
460   await introGone(page);
461   await page.getByText('ทดสอบกาแฟ').first().click();
462   await page.fill('#tx-amount', '150');
463   await button(page, 'บันทึกการแก้ไข').click();
464   check('Edit: saved', await visible(page, 'บันทึกการแก้ไขแล้ว'));
465   await page.goto(APP);
466   await introGone(page);
467   check('Edit: the balance follows (฿150 instead of ฿123)', (await balance()) === start - 15_000);
468   await page.goto(`${APP}transactions`);
469   await introGone(page);
470   await page.getByText('ทดสอบกาแฟ').first().click();
471   await button(page, 'ลบรายการ').first().click();
472   await page.getByRole('dialog').getByRole('button', { name: 'ลบรายการ' }).click();
473   check('Delete: toast with undo', await visible(page, 'เลิกทำ'));
474   await button(page, 'เลิกทำ').click();
475   await page.waitForTimeout(500);
476   check('Undo: the item is back', await visible(page, 'ทดสอบกาแฟ'));
477   // Money in without a slip: one tile on home opens the form as income.
478   await page.goto(APP);
479   await introGone(page);
480   await visible(page, 'ยอดคงเหลือ', 8000);
481   const beforeIncome = await balance();
482   // Exactly the tile (the "มีอะไรใหม่" card also mentions it).
483   await page.getByRole('button', { name: 'เพิ่มเงินเข้า', exact: true }).first().click();
484   const incomeForm = (await visible(page, 'ได้รับจาก')) && (await page.getByRole('tab', { name: 'รายรับ', exact: true }).first().getAttribute('aria-selected')) === 'true';
485   await page.getByRole('button', { name: '+฿1,000', exact: true }).first().click();
486   await page.fill('#tx-title', 'ค่าสอยพิเศษ');
487   await shot(page, '31-add-income');
```

```

488     await button(page, 'บันทึกรายการ').click();
489     check('Add money in: the form opens as income with quick amounts, and saves', incomeForm &&
(await visible(page, 'เพิ่มเงินเข้าแล้ว')));
490     await visible(page, 'ยอดคงเหลือ', 8000);
491     check('Add money in: the balance goes up by $1,000', (await balance()) === beforeIncome +
100_000);
492     await ctx.close();
493 }

```

► เปิดใน VS Code (เว็บ) [GitHub](#)

VS Code ในเครื่อง: Ctrl+P → e2e/web-auth.mjs:435

บรรทัด	ความหมาย
437–442	เปิดเบราว์เซอร์ใหม่ เปิดแอป กด "ลองใช้ด้วยข้อมูลตัวอย่าง" รอจนเห็น "ยอดคงเหลือ"
443–447	ฟังก์ชันอ่านยอดคงเหลือจากป้ายอ่านจอ (รอ 0.9 วินาทีให้ตัวเลขนับขึ้นเสร็จ) แล้วแปลงเป็นสตริง
448	จำลองตอนเริ่ม
449–452	กด "จดรายการ" แล้วกด "บันทึกรายการ" ทั้งที่ช่องเงินว่าง → ต้องยังอยู่หน้าเดิม (ไม่บันทึก)
453–456	ใส่ 123 ชื่อ "ทดสอบกาแฟ" กดบันทึก → ต้องเห็น "บันทึกรายการแล้ว"
457–458	ยอดต้องลดลง \$123 พอดี (12,300 สตางค์)
459–464	ไปแท็บรายการ แตะ "ทดสอบกาแฟ" เปลี่ยนเป็น 150 กด "บันทึกการแก้ไข" → "บันทึกการแก้ไขแล้ว"
465–467	ยอดต้องเป็นเริ่มต้น – \$150 (ไม่ใช่ – \$273)
468–473	เปิดรายการเดิม กดถึงขยะ ในหน้าต่างยืนยันกด "ลบรายการ" → ต้องเห็นปุ่ม "เลิกทำ"
474–476	กดเลิกทำ → "ทดสอบกาแฟ" ต้องกลับมา
477–486	หน้าหลัก กดปุ่ม "เพิ่มเงินเข้า" → ต้องเห็น "ได้รับจาก" และแท็บ "รายรับ" ถูกเลือก กดชิป "+\$1,000" ใส่ชื่อ "คำสอนพิเศษ" ถ่ายรูปหน้าจอไว้
488–491	บันทึก → "เพิ่มเงินเข้าแล้ว" และยอดต้องเพิ่ม \$1,000 พอดี

✕

เพิ่มเงินเข้า

รายจ่าย

รายรับ

฿ 1000.00

+฿100

+฿500

+฿1,000

+฿5,000

ได้รับจาก

คำสอนพิเศษ

หมวดหมู่

เงินจากที่บ้าน

เงินเดือน / ค่าจ้าง

งานพิเศษ

รับโอน

รายรับอื่นๆ

วันและเวลา

วันที่ (วว/ดด/ปปปป)

เวลา

09/10/2569

01:05

บันทึกเพิ่มเติม (ไม่บังคับ)

อยากทำอะไรเกี่ยวกับรายการนี้

✓ บันทึกรายการ

ภาพที่ E2E ถ่ายเองที่บรรทัด 487: หน้าเพิ่มเงินเข้า เลือกชิป +฿1,000 และกรอก 'คำสอนพิเศษ'

คำสั่งรัน unit test: npm test · E2E: npm run e2e:web

เช็กलिस्टทดสอบเองบนมือถือ (MT ในเอกสาร TRACEABILITY)

#	ทำ	ต้องเห็น
MT-02	จดยอดรายจ่าย ฿85 หมวดอาหาร	ยอดคงเหลือลด ฿85, หมวดอาหารในภาพรวมเพิ่ม, Runway เปลี่ยน
MT-03	ลบรายการ แล้วกด "เลิกทำ" ภายใน 5 วินาที	รายการกลับมา ยอดกลับเท่าเดิม
MT-10	ล็อกอินบัญชี A กับ B สลับกัน	บัญชี A ไม่เห็นรายการของ B (RLS)
เพิ่ม	พิมพ์ชื่อ "Starbucks" โดยไม่แตะหมวด	หมวดเปลี่ยนเป็น อาหารและเครื่องดื่ม เอง
เพิ่ม	กดหมวด "บันเทิง" ก่อน แล้วพิมพ์ "Starbucks"	หมวดยังเป็นบันเทิง (ระบบไม่เด้งกับ)
เพิ่ม	บนเว็บ พิมพ์วันที่พรุ่งนี้	"วันนี้หรือก่อนหน้าเท่านั้น" และกดบันทึกแล้วไม่บันทึก
เพิ่ม	กดบันทึกเร็ว ๆ 3 ครั้ง	ได้รายการเดียว

FR-1 เชื่อมกับ FR อื่นยังไ้

FR-1 คือ 'ข้อมูลดิบ' ที่ทุก FR ใช้ ถ้า FR-1 ผิด ทุกตัวเลขในแอปผิดตาม

FR	ใช้อะไรจาก FR-1	ตรงไหนในโค้ด
FR-2 ภาพรวม	ยอดคงเหลือ รายรับ/รายจ่าย หมวด ปฏิทิน จำนวนจาก txs ที่ ยืนยันแล้ว · FR-1 ยืมใช้ groupByDay และ spendCalendar ของ FR-2 ในแท็บรายการ	<code>src/domain/summary.ts</code> , <code>src/domain/recap.ts</code>
FR-3 พุดจด	หน้าเสียงบันทึกด้วย addTx ตัวเดียวกัน (source manual, confirmed) และเดาหมวดด้วย suggestCategory ของ FR-1	<code>src/app/voice.tsx:150</code> , <code>src/domain/voice.ts:144</code>
FR-4 สลิป	แบบร่างจากสลิปถูกตรวจ/ยืนยันใน หน้าเดียวกับ FR-1 · กันซ้ำด้วย DuplicateSlipError · "ยืนยันทั้งหมด" ใช้ confirmTx	<code>src/app/transaction.tsx:194</code> , <code>src/data/repo.ts:14</code>
FR-5 โค้ช	โค้ชสรุปจากรายการ (ยอดตามหมวด ค่าเฉลี่ย) ข้อมูลยังครบ คำแนะนำยังตรง	<code>src/domain/insights.ts</code>
FR-6 เงินพอถึง	ค่าเฉลี่ย 7 วันจากรายจ่ายที่จด จดเพิ่ม/ลบ จำนวนวันเปลี่ยนทันที (ข้อความในหน้าต่างลบก็เตือนเรื่องนี้)	<code>src/domain/runway.ts</code>

“ ประโยคเดียวที่สรุปความสำคัญของ FR-1

"FR-1 เป็นประตูลูกที่ข้อมูลเงินเข้าระบบ และเป็นทีี่เดียวที่ผู้ใช้ **แก้ไขข้อมูลที่ผิดได้** ไม่ว่าข้อมูลจะมาจากการพิมพ์ เสียง หรือสลิป สุดท้ายมาจากฟอร์มนี้และตาราง transactions เดียวกัน"

ทำไมฉันออกแบบแบบนี้

ทุกการตัดสินใจมีเหตุผล ใช้ตอบคำถาม 'ทำไม' ได้

หน้าเดียวทำ 4 อย่าง (จด/เงินเข้า/แก้/ตรวจสอบสลิป)	ช่องกรอก ภทตรวจ และปุ่มเหมือนกันหมด แยกเป็น 4 หน้าจะมีโค้ดซ้ำ 4 ชุด แก้บ๊ีกที่เดียวไม่พอ
เก็บเงินเป็นสตางค์จำนวนเต็ม	ทศนิยมในคอมพิวเตอร์บวกแล้วเพี้ยน (0.1 + 0.2) เงินต้องตรงทุกสตางค์
หมวดตายตัว 15 หมวด	กราฟและการเปรียบเทียบของ FR-2/FR-5 ต้องใช้หมวดชุดเดียวกัน ถ้าให้สร้างเอง "อาหาร" "ข้าว" "กิน" จะแยกกัน
เดาหมวดด้วยคำสำคัญ ไม่ใช่ AI	เร็วทันที่ระหว่างพิมพ์ ฟรี ใช้ได้ออฟไลน์ และผลแน่นอนทดสอบได้
หยุดเดาเมื่อผู้ใช้เลือกเอง	เคารพการตัดสินใจของผู้ใช้ ระบบช่วย ไม่ใช่สั่ง
ลบต้องยืนยัน + เลิกทำได้ 5 วินาที	ลบเป็นการกระทำที่ทำลายข้อมูล ป้องกัน 2 ชั้น: ถามก่อน และแก้ได้ถ้าพลาด
ห้ามวันอนาคต	รายการคือสิ่งที่ "เกิดขึ้นแล้ว" วันอนาคตจะทำให้ยอวันนี้และค่าเฉลี่ยของ FR-6 ผิด
จดเอง = confirmed กันที แต่สลิป = draft	คนพิมพ์เองรู้ตัวเลขแน่นอน แต่ AI อ่านสลิปอาจผิด จึงต้องให้คนยืนยันก่อน (ตกลงใน Charter)
ตรวจซ้ำทั้งในแอปและฐานข้อมูล	แอปตรวจเพื่อ UX ที่ดี (บอกทันที) ฐานข้อมูลตรวจเพื่อความปลอดภัย (ไม่มีใครข้ามได้)
หน้าจอเรียกผ่าน Repo เท่านั้น	สลับคลาสวต์/โหมดทดลองได้โดยไม่แก้หน้าจอ และทดสอบได้โดยไม่ต้องมีเซิร์ฟเวอร์จริง
ใส่รายการที่ได้กลับลง state กันที ไม่โหลดใหม่ทั้งหมด	เร็วและประหยัดเน็ต หน้าจอเปลี่ยนกันทีที่บันทึกสำเร็จ
ปิดหน้าด้วย useLeaveWhenDone	ถ้าผู้ใช้ปิดหน้าไปเองระหว่างรอ จะไม่ย้อนกลับซ้ำจนหลุดจากหน้าที่อยู่

ข้อจำกัด และสิ่งที่จะทำต่อ

สิ่งที่ฉันเจอจากการอ่านโค้ดทุกบรรทัด บอกตามจริง

ข้อจำกัด	ผลกระทบ	แก้ยังไง
recurring.ts ยังไม่ถูกแสดงในหน้าจอใด	ผู้ใช้ยังไม่เห็น "ค่าหอ อีก 2 วัน" (ตรรกะและเกสต์พร้อมแล้ว)	เพิ่มการดบบหน้าหลักที่เรียก findRecurring + upcomingRecurring
ข้อความผิดของช่องเงินค่างหลังพิมพ์ถูกแล้ว (ล่างตอนกดบันทึกครั้งถัดไป)	UX เล็กน้อย ผู้ใช้อาจคิดว่ายังผิด	เรียก setAmountError(null) ใน onChangeText ของ AmountField (src/app/transaction.tsx:241)
ช่องเงินลบเลขไทยทั้ง ทั้งที่ parse สองรับ	พิมพ์ "๘๕" จากคีย์บอร์ดไทยแล้วช่องว่าง	เพิ่ม o-๙ ใน regex (src/ui/inputs.tsx:189)
เลิกทำ = เพิ่มใหม่ ได้ id ใหม่	ไม่มีผลต่อผู้ใช้ แต่ถ้าในอนาคตมีตารางอื่นอ้างอิง id รายการ จะหลุด	ใช้ "ลบแบบนุ่ม" (มีคอลัมน์ deleted_at) แทนการลบจริง
คำที่มาก่อนขณะเสมอ	"Grab Food" → เดินทาง (เพราะ "grab" มาก่อน "food") แต่ "grabfood" → อาหาร	เพิ่มคำ "grab food" ในหมวดอาหาร (ยาวกว่า เริ่มที่เดียวกัน จึงชนะ)
หมวดตายตัว	ผู้ใช้สร้างหมวดเองไม่ได้ (เช่น "สัตว์เลี้ยง")	เพิ่มหมวดในรายการ หรือทำหมวดย่อยที่ผูกกับหมวดหลัก
ค้นหาด้วยชื่อหมวดภาษาไทยเท่านั้น	พิมพ์ "food" ไม่เจอหมวดอาหาร (แต่เจอรายการที่ชื่อมี food)	ค้นใน key ของหมวดด้วย
รายการประจำอาจนับข้อมูลน้อยที่บังเอิญตรง	กาแฟ 3 ครั้งใน 2 เดือนถูกนับเป็นประจำ	ต้องการอย่างน้อย 3 เดือน หรือกำหนดยอดขั้นต่ำ
ไม่มีประวัติการแก้ไข	แก้ไขแล้วไม่รู้ว่าเดิมเท่าไร	ตาราง transaction_history บันทึกค่าเก่าผ่าน trigger

คำถามที่อาจารย์อาจถาม พร้อมคำตอบ

ตอบได้ทุกข้อจากสิ่งที่อ่านในเล่มนี้

1. FR-1 ทำอะไรได้บ้าง	เพิ่ม แก้ไข จัดหมวด และลบรายการรับ-จ่าย มีค้นหา กรอง แยกตามวัน ปฏิทิน และเลือกทำหลังลบ
2. ไฟล์หลักคืออะไร	<code>src/app/transaction.tsx</code> (ฟอร์ม) และ <code>src/app/(tabs)/transactions.tsx</code> (แท็บรายการ) ตรรกะอยู่ใน <code>categories.ts</code> และ <code>money.ts</code>
3. ทำไมเก็บเงินเป็นสตางค์	กันทศนิยมเพี้ยน $0.1 + 0.2 \neq 0.3$ มีเทสต์ TC-02 พิสูจน์
4. ใส่ 0 หรือติดลบได้ไหม	ไม่ได้ 2 ชั้น: <code>parseBahtToSatang</code> คืน null แอปแสดง "ใส่จำนวนเงินมากกว่า 0" และฐานข้อมูลมี <code>check amount_satang > 0</code>
5. ระบบเดาหมวดยังงัย	เทียบคำสำคัญของ 15 หมวด คำที่เจอก่อนชนะ เริ่มที่เดียวกันคำยาวชนะ ชื่อคน (นาย/นาง) เป็น "โอนให้คนอื่น" เฉพาะเมื่อไม่มีชื่อร้าน
6. ถ้าผู้ใช้เลือกหมวดเองแล้วพิมพ์ชื่อต่อ	ระบบหยุดเดา เพราะ <code>categoryTouched</code> เป็น true
7. กดบันทึกแล้วเกิดอะไรบ้าง	ตรวจเงิน → ตรวจวัน/เวลา → รวม input → <code>addTx/updateTx</code> → Repo → Supabase insert/update → ได้แถวกลับ → ใส่ใน state → ทุกหน้าคำนวณใหม่ → toast → ปิดหน้า
8. ถ้าเน็ตหลุดตอนบันทึก	Supabase คืน error, Repo โยนต่อ, ฟอร์มจับใน catch แสดง "บันทึกไม่สำเร็จ ลองอีกครั้ง" หน้าไม่ปิด ข้อมูลที่พิมพ์ยังอยู่
9. เลิกทำงานยังงัย	<code>removeTx</code> คืนสำเนารายการที่ลบ toast มีปุ่มเลิกทำ 5 วินาที กดแล้ว <code>addTx</code> ด้วยข้อมูลเดิม (ตัด id กับ <code>createdAt</code>) ได้รายการใหม่ id ใหม่
10. ผู้ใช้คนอื่นแกล่รายการเราได้ไหม	ไม่ได้ RLS policy "own transactions" ให้เห็นและแก้ไขเฉพาะแถวที่ <code>user_id = auth.uid()</code>
11. แอปรู้ได้ยังไงว่าหน้าเป็นแบบแก้ไขหรือจอใหม่	ดู id ใน URL ถ้ามีและหาเจอใน txs = แก้ไข ถ้า status = draft = ตรวจสอบ slip ไม่มี id = จอใหม่ ดู kind=income = เพิ่มเงินเข้า
12. ทำไมบนเว็บต้องพิมพ์วันที่	ไลบรารีตัวเลือกวันที่รองรับเฉพาะ iOS/Android บนเว็บจึงให้พิมพ์ "วว/ดด/ปปปป" แล้วตรวจด้วย <code>parseSlipDate</code> ห้ามวันอนาคต
13. แบบร่างนับในยอดไหม	ไม่นับ แท็บรายการกรองเฉพาะ confirmed และทุกสตูของ FR-2/FR-6 ใช้ <code>confirmedOnly</code> แบบร่างอยู่หน้า "สลิปรอยืนยัน"
14. ทำไมแท็บรายการไม่หน่วงแม้มีหลายพันรายการ	ใช้ <code>SectionList</code> ที่วาดเฉพาะแถวที่อยู่บนจอ และ <code>useMemo</code> คำนวณกลุ่มใหม่เฉพาะเมื่อข้อมูลหรือตัวกรองเปลี่ยน
15. รายการวันที่ 7 เวลา 17:10 UTC อยู่วันไหน	วันที่ 8 เพราะเวลาไทยคือ 00:10 วันที่ 8 <code>groupByDay</code> ใช้ <code>bkkDayKey</code>
16. ทดสอบ FR-1 ยังงัย	Unit test TC-01, 02, 25, 72, 79 (Vitest) และ E2E "Money" ใน Playwright ที่เพิ่ม/แก้/ลบ/เลิกทำ แล้วเช็kyอดทุกชั้น และเช็กลิสต์ MT-02, 03, 10

17. มีอะไรที่ยังไม่ได้ทำ

รายการประจำยังไม่แสดงบนจอ, ข้อความผิดของช่องเงินค้ำ, ช่องเงินไม่รับเลขไทย,
ไม่มีประวัติการแก้ไข

แบบฝึกหัดเชิงความเข้าใจ

ลองคิดเองก่อน แล้วค่อยดูเฉลย ทุกข้อตรวจคำตอบด้วยการรันโค้ดจริงแล้ว และ "วันนี้" = 8 ต.ค. 2569

1. parseBahtToSatang ของ "1,234.5", "85,50", "12.345", "฿ 1 500", "12,345" ได้เท่าไร
2. suggestCategory แบบรายจ่ายของ "Grab Food", "grabfood", "นางสาว มานี ร้านยา", "TRUEMOVE H", "ซื้อหนังสือ SE-ED"
3. เปิด /transaction?id=abc โดยรายการ abc มี status = 'draft' หัวหน้าจอเขียนว่าจะไร และปุ่มล่างมีที่ปุ่ม
4. บนเว็บ พิมพ์วันที่ "09/10/2569", "31/02/2569" และเวลา "8.30", "25:00" แต่ละช่องแสดงข้อผิดพลาดไร
5. ใส่ช่องเงินว่างแล้วกด "บันทึกรายการ" จะเกิดอะไร และบรรทัดไหนทำ
6. มีรายการ: ข้าวมันไก่ ฿65 (8 ต.ค. 08:30), ค่าขนม +฿500 (8 ต.ค. 12:00), ชานม ฿30 (8 ต.ค. 00:10), กาแฟ ฿45 (7 ต.ค. 23:30) เก็บรายการแสดงหัวข้อย่อยอะไรบ้าง และยอดสุทธิเท่าไร
7. Netflix ฿169 จ่ายวันที่ 15 ส.ค. และ 15 ก.ย. findRecurring บอกครั้งถัดไปวันไหน dueLabel ว่าอะไร
8. ลบรายการแล้วกดเลิกทำ id ของรายการที่กลับมาเหมือนเดิมไหม เพราะอะไร

เฉลย

ข้อ	คำตอบ
1	123,450 · 8,550 · null (ทศนิยม 3 ตำแหน่ง) · 150,000 (ช่องว่างถูกลบ) · 1,234,500 (จุลภาคตามด้วย 3 หลัก = คั่นหลักพัน ไม่ใช่ทศนิยม)
2	transport (grab มาก่อน) · food (grabfood ยาวกว่า grab เริ่มที่เดียวกัน) · health (ร้านค้าชณะกฎซื้อคน) · bills (true ขึ้นต้นคำ) · study (หนังสือ)
3	"ตรวจสอบสลิป" · 2 ปุ่ม: "ยืนยันและรวมในยอดเงิน" และ "บันทึกแบบร่างไว้ก่อน" (บรรทัด 185 และ 342–346)
4	"09/10/2569" = 9 ต.ค. เป็นอนาคต → "วันนี้หรือก่อนหน้าเท่านั้น" · "31/02/2569" อ่านไม่ได้ → "เช่น 27/09/2569" · "8.30" อ่านได้เป็น 08:30 ไม่มีข้อผิดพลาด · "25:00" → "เช่น 08:30"
5	ไม่บันทึก แสดง "ใส่จำนวนเงินมากกว่า 0 เช่น 85 หรือ 85.50" ใต้ช่องเงิน (บรรทัด 89–93)
6	"วันนี้" +฿405 (ค่าขนม, ข้าวมันไก่, ชานม) และ "เมื่อวาน" -฿45 (กาแฟ)
7	15 ต.ค. 2569 · "อีก 7 วัน"
8	ไม่เหมือน เพราะเลิกทำ = addTx ใหม่โดยตัด id และ createdAt ออก ฐานข้อมูลสร้าง id ใหม่ให้ (บรรทัด 145–146)

สคริปต์นำเสนอ 1 นาที และคำศัพท์

อ่านออกเสียงได้เลย

“ สคริปต์ 1 นาที

"FR-1 คือการจัดการรายการรับจ่าย ผู้ใช้กด 'จดยรายการ' หรือ 'เพิ่มเงินเข้า' บนหน้าหลัก ใส่จำนวนเงิน ชื่อ หมวด และวันเวลา ระหว่างพิมพ์ชื่อ ระบบเดาหมวดให้จากคำสำคัญ เช่น Starbucks เป็นอาหาร แต่ถ้าผู้ใช้เลือกเองระบบจะไม่เดา กับ

เงินเก็บเป็นสตางค์จำนวนเต็มเพื่อไม่ให้ผลรวมเพี้ยน และตรวจ 2 ชั้น: แอปตรวจก่อนส่ง ฐานข้อมูลตรวจซ้ำด้วย check constraint และ RLS ทำให้แต่ละคนเห็นเฉพาะรายการของตัวเอง

เก็บรายการแสดงทุกรายการแยกตามวันไทย มีค้นหา กรอง และปฏิทิน แตะรายการเพื่อแก้หรือลบ การลบต้องยืนยันก่อนและกดเลิกทำได้ภายใน 5 วินาที

ทุกการเปลี่ยนแปลงไหลผ่าน AppProvider และ Repo ไป Supabase แล้วยอดคงเหลือ ภาพรวม และ Money Runway คำนวณใหม่ทันที เราทดสอบด้วย unit test TC-01, 02, 25, 72, 79 และ E2E ที่เพิ่ม แก้ ลบ เลิกทำ แล้วเช็kyอดเงินทุกชั้น"

คำศัพท์ของ FR-1

คำ	ความหมาย
CRUD	Create Read Update Delete = เพิ่ม อ่าน แก้ ลบ ซึ่งการกระทำพื้นฐานของข้อมูล FR-1 คือ CRUD ของรายการ
satang (สตางค์)	หน่วยที่ใช้เก็บเงิน 1 บาท = 100 สตางค์ เป็นจำนวนเต็มเสมอ
state	ความจำของหน้าจอ เปลี่ยนแล้ว React วาดใหม่
derived value	ค่าที่คำนวณจาก state ได้ ไม่ต้องเก็บแยก (เช่น dayError)
props	ข้อมูลที่ส่งเข้า component (เช่น tx ของ TxRow)
URL params	ค่าหลัง ? ใน URL เช่น ?id=abc&kind=income
modal	หน้าต่างที่ทับหน้าต่างเดิม ปิดแล้วกลับที่เดิม
toast	ข้อความแจ้งเตือนสั้น ๆ ที่หายไป
undo (เลิกทำ)	ย้อนการกระทำล่าสุด
destructive action	การกระทำที่ทำลายข้อมูล ต้องยืนยันก่อน
Repo (Repository)	ชั้นที่ซ่อนว่าข้อมูลเก็บที่ไหน (คลาวด์หรือในเครื่อง)
interface	สัญญาว่า object ต้องมีอะไรบ้าง

คำ	ความหมาย
snake_case / camelCase	amount_satang (ฐานข้อมูล) / amountSatang (TypeScript)
check constraint	กฎในฐานข้อมูลที่ปฏิเสธข้อมูลผิด เช่น amount_satang > 0
unique index	กฎว่าค่าห้ามซ้ำ (ใช้กันสลับซ้ำ)
RLS	Row Level Security กฎว่าใครเห็น/แก้ไขไหนได้
trigger	คำสั่งในฐานข้อมูลที่ทำงานเองเมื่อมีเหตุการณ์ (เช่น อัปเดต updated_at)
regular expression	รูปแบบข้อความ ใช้ค้น/ตรวจ/ลบตัวอักษร
median (มัธยฐาน)	ค่ากลางเมื่อเรียงข้อมูล ไม่โดนค่าผิดปกติดึง
SectionList	รายการยาวแบบมีหัวข้อย่อย วาดเฉพาะที่อยู่บนจอ